

ELVIR ČAJIĆ

Programiranje

Python 3 i C++17

Teorija i zbirka zadataka
za osnovne škole

Od prve naredbe do takmičarskog rješenja
Sa zadacima gradskih/općinskih takmičenja
i odabranim srednjoškolskim izazovima

RECENZIJU

Josip Vojnić

stručni savjetnik

UPINITK

Tuzla, 2026.

IMPRESUM

Programiranje – Python 3 i C++17

Teorija i zbirka zadataka za osnovne škole

Uloga	Podatak
Autor	Elvir Čajić
Recenzija	Josip Vojnić, stručni savjetnik
Programski jezici	Python 3 i C++17
Mjesto i godina	Tuzla, 2026.
Namjena	Samostalno učenje, dodatna nastava i priprema za takmičenja
Povezana arhiva	UPINITK – Udruženje profesora i nastavnika informatike u Tuzlanskom kantonu

O porijeklu zadataka

Knjiga objedinjuje razrađene vježbe iz pripremnog rukopisa i posebno označen izbor iz UPINITK biltena. Arhivski zadaci zadržavaju prepoznatljiv naziv i izvor. Njihovi tekstovi ovdje su sažeto preformulisani, a objašnjenja, novi primjeri i Python/C++ rješenja izrađeni su za ovu razradu. Nije riječ o doslovnom pretisku biltena niti o tvrdnji da je autor knjige izvorni autor svih takmičarskih postavki.

Uz svaki arhivski zadatak nalazi se oznaka izvora i fizička stranica izvornog PDF-a. Izbor obuhvata osnovnoškolska gradska/općinska takmičenja 2021–2023. i zasebno tri srednjoškolska zadatka iz 2024. godine. Vježbe bez takve oznake nisu predstavljene kao službeni zadaci tih takmičenja.

Uloga recenzenta navedena je prema autorskom podatku. Ovo izdanje ne sadrži potpisanu recenziju niti tvrdi da je formalna recenzija završena.

Predgovor

Programiranje učimo tako što povezujemo problem, postupak i rezultat. Dobar program nije samo niz naredbi koji se pokreće: on mora odgovarati tekstu zadatka, poštovati ograničenja i dati tačan odgovor i na primjerima koje unaprijed nismo vidjeli.

Ova knjiga polazi od kratkih programa i postupno uvodi grananje, petlje, funkcije, nizove, stringove i matrice. Nakon tih osnova slijede efikasnije tehnike: sortiranje, prefiksi, prozori, strukture podataka, teorija brojeva, grafovi i dinamičko programiranje. Napredne teme izdvojene su tako da početnik ne mora preskakati između nepovezanih ideja.

Teorija nije zamišljena kao spisak naziva. Svaka cjelina najprije objašnjava problem koji nova tehnika rješava, zatim prati mali primjer i tek onda uvodi sažet programski zapis. U zbirci se isti pristup nastavlja: tekst i format podataka, primjer, obrazloženje postupka, složenost i potpuna rješenja u oba jezika.

Za učenika je najvažnije da prije gledanja rješenja pokuša napraviti vlastiti mali primjer i izgovoriti ideju algoritma. Nastavniku to omogućava da razlikuje nerazumijevanje modela od nesigurnosti u sintaksi. Greška se ne ispravlja nasumičnim mijenjanjem naredbi: prvo treba pronaći pretpostavku koja nije tačna.

Posebna cjelina donosi zadatke iz lokalne UPINITK arhive. Njihova vrijednost je u vezi s stvarnim takmičarskim iskustvom. Dodatni srednjoškolski izazovi služe proširenju, a ne predstavljaju obavezni program osnovne škole. Odabir tempa i težine treba pratiti stvarno znanje učenika.

Elvir Čajić

Kako čitati i koristiti knjigu

Za prvi susret s programiranjem

Prođite teorijska poglavlja 1–5. U jednom jeziku pokrenite primjere, promijenite ulaz i ručno predvidite izlaz. Nakon toga rješavajte osnovne vježbe. Poznavanje oba jezika nije uslov za početak.

Za školsko i gradsko/općinsko takmičenje

Povežite osnovne vježbe s izdvojenim arhivskim zadacima. Uz svaki pokušaj zapišite ulaz, izlaz i razlog ispravnosti. Srednje teške zadatke o intervalima i prozorima radite nakon odgovarajuće teorije, a ne samo prema broju u zbirci.

Za napredni trening

Prije grafova, stabala i dinamičkog programiranja savladajte indeksiranje, složenost, funkcije i osnovne strukture. Vježbe srodnih algoritamskih obrazaca služe poređenju uslova; sličan naziv ne znači uvijek isti zadatak.

Kako su prikazani kodovi

U teoriji oznaka „isječak“ znači da blok prikazuje dio većeg programa i može pretpostavljati ranije definisane varijable. U zadacima su Python 3 i C++17 blokovi potpuni programi. Kada kod prelazi na narednu stranicu, oznaka jezika ponavlja se uz riječ „nastavak“. Za neposredno pokretanje priloženi paket sadrži zasebne .py i .cpp datoteke.

C++ primjeri iz rukopisa mjestimično koriste GNU zaglavlje `bits/stdc++.h`. Ono nije dio standarda C++17; u drugom prevodiocu treba uključiti konkretna standardna zaglavlja. Provjere programa obavljene su s GNU g++ u režimu C++17. Imena varijabli u kodu uglavnom ne koriste dijakritičke znakove.

Razumijevanje ima prednost nad brojem stranica

Teže nije automatski bolje. Zadovoljavajuće rješenje je ono čiju ideju učenik može objasniti, čiju složenost može opravdati i za koje može napraviti smislene testove. Jedno pažljivo analizirano rješenje vrijedi više od deset prepisanih programa.

Sadržaj

Predgovor	3
Kako čitati i koristiti knjigu	4
1. Prvi program: Python i C++	7
2. Grananje, petlje i funkcije	13
3. Složenost i pouzdanost programa	20
4. Nizovi, sortiranje, skupovi i mape	22
5. Stringovi i dvodimenzionalne mreže	27
6. Prefiksi, prozori i binarna pretraga	33
7. Stek, red i upiti nad intervalima	39
8. Teorija brojeva i binarni zapis	43
9. Pohlepni izbor i potpuno pretraživanje	50
10. Grafovi i najkraći putevi	52
11. Stabla: korijen, podstablo i preci	59
12. Dinamičko programiranje	62
13. Napredni obrasci i granice njihove primjene	67
14. Osnovne vježbe: računanje i obrada podataka	70
15. UPINITK: gradska/općinska takmičenja 2021–2023	102
16. Nizovi, stringovi i sortiranje	127
17. Intervali, prefiksi i pretraga odgovora	164
18. Mreže i potpuno pretraživanje	202

19. Djeljivost, prosti brojevi i bitovi	231
20. Strukture podataka i obrada stringova	257
21. Grafovi, povezivanje i najkraći putevi	299
22. Stabla, dinamičko programiranje i napredni izazovi	394
23. UPINITK: dodatni srednjoškolski izazovi 2024	454
24. Trening, provjera i pojmovnik	462
25. Izvori i porijeklo zadataka	464

1. Prvi program: Python i C++

U ovom poglavlju polazimo od programa koji se može odmah pokrenuti. Zatim objašnjavamo ulaz, izlaz, vrijednosti, tipove i operatore. Za početak izaberite jedan jezik; drugi koristite kao paralelan zapis iste ideje.

Od teksta zadatka do izvršavanja

Zamislamo zadatak: poznate su cijena jedne sveske i broj sveski; treba odrediti ukupan iznos. Prije pisanja programa imenujemo podatke: cijena i kolicina. Rezultat je njihov proizvod. Algoritam se sastoji od tri koraka: učitaj podatke, izračunaj proizvod, ispiši proizvod. Isti postupak važi i kada sveske zamijenimo ulaznicama ili bodovima po kolu.

Program pišemo u običnom tekstualnom dokumentu. Python izvorni kod ima nastavak .py, a C++ kod .cpp. Datoteka se ne smije završavati prikrivenim nastavkom .txt. U terminalu se Python primjer može pokrenuti naredbom `python3 prvi.py`, odnosno `python prvi.py` u okruženju koje tako imenuje interpreter. Za C++ primjer s GNU prevodiocem naredba `g++ -std=c++17 -O2 prvi.cpp -o prvi` pravi izvršni program; na Linuxu ga pokrećemo sa `./prvi`, a na Windowsu imenom nastalog .exe programa. U školskom razvojnom okruženju istu ulogu imaju komande za izgradnju i pokretanje. [5, 6]

Python red `input()` pročita tekst. `split()` ga razdvoji po razmacima, a `map(int, ...)` svaku riječ pretvori u cijeli broj. Dodjela raspoređuje dvije vrijednosti u dva imena. `print()` šalje rezultat na standardni izlaz. U C++-u cin obavlja učitavanje u prethodno deklarirane varijable; `cout` ispisuje rezultat. Funkcija `main` označava početak programa, a zaglavlje `iostream` omogućava ove ulazno-izlazne objekte.

Takmičarski program ne ispisuje „Unesite cijenu“ ako takva poruka nije tražena. Ocjenjivač očekuje odgovor prema zadatom formatu. Za ulaz `7 4` izlaz je `28`. Taj primjer provjerava da smo pravilno povezali podatke, ali još ne provjerava sve moguće ulaze.

Važno je razlikovati grešku u zapisu, grešku pri izvršavanju i pogrešan algoritam. Nedostajuća zagrada može spriječiti pokretanje. Dijeljenje nulom može prekinuti program. Zamjena proizvoda zbirom može dati uredno izvršen, ali pogrešan program. Zato nije dovoljno da se program samo pokrene.

Korak	Šta se dešava
Ulaz <code>7 4</code>	<code>cijena = 7, kolicina = 4</code>
Računanje	<code>ukupno = 7 · 4 = 28</code>
Izlaz	<code>28</code>

PYTHON 3 • vođeni primjer

```

cijena, kolicina = map(int, input().split())
ukupno = cijena * kolicina
print(ukupno)

```

C++17 • potpuni program

```

#include <iostream>
using namespace std;
int main() {
    long long cijena, kolicina;
    cin >> cijena >> kolicina;
    cout << cijena * kolicina << "\n";
    return 0;
}

```

1.1. Program, algoritam i izvršavanje

Program je zapis instrukcija, a algoritam je konačan i precizan postupak koji od ulaznih podataka vodi do izlaza. Isti algoritam može biti implementiran u različitim jezicima.

Primjena. Ovo razlikovanje je važno kada prvo tražimo ideju rješenja, a tek zatim pišemo kod.

Primjer. Za brojeve 7 i 5 algoritam „učitaj, saberi, ispiši“ daje 12.

Obratite pažnju. Ne počinjati kodirati prije nego što je jasno šta tačno treba izračunati.

PYTHON 3 • isječak

```

a, b = map(int, input().split())
print(a + b)

```

C++17 • isječak

```

#include <bits/stdc++.h>
using namespace std;
int main(){ long long a,b; cin>>a>>b; cout<<a+b; }

```

1.2. Ulaz, izlaz i specifikacija zadatka

Svaki takmičarski zadatak definiše format ulaza, format izlaza i ograničenja. Ograničenja nisu ukras: ona određuju koji algoritam može proći vremensko i memorijsko ograničenje.

Primjena. Prije rješavanja treba podvući šta je dato, šta se traži i kolike su najveće vrijednosti.

Primjer. Ako je $n = 10^5$, dvostruka petlja $O(n^2)$ obično nije prihvatljiva.

Obratite pažnju. Ignorisanje ograničenja često vodi do tačnog, ali presporog rješenja.

PYTHON 3 • isječak

```
n = int(input())
a = list(map(int, input().split()))
```

C++17 • isječak

```
int n; cin>>n;
vector<int>a(n); for(int &x:a) cin>>x;
```

1.3. Varijable i konstante

Varijabla povezuje ime s vrijednošću koju program koristi. U Pythonu ime upućuje na objekt; u C++-u deklarirana varijabla ima određen tip. Konstanta predstavlja vrijednost koju namjerno ne mijenjamo.

Primjena. Koristimo ih za čuvanje ulaza, međurezultata, brojača i granica.

Primjer. Umjesto `x1` i `x2` bolje je koristiti `suma`, `maksimum`, `broj_elementa`.

Obratite pažnju. Prekratka imena su prihvatljiva za brojače `i`, `j`, ali ne za važne pojmove algoritma.

PYTHON 3 • isječak

```
n = 10
LIMIT = 1000000007
```

C++17 • isječak

```
int n=10;
const long long MOD=1000000007LL;
```

1.4. Cijeli brojevi i 64-bitni tipovi

Cijeli brojevi nemaju decimalni dio. U Pythonu se veliki cijeli brojevi automatski proširuju, dok u C++-u tip `int` ima ograničen raspon. Za velike sume i proizvode u C++-u se koristi `long long`.

Primjena. Posebno kod suma do 10^5 elemenata vrijednosti do 10^9 , jer rezultat može biti oko 10^{14} .

Primjer. $10^5 \cdot 10^9 = 10^{14}$, što ne stane u 32-bitni `int`.

Obratite pažnju. Overflow u C++-u može dati potpuno pogrešan rezultat bez upozorenja.

Dodatno. Python `int` nema fiksnu širinu, ali računanje s velikim brojevima troši dodatnu memoriju i vrijeme. C++ `long long` nije proizvoljne preciznosti. Veličinu raspona treba provjeriti prije računanja, ne tek pri ispisu.

PYTHON 3 • isječak

```
s = sum(a)
```

C++17 • isječak

```
long long s=0;
for(long long x:a) s+=x;
```

1.5. Realni brojevi i preciznost

Realni brojevi se u računarima najčešće čuvaju približno. Zbog binarnog zapisa vrijednost poput 0.1 nije uvijek predstavljena potpuno tačno.

Primjena. Koristimo ih kada zadatak zaista zahtijeva mjerenja, prosjeke, udaljenosti ili toleranciju.

Primjer. Umjesto provjere $a == b$ za realne brojeve često provjeravamo $|a-b| < \text{eps}$.

Obratite pažnju. Ne koristiti floating point ako se sve može riješiti cijelim brojevima.

Dodatno. Apsolutna tolerancija nije univerzalna. Kod različitih razmjera brojeva često treba kombinovati apsolutnu i relativnu toleranciju. Kada zadatak dopušta cijele brojeve, njima izbjegavamo nepotrebne približne račune.

PYTHON 3 • isječak

```
eps = 1e-9
if abs(a-b) < eps: print("jednako")
```

C++17 • isječak

```
double eps=1e-9;
if(fabs(a-b)<eps) cout<<"jednako";
```

1.6. Logičke vrijednosti i uslovi

Logička vrijednost je tačno ili netačno. Složeni uslovi nastaju operatorima AND, OR i NOT. U Pythonu su to `and`, `or`, `not`, a u C++-u `&&`, `||`, `!`.

Primjena. Koristimo ih za modeliranje pravila zadatka i grananja algoritma.

Primjer. Broj x pripada intervalu $[1,10]$ ako vrijedi $x \geq 1$ i $x \leq 10$.

Obratite pažnju. Kod složenih uslova koristiti zagrade kada postoji mogućnost nejasnog prioriteta.

PYTHON 3 • isječak

```
ok = 1 <= x <= 10
if ok: print("DA")
```

C++17 • isječak

```
bool ok=(x>=1 && x<=10);
if(ok) cout<<"DA";
```

1.7. Aritmetički operatori i prioritet

Osnovni operatori su +, -, *, / i ostatak pri dijeljenju %. Cjelobrojno dijeljenje daje količnik bez ostatka. Prioritet operatora određuje redoslijed računanja.

Primjena. Važno je kod formula, cifara broja, parnosti i modularnih izraza.

Primjer. Posljednja cifra broja n je n % 10, a uklanjanje posljednje cifre je n // 10.

Obratite pažnju. Kad je formula važna, zagrade su bolja zaštita od oslanjanja na pamćenje prioriteta.

Razlika jezika. Za negativne cijele brojeve Python // zaokružuje količnik prema minus beskonačno, dok C++ cjelobrojni / skraćuje prema nuli. Zato -7 // 3 daje -3, a C++ izraz -7 / 3 daje -2. Za pozitivni modul m, C++ ostatak se može normalizirati izrazom (x % m + m) % m.

PYTHON 3 • isječak

```
zadnja = n % 10
n //= 10
```

C++17 • isječak

```
int zadnja=n%10;
n/=10;
```

1.8. Konverzija tipova

Konverzija pretvara vrijednost iz jednog tipa u drugi. Ulaz sa tastature je tekst koji se često mora pretvoriti u broj. Obrnuto, broj se nekad pretvara u string radi rada s ciframa.

Primjena. Korisno za parsiranje ulaza, rad s ciframa i kontrolu cjelobrojnog/realnog računanja.

Primjer. Broj 2026 kao string omogućava direktan pristup cifri s[0]='2'.

Obratite pažnju. Paziti da konverzija ima smisla: tekst "12a" nije ispravan cijeli broj.

PYTHON 3 • isječak

```
n = int("2026")  
s = str(n)
```

C++17 • isječak

```
int n=stoi("2026");  
string s=to_string(n);
```

2. Grananje, petlje i funkcije

Grananje bira postupak, petlja ga ponavlja, a funkcija ga izdvaja u smislenu cjelinu. Nije cilj naučiti oblike naredbi napamet, nego pratiti kako se stanje programa mijenja.

Praćenje stanja na papiru

Za niz 4, 1, 7, 2 želimo zbir samo parnih elemenata. Brojač zbir postavimo na nulu: prije obrade nema izabranih brojeva. Zatim prolazimo kroz sve elemente. Uslov $x \% 2 == 0$ odlučuje dodajemo li trenutni element. Poslije svake iteracije zbir je suma parnih brojeva u već obrađenom dijelu niza.

Ta posljednja rečenica je invarijanta petlje. Ona povezuje kod sa značenjem zadatka. Prije prve iteracije važi jer je zbir praznog skupa jednak nuli. Ako važi prije narednog elementa, dodavanje parnog ili preskakanje neparnog elementa čuva je. Na kraju je obrađen cijeli niz, pa je zbir traženi odgovor. Tako kratka tvrdnja može objasniti ispravnost mnogo jasnije od prepričavanja svake naredbe.

Uslov u while petlji provjerava se prije ulaska u tijelo. Ako je odmah netačan, tijelo se ne izvršava nijednom. Zato početna vrijednost brojača i promjena koja vodi ka završetku moraju biti namjerne. U for petlji nad range(n) dobijamo indekse 0, 1, ..., n-1; gornja granica nije uključena. Kod granica je korisno isprobati $n=0$ i $n=1$ kada ih zadatak dopušta.

Funkcija nije samo način da kod bude kraći. Njome izdvajamo ugovor: šta ulazi i šta se vraća. Funkcija paran(x) vraća logičku vrijednost, a ne ispisuje odgovor. Funkcija zbir_parnih(a) vraća broj. Pozivalac odlučuje šta će s njim. Ovakvo razdvajanje olakšava provjeru malih dijelova prije spajanja cijelog rješenja.

Rekurzija opisuje postupak koji poziva sebe za manji podproblem. Mora imati bazni slučaj i mjeru napretka ka njemu. Dubina rekurzije koristi memoriju steka; matematički tačan rekurzivni obilazak veoma dugog lanca može biti praktično nebezbedan. Iterativna petlja ili eksplicitni stek često su pouzdaniji kada dubina može biti velika.

Element	Paran?	Zbir nakon obrade
4	DA	4
1	NE	4
7	NE	4
2	DA	6

PYTHON 3 • vođeni primjer

```
def zbir_parnih(a):
    zbir = 0
    for x in a:
        if x % 2 == 0:
            zbir += x
    return zbir

print(zbir_parnih([4, 1, 7, 2]))
```

C++17 • isječak

```
long long zbir_parnih(const vector<int>& a) {
    long long zbir = 0;
    for (int x : a) {
        if (x % 2 == 0) zbir += x;
    }
    return zbir;
}
```

2.1. Jednostavni if/else uslovi

Grananje omogućava da program izvrši različite naredbe zavisno od uslova. if provjerava uslov, a else pokriva suprotan slučaj.

Primjena. Koristi se za parnost, poređenje, provjeru granica i izbor jedne od više mogućnosti.

Primjer. Ako je x paran ispisujemo PARAN, inače NEPARAN.

Obratite pažnju. Uvijek provjeriti šta se dešava na granici, npr. x=0 ili x=limit.

PYTHON 3 • isječak

```
print("PARAN" if x%2==0 else "NEPARAN")
```

C++17 • isječak

```
if(x%2==0) cout<<"PARAN"; else cout<<"NEPARAN";
```

2.2. Višestruko i ugniježđeno grananje

Kada postoje tri ili više slučajeva, koristimo elif/else ili else if. Ugniježđeni if je if unutar drugog if-a.

Primjena. Pogodno za klasifikaciju bodova, ocjena, intervala i kombinovanih pravila.

Primjer. Bodovi 90: A; 75: B; inače C.

Obratite pažnju. Redoslijed uslova je bitan: prvo se provjerava najjači ili najuži slučaj.

PYTHON 3 • isječak

```
if p>=90: o="A"
elif p>=75: o="B"
else: o="C"
```

C++17 • isječak

```
if(p>=90) o="A";
else if(p>=75) o="B";
else o="C";
```

2.3. for petlja

for petlja ponavlja naredbe poznat ili lako opisiv broj puta. U Pythonu se često koristi range, a u C++-u brojačka petlja.

Primjena. Idealna je za prolazak kroz niz, ponavljanje n puta i rad sa indeksima.

Primjer. Zbir brojeva od 1 do n dobija se dodavanjem svakog i.

Obratite pažnju. Paziti na krajnju granicu: Python range ne uključuje gornju granicu.

PYTHON 3 • isječak

```
s=0
for i in range(1,n+1): s+=i
```

C++17 • isječak

```
long long s=0;
for(int i=1;i<=n;i++) s+=i;
```

2.4. while petlja

while ponavlja blok sve dok je uslov tačan. Broj ponavljanja ne mora biti poznat unaprijed.

Primjena. Koristi se pri obradi cifara broja, simulacijama i algoritmima koji završavaju kada se stanje promijeni.

Primjer. Dok je $n > 0$ možemo uzimati cifre sa $n \% 10$.

Obratite pažnju. Petlja mora napredovati prema završetku; inače nastaje beskonačna petlja.

PYTHON 3 • isječak

```
while n>0:
    cifra=n%10
    n//=10
```

C++17 • isječak

```
while(n>0){ int cifra=n%10; n/=10; }
```

2.5. break i continue

break odmah prekida najbližu petlju. continue preskače ostatak trenutne iteracije i prelazi na sljedeću.

Primjena. Korisni su za rano zaustavljanje pretrage i preskakanje elemenata koji nas ne zanimaju.

Primjer. Pretraga prvog negativnog broja može se završiti čim ga nađemo.

Obratite pažnju. Pretjerana upotreba break/continue može učiniti tok programa teškim za praćenje.

PYTHON 3 • isječak

```
for x in a:
    if x<0:
        ans=x; break
```

C++17 • isječak

```
for(int x:a){ if(x<0){ ans=x; break; } }
```


2.6. Funkcije i modularnost

Funkcija grupiše dio algoritma pod jasnim imenom. Time se kod dijeli na manje cjeline koje je lakše testirati i ponovo koristiti.

Primjena. Koristimo ih za ponavljajuće operacije i izdvajanje logičkih koraka algoritma.

Primjer. Funkcija `kvadrat(x)` vraća $x \cdot x$.

Obratite pažnju. Funkcija treba imati jednu jasnu odgovornost, a ne raditi sve odjednom.

PYTHON 3 • isječak

```
def kvadrat(x):  
    return x*x
```

C++17 • isječak

```
long long kvadrat(long long x){ return x*x; }
```

2.7. Parametri i povratna vrijednost

Parametar je podatak koji funkcija prima. Povratna vrijednost je rezultat koji funkcija predaje pozivaocu. Funkcija može i samo mijenjati strukturu ili ispisivati, ali je često čišće vratiti rezultat.

Primjena. Pomaže da se odvoji računanje od ulaza/izlaza.

Primjer. `max2(a,b)` vraća veći od dva broja.

Obratite pažnju. Ne miješati ispis i računanje bez potrebe; funkcija koja vraća rezultat je fleksibilnija.

PYTHON 3 • isječak

```
def max2(a,b):  
    return a if a>b else b
```

C++17 • isječak

```
int max2(int a,int b){ return a>b?a:b; }
```

2.8. Doseg varijabli (scope)

Lokalna varijabla postoji unutar funkcije ili bloka, dok je globalna dostupna šire. Previše globalnih podataka otežava razumijevanje i testiranje.

Primjena. Važno je kod većih algoritama, rekurzije i grafova.

Primjer. Lokalni brojač i unutar funkcije ne utiče na istoimenu varijablu izvan funkcije.

Obratite pažnju. Globalne varijable koristiti samo kada pojednostavljaju strukturu bez skrivenih zavisnosti.

PYTHON 3 • isječak

```
def f():
    x=5
    return x
```

C++17 • isječak

```
int f(){ int x=5; return x; }
```

2.9. Rekurzija

Rekurzivna funkcija poziva samu sebe na manjem podproblemu. Mora imati bazni slučaj koji zaustavlja pozive.

Primjena. Prirodna je za stabla, DFS, generisanje kombinacija i divide-and-conquer algoritme.

Primjer. Faktorijel: $n! = n \cdot (n-1)!$, uz $0!=1$.

Obratite pažnju. Bez baznog slučaja nastaje beskonačna rekurzija; duboka rekurzija može prekoračiti stek.

PYTHON 3 • isječak

```
def fact(n):
    if n==0: return 1
    return n*fact(n-1)
```

C++17 • isječak

```
long long fact(int n){ if(n==0) return 1; return 1LL*n*fact(n-1); }
```

2.10. Invariant i tvrdnja o ispravnosti

Invariant je svojstvo koje ostaje tačno tokom izvršavanja algoritma. Dokaz ispravnosti često pokazuje da invariant važi na početku, ostaje očuvan i na kraju daje traženi rezultat.

Primjena. Koristan je kod petlji, dva pokazivača, greedy algoritama i struktura podataka.

Primjer. Kod traženja maksimuma invariant može biti: nakon obrade prvih i elemenata, $best$ je njihov maksimum.

Obratite pažnju. Ne zadovoljavati se time da algoritam „izgleda dobro“; pitati zašto ne može propustiti bolji odgovor.

PYTHON 3 • isječak

```
best=a[0]
for x in a[1:]: best=max(best,x)
```

C++17 • isječak

```
int best=a[0];
for(int x:a) best=max(best,x);
```

3. Složenost i pouzdanost programa

Dva programa mogu dati isti odgovor na malom primjeru, a potpuno se razlikovati kada ulaz poraste. Analiza složenosti služi izboru postupka, ne ukrašavanju rješenja oznakom O.

Zašto mali primjer nije dovoljan

Pretpostavimo da za svaki od n brojeva pregledamo svih n brojeva. Broj poređenja raste približno kao n^2 . Za $n=100$ to je 10 000, a za $n=200$ čak 40 000. Deset puta veći ulaz zato ne znači deset, nego približno sto puta više posla. Nasuprot tome, jedan prolaz raste linearno: svaki novi element dodaje sličnu količinu posla.

Oznaka O opisuje gornji red rasta, a ne precizno trajanje u sekundama. Broj operacija koje stanu u vremensko ograničenje zavisi od jezika, operacija, implementacije i računara. Zato nije pouzdano reći da neki algoritam „sigurno prolazi“ samo iz broja n . Ipak, prelazak s kvadratnog na linearno-logaritamski rast često je presudan.

Kod memorije razlikujemo ulaznu strukturu i dodatnu memoriju algoritma. Sortiranje po mjestu u jednom jeziku ne mora imati iste memorijske troškove kao sortiranje u drugom. Python lista čuva reference na objekte, pa procjena n puta četiri bajta nije dobra opća procjena za listu cijelih brojeva. U C++-u veličinu konkretnog tipa može pokazati sizeof, ali ukupna struktura sadrži i dodatne podatke.

Testiranje počinje prije nego što imamo brz algoritam. Najprije napravimo nekoliko malih ulaza čiji odgovor možemo pouzdano dobiti rukom. Zatim napišemo sporo, očigledno rješenje za male veličine i poredimo ga s brzim na mnogim generisanim primjerima. Neslaganje je prilika da izdvojimo najmanji primjer koji ruši pretpostavku. Saglasnost mnogih testova povećava pouzdanje, ali nije matematički dokaz.

Rubni test treba imati razlog. Za maksimum niza važni su jedan element i svi negativni brojevi. Za intervale su važni dodirivanje krajeva i isti počeci. Za graf su važni izolovan čvor i više komponenti. Za ruksak je važan predmet čija je težina veća od kapaciteta. Nasumični testovi ne zamjenjuju ove ciljane slučajeve.

Obrazac	Tipičan red rada	Pitanje za učenika
Jedan prolaz	$O(n)$	Da li svaki element obrađujem stalno puta?
Dva puna ugniježđena prolaza	$O(n^2)$	Mogu li sačuvati već izračunate podatke?
Sortiranje pa prolaz	$O(n \log n)$	Koja korist dolazi od poretka?
Prepolovljavanje prostora	$O(\log n)$	Zašto odbačena polovina ne sadrži odgovor?

3.1. Vremenska složenost

Vremenska složenost opisuje kako broj osnovnih operacija raste sa veličinom ulaza n . Najčešće koristimo $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.

Primjena. Ona povezuje ograničenja zadatka sa izborom algoritma.

Primjer. Za $n=200000$ algoritam $O(n \log n)$ je realan, a $O(n^2)$ uglavnom nije.

Obratite pažnju. Uvijek analizirati najskuplji dio algoritma, uključujući ugniježdene petlje i pozive.

PYTHON 3 • isječak

```
a.sort() # približno  $O(n \log n)$ 
```

C++17 • isječak

```
sort(a.begin(),a.end()); //  $O(n \log n)$ 
```

3.2. Memorijska složenost

Memorijska složenost procjenjuje dodatnu memoriju koju algoritam koristi. Niz od n elemenata je $O(n)$, matrica $n \times m$ je $O(nm)$.

Primjena. Važna je kod velikih grafova, DP tabela i matrica.

Primjer. Matrica 10000×10000 cijelih brojeva zahtijevala bi stotine MB memorije.

Obratite pažnju. Ne čuvati punu DP tabelu ako je za sljedeći red potreban samo prethodni.

PYTHON 3 • isječak

```
dist=[-1]*n #  $O(n)$ 
```

C++17 • isječak

```
vector<int> dist(n,-1); //  $O(n)$ 
```

4. Nizovi, sortiranje, skupovi i mape

Niz čuva redoslijed. Skup čuva pripadnost. Mapa veže ključ za vrijednost. Odabir strukture treba slijediti pitanje koje postavljamo podacima.

Jedan niz, tri različita pitanja

Neka su rezultati 7, 4, 7, 2, 4, 7. Ako tražimo ukupan broj bodova, dovoljan je zbir. Ako tražimo različite rezultate, trebaju nam vrijednosti 2, 4 i 7. Ako tražimo najčešći rezultat, moramo sačuvati i broj ponavljanja: 2 se javlja jednom, 4 dvaput, 7 triput. To su tri različita pogleda na isti ulaz.

Skup uklanja višestruka pojavljivanja, ali time gubi frekvencije. Mapa frekvencija čuva ih kao parove rezultat broj. Ako su svi rezultati između 0 i 100, niz od 101 brojača može biti jednostavniji od mape. Ako su vrijednosti velike ili tekstualne, mapa je prirodnija. „Najbolja struktura“ zato nije nezavisna od ograničenja.

Sortiranje mijenja redoslijed elemenata. Ako zadatak pita za originalni položaj učenika, prije sortiranja uz svaki rezultat treba sačuvati taj položaj. Za rangiranje može važiti više kriterija: veći broj bodova prvo, a kod jednakosti manji redni broj. U Pythonu takav ključ može biti (-bodovi, redni_broj). U C++-u komparator mora dosljedno opisivati strogi poredak; za dva potpuno jednaka elementa ne smije tvrditi da je prvi manji od drugog.

Promjena jednog objekta i pravljenje kopije nisu isto. U Pythonu `b=a` ne kopira listu: oba imena upućuju na istu listu. Za plitku kopiju koristimo `a.copy()` ili `a[:]`. Kod matrice ni kopiranje spoljne liste nije uvijek dovoljno ako želimo nezavisne unutrašnje redove. U C++-u dodjela jednog vector objekta drugom kopira njegove elemente.

Minimum i maksimum ne treba inicijalizirati nulom ako podaci mogu biti negativni. Za neprazan niz najbezbjednije je krenuti od prvog elementa. Za prazan niz moramo imati posebno definisano ponašanje ili dokaz da takav ulaz nije dopušten. Program ne smije pretpostaviti ono što u tekstu nije dato.

Vrijednost	Frekvencija
2	1
4	2
7	3

PYTHON 3 • vođeni primjer

```
a = [7, 4, 7, 2, 4, 7]
frekvencija = {}
for x in a:
    frekvencija[x] = frekvencija.get(x, 0) + 1
for x in sorted(frekvencija):
    print(x, frekvencija[x])
```

C++17 • isječak

```
vector<int> a = {7, 4, 7, 2, 4, 7};
map<int, int> frekvencija;
for (int x : a) frekvencija[x]++;
for (auto [x, broj] : frekvencija) {
    cout << x << " " << broj << "\n";
}
```

4.1. Niz/lista kao osnovna struktura

Niz čuva uređenu kolekciju elemenata kojima pristupamo indeksom. Python lista je dinamička, a C++ vector je standardna dinamička struktura.

Primjena. Koristi se za gotovo svaki skup podataka gdje je redoslijed bitan.

Primjer. `a=[4,7,2]`; `a[1]` je 7.

Obratite pažnju. Indeksi počinju od nule; posljednji indeks je `n-1`.

PYTHON 3 • isječak

```
a=[4,7,2]
a.append(9)
```

C++17 • isječak

```
vector<int>a={4,7,2};
a.push_back(9);
```

4.2. Prolazak kroz niz

Niz možemo prolaziti po vrijednostima ili po indeksima. Indeks je potreban kada odnos zavisi od pozicije ili susjednih elemenata.

Primjena. Osnova je za brojanje, provjeru uslova, gradnju novih nizova i DP.

Primjer. Prebrojimo koliko je elemenata pozitivno.

Obratite pažnju. Ne koristiti indeks ako nije potreban; kod po vrijednostima je jednostavniji.

PYTHON 3 • isječak

```
cnt=sum(1 for x in a if x>0)
```

C++17 • isječak

```
int cnt=0; for(int x:a) if(x>0) cnt++;
```

4.3. Suma, minimum i maksimum

Agregacija svodi niz na jednu vrijednost: sumu, minimum, maksimum ili broj elemenata koji ispunjavaju uslov.

Primjena. Često je prvi korak u zadatku ili dio provjere kandidata.

Primjer. Za [5,2,9] suma je 16, minimum 2, maksimum 9.

Obratite pažnju. Za prazan niz min/max nemaju smisla bez posebnog pravila.

PYTHON 3 • isječak

```
s=sum(a); mn=min(a); mx=max(a)
```

C++17 • isječak

```
long long s=accumulate(a.begin(),a.end(),0LL);
int mn=*min_element(a.begin(),a.end());
```

4.4. Frekvencijsko brojanje

Ako vrijednosti dolaze iz malog raspona ili ih mapiramo u ključeve, možemo čuvati koliko puta se svaka vrijednost pojavljuje.

Primjena. Koristi se za duplikate, anagrame, histogram vrijednosti i brze upite o broju pojavljivanja.

Primjer. U [2,2,5,2] frekvencija broja 2 je 3.

Obratite pažnju. Ako je raspon ogroman, ne praviti veliki niz frekvencija nego mapu/dict.

PYTHON 3 • isječak

```
from collections import Counter
f=Counter(a)
```


C++17 • isječak

```
unordered_map<int,int> f;
for(int x:a) f[x]++;
```

4.5. Sortiranje i zašto pomaže

Sortiranje uređuje elemente po rastućem ili opadajućem kriteriju. Cijena $O(n \log n)$ često se isplati jer poslije omogućava jednostavniji greedy, dva pokazivača ili binarnu pretragu.

Primjena. Koristi se kad redoslijed ulaza nije važan ili kada rješenje zavisi od poretka vrijednosti.

Primjer. [5,1,4] nakon sortiranja postaje [1,4,5].

Obratite pažnju. Ne sortirati ako originalni redoslijed nosi značenje, osim ako čuvamo i originalne indekse.

PYTHON 3 • isječak

```
a.sort()
```

C++17 • isječak

```
sort(a.begin(),a.end());
```

4.6. Sortiranje po prilagođenom ključu

Elementi mogu biti parovi ili objekti. Tada definišemo ključ: npr. prvo po završetku, zatim po početku.

Primjena. Važno u intervalnim zadacima, rasporedima i zadacima s više atributa.

Primjer. Parove (ime,bodovi) možemo sortirati po opadajućim bodovima pa po imenu.

Obratite pažnju. Kriterij mora biti konzistentan; u C++-u comparator mora definisati strogo slabo uređenje.

PYTHON 3 • isječak

```
v.sort(key=lambda x:(-x[1],x[0]))
```

C++17 • isječak

```
sort(v.begin(),v.end(),[](auto&a,auto&b){ if(a.second!=b.second) return
    a.second>b.second; return a.first<b.first; });
```

4.7. Skup (set)

Skup čuva jedinstvene vrijednosti i efikasno odgovara da li element postoji. Python set i C++ unordered_set prosječno nude $O(1)$ pristup.

Primjena. Koristi se za uklanjanje duplikata, provjeru posjećenosti i brzo članstvo.

Primjer. Skup elemenata [2,2,5] sadrži samo {2,5}.

Obratite pažnju. Hash strukture nemaju uređen poredak; za sortirani skup u C++-u koristi set.

PYTHON 3 • isječak

```
s=set(a)
if x in s: print("ima")
```

C++17 • isječak

```
unordered_set<int>s(a.begin(),a.end());
if(s.count(x)) cout<<"ima";
```

4.8. Mapa/dict

Mapa povezuje ključ sa vrijednošću. Python dict i C++ unordered_map su hash mape; C++ map održava ključeve sortiranim.

Primjena. Idealna je za frekvencije, indeksiranje po imenima i čuvanje stanja prema ključu.

Primjer. f[x] može čuvati koliko puta se x pojavio.

Obratite pažnju. Razlikovati prosječnu složenost hash mape od $O(\log n)$ kod uređene mape.

PYTHON 3 • isječak

```
f={}
for x in a: f[x]=f.get(x,0)+1
```

C++17 • isječak

```
unordered_map<int,int>f;
for(int x:a)f[x]++;
```

5. Stringovi i dvodimenzionalne mreže

Tekst i matrica imaju iste osnovne izazove kao niz: indeksiranje, granice i redosljed. U mreži dodajemo još jednu koordinatu i pažljivo definišemo susjedstvo.

Broj nije isto što i njegov zapis

Zapis "0072" ima četiri znaka, ali njegova brojčana vrijednost je 72. Ako zadatak pita za vodeće nule, cifre ili položaje znakova, rano pretvaranje u broj može uništiti potrebnu informaciju. Ako pita za zbir dvije količine, rad sa stringovima bez konverzije može pogrešno spojiti zapise umjesto sabrati brojeve.

Palindrom se čita jednako s oba kraja. Za riječ RADAR prvi znak poredimo s posljednjim, drugi s pretposljednjim, a srednji ne treba porediti ni s čim drugim. Dva indeksa idu jedan prema drugom. Kod pitanja da li jedno brisanje može napraviti palindrom, prvo nepoklapanje daje samo dva kandidata: izostaviti lijevi ili desni znak. Ne treba isprobavati svako brisanje od početka ako već znamo gdje je prva prepreka.

Matrica dimenzija $R \times C$ ima redove $0..R-1$ i kolone $0..C-1$. Početak (0,0) je u gornjem lijevom uglu. Pomjeranje dolje povećava red; pomjeranje desno povećava kolonu. Susjed u pravcu (dr,dc) ima koordinate (r+dr,c+dc). Prvo provjeravamo granice, a tek zatim čitamo sadržaj tog polja.

Četiri i osam susjeda nisu zamjenjivi modeli. Ako je dopušteno kretanje samo gore, dolje, lijevo i desno, dodirivanje dva polja dijagonalom ne spaja ih u istu komponentu. Kada zadatak dopušta dijagonalno kretanje, dodajemo još četiri pravca. To treba odlučiti iz teksta, ne iz navike.

U Pythonu matrica `[[0]*C]*R` ponavlja reference na isti red. Promjena jednog mjesta može se pojaviti u više redova. Zapis `[[0]*C for _ in range(R)]` pravi nezavisne redove. Ovakva greška može biti nevidljiva dok se matrica samo ispisuje, a postati ozbiljna čim se počne mijenjati.

Smjer	Promjena reda	Promjena kolone
Gore	-1	0
Dolje	+1	0
Lijevo	0	-1
Desno	0	+1

PYTHON 3 • vođeni primjer

```

r, c = 1, 2
R, C = 3, 4
for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
    nr, nc = r + dr, c + dc
    if 0 <= nr < R and 0 <= nc < C:
        print(nr, nc)

```

C++17 • isječak

```

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};
for (int i = 0; i < 4; i++) {
    int nr = r + dr[i], nc = c + dc[i];
    if (0 <= nr && nr < R && 0 <= nc && nc < C) {
        cout << nr << " " << nc << "\n";
    }
}

```

5.1. String i pristup znakovima

String je niz znakova. Možemo pristupati znakovima indeksima, prolaziti kroz njih i graditi novi string.

Primjena. Koristi se za riječi, šifre, DNA sekvence, binarne zapise i zadatke nad tekstom.

Primjer. U "ALGO" znak na indeksu 2 je G.

Obratite pažnju. Paziti na razliku između znaka i njegove brojčane vrijednosti.

PYTHON 3 • isječak

```

s="ALGO"
for c in s: pass

```

C++17 • isječak

```

string s="ALGO";
for(char c:s){}

```

5.2. Podstring i pretraga obrasca

Podstring je neprekinuti dio stringa. Mala pretraga može se raditi ugrađenim funkcijama, dok veliki zadaci zahtijevaju KMP ili hash.

Primjena. Korisno za provjeru pojavljivanja uzorka i izdvajanje segmenata.

Primjer. U "ABRACADABRA" obrazac "CAD" počinje na indeksu 4.

Obratite pažnju. Naivna pretraga može biti $O(nm)$ kada su i tekst i obrazac veliki.

PYTHON 3 • isječak

```
pos=s.find(t)
```

C++17 • isječak

```
auto pos=s.find(t);
```

5.3. Palindrom

Palindrom se čita isto s lijeva i zdesna. Provjera se može raditi poređenjem sa obrnutim stringom ili dva pokazivača.

Primjena. Čest motiv u osnovnim i naprednim string zadacima.

Primjer. "RADAR" je palindrom, "ALGORITAM" nije.

Obratite pažnju. Kod podstringova paziti na tačne granice segmenta koji provjeravamo.

PYTHON 3 • isječak

```
ok = s == s[::-1]
```

C++17 • isječak

```
string t=s; reverse(t.begin(),t.end()); bool ok=(s==t);
```

5.4. Anagram i frekvencije znakova

Dva stringa su anagrami ako sadrže iste znakove sa istim frekvencijama. Nije potrebno generisati permutacije.

Primjena. Koristi se za poređenje multiskupa znakova.

Primjer. "ROMA" i "AMOR" imaju iste frekvencije znakova.

Obratite pažnju. Prije poređenja odrediti da li velika i mala slova treba razlikovati.

PYTHON 3 • isječak

```
from collections import Counter
ok=Counter(a)==Counter(b)
```

C++17 • isječak

```
array<int,256>f{}; for(char c:a)f[(unsigned char)c]++; for(char
c:b)f[(unsigned char)c]--;
```

5.5. Leksikografski poredak

Leksikografski poredak je „rječnički“ poredak: porede se prvi različiti znakovi, a ako je jedan string prefiks drugog, kraći je manji.

Primjena. Bitno kod sortiranja imena, najmanjeg stringa i kanonskih reprezentacija.

Primjer. "ANA" < "ANJA" jer je A < J na prvom različitom mjestu.

Obratite pažnju. Poredak zavisi od kodiranja/veličine slova ako zadatak ne definiše normalizaciju.

PYTHON 3 • isječak

```
v.sort()
```

C++17 • isječak

```
sort(v.begin(),v.end());
```

5.6. Matrica i dvodimenzionalni indeks

Matrica je tabela redova i kolona. Element $a[i][j]$ određuju indeks reda i kolone.

Primjena. Koristi se za ploče, slike, mape, labirinte i DP na mreži.

Primjer. U matrici 3×4 postoje redovi 0..2 i kolone 0..3.

Obratite pažnju. Ne zamijeniti broj redova n i broj kolona m pri provjeri granica.

PYTHON 3 • isječak

```
a=[[0]*m for _ in range(n)]
```

C++17 • isječak

```
vector<vector<int>>>a(n,vector<int>(m));
```

5.7. Susjedi u mreži

Kretanje po mreži često je ograničeno na 4 ili 8 pravaca. Smjerove je praktično čuvati u malim nizovima.

Primjena. Pojednostavljuje BFS/DFS u labirintima i simulacijama.

Primjer. Za 4 susjeda koristimo (1,0),(-1,0),(0,1),(0,-1).

Obratite pažnju. Svaki novi susjed mora proći provjeru granica i zabrane/prepreke.

PYTHON 3 • isječak

```
dirs=[(1,0),(-1,0),(0,1),(0,-1)]
```

C++17 • isječak

```
int dr[4]={1,-1,0,0}, dc[4]={0,0,1,-1};
```

5.8. 2D prefiksne sume

Dvodimenzionalni prefiks omogućava sumu bilo kojeg pravougaonika u $O(1)$ nakon $O(nm)$ pripreme. Koristi princip uključivanja-isključivanja.

Primjena. Idealan je za mnogo upita sume podmatrica.

Primjer. Suma pravougaonika dobija se iz četiri prefiksne vrijednosti.

Obratite pažnju. Najčešća greška je pogrešan znak na preklapajućem pravougaoniku.

PYTHON 3 • isječak

```
p[i+1][j+1]=a[i][j]+p[i][j+1]+p[i+1][j]-p[i][j]
```

C++17 • isječak

```
p[i+1][j+1]=a[i][j]+p[i][j+1]+p[i+1][j]-p[i][j];
```

5.9. Flood fill

Flood fill obilazi cijelu povezanu oblast ćelija iste vrste ili prohodnosti. Implementira se BFS-om ili DFS-om.

Primjena. Koristi se za broj komponenti na slici, ostrva, regije i bojenje.

Primjer. Krenemo od neobiđene kopnene ćelije i označimo sve povezane kopnene ćelije.

Obratite pažnju. Svaku ćeliju označiti posjećenom čim je dodamo u obradu da je ne ubacujemo više puta.

6. Prefiksi, prozori i binarna pretraga

Ove tehnike izbjegavaju ponavljanje istog rada. Prefiksi pamte zbir prethodnog dijela, prozor mijenja samo krajeve, a binarna pretraga odbacuje dio uređenog prostora.

Zbir segmenta bez ponovnog sabiranja

Za niz 2, 5, 1, 4 napravimo prefikse 0, 2, 7, 8, 12. Prefiks $p[i]$ je zbir prvih i elemenata. Zato zbir elementa s indeksom 1 do elementa s indeksom 3, uključivo, iznosi $p[4] - p[1] = 12 - 2 = 10$.

Najlakše je dosljedno koristiti poluotvoreni zapis $[l, r)$: odgovor je $p[r] - p[l]$.

Cijena izgradnje prefiksa je jedan prolaz. Svaki kasniji upit traži samo dva pristupa i jedno oduzimanje. Za q upita ukupan red rada je $O(n+q)$, umjesto $O(nq)$. Ali promjena jednog izvornog elementa kviri mnoge prefikse. Kada se upiti i promjene miješaju, potrebna je druga struktura, kao Fenwickovo ili segmentno stablo.

Klizni prozor koristi sličnu ideju. Ako blok dužine k pomjerimo za jedno mjesto, iz zbira oduzmemo član koji izlazi i dodamo član koji ulazi. Kod promjenjive dužine prozora moramo dokazati monotonost. Za nenegativne vrijednosti proširivanje ne smanjuje zbir, a izbacivanje lijevog člana ga ne povećava. Sa negativnim vrijednostima taj argument više ne važi.

Binarna pretraga po odgovoru nije nagađanje broja. Najprije treba definisati provjeru mogućeg x . Primjer: može li se sav teret prenijeti u najviše k dana ako je dnevni kapacitet x ? Ako je odgovor DA za x , bit će DA i za veći kapacitet. Tako niz odgovora ima oblik NE, NE, ..., DA, DA. Tražimo prvo DA.

U svakom koraku održavamo granice koje zaista sadrže traženu prijelomnu tačku. Pogrešno pomjeranje granice može uzrokovati beskonačnu petlju ili preskakanje odgovora. Zbog toga su testovi s jednim mogućim odgovorom, odgovorom na lijevoj granici i odgovorom na desnoj granici obavezni za razumijevanje, ne samo za provjeru koda.

i	$a[i]$	$p[i+1]$
0	2	2
1	5	7
2	1	8
3	4	12

PYTHON 3 • vođeni primjer

```
a = [2, 5, 1, 4]
p = [0]
for x in a:
    p.append(p[-1] + x)
l, r = 1, 4
print(p[r] - p[l])
```

C++17 • isječak

```
vector<long long> a = {2, 5, 1, 4};
vector<long long> p(a.size() + 1, 0);
for (int i = 0; i < (int)a.size(); i++) {
    p[i + 1] = p[i] + a[i];
}
int l = 1, r = 4;
cout << p[r] - p[l] << "\n";
```

6.1. Prefiksne sume 1D

Prefiks $p[i]$ čuva sumu prvih i elemenata. Tada se suma intervala $[l, r]$ dobija kao $p[r+1]-p[l]$ u $O(1)$.

Primjena. Idealno kada imamo mnogo upita za sume intervala na nepromjenjivom nizu.

Primjer. Za $a=[3,1,4]$, prefiksi su $[0,3,4,8]$.

Obratite pažnju. Najčešća greška je off-by-one; prefiks dužine $n+1$ znatno pojednostavljuje formulu.

PYTHON 3 • isječak

```
p=[0]
for x in a: p.append(p[-1]+x)
ans=p[r+1]-p[l]
```

C++17 • isječak

```
vector<long long> p(n+1);
for(int i=0;i<n;i++) p[i+1]=p[i]+a[i];
```

6.2. Diferencni niz

Diferencni niz omogućava da promjenu cijelog intervala zapišemo u $O(1)$, a stvarne vrijednosti kasnije obnovimo prefiksnim sabiranjem.

Primjena. Koristan je kada imamo mnogo intervalnih dodavanja, ali konačne vrijednosti trebamo tek na kraju.

Primjer. Dodaj +5 na $[l,r]$: $\text{diff}[l] += 5$, $\text{diff}[r+1] -= 5$.

Obratite pažnju. Diferencni niz nije za proizvoljne online upite bez dodatne strukture.

PYTHON 3 • isječak

```
d=[0]*(n+1)
d[l]+=v
if r+1<n: d[r+1]-=v
```

C++17 • isječak

```
vector<long long>d(n+1);
d[l]+=v; if(r+1<n)d[r+1]-=v;
```

6.3. Dva pokazivača

Dva pokazivača održavaju dvije pozicije koje se pomjeraju monotonno. Time se često izbjegava provjera svih parova.

Primjena. Posebno korisno na sortiranom nizu ili kod prozora čiji uslov monotonno zavisi od granica.

Primjer. U sortiranom nizu tražimo par čija je suma S : ako je suma mala pomjeri lijevi, ako je velika desni.

Obratite pažnju. Metoda radi kada možemo opravdati da pomjeranjem pokazivača ne preskačemo rješenje.

PYTHON 3 • isječak

```
l,r=0,n-1
while l<r:
    s=a[l]+a[r]
    if s<S:l+=1
    elif s>S:r-=1
    else:break
```

C++17 • isječak

```
int l=0, r=n-1;
while(l<r){ long long s=a[l]+a[r]; if(s<S)l++; else if(s>S)r--; else break; }
```

6.4. Klizni prozor

Klizni prozor održava neprekinuti segment i ažurira njegovu vrijednost kada desna granica ulazi, a lijeva izlazi.

Primjena. Koristi se za fiksnu dužinu ili za najduži/najkraći segment koji zadovoljava monotoni uslov.

Primjer. Maksimalna suma k uzastopnih elemenata dobija se bez ponovnog sabiranja cijelog segmenta.

Obratite pažnju. Ako uslov nije monoton, standardni sliding window možda nije primjenjiv.

PYTHON 3 • isječak

```
s=sum(a[:k]); best=s
for i in range(k,n):
    s+=a[i]-a[i-k]; best=max(best,s)
```

C++17 • isječak

```
long long s=0; for(int i=0;i<k;i++)s+=a[i];
long long best=s; for(int i=k;i<n;i++){s+=a[i]-a[i-k]; best=max(best,s);}
```

6.5. Binarna pretraga

Binarna pretraga prepolovljava prostor pretrage i radi u $O(\log n)$. Klasično se koristi na sortiranom nizu.

Primjena. Traži vrijednost, prvi element x , posljednji x ili granicu između netačno/tačno.

Primjer. U nizu [1,4,7,9] broj 7 se nalazi prepolovljavanjem intervala indeksa.

Obratite pažnju. Prvo dokazati da je niz sortiran ili predikat monoton.

PYTHON 3 • isječak

```
import bisect
pos=bisect.bisect_left(a,x)
```

C++17 • isječak

```
auto it=lower_bound(a.begin(),a.end(),x);
```

6.6. Binarna pretraga po odgovoru

Ako možemo provjeriti da li je kandidat x moguć, a odgovor „moguć/nemoguć“ je monoton, možemo binarno tražiti najmanji ili najveći mogući x .

Primjena. Tipična je za minimalni kapacitet, maksimalnu udaljenost i optimizaciju pod ograničenjem.

Primjer. Ako kapacitet C može prenijeti teret, svaki veći kapacitet takođe može.

Obratite pažnju. Najvažnije je pravilno definisati granice i monotoni predikat $ok(x)$.

PYTHON 3 • isječak

```
lo,hi=0,10**18
while lo<hi:
    mid=(lo+hi)//2
    if ok(mid): hi=mid
    else: lo=mid+1
```

C++17 • isječak

```
long long lo=0,hi=1e18;
while(lo<hi){auto mid=(lo+hi)/2; if(ok(mid))hi=mid; else lo=mid+1;}
```

6.7. Koordinatna kompresija

Kompresija mijenja velike ili rijetke vrijednosti njihovim rangovima $0..k-1$, uz očuvanje poretka.

Primjena. Koristi se kada struktura traži mali indeks, a originalne koordinate su do 10^9 ili više.

Primjer. Vrijednosti $[100,5,1000]$ postaju rangovi $[1,0,2]$.

Obratite pažnju. Kompresija čuva poredak, ali ne čuva stvarne razlike između koordinata.

PYTHON 3 • isječak

```
vals=sorted(set(a))
rank={x:i for i,x in enumerate(vals)}
```

C++17 • isječak

```
auto v=a; sort(v.begin(),v.end()); v.erase(unique(v.begin(),v.end()),v.end());
```

7. Stek, red i upiti nad intervalima

Poredak izlaska iz strukture može biti presudan dio algoritma: posljednji uneseni, prvi uneseni ili trenutno najmanji. Naprednije strukture nad intervalima organizuju djelimične rezultate.

Koji podatak treba dobiti sljedeći?

Stek radi po principu posljednji unutra, prvi van. Kod zagrada posljednja nezatvorena lijeva zagrada mora se prva zatvoriti. Zato otvorene zagrade stavljamo na stek, a kod zatvorene provjeravamo njegov vrh. Nije dovoljno da ukupno postoji jednak broj ljevih i desnih zagrada: zapis `)` ima jednake brojeve, ali nije ispravan.

Red radi po principu prvi unutra, prvi van. U obilasku po širini prvo obrađujemo sve čvorove na udaljenosti 1, zatim 2 i tako dalje. Deque omogućava efikasne operacije na oba kraja i koristi se, između ostalog, za 0-1 BFS. Prioritetni red ne poštuje vrijeme dolaska, nego ključ: npr. najmanju privremenu udaljenost.

Uklanjanje prvog elementa Python liste metodom `pop(0)` pomjera preostale reference. Ako se to ponavlja, naizgled jednostavan algoritam može postati kvadratan. Za red je prikladniji `collections.deque`. Obrnuto, proizvoljan pristup sredini deque strukture nije zamjena za efikasan indeksirani pristup listi.

Fenwickovo stablo čuva zbirove pažljivo izabраниh blokova. Operacija `i & -i` daje veličinu bloka povezanog s indeksom `i` u uobičajenom 1-baznom zapisu. Upit ide prema ranijim blokovima, a promjena prema blokovima koji obuhvataju izmijenjeni element. Obje operacije imaju logaritamski broj koraka.

Segmentno stablo dijeli raspon na dva dijela, pa svaki dio ponovo na dva. U čvoru čuvamo podatak koji možemo spojiti iz dva djeteta: zbir, minimum, maksimum ili drugi odgovarajući sažetak. Za minimum neutralna vrijednost nije nula kada elementi mogu biti pozitivni; koristimo dovoljno veliku vrijednost. Izbor neutralnog elementa pripada matematičkoj definiciji operacije.

Struktura	Sljedeći element	Tipična primjena
Stek	Posljednji uneseni	Zagrade, poništavanje operacije
Red	Prvi uneseni	BFS
Prioritetni red	Najbolji prema ključu	Dijkstra
Fenwickovo stablo	Prefiksni zbir	Zbir uz pojedinačne promjene

7.1. Stek (stack)

Stek radi po principu LIFO: posljednji ubačen element prvi izlazi. Operacije push/pop su $O(1)$.

Primjena. Koristi se za uparivanje zagrada, DFS bez rekurzije, monotoni stek i evaluaciju izraza.

Primjer. Za "([)]" otvarajuće zagrade stavljamo na stek i skidamo odgovarajućom zatvarajućom.

Obratite pažnju. Nikad ne skidati element iz praznog steka.

PYTHON 3 • isječak

```
st=[]
st.append(x)
x=st.pop()
```

C++17 • isječak

```
stack<int>st;
st.push(x); x=st.top(); st.pop();
```

7.2. Red (queue)

Red radi po principu FIFO: prvi ubačen element prvi izlazi.

Primjena. Osnovna je struktura za BFS i simulacije događaja po redoslijedu dolaska.

Primjer. U BFS-u prvo obrađujemo vrhove na udaljenosti 1, zatim 2, itd.

Obratite pažnju. Za uklanjanje s početka Python liste ne koristiti pop(0) u velikim zadacima.

PYTHON 3 • isječak

```
from collections import deque
q=deque([s]); x=q.popleft()
```

C++17 • isječak

```
queue<int>q; q.push(s); int x=q.front(); q.pop();
```


7.3. Deque

Deque je dvostrani red: ubacivanje i skidanje moguće je na oba kraja u $O(1)$.

Primjena. Koristi se za 0-1 BFS i monotoni prozor minimuma/maksimuma.

Primjer. Element s težinom 0 u 0-1 BFS-u ide na početak, s težinom 1 na kraj.

Obratite pažnju. Deque nije isto što i prioritetni red; ne bira automatski najmanji element.

PYTHON 3 • isječak

```
from collections import deque
d=deque(); d.appendleft(x); d.append(y)
```

C++17 • isječak

```
deque<int>d; d.push_front(x); d.push_back(y);
```

7.4. Prioritetni red (heap)

Prioritetni red omogućava brzo uzimanje najmanjeg ili najvećeg elementa prema prioritetu. Heap daje $O(\log n)$ dodavanje i skidanje.

Primjena. Ključan je za Dijkstru, izbor sljedećeg najboljeg kandidata i raspoređivanje događaja.

Primjer. Min-heap uvijek daje trenutno najmanju vrijednost.

Obratite pažnju. U C++ `priority_queue` je podrazumijevano max-heap.

PYTHON 3 • isječak

```
import heapq
h=[]; heapq.heappush(h,x); x=heapq.heappop(h)
```

C++17 • isječak

```
#include <queue>
priority_queue<int,vector<int>,greater<int>> pq;
```

7.5. Fenwickovo stablo

Fenwick (BIT) podržava promjenu jedne pozicije i prefiksnu sumu u $O(\log n)$, uz $O(n)$ memorije. Indeksi se obično vode od 1.

Primjena. Odlično za dinamičke prefiksne sume i brojanje inverzija nakon kompresije.

Primjer. Suma $[l,r]=\text{prefix}(r)-\text{prefix}(l-1)$.

Obratite pažnju. Najčešća greška je miješanje 0-based i 1-based indeksa.

PYTHON 3 • isječak

```
def add(i,v):
    while i<=n: bit[i]+=v; i+=i&-i
def pref(i):
    s=0
    while i>0: s+=bit[i]; i-=i&-i
    return s
```

C++17 • isječak

```
# isti i += i&-i za update, i -= i&-i za query
```

7.6. Segmentno stablo

Segmentno stablo čuva agregat intervala i podržava upite i izmjene u $O(\log n)$. Agregat može biti suma, minimum, maksimum ili druga asocijativna operacija.

Primjena. Koristi se kada su i upiti i promjene dinamički, a Fenwick nije dovoljno opšti.

Primjer. Korišten predstavlja cijeli niz, djeca lijevu i desnu polovinu.

Obratite pažnju. Struktura je moćna, ali skuplja za implementaciju; koristiti je tek kada jednostavniji alat nije dovoljan.

8. Teorija brojeva i binarni zapis

Djeljivost, ostaci, prosti brojevi i bitovi povezuju matematiku s efikasnim programskim postupcima. Ovdje je posebno važno razlikovati brojčanu vrijednost od njenog zapisa.

Euklidov algoritam i brzo stepenovanje

Za NZD brojeva 48 i 18 primjećujemo da $48 = 2 \cdot 18 + 12$. Broj koji dijeli i 48 i 18 dijeli i 12; obrnuto, broj koji dijeli 18 i 12 dijeli i 48. Zajednički djelioci parova (48,18) i (18,12) zato su isti. Nastavljamo (12,6), pa (6,0). Posljednji nenulti član je 6.

To je razlog za ponavljanje $a, b = b, a \% b$, a ne samo recept koji treba zapamtiti. Ostatak je manji od pozitivnog djelitelja, pa se postupak približava završetku. Za NZS pozitivnih brojeva koristimo $a/\text{NZD}(a,b) \cdot b$. Dijeljenje prije množenja smanjuje rizik prekoračenja tipa, mada konačni rezultat i dalje mora stati u odabrani tip.

Da bismo provjerili je li n prost, ne moramo isprobavati sve brojeve do $n-1$. Ako n ima par netrivialnih djelilaca, barem jedan nije veći od kvadratnog korijena. U cjelobrojnom kodu možemo provjeravati $d \cdot d \leq n$, uz pažnju na moguće prekoračenje proizvoda. Brojevi 0 i 1 nisu prosti; upravo njih početničke implementacije često pogrešno prihvate.

Za a na stepen e neposredno množenje traži e koraka. Binarni zapis eksponenta omogućava ponavljano kvadriranje. Ako je trenutni eksponent neparan, jedan faktor uključimo u rezultat. Osnovu kvadriramo, a eksponent prepolovimo. Za $e=13$, binarni zapis 1101 pokazuje da su potrebni faktori a , a^4 i a^8 .

Modulo možemo primjenjivati poslije svakog sabiranja i množenja jer time čuvamo ostatak konačnog rezultata. Ne smijemo, međutim, običnim cjelobrojnim dijeljenjem zamijeniti modularno dijeljenje. Ono zahtijeva dodatne uslove i modularni inverz. U C++-u i privremeni proizvod dva ostatka mora stati u tip; modulo na kraju ne spašava već nastalo prekoračenje.

a	b	a % b
48	18	12
18	12	6
12	6	0

PYTHON 3 • vođeni primjer

```
a, b = 48, 18
while b != 0:
    a, b = b, a % b
print(a)
```

C++17 • isječak

```
long long a = 48, b = 18;
while (b != 0) {
    long long ostatak = a % b;
    a = b;
    b = ostatak;
}
cout << a << "\n";
```

8.1. Djeljivost i ostatak

a je djeljivo sa b ako je $a \% b == 0$. Ostatak pri dijeljenju često opisuje parnost, cikluse i klase kongruencije.

Primjena. Jedna od najčešćih aritmetičkih ideja u takmičarskim zadacima.

Primjer. Broj je paran ako $n \% 2 == 0$.

Obratite pažnju. Za negativne brojeve pravila ostatka mogu se razlikovati između jezika; normalizovati modulo po potrebi.

PYTHON 3 • isječak

```
if n%k==0: print("djeljiv")
```

C++17 • isječak

```
if(n%k==0) cout<<"djeljiv";
```

8.2. Najveći zajednički djelilac - NZD

$\text{NZD}(a,b)$ je najveći broj koji dijeli i a i b . Euklidov algoritam koristi činjenicu $\text{gcd}(a,b) = \text{gcd}(b, a \% b)$.

Primjena. Koristi se za skraćivanje razlomaka, periodičnost i probleme sa zajedničkim korakom.

Primjer. $\text{gcd}(48,18)=6$.

Obratite pažnju. Euklidov algoritam radi u $O(\log \min(a,b))$, pa nema potrebe isprobavati sve djelioce.

PYTHON 3 • isječak

```
import math
g=math.gcd(a,b)
```

C++17 • isječak

```
long long g=std::gcd(a,b);
```

8.3. Najmanji zajednički sadržilac - NZS

NZS(a,b) je najmanji pozitivan broj djeljiv i sa a i sa b. Veza je $\text{lcm}(a,b)=a/\text{gcd}(a,b)*b$.

Primjena. Koristi se kada se periodi, ciklusi ili događaji trebaju ponovo poklopiti.

Primjer. $\text{lcm}(6,8)=24$.

Obratite pažnju. Prvo dijeliti pa množiti da se smanji rizik overflowa.

PYTHON 3 • isječak

```
l=a//math.gcd(a,b)*b
```

C++17 • isječak

```
long long l=a/std::gcd(a,b)*b;
```

8.4. Prosti brojevi

Prost broj veći od 1 ima tačno dva pozitivna djelitelja: 1 i sebe. Za pojedinačnu provjeru dovoljno je isprobati djelioce do $\sqrt{n}$.

Primjena. Koristi se za faktORIZACIJU, djelioce i modularne probleme.

Primjer. 29 je prost; 30 nije jer je djeljiv sa 2,3,5.

Obratite pažnju. Provjera svih brojeva do n za svaki upit je prespora ako ima mnogo upita.

PYTHON 3 • isječak

```
def prime(n):
    if n<2: return False
    d=2
    while d*d<=n:
        if n%d==0: return False
        d+=1
    return True
```

C++17 • isječak

```
bool prime(long long n){ if(n<2) return false; for(long long d=2; d*d<=n; d++)
    if(n%d==0) return false; return true; }
```

8.5. Eratostenovo sito

Sito označava proste brojeve do N u približno $O(N \log \log N)$. Počinje od pretpostavke da su brojevi prosti i precrtava višekratnike.

Primjena. Najbolji izbor kada trebamo mnogo informacija o prostim brojevima do umjerenog N .

Primjer. Za $N=10$ prosti su 2,3,5,7.

Obratite pažnju. Višekratnike možemo početi od p^2 jer su manji već označeni manjim prostim faktorom.

PYTHON 3 • isječak

```
is_p=[True]*(N+1)
is_p[0]=is_p[1]=False
for p in range(2,int(N**0.5)+1):
    if is_p[p]:
        for x in range(p*p,N+1,p): is_p[x]=False
```

C++17 • isječak

```
vector<bool>is_p(N+1,true); is_p[0]=is_p[1]=false;
for(int p=2;p*p<=N;p++)if(is_p[p])for(long long
    x=1LL*p*p;x<=N;x+=p)is_p[x]=false;
```

8.6. Prosta faktorizacija

Svaki cijeli broj >1 može se jedinstveno zapisati kao proizvod prostih faktora. Probno dijeljenje do $\sqrt{n}$ je dovoljno za pojedinačne umjerene brojeve.

Primjena. Koristi se za broj djelilaca, NZD/NZS strukturu i modularne zadatke.

Primjer. $60=2^2 \cdot 3 \cdot 5$.

Obratite pažnju. Nakon petlje preostali $n>1$ je prost faktor i ne smije se zaboraviti.

PYTHON 3 • isječak

```
fac=[]
d=2
while d*d<=n:
    while n%d==0: fac.append(d); n//=d
    d+=1
if n>1: fac.append(n)
```

C++17 • isječak

```
vector<long long>fac; for(long long
    d=2; d*d<=n; d++) while(n%d==0){fac.push_back(d); n/=d;}
    if(n>1) fac.push_back(n);
```

8.7. Modularna aritmetika

Računanje modulo M zadržava samo ostatak. Vrijedi $(a+b)\%M$, $(a*b)\%M$ i slične formule, što sprečava rast ogromnih brojeva.

Primjena. Često se traži broj načina modulo 10^9+7 ili drugi modul.

Primjer. $(10^9+8) \bmod (10^9+7)=1$.

Obratite pažnju. U C++-u proizvod može overflowati prije modula ako operandi nisu dovoljno maleni ili tip nije dovoljan.

PYTHON 3 • isječak

```
ans=(a*b)%MOD
```

C++17 • isječak

```
long long ans=(a*b)%MOD;
```

8.8. Brzo modularno stepenovanje

Ekspencijacija kvadriranjem računa a^e u $O(\log e)$. Ideja: ako je bit eksponenta 1, pomnoži rezultat; bazu kvadriraj pri svakom koraku.

Primjena. Neophodna je za velike eksponente i modularne inverze kod prostog modula.

Primjer. 3^{13} koristi samo $O(\log 13)$ koraka.

Obratite pažnju. Nikad ne računati $a**e$ direktno ako su a i e ogromni, pa tek onda uzimati modulo.

PYTHON 3 • isječak

```
res=1
while e:
    if e&1: res=res*a%MOD
    a=a*a%MOD; e//=2
```

C++17 • isječak

```
long long res=1; while(e){ if(e&1)res=res*a%MOD; a=a*a%MOD; e>=1; }
```

8.9. Osnovna kombinatorika

Kombinatorika broji mogućnosti bez njihovog eksplicitnog generisanja. Permutacije uređuju sve elemente, kombinacije biraju podskup bez reda.

Primjena. Koristi se kada je pitanje „na koliko načina“.

Primjer. Broj načina da izaberemo 2 od 5 je $C(5,2)=10$.

Obratite pažnju. Prije formule utvrditi da li je redoslijed bitan i da li je ponavljanje dozvoljeno.

PYTHON 3 • isječak

```
import math
ans=math.comb(n,k)
```

8.10. Bitovski operatori

Bitovi predstavljaju cijeli broj binarno. Operatori $\&$, $|$, $\wedge$, $\sim$ i pomaci $\ll$, $\gg$ rade nad pojedinačnim bitovima.

Primjena. Korisni su za maske, parnost, skupove do ~20-25 elemenata i optimizacije.

Primjer. $x \& 1$ provjerava najmanje značajan bit, tj. parnost.

Obratite pažnju. Zagrade pomažu jer prioritet bitovskih operatora nije uvijek intuitivan.

PYTHON 3 • isječak

```
if x & 1: print("neparan")
```

C++17 • isječak

```
if(x&1) cout<<"neparan";
```


8.11. Bitmask kao mali skup

Ako imamo k malih elemenata, podskup možemo predstaviti brojem od 0 do 2^k-1 ; i -ti bit govori da li je element prisutan.

Primjena. Koristi se u bitmask DP-u, BFS-u sa stanjem ključeva i enumeraciji podskupova.

Primjer. Maska 10110_2 sadrži bitove 1, 2 i 4.

Obratite pažnju. Broj stanja raste eksponencijalno; $k=25$ već znači preko 33 miliona maski.

PYTHON 3 • isječak

```
if mask & (1<<i): pass
mask |= 1<<i
```

C++17 • isječak

```
if(mask&(1<<i)){
    mask |= 1<<i;
```

9. Pohlepni izbor i potpuno pretraživanje

Pohlepni algoritam donosi neopoziv izbor. Potpuno pretraživanje čuva više mogućnosti. Njihova razlika nije samo brzina, nego vrsta argumenta kojim dokazujemo da ništa važno nije izgubljeno.

Zašto najraniji završetak pomaže

Želimo izabrati što više radionica koje se vremenski ne preklapaju. Radionica koja najranije počinje može trajati cijeli dan i blokirati više kratkih radionica. Radionica koja najkraće traje također nije univerzalno siguran prvi izbor. Pravilo s jasnim dokazom bira onu koja najranije završava.

Uzmimo bilo koji optimalan raspored i njegovu prvu radionicu. Zamijenimo je radionicom s najranijim završetkom. Zamjena se završava najkasnije kada i originalna prva radionica, pa nijedna kasnija iz optimalnog rasporeda ne postaje nemoguća. Dakle, postoji optimalan raspored koji počinje našim izborom. Poslije toga rješavamo isti problem za preostalo vrijeme.

Granice intervala moraju biti precizne. Ako radionice zauzimaju $[s, e)$, jedna može početi tačno kada druga završi. Uslov prihvatanja je $s \geq \text{prethodni_kraj}$. Ako tekst propisuje obaveznu pauzu, uslov se mijenja. Naizgled mali znak jednakosti može odlučiti ispravnost cijelog rješenja.

Kad siguran lokalni izbor ne postoji, možemo isprobati mogućnosti. Za n malih predmeta podskup se dobija odlukom uzmi/ne uzmi za svaki predmet. Broj listova stabla odluka je 2^n . Povratno pretraživanje prekida granu čim postane jasno da krši uslove. To ubrzava mnoge primjere, ali bez dodatnog dokaza ne uklanja eksponencijalni najgori slučaj.

Kod promjenjivog stanja treba vratiti promjenu nakon rekurzivnog poziva: dodaj element, pretraži, ukloni element. Izostavljeno vraćanje znači da se dvije nezavisne grane više ne obrađuju nezavisno. Korisno je ručno nacrtati stablo odluka za samo tri elementa prije rada s velikim ulazom.

Radionica	Interval	Izabrana?
A	[1,3)	DA
B	[2,5)	NE
C	[3,4)	DA
D	[4,7)	DA

9.1. Pohlepni algoritmi

Greedy u svakom koraku bira lokalno najbolju odluku bez vraćanja. Ispravan je samo kada se može dokazati da lokalni izbor ne uništava globalni optimum.

Primjena. Čest je kod intervala, sortiranja i konstrukcija.

Primjer. Za maksimalan broj nesudarnih intervala biramo onaj koji najranije završava.

Obratite pažnju. Greedy nije „uzmi najveće“ po navici; kriterij mora imati dokaz razmjene ili invariant.

9.2. Raspoređivanje intervala

Kod izbora maksimalnog broja nepokrivajućih intervala optimalno je sortirati po završetku i uvijek uzeti prvi kompatibilni interval.

Primjena. Klasičan primjer dokaza razmjenom u greedy algoritmima.

Primjer. Interval koji ranije završava ostavlja najviše prostora za buduće izbore.

Obratite pažnju. Paziti da zadatak precizno kaže da li se intervali koji dodiruju u tački smatraju preklapljenim.

PYTHON 3 • isječak

```
intervals.sort(key=lambda x:x[1])
```

C++17 • isječak

```
sort(v.begin(),v.end(),[](auto a,auto b){return a.second<b.second;});
```

10. Grafovi i najkraći putevi

Graf je model odnosa i prelaza. Prije algoritma odredite šta predstavlja čvor, šta ivica, da li su veze usmjerene i imaju li težine. Pogrešan model ne može popraviti nijedna biblioteka.

BFS slojevi i težine ivica

Uzmimo veze 1–2, 1–3, 2–4 i 3–4. Od čvora 1 do 4 postoje dva puta dužine dvije ivice. BFS prvo označi 1 udaljenošću 0, zatim 2 i 3 udaljenošću 1, pa 4 udaljenošću 2. Čvor se označava kada se prvi put dodaje u red, ne tek kada se izvadi: tako se sprečava višestruko ubacivanje istog čvora.

Za broj ivica BFS je dovoljan jer svaki prelaz povećava udaljenost za tačno jedan. Ako jedna ivica traje 100 minuta, a dvije druge po jednu, put s manje ivica ne mora biti jeftiniji. Tada treba uzeti u obzir težine. Za nenegativne težine Dijkstrin algoritam uzima trenutno najmanju privremenu udaljenost pomoću prioritetnog reda.

U praktičnoj implementaciji isti čvor može više puta ući u prioritetni red s različitim procjenama. Stari unos ne mora se fizički tražiti i brisati. Kada ga izvadimo, uporedimo njegovu vrijednost s trenutno najboljom; ako se razlikuju, preskočimo ga. To je postupak lijenog uklanjanja zastarjelih unosa, a ne nova posjeta s konačnom udaljenošću.

Ako su težine samo 0 i 1, 0-1 BFS može koristiti deque. Poboljšanje preko ivice težine 0 stavlja se na početak, a preko ivice težine 1 na kraj. Običan BFS bez te razlike nije dovoljan. Kod svih ovih algoritama potrebno je posebno obraditi nedostižan cilj.

Za rekonstrukciju puta nije dovoljna samo udaljenost. Kada poboljšamo čvor v preko čvora u , sačuvamo roditelja $\text{parent}[v]=u$. Poslije algoritma krećemo od cilja i pratimo roditelje do izvora, a dobijeni niz obrnemo. Kada postoji više najkraćih puteva, različiti ispravni redoslijedi obilaska mogu vratiti različite puteve iste dužine; zadatak mora reći traži li poseban poredak.

DSU rješava drugačije pitanje: pripadaju li dva čvora istoj grupi dok veze dodajemo? Nije struktura za rekonstrukciju najkraćeg puta. Kruskal koristi DSU da pri dodavanju najjeftinijih ivica izbjegne ciklus. Nepovezan graf nema razapinjuće stablo svih čvorova, pa to treba provjeriti.

BFS sloj	Čvorovi	Značenje
0	1	Izvor
1	2, 3	Jedna ivica od izvora
2	4	Dvije ivice od izvora

10.1. Osnovni pojmovi grafa

Graf se sastoji od vrhova i grana. Može biti usmjeren ili neusmjeren, težinski ili netežinski. Put je niz povezanih vrhova.

Primjena. Prvi korak je prepoznati šta u tekstu zadatka predstavlja vrh, a šta granu.

Primjer. Gradovi su vrhovi, putevi grane; težina može biti dužina puta.

Obratite pažnju. Model grafa je često važniji od samog algoritma; pogrešan model daje pogrešno rješenje.

10.2. Lista susjedstva i matrica susjedstva

Lista susjedstva čuva za svaki vrh njegove susjede i koristi $O(n+m)$ memorije. Matrica susjedstva koristi $O(n^2)$, ali brzo odgovara postoji li direktna grana.

Primjena. Za rijetke takmičarske grafove lista susjedstva je standardni izbor.

Primjer. Za granu $u-v$ dodajemo v u $\text{adj}[u]$, a kod neusmjerenog i u u $\text{adj}[v]$.

Obratite pažnju. Ne zaboraviti smjer grafa; usmjerena grana se ne dodaje automatski u oba smjera.

PYTHON 3 • isječak

```
adj=[] for _ in range(n)
adj[u].append(v); adj[v].append(u)
```

C++17 • isječak

```
vector<vector<int>>>adj(n); adj[u].push_back(v); adj[v].push_back(u);
```

10.3. BFS - pretraga po širini

BFS koristi red i obilazi graf po slojevima udaljenosti. U netežinskom grafu nalazi najkraći broj grana od izvora do svakog vrha.

Primjena. Koristi se za labirinte, najkraći put u netežinskom grafu i multi-source pretragu.

Primjer. Iz izvora s svi susjedi dobijaju udaljenost 1, njihovi novi susjedi 2 itd.

Obratite pažnju. Označiti vrh posjećenim pri ubacivanju u red, ne tek pri vađenju.

PYTHON 3 • isječak

```
from collections import deque
d=[-1]*n; d[s]=0; q=deque([s])
while q:
    u=q.popleft()
    for v in adj[u]:
        if d[v]==-1: d[v]=d[u]+1; q.append(v)
```

C++17 • isječak

```
queue<int>q; vector<int>d(n,-1); d[s]=0;q.push(s); while(!q.empty()){int
    u=q.front();q.pop(); for(int v:adj[u])if(d[v]==-1){d[v]=d[u]+1;q.push(v);}}
```

10.4. DFS - pretraga po dubini

DFS ide što dublje prije povratka. Može biti rekurzivan ili iterativan sa stekom.

Primjena. Koristi se za komponente, cikluse, stabla, topološki poredak i mnoge strukturne osobine.

Primjer. Iz vrha u posjećujemo neobiđenog susjeda v i rekurzivno nastavljamo.

Obratite pažnju. Na vrlo dubokim grafovima rekurzija može prekoračiti stek; iterativni DFS je sigurnija opcija.

PYTHON 3 • isječak

```
def dfs(u):
    vis[u]=True
    for v in adj[u]:
        if not vis[v]: dfs(v)
```

C++17 • isječak

```
void dfs(int u){vis[u]=1; for(int v:adj[u])if(!vis[v])dfs(v);}
```

10.5. Povezane komponente

Povezana komponenta neusmjerenog grafa je maksimalan skup vrhova između kojih postoji put. Brojimo ih pokretanjem BFS/DFS iz svakog još neobiđenog vrha.

Primjena. Koristi se za ostrva, mreže prijateljstva i grupisanje povezanih objekata.

Primjer. Svaki novi DFS iz neobiđenog vrha povećava broj komponenti za 1.

Obratite pažnju. Izolovan vrh je komponenta veličine 1.

PYTHON 3 • isječak

```
cnt=0
for u in range(n):
    if not vis[u]: cnt+=1; dfs(u)
```

C++17 • isječak

```
int cnt=0; for(int u=0;u<n;u++)if(!vis[u]){cnt++;dfs(u);}
```

10.6. Bipartitan graf

Graf je bipartitan ako se vrhovi mogu obojiti sa dvije boje tako da svaka grana spaja različite boje. Ekvivalentno: nema neparnog ciklusa.

Primjena. Koristi se za podjelu u dvije grupe i kao uvod u uparivanja.

Primjer. BFS dodjeljuje susjedu suprotnu boju; konflikt iste boje znači da graf nije bipartitan.

Obratite pažnju. Treba provjeriti sve komponente, ne samo onu koja sadrži vrh 0.

PYTHON 3 • isječak

```
color=[-1]*n
# BFS: color[v]=color[u]^1
```

C++17 • isječak

```
vector<int>color(n,-1); // BFS sa 0/1 bojama
```

10.7. Topološko sortiranje

Topološki poredak postoji samo u usmjerenom acikličnom grafu (DAG). Svaka grana uv ide od ranijeg ka kasnijem elementu poretka.

Primjena. Modelira zavisnosti: predmet prije naprednog predmeta, zadatak prije zadatka koji od njega zavisi.

Primjer. Kahnov algoritam stalno uzima vrh ulaznog stepena 0.

Obratite pažnju. Ako topološki poredak nema svih n vrhova, graf sadrži ciklus.

PYTHON 3 • isječak

```
from collections import deque
q=deque([u for u in range(n) if indeg[u]==0])
```

C++17 • isječak

```
# queue indeg==0; skidaj i smanjuj indeg susjeda
```

10.8. Najkraći put u netežinskom grafu

Kada svaka grana ima istu cijenu, BFS daje najkraći put po broju grana. Za rekonstrukciju čuvamo roditelja svakog vrha.

Primjena. Čest zadatak traži ne samo udaljenost nego i samu putanju.

Primjer. `parent[v]=u` kada prvi put dođemo u `v`; od cilja se vraćamo do izvora.

Obratite pažnju. Roditelj se postavlja samo pri prvom otkrivanju jer je to najkraći dolazak.

PYTHON 3 • isječak

```
parent=[-1]*n
# postavi pri BFS otkrivanju
```

C++17 • isječak

```
# vector<int> parent(n,-1); postavi pri BFS otkrivanju
```

10.9. Dijkstra

Dijkstra nalazi najkraće puteve iz jednog izvora kada su težine grana nenegativne. Koristi prioritetni red i uvijek obrađuje trenutno najmanju poznatu udaljenost.

Primjena. Standardni algoritam za puteve sa pozitivnim ili nultim težinama.

Primjer. Relaksacija: ako `dist[u]+w < dist[v]`, poboljšaj `dist[v]`.

Obratite pažnju. Ne koristiti Dijkstru ako postoje negativne težine.

PYTHON 3 • isječak

```
import heapq
# heap (dist,vertex), relaksiraj susjede
```

C++17 • isječak

```
# priority_queue min-heap parova (dist,vertex)
```


10.10. 0-1 BFS

Ako su težine grana samo 0 ili 1, najkraći put se može naći deque-om u $O(n+m)$. Grana težine 0 ide na početak, težine 1 na kraj.

Primjena. Brži i jednostavniji od Dijkstre u ovom posebnom slučaju.

Primjer. Prelazak bez troška dobija prioritet nad prelaskom koji košta 1.

Obratite pažnju. Metoda ne važi kada se pojavi težina 2 ili veća.

PYTHON 3 • isječak

```
from collections import deque
# appendleft za w=0, append za w=1
```

C++17 • isječak

```
# deque; push_front za 0, push_back za 1
```

10.11. Floyd-Warshall

Floyd-Warshall računa najkraće puteve između svih parova vrhova u $O(n^3)$. Postepeno dozvoljava posredne vrhove 0..k.

Primjena. Koristi se za male grafove, tipično n do nekoliko stotina.

Primjer. $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.

Obratite pažnju. Paziti na $\text{INF} + \text{INF}$ i na to da $O(n^3)$ brzo postaje presporo za veliki n.

PYTHON 3 • isječak

```
for k in range(n):
    for i in range(n):
        for j in range(n):
            d[i][j] = min(d[i][j], d[i][k] + d[k][j])
```

C++17 • isječak

```
for(int k=0; k<n; k++) for(int i=0; i<n; i++) for(int
    j=0; j<n; j++) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

10.12. DP na DAG-u

DAG ima topološki poredak, pa se vrijednosti mogu obrađivati bez cikličnih zavisnosti. Tako računamo najduži put, broj puteva ili druge DP vrijednosti.

Primjena. Koristi se kada je stanje prirodno vrh grafa, a prijelazi prate usmjerene grane bez ciklusa.

Primjer. U topološkom poretку prenesemo `dp[u]` na sve `v` za granu `uv`.

Obratite pažnju. Bez acikličnosti obični DP po topološkom poretку nije definisan.

10.13. Disjoint Set Union - DSU

DSU održava particiju elemenata u skupove. Operacije `find` nalazi predstavnika, `union` spaja skupove. `Path compression` i `union by size/rank` daju gotovo konstantno vrijeme.

Primjena. Koristi se za dinamičko povezivanje i Kruskalov algoritam.

Primjer. Ako `find(a)==find(b)`, `a` i `b` su već u istoj komponenti.

Obratite pažnju. Bez kompresije puta i spajanja po veličini struktura može postati nepotrebno duboka.

PYTHON 3 • isječak

```
parent=list(range(n))
def find(x):
    if parent[x]!=x: parent[x]=find(parent[x])
    return parent[x]
```

C++17 • isječak

```
# parent + find sa kompresijom + union by size
```

10.14. Minimalno razapinjuće stablo - Kruskal

MST povezuje sve vrhove ukupno minimalnom težinom bez ciklusa. Kruskal sortira grane po težini i uzima granu ako spaja dvije različite DSU komponente.

Primjena. Koristi se kod projektovanja mreža, kablova i minimalne cijene povezivanja.

Primjer. Svaka prihvaćena grana smanjuje broj komponenti za 1; MST ima $n-1$ grana ako je graf povezan.

Obratite pažnju. Ako graf nije povezan, ne postoji jedno razapinjuće stablo svih vrhova.

11. Stabla: korijen, podstablo i prec

Stablo je poseban graf čija struktura omogućava jednostavnije relacije. Korijen ne mijenja ivice, nego uspostavlja odnos roditelja i djece.

Računanje odozdo prema gore

U stablu je između dva čvora jedinstven jednostavan put. Kada izaberemo korijen, svaki drugi čvor ima jednog roditelja: prethodni čvor na putu prema korijenu. Podstablo čvora obuhvata njega i sve njegove potomke. Ove definicije moraju biti jasne prije nego što pišemo DFS.

Veličina podstabla lista je 1. Za unutrašnji čvor vrijedi $velicina[u] = 1 + \text{zbir } velicina[v] \text{ po svojoj djetci } v$. Redoslijed je važan: prije računanja roditelja trebaju biti poznati odgovori djece.

Rekurzivni DFS to prirodno postiže povratkom iz poziva. Iterativni pristup može prvo zabilježiti red obilaska, a zatim ga obraditi unazad.

Ako čvor 1 ima djecu 2 i 3, a čvor 2 djecu 4 i 5, veličine su: za 3,4,5 po 1; za 2 je 3; za 1 je 5. Ne sabiramo roditelja kao dijete. U neusmjerenoj listi susjedstva on se ipak pojavljuje, pa ga treba preskočiti ili koristiti već izgrađen niz roditelja.

Najbliži zajednički predak dva čvora je njihov najdublji zajednički predak u ukorijenjenom stablu. Jedan čvor može biti predak samom sebi. Zato $LCA(u,u)=u$, a ako je u predak od v , odgovor je u . Binarno podizanje čuva pretke udaljene 1,2,4,8,... koraka i omogućava brzo izjednačavanje dubina i zajedničko penjanje.

Eulerov red ulaska daje svakom podstablu neprekinuti raspon. Uz dogovor tin uključivo, tout isključivo, čvor v je u podstablu u ako $tin[u] \leq tin[v] < tout[u]$. Time upit nad stablom postaje upit nad nizom. Promjena korijena ili drugačija definicija vremena izlaska mijenja formulu, pa oznake ne treba preuzimati bez značenja.

Čvor	Djeca	Veličina podstabla
3	nema	1
4	nema	1
5	nema	1
2	4, 5	3
1	2, 3	5

11.1. Stablo kao poseban graf

Stablo je povezan neusmjeren graf bez ciklusa. Sa n vrhova ima tačno $n-1$ grana i između svaka dva vrha postoji jedinstven put.

Primjena. Ova svojstva omogućavaju jednostavne DFS/DP algoritme bez opasnosti od alternativnih cikličkih puteva.

Primjer. Lanac 1-2-3-4 je stablo sa 4 vrha i 3 grane.

Obratite pažnju. U DFS-u stabla dovoljno je izbjegavati roditelja; opšti visited često nije potreban.

11.2. Ukorijenjeno stablo, roditelj i dubina

Odabirom korijena dobijamo roditelja, djecu i dubinu svakog vrha. Dubina je broj grana od korijena.

Primjena. Osnova je za subtree DP, LCA i mnoge upite na stablima.

Primjer. Ako je korijen 1, roditelj korijena se može postaviti na -1, dubina 0.

Obratite pažnju. Ne miješati dubinu sa visinom; visina se računa prema potomcima.

PYTHON 3 • isječak

```
def dfs(u,p):
    parent[u]=p
    for v in adj[u]:
        if v!=p: depth[v]=depth[u]+1; dfs(v,u)
```

C++17 • isječak

```
# DFS(u,p): parent[u]=p; depth[v]=depth[u]+1
```

11.3. Veličina podstabla

subtree[u] je broj vrhova u podstablu vrha u, uključujući u. Računa se postorder DFS-om: 1 plus sume podstabala djece.

Primjena. Koristi se za balansiranje, broj parova i rezanje stabla.

Primjer. List ima subtree=1.

Obratite pažnju. Vrijednost djeteta mora biti izračunata prije dodavanja roditelju.

PYTHON 3 • isječak

```
def dfs(u,p):
    sub[u]=1
    for v in adj[u]:
        if v!=p: dfs(v,u); sub[u]+=sub[v]
```

C++17 • isječak

```
# sub[u]=1; saberi sub[v] nakon DFS djeteta
```

11.4. Prečnik stabla

Prečnik je najduži najkraći put između dva vrha stabla. Za netežinsko stablo može se naći sa dva BFS/DFS prolaza: od proizvoljnog vrha do najudaljenijeg A, zatim od A do najudaljenijeg B.

Primjena. Čest zadatak traži dužinu najdužeg puta ili njegove krajeve.

Primjer. Na lancu od 5 vrhova prečnik ima 4 grane.

Obratite pažnju. Kod težinskog stabla koristi DFS sa sumom težina ili odgovarajući postupak, ne obične BFS nivoe.

11.5. LCA - najniži zajednički predak

LCA(u,v) je najdublji vrh koji je predak i u i v. Binary lifting čuva 2^k -tog pretka i odgovara na upit u $O(\log n)$ nakon pripreme.

Primjena. Koristi se za udaljenosti i mnogo upita odnosa vrhova u stablu.

Primjer. $\text{dist}(u,v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}]$.

Obratite pažnju. LOG mora biti dovoljan da 2^{LOG} pokrije maksimalnu dubinu.

11.6. Eulerov obilazak stabla

Euler/DFS red može svako podstablo pretvoriti u neprekinuti interval indeksa $[\text{tin}[u], \text{tout}[u]]$. Tada se upiti nad podstablom prevode na upite nad nizom.

Primjena. Veza je između stabala i Fenwick/segmentnog stabla.

Primjer. Svi potomci u pojavljuju se uzastopno u DFS redoslijedu ulaska.

Obratite pažnju. Tačna definicija uključivih/isključivih granica mora biti dosljedna.

12. Dinamičko programiranje

Dinamičko programiranje počinje definicijom stanja, ne pravljenjem tabele. Svako mjesto u tabeli mora imati jasno značenje, početnu vrijednost i zavisnost od ranijih stanja.

Četiri odluke prije prvog reda koda

Za stepenište po kojem se penjemo korakom dužine 1 ili 2, neka $dp[i]$ označava broj načina da stignemo na stepenik i . Do i možemo posljednjim korakom doći sa $i1$ ili sa $i2$. Te dvije grupe puteva su različite, pa se broj načina sabira: $dp[i]=dp[i1]+dp[i2]$.

Početno stanje $dp[0]=1$ ne znači da smo napravili korak. Znači da postoji jedan prazan put do početka. Za $dp[1]$ postoji samo jedan način. Ta definicija omogućava da se prelaz koristi dosljedno. Ako pogrešno postavimo $dp[0]=0$, svi kasniji brojevi mogu biti pogrešni iako formula izgleda poznato.

Prije implementacije zato odgovaramo na četiri pitanja: šta stanje znači, koje su početne vrijednosti, iz kojih stanja dolazi prijelaz i kojim redom se računa? Peto pitanje je gdje se nalazi konačni odgovor. Nije uvijek u posljednjoj ćeliji: nekad je maksimum kroz više stanja ili minimum po dopuštenim završecima.

U 0/1 ruksaku svaki predmet smije se koristiti najviše jednom. Kada tabelu svedemo na jedan niz, kapacitete obilazimo opadajuće. Tada $dp[cw]$ još ne sadrži trenutni predmet. Ako bismo išli rastuće, novodobijena vrijednost mogla bi se odmah ponovo iskoristiti, čime bismo neprimjetno prešli na problem s neograničenim brojem primjeraka.

Kod brojanja kombinacija kovanica važan je red petlji. Petlja po iznosu pa po kovanicama može brojati redoslijede, dok petlja po vrstama kovanica pa po rastućem iznosu uobičajeno broji kombinacije. Te dvije tabele ne rješavaju isto pitanje. Prije izbora formule treba odlučiti smatraju li se $1+2$ i $2+1$ istim ili različitim rješenjem.

Optimizacija memorije dolazi tek nakon jasnog osnovnog modela. Za LCS dva reda mogu zamijeniti cijelu tabelu kada tražimo samo dužinu. Za rekonstrukciju samog podniza trebaju dodatni podaci ili drugačiji postupak. Ušteda memorije ne smije neprimjetno ukloniti informacije potrebne za izlaz.

i	0	1	2	3	4	5
$dp[i]$	1	1	2	3	5	8

PYTHON 3 • vođeni primjer

```
n = 5
dp = [0] * (n + 1)
dp[0] = 1
for i in range(1, n + 1):
    dp[i] += dp[i - 1]
    if i >= 2:
        dp[i] += dp[i - 2]
print(dp[n])
```

C++17 • isječak

```
int n = 5;
vector<long long> dp(n + 1, 0);
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    dp[i] += dp[i - 1];
    if (i >= 2) dp[i] += dp[i - 2];
}
cout << dp[n] << "\n";
```

12.1. Ideja dinamičkog programiranja

DP se koristi kada se problem razlaže na preklapajuće podprobleme i optimalno rješenje zavisi od optimalnih rješenja manjih stanja. Ključ je precizno definisati stanje i prijelaz.

Primjena. Prije koda napisati: šta znači `dp[state]`, početne vrijednosti i kojim redom se stanja računaju.

Primjer. Fibonacci: $dp[i] = dp[i-1] + dp[i-2]$.

Obratite pažnju. DP nije samo tabela; bez jasnog značenja stanja kod lako postaje niz slučajnih formula.

PYTHON 3 • isječak

```
dp=[0]*(n+1); dp[0]=0; dp[1]=1
for i in range(2,n+1): dp[i]=dp[i-1]+dp[i-2]
```

C++17 • isječak

```
vector<long long>dp(n+1); dp[1]=1; for(int i=2;i<=n;i++)dp[i]=dp[i-1]+dp[i-2];
```

12.2. Memoizacija i tabulacija

Memoizacija je top-down: rekurzivno traži stanje i pamti rezultat. Tabulacija je bottom-up: računa stanja unaprijed pravilnim redom.

Primjena. Oba pristupa mogu imati istu složenost; izbor zavisi od prirodnosti zavisnosti i dubine rekurzije.

Primjer. Fibonacci sa memoizacijom svako n računa samo jednom.

Obratite pažnju. Memoizacija može imati problem duboke rekurzije; tabulacija ponekad računa i nepotrebna stanja.

PYTHON 3 • isječak

```
from functools import lru_cache
@lru_cache(None)
def f(n): return n if n<2 else f(n-1)+f(n-2)
```

C++17 • isječak

```
# memo niz ili bottom-up petlja
```

12.3. Coin change - broj ili minimum kovanica

Zavisno od zadatka $dp[x]$ može biti broj načina da dobijemo sumu x ili minimalan broj kovanica. Redoslijed petlji određuje da li se kovanice smiju ponavljati i da li se redoslijed kombinacija broji.

Primjena. Odličan primjer da ista tema ima više različitih DP definicija.

Primjer. Za minimum: $dp[x] = \min(dp[x], dp[x-c] + 1)$.

Obratite pažnju. Prvo precizirati da li se kovanice koriste neograničeno i da li se broje kombinacije ili permutacije.

PYTHON 3 • isječak

```
INF=10**9
dp=[INF]*(S+1); dp[0]=0
for x in range(1,S+1):
    for c in coins:
        if x>=c: dp[x]=min(dp[x],dp[x-c]+1)
```

C++17 • isječak

```
# analogno: dp[0]=0; relaksiraj iz dp[x-c]
```


12.4. 0/1 Knapsack

Imamo predmete težine w i vrijednosti v ; svaki predmet uzimamo najviše jednom. $dp[c]$ može biti najveća vrijednost za kapacitet c . Jednodimenzionalna optimizacija prolazi kapacitete unazad.

Primjena. Standardni model „izaberi podskup pod ograničenjem“.

Primjer. Za predmet (w,v) : $dp[c]=\max(dp[c],dp[c-w]+v)$, c od C naniže.

Obratite pažnju. Ako kapacitete prolazimo unaprijed, isti predmet možemo nenamjerno koristiti više puta.

PYTHON 3 • isječak

```
dp=[0]*(C+1)
for w,v in items:
    for c in range(C,w-1,-1): dp[c]=max(dp[c],dp[c-w]+v)
```

C++17 • isječak

```
# for item; for(c=C;c>=w;c--) dp[c]=max(dp[c],dp[c-w]+v);
```

12.5. LIS - najduži rastući podniz

LIS traži najduži podniz koji zadržava redoslijed indeksa i strogo raste. $O(n \log n)$ metoda održava najmanji mogući završetak rastućeg podniza svake dužine.

Primjena. Koristi se u zadacima o poretku, lancima i transformaciji nizova.

Primjer. Za $[3,1,2,5]$ jedan LIS je $[1,2,5]$, dužine 3.

Obratite pažnju. Niz tails nije nužno stvarni LIS, nego pomoćna struktura za njegovu dužinu.

PYTHON 3 • isječak

```
import bisect
tails=[]
for x in a:
    i=bisect.bisect_left(tails,x)
    if i==len(tails):tails.append(x)
    else:tails[i]=x
```

C++17 • isječak

```
vector<int>t; for(int x:a){auto it=lower_bound(t.begin(),t.end(),x);
    if(it==t.end())t.push_back(x); else *it=x;}
```

12.6. LCS - najduži zajednički podniz

LCS za dva stringa/niza traži najdužu sekvencu koja se pojavljuje u oba uz očuvanje redoslijeda, ali ne mora biti kontinuirana.

Primjena. Klasičan 2D DP za poređenje sekvenci.

Primjer. Ako $a[i-1]=b[j-1]$, $dp[i][j]=dp[i-1][j-1]+1$; inače maksimum lijevo/gore.

Obratite pažnju. LCS nije isto što i najduži zajednički podstring, koji mora biti kontinuiran.

C++17 • isječak

```
# 2D dp: jednak znak -> dijagonala+1, inače max gore/lijevo
```

12.7. DP na mreži

Stanje je ćelija, a prijelaz dolazi iz dozvoljenih prethodnih ćelija. Ako se krećemo samo desno/dolje, zavisnosti su aciklične i tabela se popunjava redom.

Primjena. Koristi se za broj puteva, minimalni/maksimalni zbir i prepreke.

Primjer. $dp[i][j]=a[i][j]+\min(dp[i-1][j], dp[i][j-1])$.

Obratite pažnju. Ako su dozvoljeni ciklični pokreti u svim smjerovima, problem možda više nije obični grid DP nego graf.

12.8. Intervalni DP

Intervalni DP koristi stanje $dp[l][r]$ za podproblem na kontinuiranom segmentu. Obično se računa po rastućoj dužini intervala.

Primjena. Javlja se kod spajanja, zagrada, optimalnog rezanja i igara na segmentu.

Primjer. Za svaku dužinu len prolazimo sva l , postavimo $r=l+len-1$ i pokušamo mjesta podjele k .

Obratite pažnju. Red računanja mora garantovati da su manji intervali već izračunati.

12.9. Bitmask DP

Stanje kombinuje masku izabranih elemenata i eventualno dodatni podatak poput posljednjeg vrha. Broj stanja je često $O(2^n \cdot n)$.

Primjena. Koristi se za male n , tipično do 20, kod TSP-a, rasporeda i podskupova.

Primjer. $dp[mask][v]$ može biti najbolja cijena puta koji posjećuje mask i završava u v .

Obratite pažnju. Eksponencijalna složenost je prihvatljiva samo zbog malog n ; uvijek izračunati približan broj stanja.

13. Napredni obrasci i granice njihove primjene

Ovo je izborna cjelina za učenike koji već samostalno rješavaju osnovne i srednje teške zadatke. Napredan naziv nije prednost ako jednostavniji postupak zadovoljava ograničenja.

KMP, sažeci stringova i rezidualna mreža

Ako tražimo uzorak u tekstu, naivni postupak može počinjati poređenje iznova na svakom položaju. KMP čuva informaciju o tome koji dio već upoređenog prefiksa može ostati koristan nakon nepoklapanja. Prefiks-funkcija $pi[i]$ označava dužinu najdužeg pravog prefiksa zapisa do i koji je istovremeno njegov sufiks.

Za uzorak ABABA vrijednosti su 0,0,1,2,3. Ako nakon pune pojave želimo brojati i preklapajuća pojavljivanja, ne vraćamo stanje na nulu, nego na pi posljednjeg položaja. U tekstu ABABABA uzorak ABABA javlja se na dva položaja. Ta mala provjera brzo otkriva implementaciju koja gubi preklapanja.

Sažetak stringa, odnosno hash, omogućava brzo poređenje numeričkih potpisa. Jednaki stringovi moraju imati jednak pravilno izračunat potpis, ali jednak potpis ne dokazuje uvijek jednakost stringova. Različiti stringovi mogu imati koliziju. Dvije nezavisne modularne funkcije smanjuju rizik, ali ga matematički ne uklanjaju. Kada se traži deterministička ispravnost, treba koristiti odgovarajući tačan postupak ili dodatno provjeriti poklapanje.

Trie čuva zajedničke prefikse u istim čvorovima. Za riječ MAMA slijedimo po jednu ivicu za M,A,M,A. Brojač prolazaka kroz čvor može odgovoriti koliko unesenih riječi dijeli određeni prefiks. Treba jasno odlučiti broje li se ponovljeni unosi iste riječi višestruko. Rječnik kao skup i kolekcija s duplikatima nisu isti model.

Maksimalni protok uvodi kapacitete i mogućnost popravljavanja prethodnih odluka. Kada pošaljemo tok kroz ivicu, u rezidualnoj mreži smanjujemo njen preostali kapacitet i povećavamo kapacitet obrnute ivice. Obrnuta ivica predstavlja mogućnost poništavanja dijela ranijeg toka; nije nužno fizička cijev iz originalnog zadatka. Bez nje pohlepno izabrani putevi mogu blokirati bolju raspodjelu.

Ove teme ne treba učiti kao velike blokove koda. Za KMP ručno izračunajte nekoliko prefiks-funkcija; za Trie nacrtajte tri riječi; za protok pratite jednu promjenu rezidualnih kapaciteta na maloj mreži. Tek kada su stanja jasna, implementacija dobija smisao.

Znak uzorka	A	B	A	B	A
Prefiks-funkcija	0	0	1	2	3
a					

13.1. KMP i prefiks-funkcija

KMP pretražuje obrazac u tekstu u $O(n+m)$. Prefiks-funkcija $pi[i]$ govori dužinu najdužeg pravog prefiksa koji je ujedno sufiks prefiksa do i .

Primjena. Koristi se kada naivna pretraga može biti prespora i za analizu periodičnosti stringa.

Primjer. Za obrazac "ABAB" prefiks-funkcija završava vrijednošću 2.

Obratite pažnju. Najviše grešaka dolazi iz miješanja indeksa i dužina prefiksa.

PYTHON 3 • isječak

```
pi=[0]*len(s)
for i in range(1,len(s)):
    j=pi[i-1]
    while j and s[i]!=s[j]: j=pi[j-1]
    if s[i]==s[j]: j+=1
    pi[i]=j
```

C++17 • isječak

```
vector<int>pi(s.size());
for(int i=1;i<(int)s.size();i++){int j=pi[i-1];
    while(j&& s[i]!=s[j])j=pi[j-1]; if(s[i]==s[j])j++; pi[i]=j;}
```

13.2. Rolling hash

Rolling hash pretvara podstring u brožčani potpis koji se može računati iz prefiksa.

Omogućava brzo poređenje podstringova uz malu vjerovatnoću kolizije.

Primjena. Koristi se za mnogo upita jednakosti podstringova i pretragu duplikata.

Primjer. Hash podstringa $[l,r]$ dobija se iz prefiksni hash vrijednosti i potencija baze.

Obratite pažnju. Hash nije matematički dokaz jednakosti zbog kolizija; po potrebi koristi dva modula.

Važno. Jednak hash nije matematički dokaz jednakosti stringova. Kod koji poredi samo hash predstavlja postupak s rizikom kolizije; za determinističku provjeru treba dodatna tačna provjera ili drugi algoritam.

PYTHON 3 • isječak

```
h[i+1]=(h[i]*base+ord(s[i]))%mod
```

C++17 • isječak

```
# pseudokod C++
h[i+1]=(h[i]*base+s[i])%MOD;
```

13.3. Trie (prefiksno stablo)

Trie čuva skup stringova po znakovima. Put od korijena predstavlja prefiks. Operacije zavise od dužine stringa, a ne od broja svih riječi.

Primjena. Koristi se za prefiksne upite, rječnike i broj riječi s datim prefiksom.

Primjer. Riječi ANA i ANANAS dijele put A-N-A.

Obratite pažnju. Trie može trošiti mnogo memorije; čvorove treba organizovati prema alfabetu i ograničenjima.

PYTHON 3 • isječak

```
trie={}
# svaki čvor mapira znak -> dijete
```

C++17 • isječak

```
struct Node{ int nxt[26]; bool end=false; };
```

13.4. Maksimalni protok - osnovna ideja

Mreža protoka ima izvor s , ponor t i kapacitete grana. Tražimo najveći protok koji poštuje kapacitete i očuvanje protoka u međuvrhovima. Rezidualni graf omogućava ispravljanje ranijih odluka.

Primjena. Koristi se za uparivanja, raspodjelu resursa i probleme koji se mogu modelirati kapacitetima.

Primjer. Ako postoji augmentirajući put od s do t , protok se može povećati za minimalni rezidualni kapacitet na tom putu.

Obratite pažnju. Najvažniji korak je modeliranje: šta su vrhovi, koje su grane i koji kapacitet predstavlja ograničenje.

14. Osnovne vježbe: računanje i obrada podataka

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
1	Takmičarski bodovi
2	Mini-zoo
3	Takmičarski popust
4	Četiri ekipe
5	Povoljnije putovanje
6	Temperaturni alarm
7	Slatkiši do maksimuma
8	Digitalni displej
9	Kod sekcije
10	Kružni raspored
11	Porodično glasanje
12	Kontrolna cifra
13	Najduža serija
14	Najčešći rezultat
15	Najduži strogi rast
16	Tačka ravnoteže

ZADATAK 001

Takmičarski bodovi

Cilj: Primijeniti jednostavnu formulu i pravilno učitati/ispisati cijele brojeve.

Na školskom takmičenju svako kolo nosi najviše P bodova. Učenik će nastupiti na N kola. Odredi najveći broj bodova koji može osvojiti ako na svakom kolu osvoji svih P bodova.

Ulaz. U jednom redu data su dva prirodna broja N i P.

Izlaz. Ispisati najveći mogući ukupan broj bodova.

Ograničenja. 1 N 100, 1 P 1000.

Primjer

ULAZ

6 100

IZLAZ

600

Razumijevanje i postupak

Ovdje nema potrebe za petljom. Isti broj bodova P sabrao bi se N puta, pa je rezultat proizvod $N \cdot P$. Takmičarski zadaci često počinju ovakvim direktnim modeliranjem: najvažnije je iz teksta izdvojiti dvije veličine i odnos među njima.

1. Učitati N i P.
2. Izračunati $N * P$.
3. Ispisati rezultat.

Složenost. $O(1)$ vrijeme i $O(1)$ memorija.

PYTHON 3 • potpuno rješenje

```
n, p = map(int, input().split())
print(n * p)
```

C++17 • potpuno rješenje

```
#include <iostream>
using namespace std;
int main() {
    long long n, p;
    cin >> n >> p;
    cout << n * p << "\n";
    return 0;
}
```

Važne provjere

Korištenje petlje iako nije potrebna.

Zamjena proizvoda sa zbirom.

ZADATAK 002

Mini-zoo

Cilj: Prevesti svakodnevnu situaciju u linearnu formulu.

Na školskom danu otvorenih vrata u mini-zoo prostoru nalazi se U učenika i P pasa. Pretpostavi da svaki učenik ima dvije noge, a svaki pas četiri. Koliko je ukupno nogu u prostoru?

Ulaz. U jednom redu data su U i P .

Izlaz. Ispisati ukupan broj nogu.

Ograničenja. $0 \leq U, P \leq 1000$.

Primjer

ULAZ

7 5

IZLAZ

34

Razumijevanje i postupak

Broj nogu je zbir dvije nezavisne grupe: $2 \cdot U$ za učenike i $4 \cdot P$ za pse. Nema potrebe simulirati osobu po osobu.

1. Učitati U i P .
2. Izračunati $2 \cdot U + 4 \cdot P$.
3. Ispisati rezultat.

Složenost. $O(1)$ vrijeme, $O(1)$ memorija.

PYTHON 3 • potpuno rješenje

```
u, p = map(int, input().split())
print(2 * u + 4 * p)
```

C++17 • potpuno rješenje

```
#include <iostream>
using namespace std;
int main() {
    long long u, p;
    cin >> u >> p;
    cout << 2 * u + 4 * p << "\n";
}
```

ZADATAK 003

Takmičarski popust

Cilj: Kombinovati proizvod, procenat i uslov.

Organizator kupuje N identičnih USB memorija po cijeni C KM. Ako je početni račun najmanje T KM, prodavnica odobrava popust D posto na cijeli račun. Odredi konačnu cijenu. Garantovano je da će rezultat biti cijeli broj.

Ulaz. U jednom redu dati su N , C , T i D .

Izlaz. Ispisati konačnu cijenu u KM.

Ograničenja. 1 $N, C, T \leq 100000$, 0 $D \leq 100$; rezultat je cijeli broj.

Primjer**ULAZ**

10 20 150 10

IZLAZ

180

Razumijevanje i postupak

Najprije računamo početni iznos $N \cdot C$. Tek nakon toga provjeravamo prag T . Ako je prag dostignut, ostaje $(100-D)\%$ iznosa.

1. račun = $N \cdot C$.
2. Ako račun $\geq T$, račun = račun $\cdot (100-D)/100$.
3. Ispisati račun.

Složenost. $O(1)$ vrijeme i memorija.

PYTHON 3 • potpuno rješenje

```
n, c, t, d = map(int, input().split())
iznos = n * c
if iznos >= t:
    iznos = iznos * (100 - d) // 100
print(iznos)
```

C++17 • potpuno rješenje

```
#include <iostream>
using namespace std;
int main() {
    long long n, c, t, d;
    cin >> n >> c >> t >> d;
    long long iznos = n * c;
    if (iznos >= t) iznos = iznos * (100 - d) / 100;
    cout << iznos << "\n";
}
```

ZADATAK 004

Četiri ekipe

Cilj: Prebrojati oznake, pronaći jedinstveni maksimum i obraditi neriješen rezultat.

Na završnici učestvuju ekipe A, B, C i D. Za svakog od N gledalaca poznata je oznaka ekipe za koju navija. Pobjedničkom publikom smatra se ekipa koja ima strogo više navijača od svake druge. Ako takva ekipa ne postoji, ispisati NEMA.

Ulaz. U prvom redu je N. U drugom redu je niz od N znakova iz skupa A, B, C, D.

Izlaz. Ispisati oznaku jedinstveno najzastupljenije ekipe ili NEMA.

Ograničenja. 1 $N \leq 1000$.

Primjer

ULAZ

7

A B A C A B D

IZLAZ

A

Razumijevanje i postupak

Potrebna su četiri brojača. Nakon prebrojavanja tražimo najveću vrijednost i provjeravamo pojavljuje li se samo jednom. Važno je ne proglasiti pobjednika kada dvije ekipe dijele maksimum.

1. Napraviti brojače za A,B,C,D.
2. Proći kroz N oznaka i povećavati odgovarajući brojač.
3. Naći maksimum.
4. Ako maksimum ima tačno jedna ekipa, ispisati je; inače NEMA.

Složenost. $O(N)$ vrijeme i $O(1)$ dodatna memorija.

PYTHON 3 • potpuno rješenje

```
n = int(input())
oznake = input().split()
broj = {c: 0 for c in 'ABCD'}
for c in oznake:
    broj[c] += 1
m = max(broj.values())
pobjednici = [c for c in 'ABCD' if broj[c] == m]
print(pobjednici[0] if len(pobjednici) == 1 else 'NEMA')
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <array>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    array<int, 4> cnt {
    };
    for (int i = 0; i < n; i++) {
        char c;
        cin >> c;
        cnt[c - 'A']++;
    }
    int m = *max_element(cnt.begin(), cnt.end());
    int who = -1, k = 0;
    for (int i = 0; i < 4; i++) if (cnt[i] == m) {
        who = i;
        k++;
    }
    if (k == 1) cout << char('A' + who) << "\n";
    else cout << "NEMA\n";
}
```

Važne provjere

Proglašavanje prve ekipe s maksimumom bez provjere izjednačenja.

ZADATAK 005

Povoljnije putovanje

Cilj: Uporediti dvije ukupne cijene i pravilno obraditi slučaj jednakosti.

Ekipa može putovati autobusom ili vozom. Autobuska karta košta A KM, ali uz nju je obavezan obrok od B KM. Vozna karta košta C KM. Odredi jeftiniju opciju. Ako su cijene jednake, prednost ima autobus. U drugom redu ispiši apsolutnu razliku cijena.

Ulaz. U tri reda nalaze se A, B i C.

Izlaz. U prvom redu ispisati AUTOBUS ili VOZ, a u drugom apsolutnu razliku cijena.

Ograničenja. 1 A,B,C 10 000.

Primjer

ULAZ

12
4
20

IZLAZ

AUTOBUS
4

Razumijevanje i postupak

Prvo formiramo ukupnu cijenu autobusom $A+B$. Zatim je poredimo s C. Uslov mora sadržati znak $\leq$ jer autobus pobjeđuje i kod jednakosti. Razlika cijena je apsolutna vrijednost $(A+B)-C$.

1. Izračunati $\text{bus} = A+B$.
2. Ako je $\text{bus} \leq C$, ispisati AUTOBUS, inače VOZ.
3. Ispisati $|\text{bus}-C|$.

Složenost. $O(1)$ vrijeme i $O(1)$ memorija.

PYTHON 3 • potpuno rješenje

```
a = int(input())
b = int(input())
c = int(input())
bus = a + b
print('AUTOBUS' if bus <= c else 'VOZ')
print(abs(bus - c))
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main() {
    long long a, b, c;
    cin >> a >> b >> c;
    long long bus = a + b;
    cout << (bus <= c ? "AUTOBUS" : "VOZ") << "\n";
    cout << labs(bus - c) << "\n";
    return 0;
}
```

Važne provjere

Korištenje < umjesto u slučaju jednakih cijena.

Zaboravljanje apsolutne vrijednosti.

ZADATAK 006

Temperaturni alarm

Cilj: Kreirati ispravan lanac if/elif/else uslova.

Senzor u računarskom kabinetu mjeri temperaturu T u $^{\circ}\text{C}$. Ako je $T < 15$ ispisati HLADNO, ako je $15 \leq T \leq 27$ ispisati NORMALNO, ako je $28 \leq T \leq 35$ ispisati TOPLO, a za $T > 35$ ispisati ALARM.

Ulaz. U jednom redu dat je cijeli broj T .

Izlaz. Ispisati jednu od četiri riječi.

Ograničenja. $-50 \leq T \leq 100$.

Primjer

ULAZ
36

IZLAZ
ALARM

Razumijevanje i postupak

Kod višestrukog grananja granice moraju pokriti sve moguće vrijednosti bez preklapanja i praznina. Najjednostavnije je testirati od najnižeg praga prema višem.

1. Ako $T < 15$: HLADNO.
2. Inače ako $15 \leq T \leq 27$: NORMALNO.
3. Inače ako $28 \leq T \leq 35$: TOPLO.
4. Inače: ALARM.

Složenost. $O(1)$.

PYTHON 3 • potpuno rješenje

```
t = int(input())
if t < 15:
    print('HLADNO')
elif t <= 27:
    print('NORMALNO')
elif t <= 35:
    print('TOPLO')
else:
    print('ALARM')
```


C++17 • potpuno rješenje

```
#include <iostream>
using namespace std;
int main() {
    int t;
    cin >> t;
    if (t < 15) cout << "HLADNO\n";
    else if (t <= 27) cout << "NORMALNO\n";
    else if (t <= 35) cout << "TOPLO\n";
    else cout << "ALARM\n";
}
```

ZADATAK 007

Slatkiši do maksimuma

Cilj: Pronaći maksimum niza i izračunati zbir razlika.

N učenika ima različit broj slatkiša. Nastavnik smije samo dodavati slatkiše. Koliko najmanje slatkiša mora dodati da svi na kraju imaju jednak broj?

Ulaz. U prvom redu je N, a u drugom N cijelih brojeva a_i .

Izlaz. Ispisati najmanji broj dodatnih slatkiša.

Ograničenja. 1 $N \leq 100000$, 0 $a_i \leq 10^9$.

Primjer

ULAZ

```
4
2 5 5 9
```

IZLAZ

```
15
```

Razumijevanje i postupak

Pošto slatkiše ne smijemo oduzimati, svi moraju završiti na najmanje onoliko koliko ima učenik s najvećim brojem slatkiša. Najmanji mogući cilj je zato maksimum M. Odgovor je zbir $M - a_i$.

1. Naći $M = \max(a)$.
2. Za svaki element dodati $M - a_i$ u odgovor.
3. Ispisati zbir.

Složenost. $O(N)$ vrijeme i $O(N)$ memorija ako čuvamo niz; može i $O(1)$ dodatno uz dva prolaza kada je ulaz dostupan.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
m = max(a)
print(sum((m - x for x in a)))
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > a(n);
    for (auto & x : a) cin >> x;
    long long m = * max_element(a.begin(), a.end()), ans = 0;
    for (long long x : a) ans += m - x;
    cout << ans << "\n";
}
```

ZADATAK 008

Digitalni displej

Cilj: Raditi sa ciframa kao znakovima i provjeriti dva uslova nad stringom.

Digitalni displej ostaje čitljiv nakon okretanja za 180 stepeni samo ako se broj sastoji od cifara 0, 1 i 8. U ovoj pojednostavljenoj verziji broj je „stabilan“ ako su sve cifre iz tog skupa i ako je zapis palindrom. Odredi je li zadani broj stabilan.

Ulaz. U jednom redu dat je zapis prirodnog broja bez vodećih nula.

Izlaz. Ispisati DA ili NE.

Ograničenja. Dužina zapisa je od 1 do 100000.

Primjer

ULAZ

81018

IZLAZ

DA

Razumijevanje i postupak

Broj tretiramo kao tekst. Prvi uslov provjerava da nema zabranjenih cifara. Drugi uslov provjerava jednakost stringa i njegovog obrnutog zapisa.

1. Učitati string `s`.
2. Provjeriti da je svaki znak u skupu `{0,1,8}`.
3. Provjeriti `s == obrnuti(s)`.
4. DA samo ako oba uslova vrijede.

Složenost. $O(|s|)$ vrijeme i $O(|s|)$ memorija u prikazanom Python rješenju zbog obrnutog stringa.

PYTHON 3 • potpuno rješenje

```
s = input().strip()
dozvoljene = set('018')
ok_cifre = all((c in dozvoljene for c in s))
ok_pal = s == s[::-1]
print('DA' if ok_cifre and ok_pal else 'NE')
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;
int main() {
    string s;
    cin >> s;
    bool ok = true;
    for (char c : s) if (c != '0' && c != '1' && c != '8') ok = false;
    for (int i = 0, j = (int) s.size() - 1; i < j; i++, j--) if (s[i] != s[j]) ok = false;
    cout << (ok ? "DA" : "NE") << "\n";
}
```

ZADATAK 009

Kod sekcije

Cilj: Pristupiti tačno određenim pozicijama u stringu i prebrojati zapise koji ispunjavaju uslov.

Članovi informatičke sekcije imaju šifru oblika GG-RR-B, gdje su GG dvije cifre godine, RR dvije cifre razreda, a B jedna cifra grupe, npr. 25-08-3. Prebroj koliko od N učenika ima razred najmanje 08 i grupu 3.

Ulaz. U prvom redu je N. U narednih N redova po jedna šifra tačno navedenog oblika.

Izlaz. Ispisati broj odgovarajućih učenika.

Ograničenja. 1 N 10000.

Primjer

ULAZ

4

25-08-3

25-07-3

24-09-2

26-09-3

IZLAZ

2

Razumijevanje i postupak

Format je fiksna, pa možemo izvući podstring na pozicijama 3 i 4 kao razred, te posljednji znak kao grupu. U Pythonu je jednostavno koristiti `s[3:5]`, a u C++ `substr(3,2)`.

1. Za svaku šifru izdvojiti RR.
2. Pretvoriti RR u cijeli broj.
3. Provjeriti RR ≥ 8 i posljednji znak $== 3$.
4. Povećati brojač.

Složenost. $O(N)$ vrijeme jer je dužina svake šifre konstantna.

PYTHON 3 • potpuno rješenje

```
n = int(input())
ans = 0
for _ in range(n):
    s = input().strip()
    razred = int(s[3:5])
    grupa = int(s[-1])
    if razred >= 8 and grupa == 3:
        ans += 1
print(ans)
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    int n;
    cin >> n;
    int ans = 0;
    while (n-- > 0) {
        string s;
        cin >> s;
        int razred = stoi(s.substr(3, 2));
        int grupa = s.back() - '0';
        if (razred >= 8 && grupa == 3) ans++;
    }
    cout << ans << "\n";
}
```

ZADATAK 010

Kružni raspored

Cilj: Primijeniti operator ostatka pri cikličnom kretanju.

Učenici stoje u krugu i numerisani su od 1 do N. Trenutno je na potezu učenik S. Nakon K koraka u smjeru kazaljke na satu, ko će biti na potezu? Jedan korak prelazi na sljedećeg učenika.

Ulaz. U jednom redu dati su N, S i K.

Izlaz. Ispisati broj učenika nakon K koraka.

Ograničenja. $1 \leq N \leq 10^9$, $1 \leq S \leq N$, $0 \leq K \leq 10^{18}$.

Primjer

ULAZ

8 7 4

IZLAZ

3

Razumijevanje i postupak

Numeraciju 1..N privremeno prebacimo na 0..N-1. Početak je S-1. Nakon K koraka pozicija je $(S-1+K) \bmod N$. Na kraju dodamo 1.

1. Pretvoriti S u nulti indeks S-1.
2. Dodati K i uzeti ostatak pri dijeljenju s N.
3. Vratiti se na numeraciju od 1 dodavanjem 1.

Složenost. $O(1)$ vrijeme i $O(1)$ memorija.

PYTHON 3 • potpuno rješenje

```
n, s, k = map(int, input().split())
print((s - 1 + k) % n + 1)
```

C++17 • potpuno rješenje

```
#include <iostream>
using namespace std;
int main() {
    long long n, s;
    unsigned long long k;
    cin >> n >> s >> k;
    cout << ((s - 1 + k % n) % n) + 1 << "\n";
}
```


ZADATAK 011

Porodično glasanje

Cilj: Akumulirati ponderisane glasove po kandidatu i riješiti izjednačenje.

Na izborima za predsjednika vijeća učestvuje N kandidata, označenih $1..N$. Glasanje obavlja M porodica: svaka porodica glasa za jednog kandidata, a njen glas vrijedi onoliko koliko porodica ima članova. Pobjeđuje kandidat s najviše ukupnih glasova; kod izjednačenja manji broj kandidata ima prednost.

Ulaz. U prvom redu su N i M . U narednih M redova dati su kandidat a i broj članova b .

Izlaz. Ispisati oznaku pobjednika.

Ograničenja. $2 \leq N \leq 100000$, $1 \leq M \leq 200000$, $1 \leq b \leq 100$.

Primjer

ULAZ

```
3 4
1 5
2 7
1 4
3 8
```

IZLAZ

```
1
```

Razumijevanje i postupak

Za svakog kandidata vodimo zbir njegovih pondera. Zatim prolazimo kroz kandidate redom od 1 naviše. Ako ažuriramo pobjednika samo kada nađemo strogo veći zbir, automatski zadržavamo manju oznaku pri izjednačenju.

1. Napraviti niz glasovi veličine $N+1$.
2. Za svaku porodicu dodati b u glasovi[a].
3. Proći kandidatima $1..N$ i čuvati najveći zbir.

Složenost. $O(N+M)$ vrijeme i $O(N)$ memorija.

PYTHON 3 • potpuno rješenje

```
n, m = map(int, input().split())
glasovi = [0] * (n + 1)
for _ in range(m):
    a, b = map(int, input().split())
    glasovi[a] += b
pobjednik = 1
for i in range(2, n + 1):
    if glasovi[i] > glasovi[pobjednik]:
        pobjednik = i
print(pobjednik)
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector< long long > g(n + 1, 0);
    while (m--) {
        int a, b;
        cin >> a >> b;
        g[a] += b;
    }
    int win = 1;
    for (int i = 2; i <= n; i++) if (g[i] > g[win]) win = i;
    cout << win << "\n";
}
```

ZADATAK 012

Kontrolna cifra

Cilj: Primjenjuje aritmetiku nad ciframa i pažljivo implementira formulu iz teksta.

Dat je šestocifreni identifikacioni broj N . Kontrolna vrijednost dobija se tako što se cifre, slijeva nadesno, množe težinama 1,2,3,4,5,6, rezultati saberu, a zatim uzme ostatak pri dijeljenju sa 11. Odredi kontrolnu vrijednost.

Ulaz. Jedan cijeli broj N sa tačno šest cifara.

Izlaz. Jedan cijeli broj od 0 do 10.

Ograničenja. $100000 \leq N \leq 999999$.

Primjer

ULAZ

352714

IZLAZ

10

Razumijevanje i postupak

Najjednostavnije je broj pretvoriti u string. Tada je redoslijed cifara očuvan, a težina cifre na indeksu i iznosi $i+1$. Nema potrebe za petljom sa stepenima broja 10.

1. Pretvori N u string s .
2. Za $i=0..5$ dodaj $\text{int}(s[i]) \cdot (i+1)$.
3. Ispiši zbir mod 11.

Zašto postupak daje tačan rezultat?

Petlja tačno jednom obrađuje svaku od šest cifara i primjenjuje definiciju iz zadatka, pa je dobijeni zbir upravo propisana kontrolna suma.

Složenost. Vrijeme $O(1)$, memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
s = input().strip()
ans = sum((int(ch) * (i + 1) for i, ch in enumerate(s)))
print(ans % 11)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    int ans = 0;
    for (int i = 0; i < 6; i++) ans += (s[i] - '0') * (i + 1);
    cout << ans % 11 << "\n";
}
```

Važne provjere

Težine počinju od 1, ne od 0.

Ne obrnuti redoslijed cifara pri aritmetičkom izdvajanju.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ima tačno 6 cifara; direktna obrada stringa	5
2	Puna ograničenja	15

ZADATAK 013

Najduža serija

Cilj: Jednim prolazom računa najdužu uzastopnu seriju jednakih znakova.

Tokom treninga bilježi se niz rezultata sastavljen od znakova P (poen) i G (greška). Odredi dužinu najduže uzastopne serije istog znaka.

Ulaz. Jedan string S sastavljen samo od P i G.

Izlaz. Jedan cijeli broj - najveća dužina serije.

Ograničenja. $1 \leq |S| \leq 200000$.

Primjer

ULAZ

PPPGPPPPG

IZLAZ

4

Razumijevanje i postupak

Čuvamo dužinu trenutne serije i najbolju viđenu dužinu. Kada je novi znak jednak prethodnom, serija se produžava; inače počinje nova serija dužine 1.

1. Postavi $cur=best=1$.
2. Od drugog znaka poredi $S[i]$ sa $S[i-1]$.
3. Ažuriraj cur i $best$.

Zašto postupak daje tačan rezultat?

Svaka maksimalna serija završava ili promjenom znaka ili krajem stringa. $best$ se ažurira nakon svakog proširenja, pa nijedna serija ne može biti propuštena.

Složenost. Vrijeme $O(N)$, memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
s = input().strip()
cur = best = 1
for i in range(1, len(s)):
    cur = cur + 1 if s[i] == s[i - 1] else 1
    best = max(best, cur)
print(best)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    int cur = 1, best = 1;
    for (int i = 1; i < (int) s.size(); ++ i) {
        cur = (s [i] == s [i - 1] ? cur + 1 : 1);
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

Važne provjere

Inicijalizovati best na 1 jer string nije prazan.

Ne brojati ukupan broj P ili G - traže se uzastopni znakovi.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	S ≤ 1000	5
2	Puna ograničenja	15

ZADATAK 014

Najčešći rezultat

Cilj: Koristi frekvencijsku mapu i primjenjuje jasno pravilo za izjednačenje.

Dato je N cijelih rezultata. Odredi vrijednost koja se pojavljuje najviše puta. Ako više vrijednosti ima istu najveću frekvenciju, ispiši najmanju od njih.

Ulaz. Prvi red N . Drugi red N cijelih brojeva.

Izlaz. Jedan cijeli broj - tražena vrijednost.

Ograničenja. $1 \leq N \leq 200000$, $|a_i| \leq 10^9$.

Primjer

ULAZ

```
8
4 1 4 2 2 4 2 2
```

IZLAZ

```
2
```

Razumijevanje i postupak

Direktno prebrojavanje parovima elemenata bilo bi $O(N^2)$. Mapa nam omogućava da u jednom prolazu izgradimo tačnu frekvenciju svake vrijednosti, a zatim izaberemo najbolji par (frekvencija velika, vrijednost mala).

1. Prođi niz i povećaj `cnt[x]`.
2. Čuvaj najbolju vrijednost ili nakon prebrojavanja prođi ključeve.
3. Kod izjednačenja frekvencije biraj manju vrijednost.

Zašto postupak daje tačan rezultat?

Nakon obrade prvih i elemenata mapa sadrži tačan broj pojavljivanja svake vrijednosti u tom prefiksu. Zato konačna mapa sadrži frekvencije cijelog niza.

Složenost. Vrijeme $O(N)$ očekivano, memorija $O(K)$, gdje je K broj različitih vrijednosti.

PYTHON 3 • potpuno rješenje

```
from collections import Counter
n = int(input())
a = list(map(int, input().split()))
c = Counter(a)
best = min(c, key=lambda x: (-c[x], x))
print(best)
```


C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    unordered_map < long long, int > c;
    while (n--) {
        long long x;
        cin >> x;
        c [x]++;
    }
    long long best = 0;
    int bf = - 1;
    bool first = true;
    for (auto [x, f] : c) {
        if (first || f > bf || (f == bf && x < best)) {
            first = false;
            bf = f;
            best = x;
        }
    }
    cout << best << "\n";
}

```

Važne provjere

Ne birati najveću vrijednost kod izjednačenja.

Ne pretpostaviti da su vrijednosti dovoljno male za običan frekvencijski niz.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 1000$	5
2	Sve vrijednosti nenegativne i ≤ 1000	5
3	Puna ograničenja	10

ZADATAK 015

Najduži strogi rast

Cilj: Jednim prolazom nalazi najduži uzastopni strogo rastući segment.

Dat je niz od N cijelih brojeva. Odredi dužinu najdužeg uzastopnog segmenta u kojem je svaki sljedeći element strogo veći od prethodnog.

Ulaz. N , zatim N cijelih brojeva.

Izlaz. Dužina najdužeg segmenta.

Ograničenja. $1 \leq N \leq 300000$.

Primjer

ULAZ

```
9
1 2 5 3 4 7 8 2 3
```

IZLAZ

```
4
```

Razumijevanje i postupak

Dovoljno je znati dužinu trenutnog rastućeg segmenta koji završava na i . Ako $a[i] > a[i-1]$, segment se produžava; inače novi segment počinje na i .

1. Postavi $cur=best=1$.
2. Za i od 1 do $N-1$: ako $a[i]>a[i-1]$, $cur++$; inače $cur=1$.
3. Ažuriraj $best=\max(best,cur)$.

Zašto postupak daje tačan rezultat?

Ako je stanje ispravno poslije prvih i elemenata, lokalno pravilo ga tačno ažurira nakon elementa $i+1$. Indukcijom stanje je tačno na kraju.

Složenost. Vrijeme $O(N)$, memorija $O(1)$ osim ulaznog niza.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
cur = best = 1
for i in range(1, n):
    cur = cur + 1 if a[i] > a[i - 1] else 1
    best = max(best, cur)
print(best)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < long long > a(n);
    for (auto & x : a) cin >> x;
    int cur = 1, best = 1;
    for (int i = 1; i < n; i++) {
        cur = a[i] > a[i - 1] ? cur + 1 : 1;
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

Važne provjere

Strogo rastući znači >, ne >=.

Traži se uzastopni segment, ne proizvoljan podniz.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 1000	5
2	Niz je neopadajući	5
3	Puna ograničenja	10

ZADATAK 016

Tačka ravnoteže

Cilj: Koristi ukupan zbir i tekući prefiks za pronalazak indeksa ravnoteže.

Dat je niz od N cijelih brojeva. Nađi najmanji 1-bazni indeks i takav da je zbir elemenata strogo lijevo od i jednak zbiru elemenata strogo desno od i . Ako ne postoji, ispiši -1.

Ulaz. N , zatim N cijelih brojeva.

Izlaz. Najmanji indeks ili -1.

Ograničenja. $1 \leq N \leq 300000$, $|a_i| \leq 10^9$.

Primjer

ULAZ

7

-7 1 5 2 -4 3 0

IZLAZ

4

Razumijevanje i postupak

Ako je left zbir prije i , tada je $\text{right} = \text{total} - \text{left} - a[i]$. Nema potrebe računati svaki desni zbir iznova. Prolazimo s lijeva nadesno da automatski dobijemo najmanji indeks.

1. Izračunaj total.
2. $\text{left} = 0$. Za svaki i izračunaj $\text{right} = \text{total} - \text{left} - a[i]$.
3. Ako $\text{left} == \text{right}$, ispiši $i+1$.
4. Inače $\text{left} += a[i]$.

Zašto postupak daje tačan rezultat?

Svaki element intervala $[l, r]$ pojavljuje se u $s[r+1]$, a svi elementi prije l poništavaju se oduzimanjem $s[l]$.

Složenost. Vrijeme $O(N)$, memorija $O(1)$ dodatno.

PYTHON 3 • potpuno rješenje

```

n = int(input())
a = list(map(int, input().split()))
tot = sum(a)
left = 0
for i, x in enumerate(a):
    if left == tot - left - x:
        print(i + 1)
        break
    left += x
else:
    print(-1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > a(n);
    long long tot = 0;
    for (auto & x : a) {
        cin >> x;
        tot += x;
    }
    long long l = 0;
    for (int i = 0; i < n; i++) {
        if (l == tot - l - a[i]) {
            cout << i + 1 << "\n";
            return 0;
        }
        l += a[i];
    }
    cout << - 1 << "\n";
}

```

Važne provjere

Element $a[i]$ ne pripada ni lijevom ni desnom zbiru.

Traži se najmanji indeks, zato završiti na prvom pronađenom.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 1000$	5
2	Svi $a_i \geq 0$	5
3	Puna ograničenja	10

15. UPINITK: gradska/općinska takmičenja 2021–2023

U ovoj cjelini su stvarni imenovani zadaci iz UPINITK biltena za osnovne škole, a ne vježbe predstavljene kao arhivski originali. Svaki zadatak ima sažetu postavku, oznaku izvora i novu razradu. Sljedeći primjeri ulaza i izlaza pripremljeni su za knjigu.

Izvori [A1]–[A3] navedeni su na kraju. Godine se odnose na sadržaj biltena, ne na godinu direktorija u adresi dokumenta. Tamo gdje je arhiva neprecizna, razrada izričito navodi usvojeni model, umjesto da neprimjetno mijenja zadatak.

Zadatak	Naziv
17	TRKA
18	KOVANICE
19	PIZZA
20	BROJKE I SLOVA
21	VISINE
22	PLASTENIK
23	SEKCIJA
24	BAZEN
25	DOMINE
26	SILOSI
27	DAN JEDNAKOSTI
28	BLAGO

ZADATAK 017 / UPITNIK ARHIVA

TRKA

Gradska/općinska takmičenja za osnovne škole, 2021. • [A1], PDF str. 29.

Dva završna vremena pretvori u sekunde i izračunaj razmak između njih.

Ulaz. Šest brojeva: h1, m1, s1, h2, m2, s2, redom u zasebnim redovima.

Izlaz. Razlika u sekundama.

Za ovu razradu vremena pripadaju istom danu i drugo nije ranije od prvog; minute i sekunde su 0–59. Izvorne granice i primjeri nisu potpuno usklađeni.

Primjer

ULAZ

```
9
58
40
10
1
15
```

IZLAZ

```
155
```

Do 10:00:00 prođe 80 sekundi, a zatim još 75. Ukupno je 155 sekundi.

Razumijevanje i postupak

Sat, minuta i sekunda nisu tri nezavisne veličine koje smijemo oduzimati bez prijenosa. Najprije ih svedemo na zajedničku jedinicu. Jedan sat sadrži 3600, a minuta 60 sekundi.

Za jedno vrijeme izračunamo $t = 3600 \cdot h + 60 \cdot m + s$. Nakon pretvaranja više nema posebne obrade prijelaza iz minute u sat: dovoljno je oduzimanje. Tako izbjegavamo niz uslova za „posuđivanje“ minuta i sekundi.

1. Učitaj oba vremena.
2. Izračunaj broj sekundi za svako vrijeme.
3. Ispiši drugi broj umanjen za prvi.

Zašto postupak daje tačan rezultat?

Pretvaranje čuva proteklo vrijeme jer svaki sat i svaku minutu zamjenjuje jednakim brojem sekundi. Zato razlika pretvorenih vrijednosti predstavlja upravo vremenski razmak.

Složenost. Vrijeme $O(1)$; dodatna memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
import sys
v = list(map(int, sys.stdin.read().split()))
t1 = 3600 * v[0] + 60 * v[1] + v[2]
t2 = 3600 * v[3] + 60 * v[4] + v[5]
print(t2 - t1)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long h1, m1, s1, h2, m2, s2;
    cin >> h1 >> m1 >> s1 >> h2 >> m2 >> s2;
    long long t1 = 3600 * h1 + 60 * m1 + s1;
    long long t2 = 3600 * h2 + 60 * m2 + s2;
    cout << t2 - t1 << '\n';
}
```

Važne provjere

Minute množiti sa 60, a sate sa 3600.

Ne oduzimati zasebne komponente kao decimalne cifre.

ZADATAK 018 / UPINITK ARHIVA

KOVANICE

Gradska/općinska takmičenja za osnovne škole, 2021. • [A1], PDF str. 40.

Iznos N sastavi s najmanje kovanica vrijednosti 5, 2 i 1.

Ulaz. Jedan cijeli broj N .

Izlaz. Broj petica, dvojki i jedinica, svaki u svom redu.

Ograničenja. 1 N 100.

Primjer

ULAZ

47

IZLAZ

9

1

0

Devet petica daje 45. Preostale 2 plaćamo jednom dvojkom; jedinice nisu potrebne.

Razumijevanje i postupak

Podijelimo iznos sa 5: količnik je broj petica, a ostatak još neisplaćeni dio. Preostali iznos zatim podijelimo sa 2. Nakon toga može ostati samo 0 ili 1.

Nije dovoljno reći „uvijek uzmi najveću kovanicu“: takvo pravilo nije tačno za proizvoljne apoene. Za ove apoene dvije jedinice možemo zamijeniti jednom dvojkom. Iznos 5 isplaćen manjim kovanicama traži najmanje tri kovanice; petica taj broj smanjuje.

1. Odredi $N // 5$ petica i ostatak $N \% 5$.
2. Od ostatka odredi dvojke i novi ostatak.
3. Posljednji ostatak predstavlja broj jedinica.

Zašto postupak daje tačan rezultat?

Za zadani broj petica ostatak se najkraće plaća dvojkama i eventualno jednom jedinicom. Smanjivanje broja petica za jednu povećava ostatak za 5, što traži najmanje dvije dodatne manje kovanice; ukupan broj zato ne pada. Najviše mogućih petica daje minimum.

Složenost. Vrijeme $O(1)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
petice, ostatak = divmod(n, 5)
dvojke, jedinice = divmod(ostatak, 2)
print(petice)
print(dvojke)
print(jedinice)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    cout << n / 5 << '\n';
    n %= 5;
    cout << n / 2 << '\n' << n % 2 << '\n';
}
```

Važne provjere

Ne ispisati samo ukupan broj kovanica.

Pohlepni izbor ovdje zavisi od skupa apoena.

ZADATAK 019 / UPINITK ARHIVA

PIZZA

Gradska/općinska takmičenja za osnovne škole, 2021. • [A1], PDF str. 42.

Osam količina raspoređeno je u krug. Odaberi četiri susjedne s najvećim zbirom.

Ulaz. Osam cijelih brojeva, svaki u svom redu.

Izlaz. Najveći zbir.

Ograničenja. Svaka količina je 0–50.

Primjer

ULAZ

```
9
0
0
1
2
0
0
8
```

IZLAZ

```
17
```

Blok koji obuhvata posljednji i prvi položaj može sadržavati $8 + 9 + 0 + 0 = 17$.

Razumijevanje i postupak

Početak odabranog dijela može biti bilo koji od osam položaja. Za svaki početak saberemo četiri člana. Pošto je raspored kružan, iza položaja 7 slijedi položaj 0.

Indeks $(p + \text{korak}) \% 8$ automatski obavlja taj povratak. Ne treba posebno praviti niz uslova za početke 5, 6 i 7. Broj kandidata je samo osam, pa je potpuno pretraživanje ovdje jednostavno i dovoljno.

1. Učitaj osam vrijednosti.
2. Za svaki početak p saberi članove s indeksima $(p+j) \% 8$, za j od 0 do 3.
3. Sačuvaj najveći od osam zbirova.

Zašto postupak daje tačan rezultat?

Svaki dopušteni izbor ima tačno jedan početni položaj u kružnom redoslijedu. Algoritam obrađuje svih osam početaka i za svaki računa odgovarajući zbir, pa maksimum obuhvata i optimalni izbor.

Složenost. Za osam dijelova vrijeme i memorija su $O(1)$. Za krug od n dijelova i blok dužine k ovaj neposredni postupak je $O(nk)$.

PYTHON 3 • potpuno rješenje

```
import sys
a = list(map(int, sys.stdin.read().split()))
najbolje = 0
for p in range(8):
    zbir = 0
    for j in range(4):
        zbir += a[(p + j) % 8]
    najbolje = max(najbolje, zbir)
print(najbolje)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a(8);
    for (int & x : a) cin >> x;
    int najbolje = 0;
    for (int p = 0; p < 8; p++) {
        int zbir = 0;
        for (int j = 0; j < 4; j++) zbir += a[(p + j) % 8];
        najbolje = max(najbolje, zbir);
    }
    cout << najbolje << '\n';
}
```

Važne provjere

Ne čitati početni broj N: ovdje je unaprijed poznato da ima osam vrijednosti.

Ne preskočiti blokove koji prelaze kraj zapisa.

ZADATAK 020 / UPINITK ARHIVA

BROJKE I SLOVA

Gradska/općinska takmičenja za osnovne škole, 2021. • [A1], PDF str. 43.

Iz devet slova ukloni slova svake od dvije riječi, nezavisno. Za svako uklanjanje uzmi prvo preostalo pojavljivanje.

Ulaz. Tri reda: S, prva riječ, druga riječ.

Izlaz. Dva preostala zapisa, bez promjene redoslijeda.

Ograničenja. Riječi koriste dostupna velika slova; $|S| = 9$.

Primjer

ULAZ
PROGRAMER
MORE
RAME

IZLAZ
PGRAR
POGRR

Za MORE uklanjamo redom prvo M, O, R i E. Za RAME ponovo polazimo od svih slova zapisa PROGRAMER.

Razumijevanje i postupak

Dvije riječi ne dijele isti postupak uklanjanja: obje počinju od originalnog zapisa S. To je važnije od samog načina brisanja.

Funkcija prima riječ i pravi lokalnu kopiju zapisa. Za svako slovo traži prvo trenutno pojavljivanje i briše samo njega. Metoda `replace(slovo, "", 1)` u Pythonu ograničava zamjenu na jedno pojavljivanje. U C++-u `find` određuje položaj, a `erase(p, 1)` briše jedan znak.

1. Za prvu riječ postavi `ostatak = S`.
2. Redom ukloni po jedno prvo pojavljivanje svakog njenog slova.
3. Ponovi od novog ostatka `= S` za drugu riječ.

Zašto postupak daje tačan rezultat?

Poslije obrade prvih k slova riječi uklonjeno je upravo k traženih prvih pojavljivanja, u propisanom redoslijedu. Sljedeće brisanje održava to svojstvo. Kopiranje S prije svake riječi osigurava nezavisnost rezultata.

Složenost. Za dužinu $S = n$ i ukupno m slova riječi, vrijeme $O(nm)$, memorija $O(n)$. U zadatku je $n = 9$.

PYTHON 3 • potpuno rješenje

```

s = input().strip()
r1 = input().strip()
r2 = input().strip()
def ostatak(rijec):
    rezultat = s
    for slovo in rijec:
        rezultat = rezultat.replace(slovo, "", 1)
    return rezultat
print(ostatak(r1))
print(ostatak(r2))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string s, r1, r2;
    cin >> s >> r1 >> r2;
    for (string r : {
        r1, r2
    }) {
        string ostatak = s;
        for (char c : r) {
            size_t p = ostatak.find(c);
            ostatak.erase(p, 1);
        }
        cout << ostatak << '\n';
    }
}

```

Važne provjere

Ne ukloniti sva pojavljivanja jednog slova odjednom.

Drugu riječ ne obrađivati na ostatku prve.

ZADATAK 021 / UPINITK ARHIVA

VISINE

III gradsko/općinsko takmičenje za osnovne škole, 2022. • [A2], PDF str. 39.

Četiri visine čine opadajući aritmetički niz. Iz najveće A i najmanje G odredi dvije srednje.

Ulaz. A, zatim G.

Izlaz. Dvije srednje visine, od veće prema manjoj.

Ograničenja. 125 ≤ A ≤ 190; 85 ≤ G ≤ 160; visine su cjelobrojne.

Primjer

ULAZ

180

150

IZLAZ

170

160

Razlika krajnjih visina je 30. Tri jednaka razmaka imaju po 10 cm.

Razumijevanje i postupak

Između četiri visine postoje tri jednaka razmaka, a ne četiri. Ako je razmak d, tada vrijedi $A - G = 3d$.

Prva srednja visina je $A - d$, a druga $A - 2d$. Nema potrebe isprobavati sve moguće visine: jednakost razmaka već određuje jedinstveno rješenje. Učenik može nacrtati četiri tačke i između njih označiti tri razmaka d.

1. Izračunaj $d = (A - G) // 3$.

2. Ispiši $A - d$.

3. Ispiši $A - 2d$.

Zašto postupak daje tačan rezultat?

Dobijeni niz je A, A-d, A-2d, A-3d. Razlike susjednih članova jednake su d, a posljednji član je G, pa su ispunjeni svi uslovi.

Složenost. Vrijeme $O(1)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
import sys
a, g = map(int, sys.stdin.read().split())
d = (a - g) // 3
print(a - d)
print(a - 2 * d)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a, g;
    cin >> a >> g;
    int d = (a - g) / 3;
    cout << a - d << '\n' << a - 2 * d << '\n';
}
```

Važne provjere

Dijeliti sa 3, ne sa 4.

Ne obrnuti redoslijed ispisa.

ZADATAK 022 / UPINITK ARHIVA

PLASTENIK

III gradsko/općinsko takmičenje za osnovne škole, 2022. • [A2], PDF str. 40.

Tri oznake DA/NE opisuju prozore. Prema broju otvorenih odredi stanje.

Ulaz. Tri reda, po jedna oznaka.

Izlaz. Za 3: promaha; za 1 ili 2: vjetrenje; za 0: ustajao vazduh.

Primjer

ULAZ

NE

DA

NE

IZLAZ

vjetrenje

Otvoren je jedan od tri prozora; njegovo mjesto ne mijenja kategoriju.

Razumijevanje i postupak

Tri logička podatka možemo prvo pretvoriti u jedan broj: koliko ih ima vrijednost DA. Tada osam mogućih kombinacija svodimo na tri smislene grupe.

Prvo provjeravamo poseban slučaj 3. Potom uslov otvoreni > 0 obuhvata preostale pozitivne mogućnosti. Preostali slučaj je nula. Ovakav redoslijed sprječava da slučaj tri otvorena prozora prerano završi u općoj grani.

1. Prebroj oznake DA.
2. Ako je broj 3, ispiši promaha.
3. Inače, ako je pozitivan, ispiši vjetrenje; u suprotnom ustajao vazduh.

Zašto postupak daje tačan rezultat?

Broj otvorenih prozora pripada skupu {0,1,2,3}. Tri grane programa su međusobno isključive i zajedno pokrivaju taj skup, s izlazom koji odgovara svakoj grupi.

Složenost. Vrijeme $O(1)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
otvoreni = sum(input().strip() == "DA" for _ in range(3))
if otvoreni == 3:
    print("promaha")
elif otvoreni > 0:
    print("vjetrenje")
else:
    print("ustajao vazduh")
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int otvoreni = 0;
    for (int i = 0; i < 3; i++) {
        string s;
        cin >> s;
        if (s == "DA") otvoreni++;
    }
    if (otvoreni == 3) cout << "promaha";
    else if (otvoreni > 0) cout << "vjetrenje";
    else cout << "ustajao vazduh";
    cout << '\n';
}
```

Važne provjere

Izlazne riječi moraju biti zapisane tačno kako je propisano.

Opći uslov „barem jedan“ ne provjeravati prije posebnog slučaja „sva tri“.

ZADATAK 023 / UPINITK ARHIVA

SEKCIJA

III gradsko/općinsko takmičenje za osnovne škole, 2022. • [A2], PDF str. 41.

Prebroj zapise ocjena čija je šesta ocjena barem 4, a deveta 5.

Ulaz. N, zatim N redova sa po 11 cifara ocjena.

Izlaz. Broj odgovarajućih zapisa.

Ograničenja. 1 $N \leq 200$; cifre su 1–5.

Primjer

ULAZ

```
3
11111411511
5555355555
2222522522
```

IZLAZ

```
2
```

Prvi i treći zapis prolaze oba uslova. Drugi ima matematiku 3.

Razumijevanje i postupak

Redni broj u tekstu i indeks u programu razlikuju se za jedan. Šesta cifra je `s[5]`, a deveta `s[8]`.

Oba uslova moraju biti ispunjena, zato koristimo logičko I. Pošto je svaka ocjena jedna cifra, možemo porediti znakove sa "4" i "5" ili ih pretvoriti u brojeve. Ne treba računati prosjek, niti pregledati sve ostale ocjene: za odluku su bitna samo dva mjesta.

1. Brojač postavi na nulu.
2. Za svaki zapis provjeri šestu i devetu cifru.
3. Povećaj brojač samo ako su oba uslova tačna.

Zašto postupak daje tačan rezultat?

Za svaki zapis program provjerava tačno dvije propisane ocjene. Brojač se povećava jednom i samo jednom za svaki odgovarajući zapis, pa nakon svih redova daje traženi broj.

Složenost. Vrijeme $O(N)$, jer svaki zapis ima stalnu dužinu 11; dodatna memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
broj = 0
for _ in range(n):
    s = input().strip()
    if int(s[5]) >= 4 and s[8] == "5":
        broj += 1
print(broj)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, broj = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        string s;
        cin >> s;
        if (s[5] >= '4' && s[8] == '5') broj++;
    }
    cout << broj << '\n';
}
```

Važne provjere

Ne koristiti indekse 6 i 9.

Operator OR nije zamjena za AND.

ZADATAK 024 / UPITNIK ARHIVA

BAZEN

III gradsko/općinsko takmičenje za osnovne škole, 2022. • [A2], PDF str. 42.

Za intervale prisustva $[a,b)$ nađi najveći broj istovremenih posjetilaca.

Ulaz. N, zatim N parova dolaska i odlaska.

Izlaz. Najveći broj prisutnih.

Ograničenja. 1 $N \leq 50\,000$; vremena su prirodni brojevi.

Primjer

ULAZ

```
4
1 4
4 6
2 5
3 4
```

IZLAZ

```
3
```

Od 3 do 4 prisutna su tri posjetioca. U trenutku 4 dva odlaze i jedan dolazi, pa ostaju dva.

Razumijevanje i postupak

Vrijeme može biti veliko, ali broj promjena je samo dvostruko veći od broja posjetilaca. Za dolazak zapisujemo +1, za odlazak 1. Zatim sortiramo događaje.

U istom trenutku odlazak se obrađuje prije dolaska. To nije tehnička sitnica: posjetilac u trenutku odlaska više nije prisutan. Sortiranje parova (vrijeme, promjena) upravo postavlja 1 ispred +1. Tek nakon promjene ažuriramo najveći broj prisutnih.

1. Formiraj $2N$ događaja.
2. Sortiraj po vremenu pa promjeni.
3. Sabiraj promjene i pamti najveći dostignuti zbir.

Zašto postupak daje tačan rezultat?

Poslije obrade događaja u trenutku t tekući zbir jednak je broju započetih, a nezavršenih intervala. Između događaja se ništa ne mijenja. Zato se najveća prisutnost nalazi među provjerenim stanjima.

Složenost. Vrijeme $O(N \log N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n = int(input())
dogadjaji = []
for _ in range(n):
    a, b = map(int, input().split())
    dogadjaji.append((a, 1))
    dogadjaji.append((b, -1))
dogadjaji.sort()
trenutno = najbolje = 0
for vrijeme, promjena in dogadjaji:
    trenutno += promjena
    najbolje = max(najbolje, trenutno)
print(najbolje)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < pair < long long, int >> d;
    for (int i = 0; i < n; i++) {
        long long a, b;
        cin >> a >> b;
        d.push_back({
            a, 1
        });
        d.push_back({
            b, - 1
        });
    }
    sort(d.begin(), d.end());
    int sada = 0, naj = 0;
    for (auto [t, p] : d) {
        sada += p;
        naj = max(naj, sada);
    }
    cout << naj << '\n';
}

```

Važne provjere

Ne brojati odlazećeg posjetioca u trenutku b.

Ne obilaziti svaku jedinicu velike vremenske ose.

ZADATAK 025 / UPINITK ARHIVA

DOMINE

IV gradsko/općinsko takmičenje za osnovne škole, 2023. • [A3], PDF str. 34.

Na ploču $M \times N$ postavi najviše domina 2×1 , bez preklapanja i izlaska iz ploče.

Ulaz. M i N .

Izlaz. Najveći broj domina.

Ograničenja. 1 $M, N \leq 16$; rotacija je dopuštena.

Primjer

ULAZ

5 7

IZLAZ

17

Površina je 35. Sedamnaest domina pokriva 34 polja, a jedno ostaje prazno.

Razumijevanje i postupak

Jedna domina zauzima dva polja. Zato odgovor ne može biti veći od cijelog dijela broja $MN/2$. Ali gornja granica sama po sebi nije dokaz: treba pokazati raspored koji je dostiže.

Ako je jedna dimenzija parna, ploču popunimo parovima po toj dimenziji. Ako su obje neparne, popunimo sve osim posljednje kolone, a zatim u toj koloni uparimo sva polja osim jednog. Tako u cijeloj ploči ostaje samo jedno prazno polje.

1. Izračunaj broj polja $M \cdot N$.
2. Podijeli cjelobrojno sa 2.
3. Ispiši rezultat.

Zašto postupak daje tačan rezultat?

Svaki raspored ima najviše $\text{floor}(MN/2)$ domina jer se nijedno polje ne smije ponoviti. Opisano popločavanje ostvaruje tu vrijednost i za parnu i za neparnu površinu, pa je granica optimalna.

Složenost. Vrijeme $O(1)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
import sys
m, n = map(int, sys.stdin.read().split())
print(m * n // 2)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int m, n;
    cin >> m >> n;
    cout << m * n / 2 << '\n';
}
```

Važne provjere

Ne zaokruživati naviše.

Izraz $(M//2) \cdot N$ nije dovoljan kada je M neparan, a N paran.

ZADATAK 026 / UPINITK ARHIVA

SILOSI

IV gradsko/općinsko takmičenje za osnovne škole, 2023. • [A3], PDF str. 35.

Tri silosa pune se redom. Odredi sadržaj svakog nakon dolaska količine P.

Ulaz. Kapaciteti S1,S2,S3, zatim P, svaki u svom redu.

Izlaz. Tri pohranjene količine, redom.

Bilten ne zadaje numeričke gornje granice. Razrada koristi nenegativne cijele količine; u C++-u long long.

Primjer

ULAZ

6
4
9
13

IZLAZ

6
4
3

Poslije prvog silosa ostaje 7, poslije drugog 3. Treći zato nije pun.

Razumijevanje i postupak

U jedan silos možemo smjestiti najviše njegov kapacitet, ali ne možemo smjestiti više od preostale količine. To je upravo minimum te dvije vrijednosti.

Poslije punjenja prvog silosa oduzmemo spremljenu količinu i isti postupak ponovimo za sljedeći. Jedna kratka petlja zamjenjuje više dugih i gotovo jednakih grananja. Ako ukupni kapacitet nije dovoljan, ostatak se ne pripisuje nijednom silosu.

1. Preostalo postavi na P.
2. Za svaki kapacitet uzmi minimum kapaciteta i preostalog.
3. Oduzmi pohranjeno i ispiši ga.

Zašto postupak daje tačan rezultat?

Prije svakog koraka preostalo je tačno ono što nije stalo u ranije silose. Minimum poštuje kapacitet i raspoloživu količinu, a oduzimanje održava isto svojstvo za naredni silos.

Složenost. Vrijeme $O(1)$; memorija $O(1)$, jer postoje tačno tri silosa.

PYTHON 3 • potpuno rješenje

```
import sys
v = list(map(int, sys.stdin.read().split()))
preostalo = v[3]
for kapacitet in v[:3]:
    pohranjeno = min(kapacitet, preostalo)
    print(pohranjeno)
    preostalo -= pohranjeno
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long s [3], p;
    cin >> s [0] >> s [1] >> s [2] >> p;
    for (long long kap : s) {
        long long uzmi = min(kap, p);
        cout << uzmi << '\n';
        p -= uzmi;
    }
}
```

Važne provjere

Ne nastaviti koristiti početni P nakon prvog punjenja.

Sadržaj silosa nikada ne smije biti veći od kapaciteta.

ZADATAK 027 / UPINITK ARHIVA

DAN JEDNAKOSTI

IV gradsko/općinsko takmičenje za osnovne škole, 2023. • [A3], PDF str. 36.

Samo povećavanjem vrijednosti izjednači niz uz najmanji ukupan dodatak.

Ulaz. N, zatim N vrijednosti.

Izlaz. Najmanji potreban dodatak.

Ograničenja. 1 N 100; 0 ai 1 000 000.

Primjer

ULAZ

4
3 1 6 2

IZLAZ

12

Do zajedničke vrijednosti 6 nedostaje redom 3, 5, 0 i 4. Njihov zbir je 12.

Razumijevanje i postupak

Najveća početna vrijednost ne može se smanjiti. Zato zajednička konačna vrijednost ne može biti manja od maksimuma. Izbor baš maksimuma zahtijeva najmanje dodavanja.

Za svaki element doplata je maksimum element. Zbir tih razlika možemo računati neposredno ili kao $N \cdot \text{maksimum} - \text{zbir}$. Drugi zapis otkriva zašto nema potrebe za sortiranjem: dovoljan je maksimum i suma.

1. Odredi maksimum niza.
2. Saberi razlike maksimum ai.
3. Ispiši zbir.

Zašto postupak daje tačan rezultat?

Svaki dopušten zajednički cilj x mora biti barem $\max(a_i)$. Potreban dodatak je $N \cdot x - \text{suma}(a_i)$, rastuća funkcija od x. Najmanji dopušten cilj zato daje najmanji dodatak.

Složenost. Vrijeme $O(N)$; memorija $O(N)$ u prikazanom zapisu.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
cilj = max(a)
print(sum(cilj - x for x in a))
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > a(n);
    for (auto & x : a) cin >> x;
    long long cilj = * max_element(a.begin(), a.end()), zbir = 0;
    for (long long x : a) zbir += cilj - x;
    cout << zbir << '\n';
}
```

Važne provjere

Prosjek nije dopušten cilj ako traži smanjivanje nekog elementa.

Veći zajednički cilj samo povećava trošak.

ZADATAK 028 / UPINITK ARHIVA

BLAGO

IV gradsko/općinsko takmičenje za osnovne škole, 2023. • [A3], PDF str. 37.

Prati I/Z/S/J kroz matricu O/B od gornjeg lijevog ugla i prebroj dolaske na B.

Ulaz. Upute; M N; zatim M redova matrice.

Izlaz. Broj posjeta poljima B.

Precizirani model: početno polje je O i sve upute ostaju u matrici. Biltenove Python i C++ verzije različito obrađuju početno B; zato ga ovdje isključujemo.

Primjer

```
ULAZ
IJZIS
2 3
0 B 0
B B 0
```

```
IZLAZ
5
```

Pet pomjeranja vodi na B pet puta. Povratak na ranije posjećeno polje računa se ponovo.

Razumijevanje i postupak

Stanje simulacije čine red r , kolona k i broj dosadašnjih posjeta blagu. Smjer I povećava kolonu, Z je smanjuje, J povećava red, a S ga smanjuje.

Svaka naredba mijenja položaj tačno jednom. Poslije pomjeranja pogledamo novo polje. Ne trebaju BFS ni traženje puta: put je već zadan. Razmak između znakova u ulazu uklanjamo, kako bi matrica imala jedan znak po koloni.

1. Postavi položaj na (0,0) i brojač na nulu.
2. Pročitaj naredbu i promijeni odgovarajuću koordinatu.
3. Ako novo polje sadrži B, povećaj brojač.

Zašto postupak daje tačan rezultat?

Indukcijom po broju naredbi: prije svake naredbe položaj odgovara kraju već izvršenog dijela puta, a brojač njegovim posjetama blagu. Pomjeranje i provjera novog polja održavaju oba svojstva.

Složenost. Vrijeme $O(MN + L)$, gdje je L broj naredbi; memorija $O(MN)$.

PYTHON 3 • potpuno rješenje

```

upute = input().strip()
m, n = map(int, input().split())
polje = ["".join(input().split()) for _ in range(m)]
smjer = {"I": (0, 1), "Z": (0, -1), "S": (-1, 0), "J": (1, 0)}
r = k = broj = 0
for znak in upute:
    dr, dk = smjer[znak]
    r += dr
    k += dk
    broj += polje[r][k] == "B"
print(broj)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    int m, n;
    cin >> s >> m >> n;
    vector < string > a(m, string(n, '0'));
    for (int r = 0; r < m; r++) for (int k = 0; k < n; k++) cin >> a[r][k];
    int r = 0, k = 0, broj = 0;
    for (char c : s) {
        if (c == 'I') k++;
        else if (c == 'Z') k--;
        else if (c == 'S') r--;
        else r++;
        if (a[r][k] == 'B') broj++;
    }
    cout << broj << '\n';
}

```

Važne provjere

Red raste prema dolje, ne prema gore.

Ovo je broj posjeta, ne broj različitih polja.

16. Nizovi, stringovi i sortiranje

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
29	Rang-lista
30	Pjesme na USB-u
31	Klikeri u kutijama
32	Muzejski prolaz
33	Najduži niz bez ponavljanja
34	Najbolji blok od K dana
35	Najduži budžetski segment
36	Par najbliži cilju
37	Skoro palindrom
38	Poredak takmičara
39	Pakovanje poklona
40	Najkraći skupi segment
41	Kružna smjena
42	Takmičarski parovi
43	Palindrom jednim brisanjem
44	Najkraći segment sa K različitih
45	Par najbliži cilju

ZADATAK 029

Rang-lista

Cilj: Sortirati složene zapise po više kriterija.

Za N takmičara poznati su ime, broj bodova i kazneno vrijeme. Bolji je takmičar s više bodova; ako su bodovi jednaki, bolji je onaj s manjim kaznenim vremenom; ako je i to jednako, imena se poredaju leksikografski. Ispiši konačnu rang-listu.

Ulaz. U prvom redu N . U narednih N redova: ime bodovi kazna.

Izlaz. Ispisati imena takmičara redom od prvog do posljednjeg.

Ograničenja. 1 $N \leq 100000$; ime bez razmaka; bodovi i kazna su nenegativni cijeli brojevi.

Primjer

ULAZ

```
4
Amina 300 120
Boris 300 100
Dino 250 20
Adna 300 100
```

IZLAZ

```
Adna
Boris
Amina
Dino
```

Razumijevanje i postupak

Potrebna je kompozitna sortirna ključ-funkcija. U Pythonu možemo sortirati po (-bodovi, kazna, ime). U C++ comparator vraća true kada prvi zapis treba stajati prije drugog.

1. Učitati sve zapise.
2. Sortirati: bodovi opadajuće, kazna rastuće, ime rastuće.
3. Ispisati imena.

Složenost. $O(N \log N)$ vrijeme i $O(N)$ memorija.

PYTHON 3 • potpuno rješenje

```

n = int(input())
a = []
for _ in range(n):
    ime, b, k = input().split()
    a.append((ime, int(b), int(k)))
a.sort(key=lambda x: (-x[1], x[2], x[0]))
for ime, _, _ in a:
    print(ime)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;
struct T {
    string ime;
    int bod, kazna;
}
;
int main() {
    int n;
    cin >> n;
    vector < T > a(n);
    for (auto & x : a) cin >> x.ime >> x.bod >> x.kazna;
    sort(a.begin(), a.end(), [] (const T & x, const T & y) {
        if (x.bod != y.bod) return x.bod > y.bod; if (x.kazna != y.kazna)
            return
                x.kazna < y.kazna; return x.ime < y.ime;
    });
    for (auto & x : a) cout << x.ime << "\n";
}

```

ZADATAK 030

Pjesme na USB-u

Cilj: Prepoznati greedy strategiju „uzmi najkraće prvo“ za maksimiziranje broja stavki.

USB plejer može reproducirati najviše T sekundi muzike. Na raspolaganju je N pjesama, a i -ta traje a_i sekundi. Koliko najviše cijelih pjesama možemo smjestiti ako biramo proizvoljan podskup?

Ulaz. U prvom redu N i T , u drugom N trajanja.

Izlaz. Ispisati najveći mogući broj pjesama.

Ograničenja. $1 \leq N \leq 200000$, $1 \leq a_i, T \leq 10^9$.

Primjer

ULAZ

```
6 15
7 2 4 3 9 5
```

IZLAZ

```
4
```

Razumijevanje i postupak

Ako cilj nije maksimalno ukupno trajanje nego maksimalan broj pjesama, kratke pjesme su uvijek najmanje „skupe“. Sortiramo trajanja rastuće i dodajemo dok ukupno trajanje ne pređe T . Zamjena neke izabrane kratke pjesme dužom ne može povećati broj pjesama.

1. Sortirati trajanja rastuće.
2. Dodavati jedno po jedno dok zbir + sljedeća T .
3. Brojati koliko ih je stalo.

Složenost. $O(N \log N)$ vrijeme zbog sortiranja, $O(N)$ memorija.

PYTHON 3 • potpuno rješenje

```
n, t = map(int, input().split())
a = sorted(map(int, input().split()))
s = 0
cnt = 0
for x in a:
    if s + x > t:
        break
    s += x
    cnt += 1
print(cnt)
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    long long T;
    cin >> n >> T;
    vector < long long > a(n);
    for (auto & x : a) cin >> x;
    sort(a.begin(), a.end());
    long long s = 0;
    int cnt = 0;
    for (long long x : a) {
        if (s + x > T) break;
        s += x;
        cnt++;
    }
    cout << cnt << "\n";
}
```

ZADATAK 031

Klikeri u kutijama

Cilj: Koristiti prefiksne sume za minimalan broj prenosa između susjednih pozicija.

U n kutija poredanih u nizu nalazi se a_i klikera. Ukupan broj klikera djeljiv je sa n . U jednom potezu jedan kliker može se prebaciti iz kutije i u susjednu kutiju $i-1$ ili $i+1$. Odredi najmanji broj takvih pojedinačnih prenosa da sve kutije imaju jednak broj klikera.

Ulaz. U prvom redu N , u drugom N nenegativnih cijelih brojeva.

Izlaz. Ispisati minimalan broj prenosa.

Ograničenja. $1 \leq N \leq 200000$, zbir $a_i \leq 10^{14}$ i djeljiv je sa N .

Primjer

ULAZ

4

0 4 0 4

IZLAZ

4

Razumijevanje i postupak

Neka je cilj $M = \text{zbir}/N$. Nakon prvih i kutija, višak u tom prefiksu mora preći granicu između i i $i+1$, ili manjak mora biti dopunjen preko te granice. Količina koja nužno pređe tu granicu jednaka je apsolutnoj prefiksnoj neravnoteži. Zbir tih apsolutnih vrijednosti je optimum.

1. Izračunati M .
2. Voditi $\text{bal} += a_i - M$.
3. Za svaku granicu poslije kutije i dodati $|\text{bal}|$ u odgovor ($i=0..N-2$).

Složenost. $O(N)$ vrijeme i $O(N)$ memorija za niz; $O(1)$ dodatne memorije nakon učitavanja.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
m = sum(a) // n
bal = 0
ans = 0
for i in range(n - 1):
    bal += a[i] - m
    ans += abs(bal)
print(ans)
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
#include <cstdlib>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > a(n);
    long long s = 0;
    for (auto & x : a) {
        cin >> x;
        s += x;
    }
    long long m = s / n, bal = 0, ans = 0;
    for (int i = 0; i < n - 1; i++) {
        bal += a[i] - m;
        ans += llabs(bal);
    }
    cout << ans << "\n";
}
```

ZADATAK 032

Muzejski prolaz

Cilj: Pronaći najduži uzastopni segment sa zbirom ograničenim odozgo.

Muzej ima N sala poredanih redom. Za obilazak i -te sale potrebno je t_i minuta. Učenik ima najviše K minuta i želi posjetiti što više uzastopnih sala, počevši i završavajući gdje želi. Koliko najviše sala može obići?

Ulaz. U prvom redu N i K , u drugom N pozitivnih trajanja.

Izlaz. Ispisati najveću dužinu uzastopnog segmenta čiji je zbir K .

Ograničenja. $1 \leq N \leq 200000$, $1 \leq t_i, K \leq 10^9$.

Primjer

ULAZ

7 10

2 1 5 1 1 4 2

IZLAZ

5

Razumijevanje i postupak

Svi elementi su pozitivni, pa kada zbir prozora pređe K , pomjeranje lijeve granice udesno sigurno smanjuje zbir. Zato možemo održavati klizni prozor $[l, r]$ u ukupno linearnom vremenu.

1. $l=0$, suma=0.
2. Za svaki r dodati $t[r]$.
3. Dok suma $> K$, oduzimati $t[l]$ i povećavati l .
4. Ažurirati maksimum $r-l+1$.

Složenost. $O(N)$ vrijeme i $O(N)$ memorija zbog niza; algoritam koristi $O(1)$ dodatno.

PYTHON 3 • potpuno rješenje

```

n, k = map(int, input().split())
a = list(map(int, input().split()))
l = 0
s = 0
ans = 0
for r, x in enumerate(a):
    s += x
    while s > k:
        s -= a[l]
        l += 1
    ans = max(ans, r - l + 1)
print(ans)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    long long K;
    cin >> n >> K;
    vector< long long > a(n);
    for (auto & x : a) cin >> x;
    int l = 0, ans = 0;
    long long s = 0;
    for (int r = 0; r < n; r++) {
        s += a[r];
        while (s > K) s -= a[l++];
        ans = max(ans, r - l + 1);
    }
    cout << ans << "\n";
}

```

ZADATAK 033

Najduži niz bez ponavljanja

Cilj: Koristiti mapu posljednjih pozicija za najduži podstring bez ponovljenih znakova.

Dat je string S sastavljen od malih engleskih slova. Nađi dužinu najdužeg uzastopnog dijela u kojem se nijedno slovo ne pojavljuje više od jednom.

Ulaz. U jednom redu string S .

Izlaz. Ispisati traženu maksimalnu dužinu.

Ograničenja. $1 \leq |S| \leq 200000$.

Primjer

ULAZ

abcaefgh

IZLAZ

7

Razumijevanje i postupak

Održavamo lijevu granicu l trenutnog prozora bez duplikata. Kada znak $s[r]$ već postoji unutar prozora, l pomjeramo iza njegove posljednje pozicije. Potom ažuriramo posljednju poziciju znaka i dužinu.

1. $last[znak]$ čuva posljednju poziciju.
2. Ako $last[s[r]] \neq -1$, postaviti $l = last[s[r]] + 1$.
3. $last[s[r]] = r$.
4. Ažurirati maksimum $r - l + 1$.

Složenost. $O(|S|)$ vrijeme i $O(\text{alfabet})$ memorija.

PYTHON 3 • potpuno rješenje

```
s = input().strip()
last = {}
l = 0
ans = 0
for r, c in enumerate(s):
    if c in last and last[c] >= l:
        l = last[c] + 1
    last[c] = r
    ans = max(ans, r - l + 1)
print(ans)
```


C++17 • potpuno rješenje

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    string s;
    cin >> s;
    vector<int> last(256, -1);
    int l = 0, ans = 0;
    for (int r = 0; r < (int) s.size(); r++) {
        unsigned char c = s[r];
        if (last[c] >= l) l = last[c] + 1;
        last[c] = r;
        ans = max(ans, r - l + 1);
    }
    cout << ans << "\n";
}
```

ZADATAK 034

Najbolji blok od K dana

Cilj: Učenik koristi klizni prozor za zbir fiksne dužine.

Za niz dnevnih bodova odredi najveći zbir tačno K uzastopnih dana.

Ulaz. Prvi red: N K. Drugi red: N cijelih brojeva.

Izlaz. Ispisati najveći zbir prozora dužine K.

Ograničenja. $1 \leq K \leq N \leq 2 \cdot 10^5$, $|A[i]| \leq 10^9$.

Primjer

ULAZ

```
8 3
2 -1 4 5 -2 3 1 6
```

IZLAZ

```
10
```

Razumijevanje i postupak

Umjesto da za svaki početak ponovo sabiramo K elemenata, iz starog prozora izbacimo lijevi element i dodamo novi desni.

1. Izračunaj zbir prvih K elemenata.
2. Za $i=K..N-1$ uradi $sum += A[i]-A[i-K]$.
3. Pamti najveći zbir.

Složenost. Vrijeme $O(N)$; memorija $O(1)$ pored ulaza.

PYTHON 3 • potpuno rješenje

```
N, K = map(int, input().split())
A = list(map(int, input().split()))
cur = sum(A[:K])
best = cur
for i in range(K, N):
    cur += A[i] - A[i - K]
    best = max(best, cur)
print(best)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, K;
    cin >> N >> K;
    vector < long long > A(N);
    for (auto & x : A) cin >> x;
    long long cur = 0;
    for (int i = 0; i < K; ++ i) cur += A [i];
    long long best = cur;
    for (int i = K; i < N; ++ i) {
        cur += A [i] - A [i - K];
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

Važne provjere

best se mora inicijalizirati prvim prozorom, ne nulom, jer svi brojevi mogu biti negativni.

ZADATAK 035

Najduži budžetski segment

Cilj: Učenik koristi dva pokazivača na nizu nenegativnih vrijednosti.

Dat je niz nenegativnih troškova. Nađi najveći broj uzastopnih elemenata čiji zbir ne prelazi S.

Ulaz. Prvi red: N S. Drugi red: N nenegativnih brojeva.

Izlaz. Ispisati maksimalnu dužinu segmenta.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$, $0 \leq A[i] \leq 10^9$.

Primjer

ULAZ

```
7 10
2 1 5 1 3 2 4
```

IZLAZ

```
4
```

Razumijevanje i postupak

Desni kraj samo ide naprijed. Kada zbir postane prevelik, pomjeramo lijevi kraj dok ponovo ne bude dozvoljen. Nenegativnost je ključna: uklanjanje lijevog elementa ne može povećati zbir.

1. $l=0$, $sum=0$.
2. Za svaki r dodaj $A[r]$.
3. Dok $sum > S$, oduzimaj $A[l]$ i povećavaj l.
4. Ažuriraj najbolju dužinu $r-l+1$.

Složenost. Vrijeme $O(N)$, jer svaki indeks ulazi i izlazi iz prozora najviše jednom; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
N, S = map(int, input().split())
A = list(map(int, input().split()))
l = 0
cur = 0
best = 0
for r, x in enumerate(A):
    cur += x
    while cur > S and l <= r:
        cur -= A[l]
        l += 1
    best = max(best, r - l + 1)
print(best)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    long long S;
    cin >> N >> S;
    vector < long long > A(N);
    for (auto & x : A) cin >> x;
    int l = 0, best = 0;
    long long cur = 0;
    for (int r = 0; r < N; ++ r) {
        cur += A [r];
        while (cur > S && l <= r) cur -= A [l++];
        best = max(best, r - l + 1);
    }
    cout << best << "\n";
}
```

Važne provjere

Ovaj algoritam nije ispravan ako niz sadrži negativne brojeve.

Ne vraćati l unazad.

ZADATAK 036

Par najbliži cilju

Cilj: Učenik koristi sortiranje i dva pokazivača za optimizaciju nad parovima.

Izaberi dva različita elementa niza tako da njihov zbir bude što bliži ciljnoj vrijednosti T. Ako postoji više rješenja sa istom razlikom, ispiši leksikografski manji par (manja prva, zatim manja druga vrijednost).

Ulaz. Prvi red: N T. Drugi red: N cijelih brojeva.

Izlaz. Ispisati dvije vrijednosti iz odabranog para u rastućem poretku.

Ograničenja. $2 \leq N \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
6 10
1 4 7 2 9 5
```

IZLAZ

```
1 9
```

Razumijevanje i postupak

Nakon sortiranja, ako je zbir premalen povećavamo lijevi pokazivač; ako je prevelik smanjujemo desni. Tako preskačemo cijele grupe parova koje ne mogu biti bolje u datom smjeru.

1. Sortiraj niz.
2. Postavi $l=0$, $r=N-1$.
3. Procijeni par $A[l], A[r]$ i ažuriraj najbolji uz pravilo tie-breaka.
4. Ako je $zbir < T$ povećaj l , inače smanji r .

Složenost. Vrijeme $O(N \log N)$ zbog sortiranja; nakon toga $O(N)$; memorija zavisi od sortiranja.

PYTHON 3 • potpuno rješenje

```
N, T = map(int, input().split())
A = sorted(map(int, input().split()))
l, r = (0, N - 1)
best = None
while l < r:
    pair = (A[l], A[r])
    key = (abs(A[l] + A[r] - T), pair)
    if best is None or key < best[0]:
        best = (key, pair)
```

PYTHON 3 • potpuno rješenje / nastavak

```

    if A[l] + A[r] < T:
        l += 1
    else:
        r -= 1
print(*best[1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    long long T;
    cin >> N >> T;
    vector < long long > A(N);
    for (auto & x : A) cin >> x;
    sort(A.begin(), A.end());
    int l = 0, r = N - 1;
    long long bestDiff = LLONG_MAX, bx = 0, by = 0;
    while (l < r) {
        long long x = A[l], y = A[r], diff = labs(x + y - T);
        if (diff < bestDiff || (diff == bestDiff && make_pair(x, y) <
            make_pair(bx, by))) {
            bestDiff = diff;
            bx = x;
            by = y;
        }
        if (x + y < T) ++ l;
        else -- r;
    }
    cout << bx << " " << by << "\n";
}

```

Važne provjere

Kod tie-breaka porediti uređeni par vrijednosti.

Ne koristiti $O(N^2)$ za $N=200000$.

ZADATAK 037

Skoro palindrom

Cilj: Učenik koristi dva pokazivača i razmatra mali broj mogućih odluka kada se pojavi prvo neslaganje.

Za string S odredi može li postati palindrom brisanjem najviše jednog znaka.

Ulaz. Jedan string S sastavljen od malih slova.

Izlaz. Ispisati DA ili NE.

Ograničenja. $1 \leq |S| \leq 2 \cdot 10^5$.

Primjer

ULAZ

abca

IZLAZ

DA

Razumijevanje i postupak

Dok su krajnji znakovi jednaki, odluke nema. Kod prvog neslaganja jedine realne mogućnosti su izbrisati lijevi ili desni znak. Nakon toga preostali dio mora već biti palindrom.

1. Pomjeraj l i r dok su $S[l] == S[r]$.
2. Na prvom neslaganju provjeri segment $(l+1, r)$ i segment $(l, r-1)$.
3. Ako je barem jedan palindrom, odgovor je DA.

Složenost. Vrijeme $O(N)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
s = input().strip()

def pal(l, r):
    while l < r:
        if s[l] != s[r]:
            return False
        l += 1
        r -= 1
    return True

l, r = (0, len(s) - 1)
while l < r and s[l] == s[r]:
    l += 1
    r -= 1
print('DA' if l >= r or pal(l + 1, r) or pal(l, r - 1) else 'NE')
```


C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    auto pal = [&] (int l, int r) {
        while (l < r) {
            if (s[l++] != s[r--]) return false;
        }
        return true;
    };
    int l = 0, r = (int) s.size() - 1;
    while (l < r && s[l] == s[r]) {
        ++ l;
        -- r;
    }
    cout << (l >= r || pal(l + 1, r) || pal(l, r - 1) ? "DA" : "NE") << "\n";
}
```

Važne provjere

Ne pokušavati brisanje svakog znaka posebno - to bi bilo $O(N^2)$.

Brisanje nije obavezno; već palindrom je takođe DA.

ZADATAK 038

Poredak takmičara

Cilj: Sortira rezultate i dodjeljuje rang sa jednakim mjestima za jednake bodove.

N takmičara ima bodove. Veći broj bodova znači bolji plasman. Takmičari sa jednakim brojem bodova imaju isti rang, a sljedeći rang preskače odgovarajući broj mjesta (npr. 1,2,2,4). Za svaki takmičarov rezultat u originalnom redoslijedu ispiši rang.

Ulaz. Prvi red N. Drugi red N cijelih bodova.

Izlaz. N rangova u jednom redu.

Ograničenja. $1 \leq N \leq 200000$, $0 \leq \text{bodovi} \leq 10^9$.

Primjer

ULAZ

```
6
100 80 80 60 100 50
```

IZLAZ

```
1 3 3 5 1 6
```

Razumijevanje i postupak

Sortiramo parove (-bodovi, indeks). Nakon sortiranja, element na poziciji i dobija rang i+1 samo ako mu se rezultat razlikuje od prethodnog; inače zadržava prethodni rang.

1. Formiraj (bodovi, indeks).
2. Sortiraj opadajuće po bodovima.
3. Prođi sortirani niz i odredi rang.
4. Upiši rang na originalni indeks.

Zašto postupak daje tačan rezultat?

Sortirani redoslijed je tačno redoslijed plasmana. Kod jednakih bodova zadržavamo isti rang, a kod prvog manjeg rezultata rang postaje njegova 1-bazirana pozicija, čime se automatski preskaču mjesta.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
p = sorted([(-x, i) for i, x in enumerate(a)])
ans = [0] * n
rank = 0
prev = None
```

PYTHON 3 • potpuno rješenje / nastavak

```

for pos, (neg, idx) in enumerate(p, 1):
    x = -neg
    if x != prev:
        rank = pos
        prev = x
    ans[idx] = rank
print(*ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < long long > a(n);
    vector < pair < long long, int >> p;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        p.push_back({
            -a[i], i
        });
    }
    sort(p.begin(), p.end());
    vector < int > ans(n);
    long long prev = LLONG_MIN;
    int rank = 0;
    for (int pos = 1; pos <= n; pos++) {
        auto [neg, idx] = p[pos - 1];
        long long x = -neg;
        if (pos == 1 || x != prev) {
            rank = pos;
            prev = x;
        }
        ans[idx] = rank;
    }
    for (int x : ans) cout << x << ' ';
    cout << "\n";
}

```

Važne provjere

Ne koristiti gusti rang 1,2,2,3 - tekst traži standardni takmičarski rang 1,2,2,4.

Sačuvati originalne indekse.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 2000$	10
2	Svi bodovi različiti	8
3	Puna ograničenja	12

ZADATAK 039

Pakovanje poklona

Cilj: Primjenjuje pohlepno sparivanje nakon sortiranja i argumentira izbor.

Imamo N kutija nosivosti c_i i N poklona masa w_i . Svaka kutija može sadržati najviše jedan poklon, a poklon stane ako je $w_i \leq c_j$. Odredi najveći broj poklona koje možemo spakovati.

Ulaz. Prvi red N . Drugi red mase poklona. Treći red nosivosti kutija.

Izlaz. Najveći broj uspješno spakovanih poklona.

Ograničenja. $1 \leq N \leq 200000$, vrijednosti $\leq 10^9$.

Primjer

ULAZ

```
5
4 8 2 6 3
3 5 7 4 10
```

IZLAZ

```
5
```

Razumijevanje i postupak

Sortiramo i poklone i kutije. Najlakši još nesmjешten poklon treba pokušati staviti u najmanju kutiju koja ga može primiti. Korištenje veće kutije ne može pomoći lakšem poklonu više nego što može pomoći težem.

1. Sortiraj w i c .
2. $i=0$ pokazuje najlakši neupakovan poklon.
3. Za svaku kutiju rastuće: ako $c \geq w[i]$, spakuj ga i povećaj i .
4. Odgovor je i .

Zašto postupak daje tačan rezultat?

Ako najmanja raspoloživa kutija može primiti najlakši preostali poklon, njihovo sparivanje ne može smanjiti optimum: u bilo kojem optimalnom rješenju taj poklon je ili nesparen ili u istoj/većoj kutiji, pa zamjena čuva izvodljivost.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n = int(input())
w = sorted(map(int, input().split()))
c = sorted(map(int, input().split()))
i = 0
for cap in c:
    if i < n and w[i] <= cap:
        i += 1
print(i)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < long long > w(n), c(n);
    for (auto & x : w) cin >> x;
    for (auto & x : c) cin >> x;
    sort(w.begin(), w.end());
    sort(c.begin(), c.end());
    int i = 0;
    for (long long cap : c) if (i < n && w[i] <= cap) ++ i;
    cout << i << "\n";
}

```

Važne provjere

Ne sparivati po originalnim indeksima.

Uslov je $\leq$, ne $<$.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 1000$	8
2	Nizovi sortirani	8
3	Puna ograničenja	14

ZADATAK 040

Najkraći skupi segment

Cilj: Koristi dva pokazivača za najkraći segment sa sumom barem S kod pozitivnih elemenata.

Dat je niz pozitivnih cijelih brojeva. Odredi minimalnu dužinu uzastopnog segmenta čiji je zbir najmanje S . Ako takav segment ne postoji, ispiši 0.

Ulaz. N S , zatim N pozitivnih brojeva.

Izlaz. Minimalna dužina ili 0.

Ograničenja. $1 \leq N \leq 200000$, $1 \leq a_i, S \leq 10^{18}$.

Primjer

ULAZ

```
6 11
1 2 7 3 5 2
```

IZLAZ

```
3
```

Razumijevanje i postupak

Kod pozitivnih brojeva povećavanje desne granice samo povećava zbir. Kada je zbir dovoljan, pomjeramo lijevu granicu koliko god možemo da segment skratimo.

1. $l=0, cur=0, best=N+1$.
2. Za svaki r dodaj $a[r]$.
3. Dok $cur \geq S$: ažuriraj $best$, oduzmi $a[l]$, $l++$.

Zašto postupak daje tačan rezultat?

Za svaku desnu granicu petlja uklanja sve suvišne lijeve elemente i ostavlja najkraći segment koji završava u r . Globalni minimum preko svih r je traženi odgovor.

Složenost. Vrijeme $O(N)$, memorija $O(N)$ ili $O(1)$ pored ulaza.

PYTHON 3 • potpuno rješenje

```

N, S = map(int, input().split())
a = list(map(int, input().split()))
l = 0
cur = 0
best = N + 1
for r, x in enumerate(a):
    cur += x
    while cur >= S:
        best = min(best, r - l + 1)
        cur -= a[l]
        l += 1
print(0 if best == N + 1 else best)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    long long S;
    cin >> N >> S;
    vector< long long > a(N);
    for (auto & x : a) cin >> x;
    int l = 0, best = N + 1;
    long long cur = 0;
    for (int r = 0; r < N; r++) {
        cur += a[r];
        while (cur >= S) {
            best = min(best, r - l + 1);
            cur -= a[l++];
        }
    }
    cout << (best == N + 1 ? 0 : best) << "\n";
}

```

Važne provjere

Algoritam se oslanja na pozitivnost elemenata.

best se ažurira prije uklanjanja lijevog elementa.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 2000	5
2	Puna ograničenja	15

ZADATAK 041

Kružna smjena

Cilj: Modelira kružni niz i koristi dupliranje + klizni prozor.

N učenika sjedi u krugu. Svaki ima vrijednost a_i . Odredi najveći zbir K uzastopnih učenika u kružnom poretku (segment smije preći preko kraja niza).

Ulaz. N K , zatim N cijelih vrijednosti.

Izlaz. Najveći zbir kružnog segmenta dužine K .

Ograničenja. $1 \leq K \leq N \leq 200000$, $|a_i| \leq 10^9$.

Primjer

ULAZ

```
5 3
4 -2 7 1 5
```

IZLAZ

```
13
```

Razumijevanje i postupak

Krug pretvaramo u linearni niz tako što konceptualno ponovimo prvih $K-1$ elemenata. Tada svaki kružni segment dužine K postaje običan prozor u dupliranom nizu.

1. Napraviti $b = a + a$ ili indeksirati modulo N .
2. Izračunati prvi prozor dužine K .
3. Pomjeriti početak od 1 do $N-1$.

Zašto postupak daje tačan rezultat?

Postoji bijekcija između N mogućih kružnih segmenata dužine K i N prozora koji počinju na pozicijama $0..N-1$ u dupliranom nizu.

Složenost. Vrijeme $O(N)$, memorija $O(N)$ uz dupliranje.

PYTHON 3 • potpuno rješenje

```
N, K = map(int, input().split())
a = list(map(int, input().split()))
b = a + a
cur = sum(b[:K])
best = cur
for s in range(1, N):
    cur += b[s + K - 1] - b[s - 1]
    best = max(best, cur)
print(best)
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, K;
    cin >> N >> K;
    vector < long long > a(2 * N);
    for (int i = 0; i < N; i++) {
        cin >> a[i];
        a[i + N] = a[i];
    }
    long long cur = 0;
    for (int i = 0; i < K; i++) cur += a[i];
    long long best = cur;
    for (int s = 1; s < N; s++) {
        cur += a[s + K - 1] - a[s - 1];
        best = max(best, cur);
    }
    cout << best << "\n";
}

```

Važne provjere

best ne inicijalizovati nulom zbog negativnih vrijednosti.

Razmatrati tačno N početnih pozicija.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Segment ne prelazi kraj	8
2	$N \leq 2000$	7
3	Puna ograničenja	15

ZADATAK 042

Takmičarski parovi

Cilj: Pohlepno uparuje dvije sortirane grupe uz minimalni prag razlike.

Imamo N mlađih i M starijih učenika sa nivoima znanja. Par je dozvoljen ako je apsolutna razlika nivoa najviše D . Svaki učenik može biti u najviše jednom paru. Odredi najveći broj parova.

Ulaz. N M D , zatim N nivoa prve i M nivoa druge grupe.

Izlaz. Maksimalan broj parova.

Ograničenja. $N, M \leq 200000$, $D \leq 10^9$.

Primjer

ULAZ

```
5 4 2
1 4 7 10 12
2 6 9 13
```

IZLAZ

```
4
```

Razumijevanje i postupak

Nakon sortiranja gledamo najmanje neiskorištene vrijednosti. Ako su dovoljno blizu, uparimo ih. Ako je $A[i]$ previše mali, ni sa jednim kasnijim većim B neće biti bliži, pa i možemo odbaciti; simetrično za $B[j]$.

1. Sortiraj oba niza.
2. $i=j=0$.
3. Ako $|A[i]-B[j]| \leq D$, napravi par i povećaj oba.
4. Ako $A[i] < B[j]-D$, povećaj i ; inače j .

Zašto postupak daje tačan rezultat?

Kada najmanji element jedne strane više ne može biti uparen sa najmanjim druge zbog prevelike razlike u jednom smjeru, budući elementi druge strane su još nepovoljniji. Zato je odbacivanje sigurno.

Složenost. Vrijeme $O(N \log N + M \log M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

N, M, D = map(int, input().split())
a = sorted(map(int, input().split()))
b = sorted(map(int, input().split()))
i = j = ans = 0
while i < N and j < M:
    if abs(a[i] - b[j]) <= D:
        ans += 1
        i += 1
        j += 1
    elif a[i] < b[j]:
        i += 1
    else:
        j += 1
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M;
    long long D;
    cin >> N >> M >> D;
    vector < long long > a(N), b(M);
    for (auto & x : a) cin >> x;
    for (auto & x : b) cin >> x;
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());
    int i = 0, j = 0, ans = 0;
    while (i < N && j < M) {
        if (llabs(a[i] - b[j]) <= D) {
            ans++;
            i++;
            j++;
        }
        else if (a[i] < b[j]) i++;
        else j++;
    }
    cout << ans << "\n";
}

```

Važne provjere

Nije dovoljno porediti elemente istog indeksa.

Sortiranje je dio dokaza, ne samo optimizacija.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N, M \leq 1000$	8
2	$D=0$	6
3	Puna ograničenja	16

ZADATAK 043

Palindrom jednim brisanjem

Cilj: Primjenjuje dva pokazivača i razmatra najviše dvije lokalne grane.

Dat je string S . Dozvoljeno je obrisati najviše jedan znak. Odredi može li se dobiti palindrom. Ispiši DA ili NE.

Ulaz. Jedan string S od malih slova engleske abecede.

Izlaz. DA ili NE.

Ograničenja. $1 \leq |S| \leq 300000$.

Primjer

ULAZ

abca

IZLAZ

DA

Razumijevanje i postupak

Dok su krajnji znakovi jednaki, oba pokazivača pomjeramo prema sredini. Prvi konflikt je jedino mjesto na kojem brisanje može pomoći: brišemo ili lijevi ili desni konfliktni znak i provjerimo preostali interval.

1. Postavi $l=0$, $r=n-1$ i pomjeraj dok je $s[l]==s[r]$.
2. Ako se pokazivači sretnu, odgovor je DA.
3. Na prvom konfliktu provjeri palindrom intervala $[l+1,r]$ ili $[l,r-1]$.

Zašto postupak daje tačan rezultat?

Monotonost sortiranog niza garantuje da pomjeranjem odgovarajuće granice odbacujemo samo kandidate koji ne mogu dati bolji odgovor od upravo analiziranog.

Složenost. Vrijeme $O(N)$, memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
s = input().strip()

def pal(l, r):
    while l < r:
        if s[l] != s[r]:
            return False
        l += 1
        r -= 1
    return True
```

PYTHON 3 • potpuno rješenje / nastavak

```
l, r = (0, len(s) - 1)
while l < r and s[l] == s[r]:
    l += 1
    r -= 1
print('DA' if l >= r or pal(l + 1, r) or pal(l, r - 1) else 'NE')
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    auto pal = [&] (int l, int r) {
        while (l < r) {
            if (s[l] != s[r]) return false;
            l++;
            r--;
        }
        return true;
    };
    int l = 0, r = (int) s.size() - 1;
    while (l < r && s[l] == s[r]) {
        l++;
        r--;
    }
    cout << ((l >= r || pal(l + 1, r) || pal(l, r - 1)) ? "DA" : "NE") << "\n";
}
```

Važne provjere

Nakon prvog konflikta nema smisla brisati neki udaljeni znak.

Dozvoljeno je najviše jedno brisanje, ne tačno jedno.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$ S \leq 1000$	7
2	S je već palindrom	5
3	Puna ograničenja	13

ZADATAK 044

Najkraći segment sa K različitih

Cilj: Koristi klizni prozor i frekvencijsku mapu za uslov o broju različitih vrijednosti.

Dat je niz od N cijelih brojeva i broj K . Nađi dužinu najkraćeg uzastopnog segmenta koji sadrži najmanje K različitih vrijednosti. Ako takav segment ne postoji, ispiši -1.

Ulaz. N K , zatim N cijelih brojeva.

Izlaz. Minimalna dužina ili -1.

Ograničenja. $1 \leq N \leq 300000$, $1 \leq K \leq N$.

Primjer

ULAZ

```
8 3
1 2 1 3 4 2 3 1
```

IZLAZ

```
3
```

Razumijevanje i postupak

Desnu granicu širimo dok prozor ne sadrži najmanje K različitih vrijednosti. Tada lijevu granicu guramo naprijed koliko god možemo, bilježeći svaku valjanu dužinu. Svaki indeks ulazi i izlazi iz prozora najviše jednom.

1. freq je mapa vrijednost->broj u prozoru; distinct je broj pozitivnih frekvencija.
2. Za svaki r dodaj $a[r]$.
3. Dok $\text{distinct} \geq K$, ažuriraj odgovor pa izbaci $a[l]$ i povećaj l .

Zašto postupak daje tačan rezultat?

Svaki kandidat za minimalni/maksimalni prozor razmatra se u trenutku kada desna granica prvi put zadovolji uslov; zatim lijevu guramo dok uslov više ne važi.

Složenost. Vrijeme $O(N)$ očekivano, memorija $O(D)$, gdje je D broj različitih vrijednosti u prozoru.

PYTHON 3 • potpuno rješenje

```
from collections import defaultdict
n, k = map(int, input().split())
a = list(map(int, input().split()))
f = defaultdict(int)
l = 0
d = 0
ans = n + 1
```


PYTHON 3 • potpuno rješenje / nastavak

```

for r, x in enumerate(a):
    if f[x] == 0:
        d += 1
    f[x] += 1
    while d >= k:
        ans = min(ans, r - l + 1)
        y = a[l]
        f[y] -= 1
        if f[y] == 0:
            d -= 1
        l += 1
print(-1 if ans == n + 1 else ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, k;
    cin >> n >> k;
    vector < long long > a(n);
    for (auto & x : a) cin >> x;
    unordered_map < long long, int > f;
    int l = 0, d = 0, ans = n + 1;
    for (int r = 0; r < n; r++) {
        if (f[a[r]]++ == 0) d++;
        while (d >= k) {
            ans = min(ans, r - l + 1);
            if (--f[a[l]] == 0) d--;
            l++;
        }
    }
    cout << (ans == n + 1 ? - 1 : ans) << "\n";
}

```

Važne provjere

Ne povećavati distinct pri svakom dodavanju, samo kada frekvencija prelazi 0->1.

Pri brisanju smanjiti distinct tek kada frekvencija postane 0.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 2000	8
2	K ≤ 2	7
3	Puna ograničenja	15

ZADATAK 045

Par najbliži cilju

Cilj: Sortira niz i dva pokazivača koristi za približavanje ciljanom zbiru.

Dat je niz od N cijelih brojeva i cilj X . Izaberi dva različita elementa čiji je zbir po apsolutnoj razlici najbliži X . Ispiši minimalnu razliku $|a_i + a_j - X|$.

Ulaz. N X , zatim N cijelih brojeva.

Izlaz. Minimalna apsolutna razlika.

Ograničenja. $2 \leq N \leq 300000$, $|a_i|, |X| \leq 10^9$.

Primjer

ULAZ

```
6 10
1 4 7 12 -2 8
```

IZLAZ

```
0
```

Razumijevanje i postupak

Nakon sortiranja gledamo par (l, r) . Ako je zbir manji od X , jedini način da ga povećamo bez vraćanja na već odbačene parove jeste $l++$. Ako je veći, $r--$. Svaki korak daje kandidat za odgovor.

1. Sortiraj a .
2. $l=0, r=n-1, ans=veliko$.
3. Ažuriraj ans sa trenutnim parom.
4. Ako $suma < X$ pomjeri l , inače ako $suma > X$ pomjeri r ; za jednakost odgovor je 0.

Zašto postupak daje tačan rezultat?

Monotonost sortiranog niza garantuje da pomjeranjem odgovarajuće granice odbacujemo samo kandidate koji ne mogu dati bolji odgovor od upravo analiziranog.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$ ili $O(1)$ dodatno nakon sortiranja.

PYTHON 3 • potpuno rješenje

```
n, x = map(int, input().split())
a = sorted(map(int, input().split()))
l, r = (0, n - 1)
ans = 10 ** 30
while l < r:
    s = a[l] + a[r]
    ans = min(ans, abs(s - x))
```

PYTHON 3 • potpuno rješenje / nastavak

```

if s < x:
    l += 1
elif s > x:
    r -= 1
else:
    break
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    long long x;
    cin >> n >> x;
    vector < long long > a(n);
    for (auto & v : a) cin >> v;
    sort(a.begin(), a.end());
    int l = 0, r = n - 1;
    long long ans = LLONG_MAX;
    while (l < r) {
        long long s = a[l] + a[r];
        ans = min(ans, labs(s - x));
        if (s < x) l++;
        else if (s > x) r--;
        else break;
    }
    cout << ans << "\n";
}

```

Važne provjere

Elementi moraju biti različite pozicije; uslov $l < r$ to garantuje.

Koristiti 64-bitni tip za zbir.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 2000$	8
2	Svi brojevi nenegativni	5
3	Puna ograničenja	12

17. Intervali, prefiksi i pretraga odgovora

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
46	Dvije sijalice
47	Najveća gužva u bazenu
48	Upiti o bodovima
49	Pano sa bodovima
50	Raspored učionice
51	Prvi dovoljno veliki rezultat
52	Minimalni kapacitet dostave
53	Spajanje vremenskih intervala
54	Mnogo povećanja intervala
55	Zbir pravougaonika u matrici
56	Sažmi velike koordinate
57	Zbir pravougaonika
58	Raspored radionica
59	Brzi zbir intervala
60	Najviše radionica
61	Fabrika bedževa
62	Minimalni kapacitet kamiona

ZADATAK 046

Dvije sijalice

Cilj: Izračunati dužinu unije i presjeka dva poluotvorena intervala.

Plava sijalica svijetli od trenutka P1 do P2, a žuta od Z1 do Z2. Intervali su oblika [početak, kraj], pa trajanje iznosi kraj-početak. Odredi koliko vremena svijetli samo plava, koliko obje istovremeno i koliko samo žuta.

Ulaz. U jednom redu P1 P2 Z1 Z2.

Izlaz. Ispisati tri broja: samo_plava obje samo_žuta.

Ograničenja. $0 \leq P1 < P2 \leq 10^9$, $0 \leq Z1 < Z2 \leq 10^9$.

Primjer

ULAZ

1 7 4 10

IZLAZ

3 3 3

Razumijevanje i postupak

Presjek dva intervala ima lijevu granicu $\max(P1, Z1)$ i desnu $\min(P2, Z2)$. Njegova dužina je $\max(0, \text{desna-lijeva})$. Samo plava je dužina plavog intervala minus presjek, a analogno za žutu.

1. izračunati $\text{overlap} = \max(0, \min(P2, Z2) - \max(P1, Z1))$.

2. $\text{samo_plava} = (P2 - P1) - \text{overlap}$.

3. $\text{samo_žuta} = (Z2 - Z1) - \text{overlap}$.

Složenost. $O(1)$.

PYTHON 3 • potpuno rješenje

```
p1, p2, z1, z2 = map(int, input().split())
preklop = max(0, min(p2, z2) - max(p1, z1))
print(p2 - p1 - preklop, preklop, z2 - z1 - preklop)
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    long long p1, p2, z1, z2;
    cin >> p1 >> p2 >> z1 >> z2;
    long long pre = max(0LL, min(p2, z2) - max(p1, z1));
    cout << p2 - p1 - pre << " " << pre << " " << z2 - z1 - pre << "\n";
}
```

ZADATAK 047

Najveća gužva u bazenu

Cilj: Pretvoriti intervale prisustva u događaje i pronaći maksimum aktivnih intervala.

Za svakog od N posjetilaca poznato je vrijeme dolaska A_i i odlaska B_i . Posjetilac je prisutan u intervalu $[A_i, B_i)$, tj. onaj koji odlazi u trenutku t više nije prisutan kada drugi dođe u t . Odredi najveći broj istovremeno prisutnih.

Ulaz. U prvom redu N , u narednih N redova $A_i B_i$.

Izlaz. Ispisati maksimalan broj prisutnih.

Ograničenja. $1 \leq N \leq 200000$, $0 \leq A_i < B_i \leq 10^9$.

Primjer

ULAZ

```
5
1 5
2 6
4 8
5 7
9 10
```

IZLAZ

```
3
```

Razumijevanje i postupak

Za svaki dolazak pravimo događaj $+1$, a za svaki odlazak -1 . Događaje sortiramo po vremenu; za isti trenutak -1 mora biti obrađen prije $+1$ zbog poluotvorenog intervala. U paru (vrijeme, delta), standardno sortiranje stavlja -1 prije $+1$.

1. Dodati $(A_i, +1)$ i $(B_i, -1)$.
2. Sortirati događaje.
3. Prolazom akumulirati trenutno prisutne i čuvati maksimum.

Složenost. $O(N \log N)$ vrijeme, $O(N)$ memorija.

PYTHON 3 • potpuno rješenje

```

n = int(input())
e = []
for _ in range(n):
    a, b = map(int, input().split())
    e.append((a, 1))
    e.append((b, -1))
e.sort()
tren = ans = 0
for _, d in e:
    tren += d
    ans = max(ans, tren)
print(ans)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < pair < long long, int >> e;
    e.reserve(2 * n);
    for (int i = 0; i < n; i++) {
        long long a, b;
        cin >> a >> b;
        e.push_back({
            a, 1
        });
        e.push_back({
            b, - 1
        });
    }
    sort(e.begin(), e.end());
    int cur = 0, ans = 0;
    for (auto [t, d] : e) {
        cur += d;
        ans = max(ans, cur);
    }
    cout << ans << "\n";
}

```


ZADATAK 048

Upiti o bodovima

Cilj: Odgovarati na veliki broj upita o zbiru intervala u $O(1)$.

Tokom N dana učenik osvaja a_i bodova. Zatim dobija Q upita: za svaki interval $[L,R]$ treba izračunati koliko je bodova osvojio od L -tog do R -tog dana uključivo.

Ulaz. U prvom redu N i Q , u drugom N brojeva a_i , zatim Q redova L R .

Izlaz. Za svaki upit ispisati zbir na intervalu.

Ograničenja. $1 \leq N, Q \leq 200000$, $|a_i| \leq 10^9$.

Primjer

ULAZ

```
5 3
3 1 4 1 5
1 3
2 5
4 4
```

IZLAZ

```
8
11
1
```

Razumijevanje i postupak

Prefiks $p[i]$ je zbir prvih i elemenata. Tada je zbir $L..R$ jednak $p[R]-p[L-1]$. Izgradnja traje $O(N)$, a svaki upit $O(1)$, što je mnogo brže od ponovnog sabiranja.

1. Napraviti $p[0]=0$.
2. Za $i=1..N$: $p[i]=p[i-1]+a[i-1]$.
3. Za upit L,R ispisati $p[R]-p[L-1]$.

Složenost. $O(N+Q)$ vrijeme i $O(N)$ memorija.

PYTHON 3 • potpuno rješenje

```
n, q = map(int, input().split())
a = list(map(int, input().split()))
p = [0]
for x in a:
    p.append(p[-1] + x)
for _ in range(q):
    l, r = map(int, input().split())
    print(p[r] - p[l - 1])
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n, q;
    cin >> n >> q;
    vector< long long > p(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        p[i] = p[i - 1] + x;
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << p[r] - p[l - 1] << "\n";
    }
}
```

ZADATAK 049

Pano sa bodovima

Cilj: Proširiti prefiksne sume sa niza na matricu.

Na panou $R \times C$ svaki element predstavlja broj bodova u jednoj ćeliji. Za Q pravougaonih upita $(r1, c1, r2, c2)$ treba izračunati zbir svih vrijednosti unutar pravougaonika, uključujući granice.

Ulaz. U prvom redu R C Q . Slijedi R redova matrice. Zatim Q upita.

Izlaz. Za svaki upit ispisati zbir pravougaonika.

Ograničenja. $1 \leq R, C \leq 1000, 1 \leq Q \leq 200000, |a_{ij}| \leq 10^6$.

Primjer

ULAZ

```
3 4 2
1 2 3 4
5 6 7 8
9 1 2 3
1 2 2 4
2 1 3 2
```

IZLAZ

```
30
21
```

Razumijevanje i postupak

2D prefiks $P[i][j]$ je zbir pravougaonika od $(1,1)$ do (i,j) . Pri izračunu pravougaonika koristimo princip uključivanja-isključivanja: $P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1]$.

1. Izgraditi 2D prefiks.
2. Za svaki upit primijeniti formulu uključivanja-isključivanja.

Složenost. $O(R \cdot C + Q)$ vrijeme i $O(R \cdot C)$ memorija.

PYTHON 3 • potpuno rješenje

```

r, c, q = map(int, input().split())
p = [[0] * (c + 1) for _ in range(r + 1)]
for i in range(1, r + 1):
    row = list(map(int, input().split()))
    for j in range(1, c + 1):
        p[i][j] = row[j - 1] + p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1]
for _ in range(q):
    r1, c1, r2, c2 = map(int, input().split())
    ans = p[r2][c2] - p[r1 - 1][c2] - p[r2][c1 - 1] + p[r1 - 1][c1 - 1]
    print(ans)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
using namespace std;
int main() {
    int R, C, Q;
    cin >> R >> C >> Q;
    vector < vector < long long >> p(R + 1, vector < long long > (C + 1, 0));
    for (int i = 1; i <= R; i++) for (int j = 1; j <= C; j++) {
        long long x;
        cin >> x;
        p[i][j] = x + p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1];
    }
    while (Q--) {
        int r1, c1, r2, c2;
        cin >> r1 >> c1 >> r2 >> c2;
        cout << p[r2][c2] - p[r1 - 1][c2] - p[r2][c1 - 1] + p[r1 - 1][c1 - 1]
            << "\n";
    }
}

```

ZADATAK 050

Raspored učionice

Cilj: Izabrati maksimalan broj međusobno nepreklapajućih termina.

Za N radionica poznati su početak s_i i kraj e_i . U jednoj učionici može se održati samo jedna radionica u isto vrijeme. Radionica koja završava u trenutku t ne preklapa se s onom koja počinje u t . Odredi najveći broj radionica koje se mogu održati u jednoj učionici.

Ulaz. U prvom redu N , zatim N redova $s_i e_i$.

Izlaz. Ispisati maksimalan broj radionica.

Ograničenja. $1 \leq N \leq 200000$, $0 \leq s_i < e_i \leq 10^9$.

Primjer

ULAZ

```
5
1 3
2 5
3 4
4 7
6 8
```

IZLAZ

```
3
```

Razumijevanje i postupak

Optimalna greedy strategija je uvijek izabrati među dostupnim terminima onaj koji najranije završava. Time ostavljamo najviše prostora za buduće radionice. Sortiramo po vremenu završetka i uzimamo termin ako počinje nakon ili tačno kada je prethodni završio.

1. Sortirati intervale po e_i rastuće.
2. Voditi `last_end`.
3. Ako $s_i \geq \text{last_end}$, izabrati radionicu i postaviti `last_end = e_i`.

Složenost. $O(N \log N)$ vrijeme, $O(N)$ memorija.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = []
for _ in range(n):
    s, e = map(int, input().split())
    a.append((e, s))
a.sort()
```

PYTHON 3 • potpuno rješenje / nastavak

```

last = -10 ** 30
ans = 0
for e, s in a:
    if s >= last:
        ans += 1
        last = e
print(ans)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < pair < long long, long long >> a;
    for (int i = 0; i < n; i++) {
        long long s, e;
        cin >> s >> e;
        a.push_back({
            e, s
        });
    }
    sort(a.begin(), a.end());
    long long last = - (1LL << 60);
    int ans = 0;
    for (auto [e, s] : a) if (s >= last) {
        ans++;
        last = e;
    }
    cout << ans << "\n";
}

```

ZADATAK 051

Prvi dovoljno veliki rezultat

Cilj: Učenik primjenjuje binarno pretraživanje na sortiranom nizu i nalazi prvu poziciju koja zadovoljava uslov.

Dat je neopadajuće sortiran niz A i vrijednost X . Nađi prvu poziciju na kojoj je $A[i] \geq X$. Pozicije su numerisane od 1. Ako takva pozicija ne postoji, ispiši -1.

Ulaz. Prvi red: N X . Drugi red: N cijelih brojeva.

Izlaz. Ispisati traženu 1-baziranu poziciju ili -1.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
7 8
1 3 5 8 8 10 14
```

IZLAZ

```
4
```

Razumijevanje i postupak

Ne tražimo samo da li X postoji. Tražimo granicu: lijevo su vrijednosti $< X$, a od odgovora nadalje vrijednosti su $\geq X$. To je tipičan `lower_bound` obrazac.

1. Postavi $l=0$, $r=N$.
2. Dok $l < r$, uzmi $m=(l+r)//2$.
3. Ako $A[m] \geq X$, pomjeri $r=m$; inače $l=m+1$.
4. Ako $l==N$, nema odgovora; inače odgovor je $l+1$.

Složenost. Vrijeme $O(\log N)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
N, X = map(int, input().split())
A = list(map(int, input().split()))
l, r = (0, N)
while l < r:
    m = (l + r) // 2
    if A[m] >= X:
        r = m
    else:
        l = m + 1
print(-1 if l == N else l + 1)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    long long X;
    cin >> N >> X;
    vector < long long > A(N);
    for (auto & v : A) cin >> v;
    int l = 0, r = N;
    while (l < r) {
        int m = (l + r) / 2;
        if (A [m] >= X) r = m;
        else l = m + 1;
    }
    cout << (l == N ? - 1 : l + 1) << "\n";
}
```

Važne provjere

Binarno pretraživanje zahtijeva monotoni poredak.

Granice [l,r) smanjuju rizik od beskonačne petlje.

ZADATAK 052

Minimalni kapacitet dostave

Cilj: Učenik prepoznaje binarno pretraživanje po odgovoru i gradi monotoni test izvodljivosti.

Paketi moraju biti poslani zadanim redoslijedom u najviše D dana. Svaki dan kamion može ponijeti uzastopne pakete ukupne mase najviše C . Nađi najmanji mogući kapacitet C .

Ulaz. Prvi red: N D . Drugi red: mase N paketa.

Izlaz. Ispisati minimalni kapacitet.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$, $1 \leq D \leq N$, mase $\leq 10^9$.

Primjer

ULAZ

```
5 3
4 2 7 3 5
```

IZLAZ

```
8
```

Razumijevanje i postupak

Ako kapacitet C radi, svaki veći kapacitet takođe radi. To je monotono svojstvo. Donja granica je najteži paket, gornja zbir svih masa.

1. Napiši funkciju `days_needed(C)` koja pohlepno puni svaki dan.
2. Binarno pretražuj C između `max(masa)` i `sum(masa)`.
3. Ako je broj dana $\leq D$, pokušaj manji C ; inače veći.

Složenost. Vrijeme $O(N \log(\sum A))$; memorija $O(1)$ pored ulaza.

PYTHON 3 • potpuno rješenje

```
N, D = map(int, input().split())
w = list(map(int, input().split()))
```

```
def ok(C):
    days, cur = (1, 0)
    for x in w:
        if cur + x > C:
            days += 1
            cur = 0
        cur += x
    return days <= D
l, r = (max(w), sum(w))
while l < r:
    m = (l + r) // 2
```

PYTHON 3 • potpuno rješenje / nastavak

```

    if ok(m):
        r = m
    else:
        l = m + 1
print(l)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, D;
    cin >> N >> D;
    vector < long long > w(N);
    long long l = 0, r = 0;
    for (auto & x : w) {
        cin >> x;
        l = max(l, x);
        r += x;
    }
    auto ok = [&] (long long C) {
        int days = 1;
        long long cur = 0;
        for (long long x : w) {
            if (cur + x > C) {
                ++ days;
                cur = 0;
            }
            cur += x;
        }
        return days <= D;
    }
    ;
    while (l < r) {
        long long m = l + (r - l) / 2;
        if (ok(m)) r = m;
        else l = m + 1;
    }
    cout << l << "\n";
}

```

Važne provjere

Test izvodljivosti mora biti $O(N)$, ne još jedno binarno pretraživanje.

Donja granica mora biti barem najveća pojedinačna masa.

ZADATAK 053

Spajanje vremenskih intervala

Cilj: Učenik sortira intervale i računa dužinu unije bez dvostrukog brojanja preklapanja.

Dato je N zatvorenih vremenskih intervala $[l, r]$ na realnoj osi. Izračunaj ukupnu dužinu vremena pokrivenu barem jednim intervalom.

Ulaz. Prvi red N . Zatim N redova $l\ r$, gdje je $l \leq r$.

Izlaz. Ispisati ukupnu dužinu unije intervala.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$, koordinate su cijeli brojevi.

Primjer

ULAZ

```
4
1 4
3 7
10 12
11 15
```

IZLAZ

```
11
```

Razumijevanje i postupak

Sortiranjem po lijevoj granici svi intervali koji mogu produžiti trenutni spojeni blok dolaze uzastopno. Ako novi interval počinje prije ili na kraju trenutnog, spajamo ga; inače zatvaramo blok.

1. Sortiraj intervale po l pa r .
2. Drži trenutni $[L, R]$.
3. Ako $l \leq R$, postavi $R = \max(R, r)$.
4. Inače dodaj $R - L$ rezultatu i započni novi blok.
5. Na kraju dodaj posljednji blok.

Složenost. Vrijeme $O(N \log N)$; memorija $O(N)$ za intervale.

PYTHON 3 • potpuno rješenje

```
N = int(input())
seg = [tuple(map(int, input().split())) for _ in range(N)]
seg.sort()
L, R = seg[0]
ans = 0
for l, r in seg[1:]:
    if l <= R:
```

PYTHON 3 • potpuno rješenje / nastavak

```

    R = max(R, r)
else:
    ans += R - L
    L, R = (l, r)
ans += R - L
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    cin >> N;
    vector < pair < long long, long long >> s(N);
    for (auto & p : s) cin >> p.first >> p.second;
    sort(s.begin(), s.end());
    long long L = s [0].first, R = s [0].second, ans = 0;
    for (int i = 1; i < N; ++ i) {
        auto [l, r] = s [i];
        if (l <= R) R = max(R, r);
        else {
            ans += R - L;
            L = l;
            R = r;
        }
    }
    ans += R - L;
    cout << ans << "\n";
}

```

Važne provjere

Dužina realnog intervala $[L,R]$ je $R-L$, ne $R-L+1$.

Ne zaboraviti posljednji otvoreni blok.

ZADATAK 054

Mnogo povećanja intervala

Cilj: Učenik koristi diferencni niz za veliki broj intervalnih ažuriranja.

Dat je početni niz A i Q operacija. Svaka operacija $l\ r\ x$ dodaje x svim elementima od l do r . Nakon svih operacija ispiši konačni niz.

Ulaz. Prvi red $N\ Q$. Drugi red N brojeva. Zatim Q redova $l\ r\ x$.

Izlaz. Ispisati konačni niz.

Ograničenja. $1 \leq N, Q \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
5 3
1 2 3 4 5
2 4 10
1 2 -1
5 5 7
```

IZLAZ

```
0 11 13 14 12
```

Razumijevanje i postupak

Umjesto da svako ažuriranje dira svaki element intervala, u diferencnom nizu označimo početak promjene sa $+x$ i mjesto odmah poslije intervala sa $-x$. Jedan završni prefiks pretvara razlike u stvarne dodatke.

1. Napravi diff veličine $N+1$.
2. Za update l, r, x : $\text{diff}[l-1] += x$; ako $r < N$, $\text{diff}[r] -= x$.
3. Prefiksno saberi diff i dodaj tekući zbir $A[i]$.

Složenost. Vrijeme $O(N+Q)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

N, Q = map(int, input().split())
A = list(map(int, input().split()))
d = [0] * (N + 1)
for _ in range(Q):
    l, r, x = map(int, input().split())
    d[l - 1] += x
    d[r] -= x
cur = 0
for i in range(N):
    cur += d[i]
    A[i] += cur
print(*A)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, Q;
    cin >> N >> Q;
    vector< long long > A(N), d(N + 1);
    for (auto & x : A) cin >> x;
    while (Q--) {
        int l, r;
        long long x;
        cin >> l >> r >> x;
        d[l - 1] += x;
        d[r] -= x;
    }
    long long cur = 0;
    for (int i = 0; i < N; ++ i) {
        cur += d[i];
        A[i] += cur;
        if (i) cout << ' ';
        cout << A[i];
    }
    cout << "\n";
}

```

Važne provjere

Diferencni niz je posebno koristan kada odgovori trebaju tek nakon svih ažuriranja.

Indeksi l,r iz ulaza su 1-bazirani.

ZADATAK 055

Zbir pravougaonika u matrici

Cilj: Učenik koristi 2D prefiksne sume i princip uključenja-isključenja.

Za matricu $R \times C$ odgovori na Q upita: koliki je zbir elemenata u pravougaoniku od $(r1, c1)$ do $(r2, c2)$, uključivo.

Ulaz. Prvi red $R \ C \ Q$. Zatim matrica. Nakon toga Q redova $r1 \ c1 \ r2 \ c2$.

Izlaz. Za svaki upit ispisati zbir.

Ograničenja. $1 \leq R, C \leq 1000$, $1 \leq Q \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
3 3 2
1 2 3
4 5 6
7 8 9
1 1 2 2
2 2 3 3
```

IZLAZ

```
12
28
```

Razumijevanje i postupak

$P[r][c]$ čuva zbir pravougaonika od $(1,1)$ do (r,c) . Željeni pravougaonik dobijamo kao veliki prefiks minus gornji pojas minus lijevi pojas plus dio koji smo oduzeli dvaput.

1. Izgradi P sa dodatnim nultim redom i kolonom.
2. $P[r][c] = A + P[r-1][c] + P[r][c-1] - P[r-1][c-1]$.
3. $\text{Upit} = P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1]$.

Složenost. Izgradnja $O(R \cdot C)$; svaki upit $O(1)$; memorija $O(R \cdot C)$.

PYTHON 3 • potpuno rješenje

```
R, C, Q = map(int, input().split())
P = [[0] * (C + 1) for _ in range(R + 1)]
for r in range(1, R + 1):
    row = list(map(int, input().split()))
    for c in range(1, C + 1):
        P[r][c] = row[c - 1] + P[r - 1][c] + P[r][c - 1] - P[r - 1][c - 1]
for _ in range(Q):
    r1, c1, r2, c2 = map(int, input().split())
    print(P[r2][c2] - P[r1 - 1][c2] - P[r2][c1 - 1] + P[r1 - 1][c1 - 1])
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int R, C, Q;
    cin >> R >> C >> Q;
    vector < vector < long long >> P(R + 1, vector < long long > (C + 1));
    for (int r = 1; r <= R; ++ r) for (int c = 1; c <= C; ++ c) {
        long long x;
        cin >> x;
        P [r] [c] = x + P [r - 1] [c] + P [r] [c - 1] - P [r - 1] [c - 1];
    }
    while (Q--) {
        int r1, c1, r2, c2;
        cin >> r1 >> c1 >> r2 >> c2;
        cout << P [r2] [c2] - P [r1 - 1] [c2] - P [r2] [c1 - 1] + P [r1 -
            1] [c1 - 1]
            << "\n";
    }
}

```

Važne provjere

Zadnji presjek mora se dodati jer je oduzet dvaput.

Nulti rub prefiksne matrice znatno pojednostavljuje formule.

ZADATAK 056

Sažmi velike koordinate

Cilj: Učenik primjenjuje koordinatnu kompresiju bez gubitka poretka i jednakosti.

Dat je niz cijelih brojeva koji mogu biti veoma veliki ili negativni. Zamijeni svaku vrijednost njenim rangom među različitim vrijednostima niza, počevši od 1.

Ulaz. Prvi red N. Drugi red N cijelih brojeva.

Izlaz. Ispisati N kompresovanih rangova.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$, $|A[i]| \leq 10^{18}$.

Primjer

ULAZ

```
5
100 -5 100 7 20
```

IZLAZ

```
4 1 4 2 3
```

Razumijevanje i postupak

Kompresija ne čuva razlike između vrijednosti, ali čuva njihov poredak i jednakost. Sortiramo jedinstvene vrijednosti i svakoj dodijelimo indeks.

1. `unique=sort(set(A))`.
2. Napravi mapu vrijednost -> pozicija+1.
3. Svaki `A[i]` zamijeni njegovim rangom.

Složenost. Vrijeme $O(N \log N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N = int(input())
A = list(map(int, input().split()))
vals = sorted(set(A))
rank = {x: i + 1 for i, x in enumerate(vals)}
print(*(rank[x] for x in A))
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    cin >> N;
    vector < long long > A(N), v;
```

C++17 • potpuno rješenje / nastavak

```
for (auto & x : A) cin >> x;
v = A;
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
for (int i = 0; i < N; ++ i) {
    if (i) cout << ' ';
    cout << lower_bound(v.begin(), v.end(), A [i]) - v.begin() + 1;
}
cout << "\n";
}
```

Važne provjere

Jednake originalne vrijednosti moraju dobiti isti rang.

Kompresija nije normalizacija na interval [0,1]; važan je samo redoslijed.

ZADATAK 057

Zbir pravougaonika

Cilj: Koristi 2D prefiksne sume za mnogo pravougaonih upita.

Data je matrica bodova $R \times C$ i Q upita. Svaki upit daje $r1, c1, r2, c2$ (1-bazirano) i traži zbir svih elemenata unutar tog pravougaonika.

Ulaz. $R \ C \ Q$, zatim R redova matrice, zatim Q upita.

Izlaz. Za svaki upit ispisati zbir.

Ograničenja. $1 \leq R, C \leq 700$, $Q \leq 200000$, $|a| \leq 10^6$.

Primjer

ULAZ

```
3 4 2
1 2 3 4
5 6 7 8
9 10 11 12
1 2 2 4
2 1 3 2
```

IZLAZ

```
30
30
```

Razumijevanje i postupak

$pref[r][c]$ čuva zbir pravougaonika od $(1,1)$ do (r,c) . Zbir proizvoljnog pravougaonika dobija se uključenjem-isključenjem četiri prefiksa.

1. Izgradi $pref$ sa dodatnim nultim redom i kolonom.
2. Za upit koristi $P[r2][c2]-P[r1-1][c2]-P[r2][c1-1]+P[r1-1][c1-1]$.

Zašto postupak daje tačan rezultat?

Svaki element se u prefiksnoj matrici uključi tačno prema definiciji. Formula uklanja područja iznad i lijevo od upita, a njihov preklop vraća jednom.

Složenost. Predobrada $O(RC)$, svaki upit $O(1)$, memorija $O(RC)$.

PYTHON 3 • potpuno rješenje

```

R, C, Q = map(int, input().split())
p = [[0] * (C + 1) for _ in range(R + 1)]
for r in range(1, R + 1):
    row = list(map(int, input().split()))
    for c, x in enumerate(row, 1):
        p[r][c] = x + p[r - 1][c] + p[r][c - 1] - p[r - 1][c - 1]
for _ in range(Q):
    r1, c1, r2, c2 = map(int, input().split())
    print(p[r2][c2] - p[r1 - 1][c2] - p[r2][c1 - 1] + p[r1 - 1][c1 - 1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int R, C, Q;
    cin >> R >> C >> Q;
    vector < vector < long long >> p(R + 1, vector < long long > (C + 1));
    for (int r = 1; r <= R; r++) for (int c = 1; c <= C; c++) {
        long long x;
        cin >> x;
        p[r][c] = x + p[r - 1][c] + p[r][c - 1] - p[r - 1][c - 1];
    }
    while (Q--) {
        int r1, c1, r2, c2;
        cin >> r1 >> c1 >> r2 >> c2;
        cout << p[r2][c2] - p[r1 - 1][c2] - p[r2][c1 - 1] + p[r1 - 1][c1 - 1]
              << "\n";
    }
}

```

Važne provjere

Paziti na 1-bazirane koordinate i dodatnu nultu granicu.

Koristiti 64-bitni zbir.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	R,C<=50 i Q<=50	5
2	Q<=1000	5
3	Puna ograničenja	10

ZADATAK 058

Raspored radionica

Cilj: Rješava težinski interval scheduling pomoću sortiranja, binarnog pretraživanja i DP-a.

Svaka od N radionica ima početak s_i , kraj e_i i vrijednost v_i . Ne možemo pohađati preklapajuće radionice. Ako jedna završava u trenutku kada druga počinje, mogu se obje pohađati. Odredi maksimalnu ukupnu vrijednost.

Ulaz. N , zatim N redova s e v .

Izlaz. Maksimalna vrijednost.

Ograničenja. $N \leq 200000$, vremena i vrijednosti $\leq 1e9$.

Primjer

ULAZ

```
5
1 3 5
2 5 6
4 6 5
6 7 4
5 8 11
```

IZLAZ

```
17
```

Razumijevanje i postupak

Sortiramo radionice po vremenu završetka. Za i -tu radionicu binarnim pretraživanjem nađemo posljednju radionicu koja završava $\leq s_i$. DP odlučuje: preskoči i ili uzmi i + najbolji kompatibilni prefiks.

1. Sortiraj po e .
2. Napravi niz završetaka $ends$.
3. Za svaki i nađi $j = \text{bisect_right}(ends, s_i, 0, i) - 1$.
4. $dp[i+1] = \max(dp[i], v_i + dp[j+1])$.

Zašto postupak daje tačan rezultat?

U optimalnom rješenju za prvih i radionica posljednja sortirana radionica ili nije izabrana, ili jeste. Ako jeste, sve ostale izabrane moraju završiti prije njenog početka, a najbolja vrijednost tog kompatibilnog prefiksa je već u DP-u.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

from bisect import bisect_right
n = int(input())
a = []
for _ in range(n):
    s, e, v = map(int, input().split())
    a.append((e, s, v))
a.sort()
ends = [x[0] for x in a]
dp = [0] * (n + 1)
for i, (e, s, v) in enumerate(a):
    j = bisect_right(ends, s, 0, i) - 1
    dp[i + 1] = max(dp[i], v + dp[j + 1])
print(dp[n])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < tuple < long long, long long, long long >> a;
    for (int i = 0; i < n; i++) {
        long long s, e, v;
        cin >> s >> e >> v;
        a.push_back({
            e, s, v
        });
    }
    sort(a.begin(), a.end());
    vector < long long > ends(n), dp(n + 1);
    for (int i = 0; i < n; i++) ends [i] = get < 0 > (a [i]);
    for (int i = 0; i < n; i++) {
        auto [e, s, v] = a [i];
        int j = upper_bound(ends.begin(), ends.begin() + i, s) -
            ends.begin() - 1;
        dp [i + 1] = max(dp [i], v + dp [j + 1]);
    }
    cout << dp [n] << "\n";
}

```

Važne provjere

Binarna pretraga je po krajevima već sortiranog prefiksa.

Kompatibilnost je $e \leq s$, ne $e < s$.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 25$	5
2	$N \leq 3000$	5
3	Sve vrijednosti=1	5
4	Puna ograničenja	10

ZADATAK 059

Brzi zbir intervala

Cilj: Predobrađuje prefiksne sume i odgovara na mnogo intervalnih upita.

Dat je niz od N cijelih brojeva i Q upita. Svaki upit daje 1-bazne indekse l i r . Za svaki upit ispiši zbir $a_l + \dots + a_r$.

Ulaz. N Q , zatim N brojeva, zatim Q redova l r .

Izlaz. Po jedan zbir za svaki upit.

Ograničenja. $1 \leq N, Q \leq 300000$, $|a_i| \leq 10^9$.

Primjer

ULAZ

```
5 3
2 -1 4 7 3
1 3
2 5
4 4
```

IZLAZ

```
5
13
7
```

Razumijevanje i postupak

Jedan upit se može sabirati $O(N)$, ali Q može biti velik. Prefiks $s[i] =$ zbir prvih i elemenata omogućava odgovor $s[r] - s[l-1]$ u $O(1)$.

1. Napravi s dužine $N+1$ sa $s[0]=0$.
2. $s[i]=s[i-1]+a[i-1]$.
3. Za svaki 1-bazni l, r ispiši $s[r]-s[l-1]$.

Zašto postupak daje tačan rezultat?

Svaki element intervala $[l, r]$ pojavljuje se u $s[r+1]$, a svi elementi prije l poništavaju se oduzimanjem $s[l]$.

Složenost. Predobrada $O(N)$, svaki upit $O(1)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n, q = map(int, input().split())
a = list(map(int, input().split()))
s = [0]
for x in a:
    s.append(s[-1] + x)
for _ in range(q):
    l, r = map(int, input().split())
    print(s[r] - s[l - 1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector< long long > s(n + 1);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        s[i] = s[i - 1] + x;
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << s[r] - s[l - 1] << "\n";
    }
}

```

Važne provjere

U C++ zbir mora biti long long.

Ne koristiti s[r]-s[l] za 1-bazne granice.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N, Q ≤ 1000	5
2	Svi upiti počinju sa l=1	5
3	Puna ograničenja	10

ZADATAK 060

Najviše radionica

Cilj: Primjenjuje i dokazuje klasični greedy izbor najranijeg završetka.

Dato je N radionica sa vremenima početka s_i i završetka e_i . Jedna osoba može prisustvovati radionici ako se ne preklapa sa prethodno izabranom; radionica koja počinje tačno kada druga završava je dozvoljena. Odredi najveći broj radionica.

Ulaz. N , zatim N redova s e .

Izlaz. Maksimalan broj izabranih radionica.

Ograničenja. $1 \leq N \leq 300000$, $0 \leq s_i < e_i \leq 10^9$.

Primjer

ULAZ

```
6
1 4
3 5
0 6
5 7
8 9
5 9
```

IZLAZ

```
3
```

Razumijevanje i postupak

Sortiranje po završetku ostavlja što više prostora za budućnost. Uzimamo prvi interval, zatim svaki naredni čiji je početak $\geq$ kraj posljednjeg izabranog.

1. Sortiraj parove po e rastuće.
2. $last = -\infty$, $ans = 0$.
3. Ako $s \geq last$, izaberi interval: $ans++$, $last = e$.

Zašto postupak daje tačan rezultat?

Exchange argument: ako optimalno rješenje prvo uzima interval koji završava kasnije, možemo ga zamijeniti najranije završavajućim bez smanjenja broja budućih mogućnosti.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n = int(input())
a = []
for _ in range(n):
    s, e = map(int, input().split())
    a.append((e, s))
a.sort()
last = -10 ** 30
ans = 0
for e, s in a:
    if s >= last:
        ans += 1
        last = e
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < pair < long long, long long >> a;
    while (n--) {
        long long s, e;
        cin >> s >> e;
        a.push_back({
            e, s
        });
    }
    sort(a.begin(), a.end());
    long long last = LLONG_MIN;
    int ans = 0;
    for (auto [e, s] : a) if (s >= last) {
        ans++;
        last = e;
    }
    cout << ans << "\n";
}

```

Važne provjere

Sortirati po završetku, ne po početku.

Kompatibilnost je `s >= last_end`.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 20$	5
2	$N \leq 2000$	7
3	Puna ograničenja	13

ZADATAK 061

Fabrika bedževa

Cilj: Binarno pretražuje najmanje vrijeme koje omogućava proizvodnju najmanje K komada.

Imamo N mašina. Mašina i napravi jedan bedž svakih t_i sekundi i sve počinju u trenutku 0. Odredi najmanje cijelo vrijeme T nakon kojeg je proizvedeno najmanje K bedževa.

Ulaz. N K, zatim N pozitivnih vremena t_i .

Izlaz. Minimalno vrijeme T.

Ograničenja. $1 \leq N \leq 200000$, $1 \leq K \leq 10^{18}$, $1 \leq t_i \leq 10^9$.

Primjer

ULAZ

```
3 10
2 3 7
```

IZLAZ

```
12
```

Razumijevanje i postupak

Za fiksno vrijeme T mašina i proizvede $\text{floor}(T/t_i)$ komada. Predikat “ukupno $\geq K$ ” je monoton: kada jednom postane tačan, ostaje tačan za svako veće vrijeme.

1. $\text{feasible}(T) = \sum(T/t_i) \geq K$; prekini sumu čim dosegne K.
2. Donja granica 0, gornja $\min(t_i) * K$.
3. Klasično traži prvu vrijednost za koju je feasible tačan.

Zašto postupak daje tačan rezultat?

Ako je rješenje moguće sa X, moguće je i sa svakim većim X; zato skup odgovora ima oblik $F \dots F T \dots T$ i binarna pretraga nalazi prvu T poziciju.

Složenost. Vrijeme $O(N \log(\min(t_i) * K))$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
n, k = map(int, input().split())
t = list(map(int, input().split()))
lo, hi = (0, min(t) * k)
while lo < hi:
    mid = (lo + hi) // 2
    s = 0
    for x in t:
        s += mid // x
        if s >= k:
            break
```

PYTHON 3 • potpuno rješenje / nastavak

```

    if s >= k:
        hi = mid
    else:
        lo = mid + 1
print(lo)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
using il28 = __int128_t;
int main() {
    int n;
    unsigned long long k;
    cin >> n >> k;
    vector < unsigned long long > t(n);
    for (auto & x : t) cin >> x;
    unsigned long long mn = * min_element(t.begin(), t.end()), lo = 0, hi =
        mn * k;
    while (lo < hi) {
        auto mid = lo + (hi - lo) / 2;
        il28 s = 0;
        for (auto x : t) {
            s += mid / x;
            if (s >= k) break;
        }
        if (s >= k) hi = mid;
        else lo = mid + 1;
    }
    cout << lo << "\n";
}

```

Važne provjere

Ne binarno pretraživati broj bedževa nego vrijeme.

Gornja granica $\min(t) \cdot K$ može tražiti 64-bitni tip.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$K \leq 10^6$	5
2	$N \leq 1000$	8
3	Puna ograničenja	17

ZADATAK 062

Minimalni kapacitet kamiona

Cilj: Binarno pretražuje minimalni kapacitet uz greedy provjeru broja potrebnih dana/vožnji.

Paketi težina $a_1..a_N$ moraju se prevesti redom, bez promjene poretka. U jednoj vožnji kamion uzima uzastopne pakete ukupne težine najviše C . Dozvoljeno je najviše D vožnji. Odredi minimalni kapacitet C .

Ulaz. N D , zatim N pozitivnih težina.

Izlaz. Minimalni kapacitet.

Ograničenja. $1 \leq D \leq N \leq 300000$, $a_i \leq 10^9$.

Primjer

ULAZ

```
5 3
7 2 5 10 8
```

IZLAZ

```
14
```

Razumijevanje i postupak

Za fiksni C greedy puni svaku vožnju što je više moguće redom. To minimizira broj vožnji za taj C . Ako treba $\leq D$ vožnji, svaki veći kapacitet je takođe moguć.

1. $lo = \max(a)$, $hi = \text{sum}(a)$.
2. `feasible(C)`: prolazi niz i i počinje novu vožnju kada bi sljedeći paket prešao C .
3. Ako broj vožnji $\leq D$, smanji hi ; inače povećaj lo .

Zašto postupak daje tačan rezultat?

Ako je rješenje moguće sa X , moguće je i sa svakim većim X ; zato skup odgovora ima oblik $F...F T...T$ i binarna pretraga nalazi prvu T poziciju.

Složenost. Vrijeme $O(N \log(\text{sum } a))$, memorija $O(N)$ ili $O(1)$ dodatno.

PYTHON 3 • potpuno rješenje

```
n, d = map(int, input().split())
a = list(map(int, input().split()))
lo = max(a)
hi = sum(a)

def ok(c):
    rides = 1
    s = 0
```

PYTHON 3 • potpuno rješenje / nastavak

```

    for x in a:
        if s + x > c:
            rides += 1
            s = 0
        s += x
    return rides <= d
while lo < hi:
    m = (lo + hi) // 2
    if ok(m):
        hi = m
    else:
        lo = m + 1
print(lo)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, d;
    cin >> n >> d;
    vector< long long > a(n);
    long long lo = 0, hi = 0;
    for (auto & x : a) {
        cin >> x;
        lo = max(lo, x);
        hi += x;
    }
    auto ok = [&] (long long c) {
        int rides = 1;
        long long s = 0;
        for (long long x : a) {
            if (s + x > c) {
                rides++;
                s = 0;
            }
            s += x;
        }
        return rides <= d;
    };
    while (lo < hi) {
        long long m = lo + (hi - lo) / 2;
        if (ok(m)) hi = m;
        else lo = m + 1;
    }
    cout << lo << "\n";
}

```

Važne provjere

Redoslijed paketa se ne smije mijenjati.

Greedy provjera računa minimalan broj vožnji za dati C.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 1000$	7
2	$D=N$	5
3	Puna ograničenja	18

18. Mreže i potpuno pretraživanje

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
63	Latinski kvadrat - provjera
64	Izlaz iz labirinta
65	Izaberi tačno K brojeva
66	N kraljica
67	Riječ kroz mrežu
68	Binarni nizovi bez susjednih jedinica
69	Tačan zbir za četrdeset brojeva
70	Najjeftiniji put kroz tabelu
71	Putevi kroz mrežu sa preprekama
72	Evakuacija škole
73	Robot i jedan zid
74	Najbliži izlaz iz labirinta

ZADATAK 063

Latinski kvadrat - provjera

Cilj: Provjeriti ponavljanja u svakom redu i stupcu matrice.

Data je kvadratna matrica $N \times N$. Svako popunjeno polje sadrži broj od 1 do N , a 0 znači prazno. Trenutno stanje je ispravno ako se nijedan nenulti broj ne ponavlja u istom redu niti u istom stupcu. Odredi postoji li greška.

Ulaz. U prvom redu N , zatim N redova sa po N cijelih brojeva.

Izlaz. Ispisati OK ako nema ponavljanja, inače GRESKA.

Ograničenja. 1 N 200.

Primjer

ULAZ

```
4
1 2 0 4
3 4 1 0
2 0 4 3
4 3 2 1
```

IZLAZ

```
OK
```

Razumijevanje i postupak

Provjera reda i stupca je ista operacija: vodimo skup već viđenih nenulatih vrijednosti. Ako naiđemo na broj koji je već u skupu, stanje je pogrešno. Nula se ignoriše.

1. Za svaki red provjeriti duplikate nenulatih brojeva.
2. Za svaki stupac provjeriti duplikate.
3. Ako bilo gdje postoji duplikat, ispisati GRESKA.

Složenost. $O(N^2)$ vrijeme i $O(N)$ pomoćne memorije.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = [list(map(int, input().split())) for _ in range(n)]

def valid(v):
    s = set()
    for x in v:
        if x == 0:
            continue
        if x in s:
            return False
```

PYTHON 3 • potpuno rješenje / nastavak

```

        s.add(x)
    return True
ok = all((valid(row) for row in a))
if ok:
    for j in range(n):
        if not valid((a[i][j] for i in range(n))):
            ok = False
            break
print('OK' if ok else 'GRESKA')
```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < vector < int >> a(n, vector < int > (n));
    for (auto & r : a) for (int & x : r) cin >> x;
    bool ok = true;
    for (int i = 0; i < n; i++) {
        vector < char > seen(n + 1, 0);
        for (int j = 0; j < n; j++) if (a [i] [j]) {
            if (seen [a [i] [j]]) ok = false;
            seen [a [i] [j]] = 1;
        }
    }
    for (int j = 0; j < n; j++) {
        vector < char > seen(n + 1, 0);
        for (int i = 0; i < n; i++) if (a [i] [j]) {
            if (seen [a [i] [j]]) ok = false;
            seen [a [i] [j]] = 1;
        }
    }
    cout << (ok ? "OK" : "GRESKA") << "\n";
}
```

ZADATAK 064

Izlaz iz labirinta

Cilj: Primijeniti BFS za najkraći put u neponderisanoj mreži.

Data je mreža $R \times C$. Znak S je početak, E izlaz, # zid, a . slobodno polje. U jednom koraku može se ići gore, dolje, lijevo ili desno. Odredi najmanji broj koraka od S do E ili -1 ako izlaz nije dostupan.

Ulaz. U prvom redu R i C, zatim R redova mreže.

Izlaz. Ispisati dužinu najkraćeg puta ili -1.

Ograničenja. 1 $R, C \leq 1000$; postoji tačno jedno S i jedno E.

Primjer

ULAZ

```
4 5
S...#
##...#
...#.
...E.
```

IZLAZ

```
6
```

Razumijevanje i postupak

Svaki potez ima jednaku cijenu 1, pa BFS obilazi polja po rastućoj udaljenosti od starta. Prvi put kada dođemo u polje, pronašli smo najkraći put do njega.

1. Pronaći koordinate S.
2. Staviti S u red i $\text{dist}[S]=0$.
3. Dok red nije prazan, širiti se na validne neobičene susjede.
4. Ispisati $\text{dist}[E]$ ili -1.

Složenost. $O(R \cdot C)$ vrijeme i $O(R \cdot C)$ memorija.

PYTHON 3 • potpuno rješenje

```
from collections import deque
r, c = map(int, input().split())
g = [list(input().strip()) for _ in range(r)]
for i in range(r):
    for j in range(c):
        if g[i][j] == 'S':
            sr, sc = (i, j)
        if g[i][j] == 'E':
```

PYTHON 3 • potpuno rješenje / nastavak

```

        er, ec = (i, j)
dist = [[-1] * c for _ in range(r)]
dist[sr][sc] = 0
q = deque([(sr, sc)])
for_pop = [(1, 0), (-1, 0), (0, 1), (0, -1)]
while q:
    x, y = q.popleft()
    for dx, dy in for_pop:
        nx, ny = (x + dx, y + dy)
        if 0 <= nx < r and 0 <= ny < c and (g[nx][ny] != '#') and \
            (dist[nx][ny] == -1):
            dist[nx][ny] = dist[x][y] + 1
            q.append((nx, ny))
print(dist[er][ec])

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>
using namespace std;
int main() {
    int R, C;
    cin >> R >> C;
    vector < string > g(R);
    for (auto & s : g) cin >> s;
    int sr = 0, sc = 0, er = 0, ec = 0;
    for (int i = 0; i < R; i++) for (int j = 0; j < C; j++) {
        if (g[i][j] == 'S') sr = i, sc = j;
        if (g[i][j] == 'E') er = i, ec = j;
    }
    vector < vector < int >> d(R, vector < int > (C, - 1));
    queue < pair < int, int >> q;
    q.push({
        sr, sc
    });
    d[sr][sc] = 0;
    int dx[4] = {
        1, - 1, 0, 0
    }, dy[4] = {
        0, 0, 1, - 1
    };
    while (! q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (int k = 0; k < 4; k++) {
            int nx = x + dx[k], ny = y + dy[k];
            if (nx >= 0 && nx < R && ny >= 0 && ny < C && g[nx][ny] !=
                '#' && d

```

C++17 • potpuno rješenje / nastavak

```
        [nx] [ny] == - 1) {
            d [nx] [ny] = d [x] [y] + 1;
            q.push({
                nx, ny
            });
        }
    }
}
cout << d [er] [ec] << "\n";
}
```

ZADATAK 065

Izaberi tačno K brojeva

Cilj: Učenik oblikuje backtracking sa parametrima pozicije, broja izabranih elemenata i trenutnog zbira.

Dat je niz N različitih pozitivnih brojeva. Prebroj koliko podskupova od tačno K elemenata ima zbir S.

Ulaz. Prvi red: N K S. Drugi red: N različitih pozitivnih brojeva.

Izlaz. Ispisati broj odgovarajućih izbora.

Ograničenja. $1 \leq N \leq 25$, $0 \leq K \leq N$, $S \leq 10^9$.

Primjer

ULAZ

```
5 2 7
1 2 3 4 5
```

IZLAZ

```
2
```

Razumijevanje i postupak

Na svakoj poziciji imamo dvije odluke: preskočiti element ili ga uzeti. Stanje je dovoljno opisati sa i, chosen i sum. Pozitivnost omogućava rez ako $sum > S$.

1. Ako $chosen == K$, doprinos je 1 samo ako $sum == S$.
2. Ako $i == N$ ili $chosen > K$ ili $sum > S$, nema rješenja.
3. Rekurzivno saberi granu bez $A[i]$ i granu sa $A[i]$.

Složenost. Najgore $O(2^N)$, praktično manje zbog K i rezova; dubina $O(N)$.

PYTHON 3 • potpuno rješenje

```
N, K, S = map(int, input().split())
A = list(map(int, input().split()))

def dfs(i, chosen, total):
    if chosen == K:
        return 1 if total == S else 0
    if i == N or total > S or chosen + (N - i) < K:
        return 0
    return dfs(i + 1, chosen, total) + dfs(i + 1, chosen + 1, total + A[i])
print(dfs(0, 0, 0))
```


C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int N, K;
long long S;
vector < long long > A;
long long dfs(int i, int chosen, long long sum) {
    if (chosen == K) return sum == S;
    if (i == N || sum > S || chosen + (N - i) < K) return 0;
    return dfs(i + 1, chosen, sum) + dfs(i + 1, chosen + 1, sum + A[i]);
}
int main() {
    cin >> N >> K >> S;
    A.resize(N);
    for (auto & x : A) cin >> x;
    cout << dfs(0, 0, 0) << "\n";
}
```

Važne provjere

Rez $sum > S$ važi zato što su svi brojevi pozitivni.

Ne brojati permutacije istog podskupa; indeks uvijek ide naprijed.

ZADATAK 066

N kraljica

Cilj: Učenik koristi backtracking sa skupovima zauzetih kolona i dijagonala.

Na šahovsku tablu $N \times N$ postavi N kraljica tako da se nijedne dvije ne napadaju. Odredi broj različitih rasporeda.

Ulaz. Jedan broj N .

Izlaz. Ispisati broj rasporeda.

Ograničenja. $1 \leq N \leq 12$.

Primjer

ULAZ

4

IZLAZ

2

Razumijevanje i postupak

Postavljamo po jednu kraljicu u svaki red. Za polje (r, c) moraju biti slobodni kolona c , dijagonala $r - c$ i dijagonala $r + c$. Time se broj grana dramatično smanjuje.

1. Rekurzija obrađuje red r .
2. Probaj svaku kolonu c koja nije zauzeta ni po jednoj dijagonali.
3. Označi, pozovi sljedeći red, zatim poništi oznake.
4. Kada $r == N$, pronađen je raspored.

Složenost. Eksponencijalno, ali uz jake rezove dovoljno za $N \leq 12$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N = int(input())
cols, d1, d2 = (set(), set(), set())

def dfs(r):
    if r == N:
        return 1
    ans = 0
    for c in range(N):
        if c in cols or r - c in d1 or r + c in d2:
            continue
        cols.add(c)
        d1.add(r - c)
        d2.add(r + c)
        ans += dfs(r + 1)
    return ans
```

PYTHON 3 • potpuno rješenje / nastavak

```

        cols.remove(c)
        d1.remove(r - c)
        d2.remove(r + c)
    return ans
print(dfs(0))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int N;
bool col [20], d1 [40], d2 [40];
long long dfs(int r) {
    if (r == N) return 1;
    long long ans = 0;
    for (int c = 0; c < N; ++ c) {
        int a = r - c + N, b = r + c;
        if (col [c] || d1 [a] || d2 [b]) continue;
        col [c] = d1 [a] = d2 [b] = true;
        ans += dfs(r + 1);
        col [c] = d1 [a] = d2 [b] = false;
    }
    return ans;
}
int main() {
    cin >> N;
    cout << dfs(0) << "\n";
}

```

Važne provjere

Obavezno vratiti stanje nakon rekurzivnog poziva - backtracking znači “uradi pa poništi”.

Dijagonale se mogu kodirati sa $r-c$ i $r+c$.

ZADATAK 067

Riječ kroz mrežu

Cilj: Učenik kombinuje DFS/backtracking, granice matrice i privremeno označavanje posjećenih polja.

U matrici slova pronađi da li se zadana riječ može pročitati kretanjem gore, dolje, lijevo ili desno. Isto polje se u jednoj putanji ne smije koristiti dvaput.

Ulaz. Prvi red R C . Zatim R redova mreže. Posljednji red je riječ W .

Izlaz. Ispisati DA ili NE.

Ograničenja. $1 \leq R, C \leq 20$, $|W| \leq R * C$.

Primjer

ULAZ

```
3 4
ABCE
SFCS
ADEE
ABCCED
```

IZLAZ

```
DA
```

Razumijevanje i postupak

DFS stanje je (r, c, k) : nalazimo se na polju (r, c) i očekujemo $W[k]$. Polje privremeno označimo kao iskorišteno, istražimo četiri susjeda, pa ga vratimo.

1. Pokreni DFS iz svakog polja jednakog prvom slovu.
2. Ako znak ne odgovara ili je polje van mreže, grana ne uspijeva.
3. Ako je k posljednji indeks i znak odgovara, uspjeh.
4. Privremeno označi polje, probaj 4 smjera, zatim vrati znak.

Složenost. Najgore $O(R * C * 4^{|W|})$, uz rezove obično znatno manje; memorija $O(|W|)$.

PYTHON 3 • potpuno rješenje

```
R, C = map(int, input().split())
g = [list(input().strip()) for _ in range(R)]
w = input().strip()

def dfs(r, c, k):
    if r < 0 or r >= R or c < 0 or (c >= C) or (g[r][c] != w[k]):
        return False
    if k == len(w) - 1:
        return True
    ch = g[r][c]
```

PYTHON 3 • potpuno rješenje / nastavak

```

    g[r][c] = '#'
    ok = dfs(r + 1, c, k + 1) or dfs(r - 1, c, k + 1) or dfs(r, c + 1, k + 1) or dfs(r, c - 1, k + 1)
    g[r][c] = ch
    return ok
ans = any((dfs(r, c, 0) for r in range(R) for c in range(C)))
print('DA' if ans else 'NE')

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int R, C;
vector < string > g;
string w;
bool dfs(int r, int c, int k) {
    if (r < 0 || r >= R || c < 0 || c >= C || g[r][c] != w[k]) return false;
    if (k == (int) w.size() - 1) return true;
    char ch = g[r][c];
    g[r][c] = '#';
    bool ok = dfs(r + 1, c, k + 1) || dfs(r - 1, c, k + 1) || dfs(r, c + 1, k + 1) || dfs(r, c - 1, k + 1);
    g[r][c] = ch;
    return ok;
}
int main() {
    cin >> R >> C;
    g.resize(R);
    for (auto & s : g) cin >> s;
    cin >> w;
    for (int r = 0; r < R; ++r) for (int c = 0; c < C; ++c) if (dfs(r, c, 0)) {
        cout << "DA\n";
        return 0;
    }
    cout << "NE\n";
}

```

Važne provjere

Ako se stanje ne vrati nakon DFS-a, kasniji početci dobijaju pokvarenu mrežu.

Ne dozvoliti dijagonalno kretanje.

ZADATAK 068

Binarni nizovi bez susjednih jedinica

Cilj: Učenik generiše kombinatorne objekte uz ograničenje i razlikuje broj posjećenih stanja od 2^N .

Odredi koliko binarnih stringova dužine N ne sadrži podstring 11.

Ulaz. Jedan broj N.

Izlaz. Ispisati broj takvih stringova.

Ograničenja. $1 \leq N \leq 40$.

Primjer

ULAZ

3

IZLAZ

5

Razumijevanje i postupak

Rekurzivno gradimo string slijeva nadesno. Nulu uvijek smijemo dodati. Jedinicu smijemo dodati samo ako prethodni znak nije 1. Isto stanje zavisi samo od pozicije i prethodnog bita, pa ga možemo memoizirati.

1. Definiši $f(pos, prev1)$.
2. Ako $pos == N$, vrati 1.
3. Dodaj granu sa 0; ako $prev1$ nije postavljen, dodaj i granu sa 1.
4. Memoiziraj stanje.

Složenost. S memoizacijom $O(N)$ stanja i $O(N)$ memorije.

PYTHON 3 • potpuno rješenje

```
from functools import lru_cache
N = int(input())

@lru_cache(None)
def f(pos, prev1):
    if pos == N:
        return 1
    ans = f(pos + 1, False)
    if not prev1:
        ans += f(pos + 1, True)
    return ans
print(f(0, False))
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int N;
long long memo [45] [2];
long long f(int pos, int prev1) {
    if (pos == N) return 1;
    long long & res = memo [pos] [prev1];
    if (res != - 1) return res;
    res = f(pos + 1, 0);
    if (! prev1) res += f(pos + 1, 1);
    return res;
}
int main() {
    cin >> N;
    memset(memo, - 1, sizeof(memo));
    cout << f(0, 0) << "\n";
}
```

Važne provjere

Bez memoizacije isti sufiksni problem se računa mnogo puta.

Za N=1 odgovori su 0 i 1, dakle 2.

ZADATAK 069

Tačan zbir za četrdeset brojeva

Cilj: Učenik upoznaje meet-in-the-middle tehniku za smanjenje 2^N na približno $2^{(N/2)}$.

Za N cijelih brojeva odredi postoji li podskup čiji je zbir tačno S. Svaki element se može uzeti najviše jednom.

Ulaz. Prvi red: N S. Drugi red: N cijelih brojeva.

Izlaz. Ispisati DA ili NE.

Ograničenja. $1 \leq N \leq 40$, $|A[i]| \leq 10^9$.

Primjer

ULAZ

```
5 11
2 4 7 9 12
```

IZLAZ

```
DA
```

Razumijevanje i postupak

Direktnih 2^{40} podskupova je previše. Podijelimo niz na dvije polovine. Generišemo sve sume svake polovine; zatim za svaku lijevu sumu x tražimo $S-x$ među desnim sumama.

1. Podijeli niz na L i R.
2. Generiši sve podskupne sume L i R.
3. Sortiraj desne sume.
4. Za svaku x iz lijevih suma binarno potraži $S-x$.

Složenost. $O(2^{(N/2)} * N + 2^{(N/2)} \log 2^{(N/2)})$; memorija $O(2^{(N/2)})$.

PYTHON 3 • potpuno rješenje

```
from bisect import bisect_left
N, S = map(int, input().split())
A = list(map(int, input().split()))
mid = N // 2

def sums(arr):
    res = [0]
    for x in arr:
        res += [v + x for v in res]
    return res
L = sums(A[:mid])
R = sorted(sums(A[mid:]))
ok = False
```


PYTHON 3 • potpuno rješenje / nastavak

```

for x in L:
    y = S - x
    i = bisect_left(R, y)
    if i < len(R) and R[i] == y:
        ok = True
        break
print('DA' if ok else 'NE')

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
vector < long long > makeSums(const vector < long long > & a) {
    vector < long long > r {
        0
    }
    ;
    for (long long x : a) {
        int sz = r.size();
        for (int i = 0; i < sz; ++ i) r.push_back(r [i] + x);
    }
    return r;
}
int main() {
    int N;
    long long S;
    cin >> N >> S;
    vector < long long > A(N);
    for (auto & x : A) cin >> x;
    int m = N / 2;
    vector < long long > L(A.begin(), A.begin() + m), Rpart(A.begin() + m,
        A.end());
    auto X = makeSums(L), Y = makeSums(Rpart);
    sort(Y.begin(), Y.end());
    for (long long x : X) if (binary_search(Y.begin(), Y.end(), S - x)) {
        cout << "DA\n";
        return 0;
    }
    cout << "NE\n";
}

```

Važne provjere

Ne pokušavati 2^{40} eksplicitno.

Kod generisanja suma paziti da se iterira samo kroz staru veličinu liste prije dodavanja novih elemenata.

ZADATAK 070

Najjeftiniji put kroz tabelu

Cilj: Učenik formira 2D dinamičko programiranje sa lokalnim prijelazom iz gornjeg i lijevog polja.

U matrici pozitivnih troškova kreće se iz gornjeg lijevog u donje desno polje, samo desno ili dolje. Trošak puta je zbir svih posjećenih polja. Nađi minimalni trošak.

Ulaz. Prvi red R C . Zatim R redova sa C troškova.

Izlaz. Ispisati minimalni trošak.

Ograničenja. $1 \leq R, C \leq 1000$.

Primjer

ULAZ

```
3 3
1 3 1
1 5 1
4 2 1
```

IZLAZ

```
7
```

Razumijevanje i postupak

Svaki put do (r, c) mora posljednjim potezom doći iz $(r-1, c)$ ili $(r, c-1)$. Zato je $dp[r][c] = cost[r][c] + \min(gore, lijevo)$.

1. Inicijalizuj $dp[0][0]$.
2. Prvi red može doći samo slijeva, prva kolona samo odozgo.
3. Ostala polja koriste minimum dvije prethodne vrijednosti.

Složenost. Vrijeme $O(R \cdot C)$; memorija $O(C)$ u optimizovanoj verziji.

PYTHON 3 • potpuno rješenje

```
R, C = map(int, input().split())
dp = [10 ** 30] * C
for r in range(R):
    row = list(map(int, input().split()))
    for c in range(C):
        if r == 0 and c == 0:
            dp[c] = row[c]
        elif r == 0:
            dp[c] = dp[c - 1] + row[c]
```

PYTHON 3 • potpuno rješenje / nastavak

```

    elif c == 0:
        dp[c] = dp[c] + row[c]
    else:
        dp[c] = min(dp[c], dp[c - 1]) + row[c]
print(dp[-1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int R, C;
    cin >> R >> C;
    const long long INF = 4e18;
    vector < long long > dp(C, INF);
    for (int r = 0; r < R; ++ r) {
        for (int c = 0; c < C; ++ c) {
            long long x;
            cin >> x;
            if (r == 0 && c == 0) dp [c] = x;
            else if (r == 0) dp [c] = dp [c - 1] + x;
            else if (c == 0) dp [c] = dp [c] + x;
            else dp [c] = min(dp [c], dp [c - 1]) + x;
        }
    }
    cout << dp [C - 1] << "\n";
}

```

Važne provjere

Kod 1D optimizacije `dp[c]` prije ažuriranja znači vrijednost odozgo, a `dp[c-1]` već znači vrijednost slijeva.

Početno polje se računa u trošak.

ZADATAK 071

Putevi kroz mrežu sa preprekama

Cilj: Učenik koristi DP na mreži sa zabranjenim stanjima i modularnim brojanjem.

U mreži $R \times C$ znak `.` označava slobodno polje, a `#` prepreku. Iz $(1,1)$ se kreće samo desno ili dolje do (R,C) . Odredi broj različitih puteva modulo 1 000 000 007.

Ulaz. Prvi red R C . Zatim R redova mreže.

Izlaz. Ispisati broj puteva modulo 1 000 000 007.

Ograničenja. $1 \leq R, C \leq 2000$; početno i završno polje su slobodni.

Primjer

ULAZ

```
3 4
....
.#..
....
```

IZLAZ

```
4
```

Razumijevanje i postupak

Do slobodnog polja možemo doći samo odozgo ili slijeva. Prepreka ima 0 puteva. Zbog velikog broja puteva sabiranje se radi modulo M .

1. $dp[0]=1$ na početku.
2. Prolazi red po red. Ako je `#`, $dp[c]=0$.
3. Inače za $c>0$: $dp[c]=(dp[c]+dp[c-1]) \bmod M$.
4. Na kraju $dp[C-1]$ je odgovor.

Složenost. Vrijeme $O(R \cdot C)$; memorija $O(C)$.

PYTHON 3 • potpuno rješenje

```

MOD = 1000000007
R, C = map(int, input().split())
dp = [0] * C
dp[0] = 1
for _ in range(R):
    row = input().strip()
    for c in range(C):
        if row[c] == '#':
            dp[c] = 0
        elif c > 0:
            dp[c] = (dp[c] + dp[c - 1]) % MOD
print(dp[-1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    const long long MOD = 1000000007;
    int R, C;
    cin >> R >> C;
    vector < long long > dp(C);
    dp [0] = 1;
    for (int r = 0; r < R; ++ r) {
        string s;
        cin >> s;
        for (int c = 0; c < C; ++ c) {
            if (s [c] == '#') dp [c] = 0;
            else if (c > 0) dp [c] = (dp [c] + dp [c - 1]) % MOD;
        }
    }
    cout << dp [C - 1] << "\n";
}

```

Važne provjere

Kod prepreke `dp[c]` se mora resetovati na 0, jer vrijednost odozgo više ne smije prolaziti.

Modul se primjenjuje nakon sabiranja.

ZADATAK 072

Evakuacija škole

Cilj: Primjenjuje multi-source BFS na mreži sa preprekama.

Data je mapa škole $R \times C$. Znak # je zid, . prolaz, U učenik, a I izlaz. Svi učenici se kreću istovremeno po susjednim poljima gore/dolje/lijevo/desno. Odredi minimalan broj sekundi nakon kojeg svi učenici mogu stići do nekog izlaza. Ako neki učenik ne može do izlaza, ispiši -1.

Ulaz. R C , zatim R redova mape.

Izlaz. Maksimalna udaljenost učenika do najbližeg izlaza ili -1.

Ograničenja. $R \cdot C \leq 500000$.

Primjer

ULAZ

```
4 6
I...#.
.##.U.
..U...
#...I.
```

IZLAZ

```
3
```

Razumijevanje i postupak

Umjesto BFS-a od svakog učenika, pokrenemo jedan BFS od svih izlaza odjednom. Dobijena distanca svake ćelije je udaljenost do najbližeg izlaza.

1. Sve izlaze stavi u red sa $dist=0$.
2. BFS kroz sve nezidne ćelije.
3. Za svaku U provjeri $dist$; uzmi maksimum.

Zašto postupak daje tačan rezultat?

Multi-source BFS je ekvivalentan dodavanju zamišljenog super-izvora spojenog nultim ivicama sa svim izlazima. Prvi dolazak u ćeliju daje najkraću udaljenost do bilo kojeg izlaza.

Složenost. Vrijeme $O(RC)$, memorija $O(RC)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
R, C = map(int, input().split())
g = [input().strip() for _ in range(R)]
d = [[-1] * C for _ in range(R)]
q = deque()
for r in range(R):
    for c in range(C):
        if g[r][c] == 'I':
            d[r][c] = 0
            q.append((r, c))
while q:
    r, c = q.popleft()
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = (r + dr, c + dc)
        if 0 <= nr < R and 0 <= nc < C and (g[nr][nc] != '#' and \
            (d[nr][nc] == -1)):
            d[nr][nc] = d[r][c] + 1
            q.append((nr, nc))
ans = 0
for r in range(R):
    for c in range(C):
        if g[r][c] == 'U':
            if d[r][c] == -1:
                print(-1)
                raise SystemExit
            ans = max(ans, d[r][c])
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int R, C;
    cin >> R >> C;
    vector < string > g(R);
    for (auto & s : g) cin >> s;
    vector < vector < int >> d(R, vector < int > (C, - 1));
    queue < pair < int, int >> q;
    for (int r = 0; r < R; r++) for (int c = 0; c < C; c++) if (g[r][c]
        == 'I') {
        d[r][c] = 0;
        q.push({
            r, c
        });
    }
    int dr[4] = {
        1, - 1, 0, 0
    }
    , dc[4] = {
        0, 0, 1, - 1
    }
}

```

C++17 • potpuno rješenje / nastavak

```

;
while (! q.empty()) {
    auto [r, c] = q.front();
    q.pop();
    for (int k = 0; k < 4; k++) {
        int nr = r + dr [k], nc = c + dc [k];
        if (nr >= 0 && nr < R && nc >= 0 && nc < C && g [nr] [nc] !=
            '#' && d
                [nr] [nc] == - 1) {
            d [nr] [nc] = d [r] [c] + 1;
            q.push({
                nr, nc
            });
        }
    }
}
}
int ans = 0;
for (int r = 0; r < R; r++) for (int c = 0; c < C; c++) if (g [r] [c]
    == 'U') {
    if (d [r] [c] < 0) {
        cout << - 1 << "\n";
        return 0;
    }
    ans = max(ans, d [r] [c]);
}
cout << ans << "\n";
}

```

Važne provjere

Ne pokretati BFS posebno za svakog učenika.

Zidovi se ne smiju posjećivati.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Jedan izlaz	5
2	Bez zidova	5
3	Puna ograničenja	10

ZADATAK 073

Robot i jedan zid

Cilj: Proširuje BFS stanje dodatnom binarnom informacijom.

Robot ide od S do T u mreži. Može se kretati 4-smjerno. Znak # je zid, ali robot tokom cijelog puta smije srušiti najviše jedan zid. Odredi minimalan broj koraka ili -1.

Ulaz. R C, zatim mapa sa S,T,,#.

Izlaz. Minimalan broj koraka.

Ograničenja. $R \cdot C \leq 300000$.

Primjer

ULAZ

```
4 5
S.#..
##.#.
...#.
.#..T
```

IZLAZ

```
7
```

Razumijevanje i postupak

Ista ćelija nije jedno stanje: dolazak bez potrošenog rušenja je vredniji od dolaska nakon rušenja. Zato stanje mora biti (r,c,used), gdje used=0/1.

1. BFS od (S,0).
2. Prelaz na . ili T zadržava used.
3. Prelaz na # dozvoljen je samo kada used=0 i tada postaje 1.
4. Prvi dolazak do T u BFS-u je optimalan.

Zašto postupak daje tačan rezultat?

Graf stanja ima najviše $2RC$ čvorova i sve ivice težine 1. Standardni BFS zato daje najkraći put među svim legalnim sekvencama, uključujući izbor zida koji se ruši.

Složenost. Vrijeme $O(RC)$, memorija $O(RC)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
R, C = map(int, input().split())
g = [input().strip() for _ in range(R)]
for r in range(R):
    for c in range(C):
        if g[r][c] == 'S':
            sr, sc = (r, c)
        if g[r][c] == 'T':
            tr, tc = (r, c)
d = [[[-1] * 2 for _ in range(C)] for __ in range(R)]
d[sr][sc][0] = 0
q = deque([(sr, sc, 0)])
while q:
    r, c, u = q.popleft()
    if (r, c) == (tr, tc):
        print(d[r][c][u])
        break
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = (r + dr, c + dc)
        if 0 <= nr < R and 0 <= nc < C:
            nu = u + (g[nr][nc] == '#')
            if nu <= 1 and d[nr][nc][nu] == -1:
                d[nr][nc][nu] = d[r][c][u] + 1
                q.append((nr, nc, nu))
else:
    print(-1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct S {
    int r, c, u;
}
;
int main() {
    int R, C;
    cin >> R >> C;
    vector < string > g(R);
    int sr = 0, sc = 0, tr = 0, tc = 0;
    for (int r = 0; r < R; r++) {
        cin >> g[r];
        for (int c = 0; c < C; c++) {
            if (g[r][c] == 'S') sr = r, sc = c;
            if (g[r][c] == 'T') tr = r, tc = c;
        }
    }
    vector < vector < array < int, 2 >> > d(R, vector < array < int, 2 >>
        (C, array <
            int, 2 > {
                - 1, - 1
            }
        ));

```

C++17 • potpuno rješenje / nastavak

```

queue < S > q;
d [sr] [sc] [0] = 0;
q.push({
    sr, sc, 0
});
int dr [4] = {
    1, - 1, 0, 0
}
, dc [4] = {
    0, 0, 1, - 1
}
;
while (! q.empty()) {
    auto [r, c, u] = q.front();
    q.pop();
    if (r == tr && c == tc) {
        cout << d [r] [c] [u] << "\n";
        return 0;
    }
    for (int k = 0; k < 4; k++) {
        int nr = r + dr [k], nc = c + dc [k];
        if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
        int nu = u + (g [nr] [nc] == '#');
        if (nu <= 1 && d [nr] [nc] [nu] == - 1) {
            d [nr] [nc] [nu] = d [r] [c] [u] + 1;
            q.push({
                nr, nc, nu
            });
        }
    }
}
cout << - 1 << "\n";
}

```

Važne provjere

visited[r][c] bez used informacije nije dovoljno.

Ne dozvoliti rušenje S ili T; samo # povećava used.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Bez zidova	5
2	$R \cdot C \leq 5000$	10
3	Puna ograničenja	15

ZADATAK 074

Najbliži izlaz iz labirinta

Cilj: Modelira mrežu kao neponderisani graf i koristi BFS za najkraći broj koraka.

Data je mreža $R \times C$ sa početkom S, prolazima . i zidovima #. Izlaz je bilo koje prohodno polje na ivici mreže (uključujući S ako je na ivici). Odredi minimalan broj poteza do izlaza.

Ulaz. R C , zatim R redova mreže.

Izlaz. Minimalan broj poteza ili -1.

Ograničenja. $R \cdot C \leq 500000$.

Primjer

ULAZ

```
4 5
#####
#S...#
#...#
##...#
```

IZLAZ

```
3
```

Razumijevanje i postupak

Svaka prohodna ćelija je čvor, a potez u četiri smjera ivica težine 1. BFS od S prvi put dostiže svaku ćeliju najkraćim brojem poteza; čim prvi put dođemo na prohodnu ivicu, imamo optimalan izlaz.

1. Nađi koordinate S.
2. $\text{dist}[S]=0$ i stavi S u red.
3. BFS kroz četiri susjeda koji nisu # i nisu posjećeni.
4. Kada izvadimo ćeliju na ivici, ispiši njenu udaljenost.

Zašto postupak daje tačan rezultat?

Kada se čvor prvi put izvadi/dohvati, nijedan kasniji put sa više ivica ne može biti kraći; slojevi redom predstavljaju udaljenosti 0,1,2,...

Složenost. Vrijeme $O(RC)$, memorija $O(RC)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
r, c = map(int, input().split())
g = [input().strip() for _ in range(r)]
for i in range(r):
    for j in range(c):
        if g[i][j] == 'S':
            s = (i, j)
d = [[-1] * c for _ in range(r)]
d[s[0]][s[1]] = 0
q = deque([s])
while q:
    x, y = q.popleft()
    if x == 0 or x == r - 1 or y == 0 or (y == c - 1):
        print(d[x][y])
        break
    for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nx, ny = (x + dx, y + dy)
        if 0 <= nx < r and 0 <= ny < c and (g[nx][ny] != '#' and \
            (d[nx][ny] < 0)):
            d[nx][ny] = d[x][y] + 1
            q.append((nx, ny))
else:
    print(-1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int R, C;
    cin >> R >> C;
    vector < string > g(R);
    pair < int, int > s;
    for (int i = 0; i < R; i++) {
        cin >> g[i];
        for (int j = 0; j < C; j++) if (g[i][j] == 'S') s = {
            i, j
        };
    }
    vector < vector < int >> d(R, vector < int > (C, -1));
    queue < pair < int, int >> q;
    d[s.first][s.second] = 0;
    q.push(s);
    int dx[4] = {
        1, -1, 0, 0
    }, dy[4] = {
        0, 0, 1, -1
    };
    while (!q.empty()) {
        auto [x, y] = q.front();

```

C++17 • potpuno rješenje / nastavak

```

    q.pop();
    if (x == 0 || x == R - 1 || y == 0 || y == C - 1) {
        cout << d [x] [y] << "\n";
        return 0;
    }
    for (int k = 0; k < 4; k++) {
        int nx = x + dx [k], ny = y + dy [k];
        if (nx >= 0 && nx < R && ny >= 0 && ny < C && g [nx] [ny] !=
            '#' && d
                [nx] [ny] < 0) {
            d [nx] [ny] = d [x] [y] + 1;
            q.push({
                nx, ny
            });
        }
    }
}
cout << - 1 << "\n";
}

```

Važne provjere

Izlaz nije poseban znak nego prohodna ćelija na ivici.

Ne zaboraviti da S može već biti izlaz.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	R,C <= 30	5
2	Bez zidova	5
3	Puna ograničenja	10

19. Djeljivost, prosti brojevi i bitovi

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
75	Najveći zajednički djelilac
76	Kada se signali ponovo poklope
77	Da li je broj prost?
78	Brzi odgovori o prostim brojevima
79	Koliko djelilaca ima broj?
80	Jedinice u binarnom zapisu
81	Brzo stepenovanje po modulu
82	Rastavljanje na proste faktore
83	Koliko je prostih?
84	NZD bez jednog broja
85	Veliki stepen modulo M
86	Posljednja nenulta cifra faktoriijela

ZADATAK 075

Najveći zajednički djelilac

Cilj: Učenik primjenjuje Euklidov algoritam i razlikuje efikasan algoritam od provjere svih djelilaca.

Za data pozitivna cijela broja a i b odredi njihov najveći zajednički djelilac (NZD).

Ulaz. Jedan red sadrži dva cijela broja a i b .

Izlaz. Ispisati NZD(a,b).

Ograničenja. $1 \leq a, b \leq 10^{18}$.

Primjer

ULAZ

48 18

IZLAZ

6

Razumijevanje i postupak

Ako je $a = qb + r$, svaki zajednički djelilac brojeva a i b dijeli i ostatak r . Zato par (a,b) možemo zamijeniti parom $(b, a \bmod b)$. Ostatak se smanjuje i postupak brzo završava.

1. Dok je b različit od nule izračunaj $a \bmod b$.
2. Postavi $(a,b) = (b, a \bmod b)$.
3. Kada b postane 0, a je NZD.

Složenost. Vrijeme $O(\log \min(a,b))$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
a, b = map(int, input().split())
while b:
    a, b = (b, a % b)
print(a)
```


C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long a, b;
    cin >> a >> b;
    while (b) {
        long long r = a % b;
        a = b;
        b = r;
    }
    cout << a << "\n";
}
```

Važne provjere

Petlja koja ide do $\min(a,b)$ je nepotrebno spora za velike brojeve.

Ne zamijeniti redoslijed ažuriranja a i b pogrešno.

ZADATAK 076

Kada se signali ponovo poklope

Cilj: Učenik povezuje NZS sa periodičnim događajima i koristi NZD za sigurno računanje NZS-a.

Dva signalna uređaja trepnu istovremeno u trenutku 0. Prvi zatim trepće svakih a sekundi, a drugi svakih b sekundi. Nakon koliko sekundi će ponovo trepnuti istovremeno?

Ulaz. Jedan red sadrži a i b .

Izlaz. Ispisati najmanji pozitivan zajednički trenutak.

Ograničenja. $1 \leq a, b \leq 10^9$.

Primjer

ULAZ

6 8

IZLAZ

24

Razumijevanje i postupak

Traži se najmanji zajednički sadržalac. Formula $NZS(a,b)=a/NZD(a,b)*b$ izbjegava nepotrebno rano množenje velikih brojeva.

1. Izračunaj $g=NZD(a,b)$.
2. Izračunaj $a/g*b$.
3. Ispiši rezultat.

Složenost. Vrijeme $O(\log \min(a,b))$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
import math
a, b = map(int, input().split())
g = math.gcd(a, b)
print(a // g * b)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long a, b;
    cin >> a >> b;
    long long g = std::gcd(a, b);
    cout << (a / g) * b << "\n";
}
```

Važne provjere

Ne koristiti $a*b/g$ ako postoji rizik prelijevanja u drugim zadacima.

Ne zamijeniti NZD i NZS.

ZADATAK 077

Da li je broj prost?

Cilj: Učenik provjerava prostost jednog broja u $O(\sqrt{n})$ i pravilno obrađuje $n < 2$.

Za dati cijeli broj n odredi da li je prost. Prost broj ima tačno dva pozitivna djelitelja: 1 i samog sebe.

Ulaz. Jedan broj n .

Izlaz. Ispisati DA ako je n prost, inače NE.

Ograničenja. $0 \leq n \leq 2 \cdot 10^9$.

Primjer**ULAZ**

97

IZLAZ

DA

Razumijevanje i postupak

Ako n ima djelilac veći od $\sqrt{n}$, uz njega postoji odgovarajući djelilac manji od $\sqrt{n}$. Zato je dovoljno tražiti djelioce samo do kvadratnog korijena.

1. Ako je $n < 2$, odgovor je NE.
2. Provjeri djelioce d od 2 dok $d \cdot d \leq n$.
3. Ako neki d dijeli n , broj nije prost; inače jeste.

Složenost. Vrijeme $O(\sqrt{n})$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
if n < 2:
    print('NE')
else:
    d = 2
    prime = True
    while d * d <= n:
        if n % d == 0:
            prime = False
            break
        d += 1
    print('DA' if prime else 'NE')
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long n;
    cin >> n;
    if (n < 2) {
        cout << "NE\n";
        return 0;
    }
    for (long long d = 2; d * d <= n; ++d) if (n % d == 0) {
        cout << "NE\n";
        return 0;
    }
    cout << "DA\n";
}
```

Važne provjere

Broj 1 nije prost.

Uslov $d*d \leq n$ mora uključiti jednakost zbog savršenih kvadrata.

ZADATAK 078

Brzi odgovori o prostim brojevima

Cilj: Učenik koristi Eratostenovo sito kada se postavlja mnogo upita o prostosti.

Za sve brojeve od 0 do N unaprijed odredi prostost, a zatim odgovori na Q upita.

Ulaz. Prvi red: N Q. Drugi red: Q brojeva x_i .

Izlaz. Za svaki upit ispisati DA ili NE u posebnom redu.

Ograničenja. $2 \leq N \leq 10^6$, $1 \leq Q \leq 2 \cdot 10^5$, $0 \leq x_i \leq N$.

Primjer

ULAZ

```
20 5
2 9 17 20 19
```

IZLAZ

```
DA
NE
DA
NE
DA
```

Razumijevanje i postupak

Kod mnogo upita nije dobro svaki broj zasebno testirati do $\sqrt{x}$. Sito jednom označi sve složene brojeve, a svaki kasniji upit postaje $O(1)$.

1. Kreiraj niz prime veličine $N+1$ i inicijalno ga postavi na True.
2. 0 i 1 postavi na False.
3. Za p od 2 do $\sqrt{N}$, ako je p prost, označi $p \cdot p$, $p \cdot p + p$, ... kao složene.
4. Odgovori na upite direktnim pristupom nizu.

Složenost. Priprema $O(N \log \log N)$; svaki upit $O(1)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

N, Q = map(int, input().split())
queries = list(map(int, input().split()))
prime = [True] * (N + 1)
prime[0] = prime[1] = False
p = 2
while p * p <= N:
    if prime[p]:
        for x in range(p * p, N + 1, p):
            prime[x] = False
        p += 1
for x in queries:
    print('DA' if prime[x] else 'NE')

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, Q;
    cin >> N >> Q;
    vector< bool > prime(N + 1, true);
    prime[0] = prime[1] = false;
    for (long long p = 2; p * p <= N; ++p) if (prime[p]) for (long long x
        = p * p; x
        <= N; x += p) prime[x] = false;
    while (Q--) {
        int x;
        cin >> x;
        cout << (prime[x] ? "DA" : "NE") << "\n";
    }
}

```

Važne provjere

Označavanje višekratnika može početi od $p \cdot p$, jer su manji već obrađeni.

Ne zaboraviti da 0 i 1 nisu prosti.

ZADATAK 079

Koliko djelilaca ima broj?

Cilj: Učenik koristi parove djelilaca i pravilno obrađuje savršene kvadrate.

Za dati pozitivan cijeli broj n odredi broj njegovih pozitivnih djelilaca.

Ulaz. Jedan broj n .

Izlaz. Ispisati broj pozitivnih djelilaca broja n .

Ograničenja. $1 \leq n \leq 10^{12}$.

Primjer**ULAZ**

36

IZLAZ

9

Razumijevanje i postupak

Djelitelji dolaze u parovima d i n/d . Za svaki $d \leq \sqrt{n}$ dobijamo dva djelitelja. Ako je n savršen kvadrat, $\sqrt{n}$ se računa samo jednom.

1. Postavi brojač na 0.
2. Za $d=1$ dok $d*d \leq n$, ako d dijeli n dodaj 2.
3. Ako je $d*d == n$ dodaj samo 1.

Složenost. Vrijeme $O(\sqrt{n})$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
ans = 0
d = 1
while d * d <= n:
    if n % d == 0:
        ans += 1 if d * d == n else 2
    d += 1
print(ans)
```


C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long n;
    cin >> n;
    long long ans = 0;
    for (long long d = 1; d * d <= n; ++d) if (n % d == 0) ans += (d * d ==
        n ? 1 : 2);
    cout << ans << "\n";
}
```

Važne provjere

Za 36 parovi su (1,36),(2,18),(3,12),(4,9) i srednji djelilac 6.

Ne brojati sqrt(n) dvaput.

ZADATAK 080

Jedinice u binarnom zapisu

Cilj: Učenik razumije binarni zapis i osnovnu bitovsku operaciju $n \& 1$.

Za nenegativan cijeli broj n odredi koliko jedinica sadrži njegov binarni zapis bez vodećih nula.

Ulaz. Jedan broj n .

Izlaz. Ispisati broj jedinica u binarnom zapisu.

Ograničenja. $0 \leq n < 2^{63}$.

Primjer**ULAZ**

45

IZLAZ

4

Razumijevanje i postupak

Najniži bit broja n je 1 upravo kada je n neparan. Operacija $n \& 1$ čita taj bit, a pomjeranje $n \gg= 1$ uklanja ga.

1. Postavi brojač na 0.
2. Dok je $n > 0$, dodaj $n \& 1$.
3. Pomjeri n za jedan bit udesno.

Složenost. Vrijeme $O(\log n)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
ans = 0
while n:
    ans += n & 1
    n >>= 1
print(ans)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    unsigned long long n;
    cin >> n;
    int ans = 0;
    while (n) {
        ans += (n & 1ULL);
        n >>= 1;
    }
    cout << ans << "\n";
}
```

Važne provjere

Za $n=0$ odgovor je 0 i petlja se ne izvršava.

Bitovsko & nije isto što i logičko &&.

ZADATAK 081

Brzo stepenovanje po modulu

Cilj: Učenik koristi binarno stepenovanje i svodi broj množenja sa $O(b)$ na $O(\log b)$.

Za date cijele brojeve a , b i m izračunaj $a^b \bmod m$.

Ulaz. Jedan red sadrži a b m .

Izlaz. Ispisati ostatak a^b pri dijeljenju sa m .

Ograničenja. $0 \leq a \leq 10^{18}$, $0 \leq b \leq 10^{18}$, $1 \leq m \leq 10^{18}$.

Primjer

ULAZ

3 13 1000

IZLAZ

323

Razumijevanje i postupak

Eksponent pišemo binarno. Ako je trenutni bit eksponenta 1, trenutnu bazu uključujemo u rezultat. Nakon svakog koraka bazu kvadiramo, a eksponent prepolovimo.

1. $result = 1 \bmod m$, $base = a \bmod m$.
2. Dok je $b > 0$: ako je b neparan, $result = result * base \bmod m$.
3. $base = base * base \bmod m$; $b //= 2$.

Složenost. Vrijeme $O(\log b)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
a, b, m = map(int, input().split())
res = 1 % m
a %= m
while b:
    if b & 1:
        res = res * a % m
    a = a * a % m
    b >>= 1
print(res)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
using u128 = __uint128_t;
int main() {
    unsigned long long a, b, m;
    cin >> a >> b >> m;
    unsigned long long res = 1 % m;
    a %= m;
    while (b) {
        if (b & 1) res = (u128) res * a % m;
        a = (u128) a * a % m;
        b >>= 1;
    }
    cout << res << "\n";
}
```

Važne provjere

Za $b=0$ rezultat je $1 \bmod m$.

Kod C++ za vrlo veliki m proizvod dva 64-bitna broja zahtijeva širi međutip; ovdje se koristi `__uint128_t`.

ZADATAK 082

Rastavljanje na proste faktore

Cilj: Učenik rastavlja broj na proste faktore i evidentira eksponente.

Rastavi pozitivan cijeli broj $n > 1$ na proste faktore. Faktore ispiši rastuće u obliku p^e .

Ulaz. Jedan broj n .

Izlaz. Ispisati faktore odvojene razmakom.

Ograničenja. $2 \leq n \leq 10^{12}$.

Primjer

ULAZ

360

IZLAZ

2³ 3² 5¹

Razumijevanje i postupak

Kada pronađemo djelilac p , dijelimo n sa p sve dok je djeljiv i brojimo eksponent. Nakon što isprobamo sve p sa $p \cdot p \leq n$, eventualni preostali $n > 1$ mora biti prost.

1. Za p od 2 naviše dok $p \cdot p \leq n$.
2. Ako p dijeli n , broj ponavljanja čuvaj u e i potpuno ukloni p iz n .
3. Na kraju, ako je $n > 1$, dodaj n^1 .

Složenost. Vrijeme $O(\sqrt{n})$ u najgorem slučaju; memorija $O(\text{broja različitih prostih faktora})$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
ans = []
p = 2
while p * p <= n:
    if n % p == 0:
        e = 0
        while n % p == 0:
            n //= p
            e += 1
        ans.append((p, e))
    p += 1
if n > 1:
    ans.append((n, 1))
print(' '.join('{p}^{e}'.format(p=p, e=e) for p, e in ans))
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long n;
    cin >> n;
    vector < pair < long long, int >> a;
    for (long long p = 2; p * p <= n; ++ p) if (n % p == 0) {
        int e = 0;
        while (n % p == 0) {
            n /= p;
            ++ e;
        }
        a.push_back({
            p, e
        });
    }
    if (n > 1) a.push_back({
        n, 1
    });
    for (int i = 0; i < (int) a.size(); ++ i) {
        if (i) cout << ' ';
        cout << a [i].first << '^' << a [i].second;
    }
    cout << "\n";
}
```

Važne provjere

Uslov $p * p \leq n$ se mijenja dok se n smanjuje - to je ispravno i ubrzava rad.

Preostali $n > 1$ se ne smije zaboraviti.

ZADATAK 083

Koliko je prostih?

Cilj: Gradi Eratostenovo sito i prefiks broja prostih za veliki broj upita.

Dato je Q upita. Svaki upit sadrži n i traži broj prostih brojeva $\leq n$.

Ulaz. Q , zatim Q brojeva n .

Izlaz. Za svaki upit jedan broj.

Ograničenja. $1 \leq Q \leq 200000$, $0 \leq n \leq 2000000$.

Primjer

ULAZ

```
4
1
10
20
2
```

IZLAZ

```
0
4
8
1
```

Razumijevanje i postupak

Najveći n određuje granicu predobrade. Sito jednom označi sve proste do MAX , a prefiks `prime_count[i]` daje broj prostih $\leq i$ u $O(1)$ po upitu.

1. Pročitaj sve upite i nađi $M = \max(n)$.
2. Napravi `is_prime[0..M]`, označi 0 i 1 kao složene.
3. Za p do $\sqrt{M}$ precrtaj višekratnike od $p \cdot p$.
4. Napravi prefiks broja `True` vrijednosti.

Zašto postupak daje tačan rezultat?

Najmanji prosti djelilac složenog broja p označiće taj broj kada obrađujemo p , pa nijedan složen broj ne ostaje označen kao prost.

Složenost. Vrijeme $O(M \log \log M + Q)$, memorija $O(M)$.

PYTHON 3 • potpuno rješenje

```

q = int(input())
qs = [int(input()) for _ in range(q)]
m = max(qs, default=0)
p = [True] * (m + 1)
if m >= 0:
    if m >= 0 and len(p) > 0:
        p[0] = False
    if m >= 1:
        p[1] = False
i = 2
while i * i <= m:
    if p[i]:
        p[i * i:m + 1:i] = [False] * ((m - i * i) // i + 1)
    i += 1
pref = [0] * (m + 1)
for i in range(1, m + 1):
    pref[i] = pref[i - 1] + p[i]
for x in qs:
    print(pref[x])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int q;
    cin >> q;
    vector< int > a(q);
    int M = 0;
    for (int & x : a) {
        cin >> x;
        M = max(M, x);
    }
    vector< char > p(M + 1, 1);
    if (M >= 0) p[0] = 0;
    if (M >= 1) p[1] = 0;
    for (long long i = 2; i * i <= M; i++) if (p[i]) for (long long j = i
        * i; j <=
        M; j += i) p[j] = 0;
    vector< int > pref(M + 1);
    for (int i = 1; i <= M; i++) pref[i] = pref[i - 1] + p[i];
    for (int x : a) cout << pref[x] << "\n";
}

```

Važne provjere

Broj 1 nije prost.

Kod precrtavanja početi od $p \cdot p$, ali koristiti 64-bitni proizvod.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$n \leq 10000$	7
2	$Q \leq 1000$	5
3	Puna ograničenja	13

ZADATAK 084

NZD bez jednog broja

Cilj: Kombinuje prefiksni i sufixni NZD da obradi "sve osim jednog" u linearnom vremenu.

Dat je niz pozitivnih cijelih brojeva. Za svaki izbor jednog elementa koji izbacimo posmatramo NZD svih preostalih. Ispiši najveći mogući NZD.

Ulaz. N, zatim N pozitivnih brojeva.

Izlaz. Najveći NZD nakon izbacivanja tačno jednog elementa.

Ograničenja. $2 \leq N \leq 300000$, $a_i \leq 10^9$.

Primjer

ULAZ

```
5
12 18 24 30 42
```

IZLAZ

```
6
```

Razumijevanje i postupak

Za svaku poziciju i trebaju nam svi elementi prije i i svi poslije i. Prefiksni gcd $L[i]$ i sufixni gcd $R[i]$ omogućavaju kandidat $\text{gcd}(L[i], R[i+1])$ bez ponovnog prolaska kroz niz.

1. $L[0]=0$; $L[i+1]=\text{gcd}(L[i], a[i])$.
2. $R[N]=0$; $R[i]=\text{gcd}(R[i+1], a[i])$.
3. Za svako i kandidat je $\text{gcd}(L[i], R[i+1])$; uzmi maksimum.

Zašto postupak daje tačan rezultat?

Asocijativnost gcd operacije omogućava da sve elemente osim $a[i]$ podijelimo u dvije grupe i spojimo njihove NZD vrijednosti.

Složenost. Vrijeme $O(N \log A)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
from math import gcd
n = int(input())
a = list(map(int, input().split()))
L = [0] * (n + 1)
R = [0] * (n + 1)
for i, x in enumerate(a):
    L[i + 1] = gcd(L[i], x)
for i in range(n - 1, -1, -1):
    R[i] = gcd(R[i + 1], a[i])
print(max((gcd(L[i], R[i + 1]) for i in range(n))))
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > a(n), L(n + 1), R(n + 1);
    for (auto & x : a) cin >> x;
    for (int i = 0; i < n; i++) L [i + 1] = gcd(L [i], a [i]);
    for (int i = n - 1; i >= 0; i--) R [i] = gcd(R [i + 1], a [i]);
    long long ans = 0;
    for (int i = 0; i < n; i++) ans = max(ans, gcd(L [i], R [i + 1]));
    cout << ans << "\n";
}
```

Važne provjere

$\text{gcd}(0, x) = x$ je korisna baza za praznu stranu.

Traži se tačno jedno izbacivanje.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 2000$	8
2	Svi brojevi ≤ 1000	7
3	Puna ograničenja	15

ZADATAK 085

Veliki stepen modulo M

Cilj: Implementira binarno stepenovanje i sprječava eksploziju veličine brojeva modularnim računanjem.

Data su cijela A, B i M. Izračunaj $A^B \bmod M$.

Ulaz. A B M.

Izlaz. Jedan cijeli broj.

Ograničenja. $0 \leq A \leq 10^{18}$, $0 \leq B \leq 10^{18}$, $1 \leq M \leq 10^9+7$.

Primjer**ULAZ**

7 13 1000

IZLAZ

407

Razumijevanje i postupak

Množenje baze B puta je presporo. Eksponent prepolovljavamo: ako je bit neparan, trenutnu bazu uključimo u rezultat; zatim bazu kvadriramo modulo M.

1. $a \% m$; $res = 1 \% m$.
2. Dok $b > 0$: ako $b \& 1$, $res = res * a \% m$.
3. $a = a * a \% m$; $b //= 2$.

Zašto postupak daje tačan rezultat?

Invariant je $result * base^{exp}$ početna_baza^{početni_exp} (mod M). Svaki korak ga čuva, a kada exp postane 0 result je odgovor.

Složenost. Vrijeme $O(\log B)$, memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
a, b, m = map(int, input().split())
res = 1 % m
a %= m
while b:
    if b & 1:
        res = res * a % m
    a = a * a % m
    b //= 2
print(res)
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
using i128 = __int128_t;
int main() {
    unsigned long long a, b, m;
    cin >> a >> b >> m;
    unsigned long long r = 1 % m;
    a %= m;
    while (b) {
        if (b & 1) r = (unsigned long long) ((i128) r * a % m);
        a = (unsigned long long) ((i128) a * a % m);
        b >>= 1;
    }
    cout << r << "\n";
}

```

Važne provjere

Ne računati A^B prije uzimanja modula.

Paziti na overflow pri množenju u C++; koristiti `__int128`.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$B \leq 100000$	5
2	$A, M \leq 10^9$	5
3	Puna ograničenja	10

ZADATAK 086

Posljednja nenulta cifra faktoriijela

Cilj: Kontrolirajte faktore 2 i 5 da ukloni završne nule bez izgradnje ogromnog faktoriijela.

Za dato N odredi posljednju nenultu decimalnu cifru broja $N!$.

Ulaz. Jedan cijeli broj N .

Izlaz. Jedna cifra 1-9 (za $N=0$ odgovor je 1).

Ograničenja. $0 \leq N \leq 1000000$.

Primjer

ULAZ

10

IZLAZ

8

Razumijevanje i postupak

Završna nula nastaje parom $2 \cdot 5$. Pri množenju $1..N$ iz svakog faktora izbacujemo sve dvojke i petice, množimo ostatke modulo 10, a na kraju vraćamo višak dvojki.

1. $\text{cnt2}=\text{cnt5}=0$, $\text{res}=1$.
2. Za svaki x izdvoji faktore 2 i 5, ostatak pomnoži u res modulo 10.
3. Upari $\min(\text{cnt2}, \text{cnt5})$; ostaje $\text{cnt2}-\text{cnt5}$ dvojki.
4. Pomnoži res sa $2^{(\text{višak})}$ modulo 10.

Zašto postupak daje tačan rezultat?

Ako je stanje ispravno poslije prvih i elemenata, lokalno pravilo ga tačno ažurira nakon elementa $i+1$. Indukcijom stanje je tačno na kraju.

Složenost. Vrijeme $O(N \log N)$ u najgorem faktorizacionom smislu, praktično $O(N \log N)$ sa vrlo malom konstantom; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
r = 1
c2 = c5 = 0
for x in range(2, n + 1):
    y = x
    while y % 2 == 0:
        c2 += 1
        y //= 2
    while y % 5 == 0:
```

PYTHON 3 • potpuno rješenje / nastavak

```

    c5 += 1
    y //= 5
    r = r * (y % 10) % 10
for _ in range(c2 - c5):
    r = r * 2 % 10
print(r)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    int r = 1, c2 = 0, c5 = 0;
    for (int x = 2; x <= n; x++) {
        int y = x;
        while (y % 2 == 0) {
            c2++;
            y /= 2;
        }
        while (y % 5 == 0) {
            c5++;
            y /= 5;
        }
        r = r * (y % 10) % 10;
    }
    for (int k = 0; k < c2 - c5; k++) r = r * 2 % 10;
    cout << r << "\n";
}

```

Važne provjere

Ne brisati nule tek na kraju iz ogromnog broja.

Višak faktora 2 je uvijek nenegativan u faktoriјelu.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N <= 20	5
2	N <= 10000	8
3	Puna ograničenja	17

20. Strukture podataka i obrada stringova

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
87	Ispravno ugniježdene zagrade
88	Prvi veći desno
89	Poništi posljednju operaciju
90	Postfiksni kalkulator
91	Dinamički zbir intervala
92	Minimum uz promjene
93	Koliko riječi počinje prefiksom?
94	Koliko puta se uzorak pojavljuje?
95	Najkraći period poruke
96	Dinamički zbir intervala
97	Minimum na intervalu
98	Rječnik prefiksa
99	Pojavljivanja uzorka
100	Jednaki podstringovi — tačno poređenje
101	Sljedeći veći element
102	Maksimum kliznog prozora

ZADATAK 087

Ispravno ugniježdene zagrade

Cilj: Učenik koristi stek za provjeru pravilnog ugniježđivanja tri vrste zagrada.

Dat je string sastavljen samo od znakova (), [] i {}. Odredi da li je zapis pravilno ugniježđen.

Ulaz. Jedan string S.

Izlaz. Ispisati DA ili NE.

Ograničenja. $1 \leq |S| \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
{[()()]}
```

IZLAZ

DA

Razumijevanje i postupak

Zatvorena zagrada mora odgovarati posljednjoj još nezatvorenoj otvorenoj zagradi. Upravo to radi LIFO struktura - stek.

1. Za otvorenu zgradu stavi znak na stek.
2. Za zatvorenu provjeri da stek nije prazan i da vrh odgovara.
3. Na kraju stek mora biti prazan.

Složenost. Vrijeme $O(N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
s = input().strip()
st = []
match = {')': '(', ']': '[', '}': '{'}
ok = True
for ch in s:
    if ch in '([{':
        st.append(ch)
    else:
        if not st or st[-1] != match[ch]:
            ok = False
            break
        st.pop()
print('DA' if ok and (not st) else 'NE')
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    vector< char > st;
    bool ok = true;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push_back(c);
        else {
            char need = (c == ')'? '(' : (c == ']'? '[' : '{'));
            if (st.empty() || st.back() != need) {
                ok = false;
                break;
            }
            st.pop_back();
        }
    }
    cout << (ok && st.empty() ? "DA" : "NE") << "\n";
}
```

Važne provjere

Samo prebrojavanje broja otvorenih i zatvorenih zagrada nije dovoljno.

Provjeriti praznost steka prije čitanja vrha.

ZADATAK 088

Prvi veći desno

Cilj: Učenik primjenjuje monotoni stek i svodi kvadratnu pretragu na linearno vrijeme.

Za svaki element niza pronađi prvi element desno od njega koji je strogo veći. Ako ga nema, ispiši -1.

Ulaz. Prvi red N. Drugi red N cijelih brojeva.

Izlaz. Ispisati N odgovora.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$.

Primjer**ULAZ**

```
5
2 1 4 3 5
```

IZLAZ

```
4 4 5 5 -1
```

Razumijevanje i postupak

Na steku čuvamo indekse elemenata koji još čekaju prvi veći element. Kada stigne nova vrijednost x , ona rješava sve vrhove steka čija je vrijednost manja od x .

1. ans inicijalno -1.
2. Za svaki i : dok stek nije prazan i $A[\text{vrh}] < A[i]$, postavi $\text{ans}[\text{vrh}] = A[i]$ i skini vrh.
3. Stavi i na stek.

Složenost. Vrijeme $O(N)$, jer svaki indeks ulazi i izlazi iz steka najviše jednom; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N = int(input())
A = list(map(int, input().split()))
ans = [-1] * N
st = []
for i, x in enumerate(A):
    while st and A[st[-1]] < x:
        ans[st.pop()] = x
    st.append(i)
print(*ans)
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    cin >> N;
    vector< long long > A(N), ans(N, - 1);
    for (auto & x : A) cin >> x;
    vector< int > st;
    for (int i = 0; i < N; ++ i) {
        while (! st.empty() && A [st.back()] < A [i]) {
            ans [st.back()] = A [i];
            st.pop_back();
        }
        st.push_back(i);
    }
    for (int i = 0; i < N; ++ i) {
        if (i) cout << ' ';
        cout << ans [i];
    }
    cout << "\n";
}

```

Važne provjere

Uslov je strogo veći, zato se jednaki elementi ne skidaju.

Na steku su indeksi, ne nužno same vrijednosti.

ZADATAK 089

Poništi posljednju operaciju

Cilj: Učenik modelira undo mehanizam stekom.

Dato je N operacija. Pozitivan ili negativan broj x različit od 0 znači “dodaj x ”. Vrijednost 0 znači “poništi posljednje još neponišteno dodavanje”, ako postoji. Na kraju ispiši zbir preostalih vrijednosti.

Ulaz. Prvi red N . Drugi red N cijelih brojeva koji opisuju operacije.

Izlaz. Ispisati konačni zbir.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
7
5 3 0 8 2 0 4
```

IZLAZ

```
17
```

Razumijevanje i postupak

Undo uvijek djeluje na posljednju aktivnu operaciju. To je ponovo LIFO obrazac. Zbir možemo održavati zajedno sa stekom da ne sabiramo sve na kraju.

1. Ako je $x \neq 0$, stavi x na stek i dodaj ga sumi.
2. Ako je $x = 0$ i stek nije prazan, skini vrh i oduzmi ga od sume.
3. Ispiši sumu.

Složenost. Vrijeme $O(N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N = int(input())
ops = list(map(int, input().split()))
st = []
total = 0
for x in ops:
    if x != 0:
        st.append(x)
        total += x
    elif st:
        total -= st.pop()
print(total)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    cin >> N;
    vector < long long > st;
    long long sum = 0;
    while (N--) {
        long long x;
        cin >> x;
        if (x != 0) {
            st.push_back(x);
            sum += x;
        }
        else if (! st.empty()) {
            sum -= st.back();
            st.pop_back();
        }
    }
    cout << sum << "\n";
}
```

Važne provjere

Nula je naredba, ne podatak koji se stavlja na stek.

Poništavanje praznog steka se samo ignoriše prema tekstu zadatka.

ZADATAK 090

Postfiksni kalkulator

Cilj: Učenik evaluira postfiksni izraz pomoću steka i pravilno poštuje redoslijed operanada.

Dat je ispravan postfiksni izraz od cijelih brojeva i operatora +, - i *. Izračunaj njegovu vrijednost.

Ulaz. Prvi red N - broj tokena. Drugi red sadrži N tokena razdvojenih razmakom.

Izlaz. Ispisati vrijednost izraza.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$; rezultat staje u 64-bitni cijeli broj.

Primjer

ULAZ

```
7
3 4 + 2 * 5 -
```

IZLAZ

```
9
```

Razumijevanje i postupak

Kada čitamo broj, stavljamo ga na stek. Kada čitamo operator, skidamo desni operand b pa lijevi operand a i vraćamo rezultat a op b.

1. Za broj: push.
2. Za operator: b=pop(), a=pop().
3. Izračunaj a op b i push rezultat.
4. Na kraju je na steku tačno jedna vrijednost.

Složenost. Vrijeme $O(N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N = int(input())
tokens = input().split()
st = []
for t in tokens:
    if t not in ['+', '-', '*']:
        st.append(int(t))
    else:
        b = st.pop()
        a = st.pop()
        st.append(a + b if t == '+' else a - b if t == '-' else a * b)
print(st[-1])
```


C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    cin >> N;
    vector < long long > st;
    string t;
    while (N--) {
        cin >> t;
        if (t == "+" || t == "-" || t == "*") {
            long long b = st.back();
            st.pop_back();
            long long a = st.back();
            st.pop_back();
            st.push_back(t == "+" ? a + b : t == "-" ? a - b : a * b);
        }
        else st.push_back(stoll(t));
    }
    cout << st.back() << "\n";
}
```

Važne provjere

Kod oduzimanja redoslijed je a-b, ne b-a.

Token -5 je broj, ali u ovom zadatku se negativni brojevi mogu pojaviti; provjera t tačno protiv operatora to ispravno razlikuje.

ZADATAK 091

Dinamički zbir intervala

Cilj: Učenik koristi Fenwickovo stablo za tačkasto ažuriranje i upite prefiksnog/intervalnog zbira.

Dat je niz od N brojeva i Q upita. Upit ADD i x povećava $A[i]$ za x . Upit SUM l r traži zbir $A[l]+...+A[r]$.

Ulaz. Prvi red N Q . Drugi red početni niz. Zatim Q redova sa ADD i x ili SUM l r.

Izlaz. Za svaki SUM upit ispisati odgovor u posebnom redu.

Ograničenja. $1 \leq N, Q \leq 2 \cdot 10^5$, vrijednosti staju u 64-bitni cijeli broj.

Primjer**ULAZ**

```
5 4
1 2 3 4 5
SUM 2 4
ADD 3 5
SUM 1 3
SUM 4 5
```

IZLAZ

```
9
11
9
```

Razumijevanje i postupak

Prefiksna suma je odlična kada se niz ne mijenja, ali nakon svakog ADD morali bismo prepraviti mnogo prefiksa. Fenwickovo stablo čuva parcijalne sume tako da i ažuriranje i prefiksni upit traju $O(\log N)$.

1. add(i,x): dok $i \leq N$ dodaj x u bit[i] i uradi $i += i \& -i$.
2. sum(i): dok $i > 0$ saberi bit[i] i uradi $i -= i \& -i$.
3. SUM(l,r)=sum(r)-sum(l-1).
4. Početni niz ubaci pozivima add.

Složenost. Izgradnja $O(N \log N)$; svaki upit $O(\log N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N, Q = map(int, input().split())
A = list(map(int, input().split()))
bit = [0] * (N + 1)

def add(i, x):
    while i <= N:
```

PYTHON 3 • potpuno rješenje / nastavak

```

        bit[i] += x
        i += i & -i

def pref(i):
    s = 0
    while i > 0:
        s += bit[i]
        i -= i & -i
    return s
for i, x in enumerate(A, 1):
    add(i, x)
for _ in range(Q):
    t, a, b = input().split()
    a = int(a)
    b = int(b)
    if t == 'ADD':
        add(a, b)
    else:
        print(pref(b) - pref(a - 1))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct Fenwick {
    int n;
    vector < long long > b;
    Fenwick(int n) : n(n), b(n + 1) {}
    void add(int i, long long x) {
        for (; i <= n; i += i & - i) b[i] += x;
    }
    long long sum(int i) {
        long long s = 0;
        for (; i > 0; i -= i & - i) s += b[i];
        return s;
    }
};
int main() {
    int N, Q;
    cin >> N >> Q;
    Fenwick f(N);
    for (int i = 1; i <= N; ++ i) {
        long long x;
        cin >> x;
        f.add(i, x);
    }
    while (Q--) {
        string t;
        int a;
        long long b;

```

C++17 • potpuno rješenje / nastavak

```
    cin >> t >> a >> b;  
    if (t == "ADD") f.add(a, b);  
    else cout << f.sum((int) b) - f.sum(a - 1) << "\n";  
  }  
}
```

Važne provjere

Fenwick se najjednostavnije koristi sa indeksima od 1.

U SUM upitu treći broj je indeks r , iako se u parseru privremeno čuva u 64-bitnoj varijabli.

ZADATAK 092

Minimum uz promjene

Cilj: Učenik koristi segmentno stablo za tačkasta ažuriranja i minimum na intervalu.

Dat je niz i Q upita. SET i x postavlja $A[i]=x$. MIN l r traži najmanju vrijednost na intervalu $[l,r]$.

Ulaz. Prvi red N Q. Drugi red niz. Zatim Q upita.

Izlaz. Za svaki MIN ispisati odgovor.

Ograničenja. $1 \leq N, Q \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
5 4
5 2 7 1 6
MIN 2 5
SET 4 8
MIN 2 5
MIN 1 3
```

IZLAZ

```
1
2
2
```

Razumijevanje i postupak

Segmentno stablo hijerarhijski dijeli niz na intervale. Svaki čvor čuva minimum svog intervala. Ažuriranje mijenja jedan list i $O(\log N)$ predaka; upit obilazi samo $O(\log N)$ relevantnih čvorova.

1. Izaberi veličinu size kao prvi stepen dvojke $\geq N$.
2. Listovi počinju na size, roditelj čuva min djece.
3. SET ažurira list pa se penje prema korijenu.
4. MIN koristi iterativni interval $[l,r]$ i pomjera granice prema gore.

Složenost. Izgradnja $O(N)$; svaki upit $O(\log N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N, Q = map(int, input().split())
A = list(map(int, input().split()))
size = 1
while size < N:
    size *= 2
INF = 10 ** 30
```

PYTHON 3 • potpuno rješenje / nastavak

```

seg = [INF] * (2 * size)
for i, x in enumerate(A):
    seg[size + i] = x
for i in range(size - 1, 0, -1):
    seg[i] = min(seg[2 * i], seg[2 * i + 1])

def setv(i, x):
    i += size
    seg[i] = x
    i //= 2
    while i:
        seg[i] = min(seg[2 * i], seg[2 * i + 1])
        i //= 2

def query(l, r):
    l += size
    r += size
    ans = INF
    while l < r:
        if l & 1:
            ans = min(ans, seg[l])
            l += 1
        if r & 1:
            r -= 1
            ans = min(ans, seg[r])
        l //= 2
        r //= 2
    return ans

for _ in range(Q):
    t, a, b = input().split()
    a = int(a)
    b = int(b)
    if t == 'SET':
        setv(a - 1, b)
    else:
        print(query(a - 1, b))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, Q;
    cin >> N >> Q;
    int s = 1;
    while (s < N) s *= 2;
    const long long INF = 4e18;
    vector < long long > seg(2 * s, INF);
    for (int i = 0; i < N; ++i) cin >> seg[s + i];
    for (int i = s - 1; i; --i) seg[i] = min(seg[2 * i], seg[2 * i + 1]);
    auto setv = [&] (int i, long long x) {
        i += s;
        seg[i] = x;
    };

```

C++17 • potpuno rješenje / nastavak

```

    for (i /= 2; i; i /= 2) seg [i] = min(seg [2 * i], seg [2 * i + 1]);
}
;
auto qry = [&] (int l, int r) {
    long long ans = INF;
    for (l += s, r += s; l < r; l /= 2, r /= 2) {
        if (l & 1) ans = min(ans, seg [l++]);
        if (r & 1) ans = min(ans, seg [-- r]);
    }
    return ans;
}
;
while (Q--) {
    string t;
    int a;
    long long b;
    cin >> t >> a >> b;
    if (t == "SET") setv(a - 1, b);
    else cout << qry(a - 1, (int) b) << "\n";
}
}

```

Važne provjere

Upit se često implementira kao poluotvoren interval $[l,r)$.

Fenwick je jednostavniji za zbir; segmentno stablo je fleksibilnije za minimum i druge asocijativne operacije.

ZADATAK 093

Koliko riječi počinje prefiksom?

Cilj: Učenik koristi trie za skup stringova i prefiksne upite.

Dato je N riječi malim slovima. Za svaki od Q prefiksa odredi koliko unesenih riječi počinje tim prefiksom.

Ulaz. Prvi red N . Zatim N riječi. Zatim Q i Q prefiksa, svaki u svom redu.

Izlaz. Za svaki prefiks ispisati broj riječi.

Ograničenja. Ukupna dužina svih riječi i upita $\leq 5 \cdot 10^5$.

Primjer

ULAZ

```
5
apple
app
ape
bat
bath
4
ap
app
ba
cat
```

IZLAZ

```
3
2
2
0
```

Razumijevanje i postupak

Trie je stablo znakova. Svaki prolazak kroz čvor povećava cnt tog prefiksa. Upit samo prati znakove prefiksa; ako veza nedostaje, odgovor je 0.

1. Korijen je prazan prefiks.
2. Pri dodavanju riječi kreiraj nedostajuće dijete i povećaj cnt svakog posjećenog čvora.
3. Za upit prati znakove; rezultat je cnt posljednjeg čvora.

Složenost. Vrijeme $O(\text{ukupnog broja znakova})$; memorija $O(\text{ukupnog broja znakova})$.

PYTHON 3 • potpuno rješenje

```

N = int(input())
nxt = [{}]
cnt = [0]
for _ in range(N):
    s = input().strip()
    u = 0
    for ch in s:
        if ch not in nxt[u]:
            nxt[u][ch] = len(nxt)
            nxt.append({})
            cnt.append(0)
        u = nxt[u][ch]
        cnt[u] += 1
Q = int(input())
for _ in range(Q):
    p = input().strip()
    u = 0
    ok = True
    for ch in p:
        if ch not in nxt[u]:
            ok = False
            break
        u = nxt[u][ch]
    print(cnt[u] if ok else 0)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct Node {
    array < int, 26 > n;
    int cnt = 0;
    Node() {
        n.fill(- 1);
    }
};
;
int main() {
    int N;
    cin >> N;
    vector < Node > tr(1);
    while (N-- > 0) {
        string s;
        cin >> s;
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (tr[u].n[c] == - 1) {
                tr[u].n[c] = tr.size();
                tr.emplace_back();
            }
            u = tr[u].n[c];
            ++ tr[u].cnt;
        }
    }
}

```

C++17 • potpuno rješenje / nastavak

```

    }
}
int Q;
cin >> Q;
while (Q--) {
    string p;
    cin >> p;
    int u = 0;
    bool ok = true;
    for (char ch : p) {
        int c = ch - 'a';
        if (tr[u].n[c] == -1) {
            ok = false;
            break;
        }
        u = tr[u].n[c];
    }
    cout << (ok ? tr[u].cnt : 0) << "\n";
}
}

```

Važne provjere

Brojač se povećava na čvoru prefiksa nakon prelaska znaka.

Trie troši više memorije od običnog seta, ali daje direktne prefiksne upite.

ZADATAK 094

Koliko puta se uzorak pojavljuje?

Cilj: Učenik koristi KMP prefiks-funkciju za linearno traženje uzorka sa preklapanjima.

Dat je tekst T i uzorak P . Odredi koliko puta se P pojavljuje u T , uključujući preklapajuća pojavljivanja.

Ulaz. Prvi red T , drugi red P .

Izlaz. Ispisati broj pojavljivanja.

Ograničenja. $1 \leq |T|, |P| \leq 5 \cdot 10^5$.

Primjer

ULAZ

```
aaaaa
aa
```

IZLAZ

```
4
```

Razumijevanje i postupak

KMP ne vraća pokazivač teksta unazad. Prefiks-funkcija $pi[q]$ govori koliki najduži pravi prefiks uzorka je ujedno sufiks već poklopljenog dijela. Poslije punog poklapanja možemo nastaviti od $pi[m-1]$ i tako brojati preklapanja.

1. Izračunaj pi za P .
2. Prolazi kroz T sa dužinom trenutnog poklapanja j .
3. Kod neslaganja vraćaj $j=pi[j-1]$.
4. Kod $j==|P|$ povećaj odgovor i postavi $j=pi[j-1]$.

Složenost. Vrijeme $O(|T| + |P|)$; memorija $O(|P|)$.

PYTHON 3 • potpuno rješenje

```
T = input().strip()
P = input().strip()
m = len(P)
pi = [0] * m
for i in range(1, m):
    j = pi[i - 1]
    while j and P[i] != P[j]:
        j = pi[j - 1]
    if P[i] == P[j]:
        j += 1
    pi[i] = j
ans = 0
```

PYTHON 3 • potpuno rješenje / nastavak

```

j = 0
for ch in T:
    while j and ch != P[j]:
        j = pi[j - 1]
    if ch == P[j]:
        j += 1
    if j == m:
        ans += 1
        j = pi[j - 1]
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string T, P;
    cin >> T >> P;
    int m = P.size();
    vector<int> pi(m);
    for (int i = 1; i < m; ++i) {
        int j = pi[i - 1];
        while (j && P[i] != P[j]) j = pi[j - 1];
        if (P[i] == P[j]) ++j;
        pi[i] = j;
    }
    long long ans = 0;
    int j = 0;
    for (char ch : T) {
        while (j && ch != P[j]) j = pi[j - 1];
        if (ch == P[j]) ++j;
        if (j == m) {
            ++ans;
            j = pi[j - 1];
        }
    }
    cout << ans << "\n";
}

```

Važne provjere

Nakon punog poklapanja ne postavljati $j=0$, jer se mogu izgubiti preklapanja.

Prefiks-funkcija se računa samo nad uzorkom.

ZADATAK 095

Najkraći period poruke

Cilj: Koristi prefiks-funkciju za pronalazak najmanjeg perioda stringa.

Za string S odredi najmanju pozitivnu dužinu p takvu da se S može dobiti ponavljanjem svog prefiksa dužine p jedan ili više puta. Ako nema kraćeg perioda, odgovor je $|S|$.

Ulaz. Jedan string S .

Izlaz. Najmanja dužina perioda.

Ograničenja. $1 \leq |S| \leq 1000000$.

Primjer

ULAZ

abcbabcabcabc

IZLAZ

3

Razumijevanje i postupak

Prefiks-funkcija $\pi[n-1]$ daje dužinu najdužeg pravog prefiksa koji je i sufiks. Kandidat perioda je $p=n-\pi[n-1]$. On je pravi period samo ako $n\%p==0$.

1. Izračunaj KMP prefiks-funkciju.
2. $p=n-\pi[-1]$.
3. Ako $n\%p==0$ ispiši p , inače n .

Zašto postupak daje tačan rezultat?

Ako string ima period p , tada prefiks dužine $n-p$ jednak je sufiksu iste dužine, pa $\pi[n-1] \geq n-p$. Najmanji kandidat iz najdužeg bordera daje osnovni period; djeljivost osigurava cijeli broj ponavljanja.

Složenost. Vrijeme $O(N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

s = input().strip()
n = len(s)
pi = [0] * n
for i in range(1, n):
    j = pi[i - 1]
    while j and s[i] != s[j]:
        j = pi[j - 1]
    if s[i] == s[j]:
        j += 1
    pi[i] = j
p = n - pi[-1]
print(p if n % p == 0 else n)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    int n = s.size();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    int p = n - pi[n - 1];
    cout << (n % p == 0 ? p : n) << "\n";
}

```

Važne provjere

Kandidat p bez provjere $n\%p$ može biti lažan period.

pi je dužina, ne indeks.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$ S \leq 5000$	5
2	S je sigurno periodičan	10
3	Puna ograničenja	15

ZADATAK 096

Dinamički zbir intervala

Cilj: Fenwickovim stablom podržava tačkaste promjene i intervalne sume u $O(\log N)$.

Dat je niz od N brojeva i Q operacija. Operacija 1 i x postavlja $a_i = x$. Operacija 2 l r traži zbir $a_l + \dots + a_r$. Ispiši odgovor za svaku operaciju tipa 2.

Ulaz. N Q , početni niz, zatim Q operacija.

Izlaz. Odgovori na upite tipa 2.

Ograničenja. $N, Q \leq 300000$, $|a_i|, |x| \leq 10^9$.

Primjer

ULAZ

```
5 5
1 2 3 4 5
2 1 5
1 3 10
2 2 4
1 1 -2
2 1 2
```

IZLAZ

```
15
16
0
```

Razumijevanje i postupak

Obične prefiksne sume pucaju kada se elementi mijenjaju. Fenwick podržava `add(delta)` i `prefix(i)`. Za postavljanje $a_i = x$ računamo $\text{delta} = x - a_i$, ažuriramo Fenwick i samu vrijednost.

1. Izgradi Fenwick dodavanjem početnih $a[i]$.
2. `update(i,delta): i+=i&-i.`
3. `prefix(i): s+=bit[i], i-=i&-i.`
4. `Range sum = prefix(r)-prefix(l-1).`

Zašto postupak daje tačan rezultat?

Blokovi koje `prefix` posjeti disjunktno pokrivaju tačno prefiks $1..i$; `update` mijenja tačno sve blokove koji sadrže promijenjenu poziciju.

Složenost. Izgradnja $O(N \log N)$, svaka operacija $O(\log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n, q = map(int, input().split())
a = [0] + list(map(int, input().split()))
bit = [0] * (n + 1)

def addb(i, d):
    while i <= n:
        bit[i] += d
        i += i & -i

def pref(i):
    s = 0
    while i:
        s += bit[i]
        i -= i & -i
    return s

for i in range(1, n + 1):
    addb(i, a[i])
for _ in range(q):
    t, x, y = map(int, input().split())
    if t == 1:
        addb(x, y - a[x])
        a[x] = y
    else:
        print(pref(y) - pref(x - 1))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct F {
    int n;
    vector < long long > b;
    F(int n) : n(n), b(n + 1) {
    }
    void add(int i, long long d) {
        for (; i <= n; i += i & - i) b [i] += d;
    }
    long long sum(int i) {
        long long s = 0;
        for (; i; i -= i & - i) s += b [i];
        return s;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector < long long > a(n + 1);
    F f(n);
    for (int i = 1; i <= n; i++) {
        cin >> a [i];
    }
}

```


C++17 • potpuno rješenje / nastavak

```

        f.add(i, a [i]);
    }
    while (q--) {
        int t, x;
        long long y;
        cin >> t >> x >> y;
        if (t == 1) {
            f.add(x, y - a [x]);
            a [x] = y;
        }
        else cout << f.sum((int) y) - f.sum(x - 1) << "\n";
    }
}

```

Važne provjere

Fenwick koristi 1-bazne indekse.

Operacija postavljanja nije isto što i dodavanje; zato računamo delta.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Q samo tipa 2	5
2	N,Q ≤ 3000	8
3	Puna ograničenja	17

ZADATAK 097

Minimum na intervalu

Cilj: Segmentnim stablom podržava tačkasto postavljanje i minimum na intervalu.

Dat je niz i Q operacija. 1 i x postavlja $a_i = x$. 2 l r traži minimalnu vrijednost na indeksima l..r.

Ulaz. N Q, početni niz, zatim Q operacija.

Izlaz. Odgovori za upite tipa 2.

Ograničenja. $N, Q \leq 300000$, $|vrijednost| \leq 10^9$.

Primjer

ULAZ

```
5 4
7 2 9 4 6
2 2 5
1 2 8
2 1 3
2 4 4
```

IZLAZ

```
2
7
4
```

Razumijevanje i postupak

Koristimo iterativno segmentno stablo sa bazom size kao sljedećim stepenom dvojke. Listovi čuvaju elemente, a roditelj minimum djece. Upit $[l, r]$ pretvaramo u poluotvoreni $[l, r+1)$.

1. Izaberi $S = 2^k \geq N$ i napuni $tree[S+i]$.
2. Izgradi roditelje $\min(tree[2p], tree[2p+1])$.
3. Update mijenja list pa sve pretke.
4. Query skuplja rubne čvorove dok se l i r penju.

Zašto postupak daje tačan rezultat?

Agregat roditelja je tačno kombinacija agregata djece. Nakon ažuriranja svih predaka i spajanja pokrivenih čvorova dobijamo tačan odgovor.

Složenost. Izgradnja $O(N)$, update $O(\log N)$, query $O(\log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n, q = map(int, input().split())
a = list(map(int, input().split()))
S = 1
while S < n:
    S *= 2
INF = 10 ** 30
t = [INF] * (2 * S)
for i, x in enumerate(a):
    t[S + i] = x
for i in range(S - 1, 0, -1):
    t[i] = min(t[2 * i], t[2 * i + 1])

def upd(i, x):
    i = S + i - 1
    t[i] = x
    i //= 2
    while i:
        t[i] = min(t[2 * i], t[2 * i + 1])
        i //= 2

def qry(l, r):
    l = S + l - 1
    r = S + r
    ans = INF
    while l < r:
        if l & 1:
            ans = min(ans, t[l])
            l += 1
        if r & 1:
            r -= 1
            ans = min(ans, t[r])
        l //= 2
        r //= 2
    return ans
for _ in range(q):
    z, x, y = map(int, input().split())
    if z == 1:
        upd(x, y)
    else:
        print(qry(x, y))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    int S = 1;
    while (S < n) S <= 1;
    const long long INF = 4e18;

```

C++17 • potpuno rješenje / nastavak

```

vector < long long > t(2 * S, INF);
for (int i = 0; i < n; i++) cin >> t [S + i];
for (int i = S - 1; i; i--) t [i] = min(t [2 * i], t [2 * i + 1]);
auto upd = [&] (int p, long long x) {
    p = S + p - 1;
    t [p] = x;
    for (p >= 1; p; p >= 1) t [p] = min(t [2 * p], t [2 * p + 1]);
}
;
auto qry = [&] (int l, int r) {
    long long ans = INF;
    l = S + l - 1;
    r = S + r;
    while (l < r) {
        if (l & 1) ans = min(ans, t [l++]);
        if (r & 1) ans = min(ans, t [-- r]);
        l >>= 1;
        r >>= 1;
    }
    return ans;
}
;
while (q--) {
    int z, x;
    long long y;
    cin >> z >> x >> y;
    if (z == 1) upd(x, y);
    else cout << qry(x, (int) y) << "\n";
}
}

```

Važne provjere

Neutralni element za min mora biti +INF, ne 0.

Paziti na poluotvoreni interval u iterativnom upitu.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N, Q ≤ 3000	8
2	N je stepen dvojke	5
3	Puna ograničenja	17

ZADATAK 098

Rječnik prefiksa

Cilj: Gradi Trie i odgovara koliko riječi počinje datim prefiksom.

Dato je N riječi malih slova i Q prefiksnih upita. Za svaki prefiks P odredi koliko od N riječi počinje sa P. Duplikati riječi računaju se posebno.

Ulaz. N Q, zatim N riječi, zatim Q prefiksa.

Izlaz. Po jedan broj za svaki prefiks.

Ograničenja. Ukupan broj znakova u svim riječima i upitima ≤ 1000000 .

Primjer

ULAZ

```
5 4
ana
ananas
anto
ban
ana
an
ana
b
x
```

IZLAZ

```
4
3
1
0
```

Razumijevanje i postupak

Svaki Trie čvor predstavlja prefiks. Pri umetnju riječi povećamo cnt u svakom čvoru kroz koji riječ prolazi. Upit samo prati znakove prefiksa; ako put ne postoji odgovor je 0, inače cnt posljednjeg čvora.

1. Korijen je čvor 0.
2. Za svaku riječ prolazi znakove, kreiraj nedostajuće djetete i povećaj cnt tog djeteta.
3. Za upit prati put bez kreiranja.
4. Ispiši cnt na kraju ili 0.

Zašto postupak daje tačan rezultat?

Umetanje povećava brojač tačno na čvorovima svih prefiksa riječi. Zato čvor do kojeg dođemo nakon čitanja prefiksa sadrži tačan broj riječi sa tim prefiksom.

Složenost. Vrijeme $O(\text{ukupnom broju znakova})$, memorija $O(\text{ukupnom broju različitih prefiksa})$.

PYTHON 3 • potpuno rješenje

```
n, q = map(int, input().split())
nxt = [{}]
```

```
cnt = [0]
```

```
for _ in range(n):
    u = 0
    for ch in input().strip():
        if ch not in nxt[u]:
            nxt[u][ch] = len(nxt)
            nxt.append({})
            cnt.append(0)
        u = nxt[u][ch]
    cnt[u] += 1
```

```
for _ in range(q):
    u = 0
    ok = True
    for ch in input().strip():
        if ch not in nxt[u]:
            ok = False
            break
        u = nxt[u][ch]
    print(cnt[u] if ok else 0)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
struct N {
    array < int, 26 > to {
    }
    ;
    int c = 0;
}
;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector < N > tr(1);
    while (n--) {
        string s;
        cin >> s;
        int u = 0;
        for (char ch : s) {
            int x = ch - 'a';
            if (! tr [u].to [x]) {
                tr [u].to [x] = tr.size();
                tr.push_back(N());
            }
            u = tr [u].to [x];
        }
    }
```

C++17 • potpuno rješenje / nastavak

```

        tr [u].c++;
    }
}
while (q--) {
    string s;
    cin >> s;
    int u = 0;
    bool ok = 1;
    for (char ch : s) {
        int x = ch - 'a';
        if (! tr [u].to [x]) {
            ok = 0;
            break;
        }
        u = tr [u].to [x];
    }
    cout << (ok ? tr [u].c : 0) << "\n";
}
}

```

Važne provjere

Brojač se povećava pri prolasku svakog umetnutog primjerka.

Upit ne smije slučajno kreirati nove čvorove.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Ukupno znakova <= 5000	7
2	Nema duplikata	5
3	Puna ograničenja	18

ZADATAK 099

Pojavljivanja uzorka

Cilj: KMP algoritmom broji sva, uključujući preklapajuća, pojavljivanja uzorka u tekstu.

Data su stringovi T (tekst) i P (uzorak). Odredi koliko puta se P pojavljuje u T kao uzastopni podstring. Pojavljivanja se smiju preklapati.

Ulaz. Tekst T , zatim uzorak P .

Izlaz. Broj pojavljivanja.

Ograničenja. $1 \leq |P| \leq |T|$, $|T| \leq 1000000$.

Primjer

ULAZ

```
aaaaa
aa
```

IZLAZ

```
4
```

Razumijevanje i postupak

KMP održava j - dužinu trenutno uparenog prefiksa uzorka. Pri nepodudaranju vraća j na $pi[j-1]$, bez vraćanja indeksa teksta. Kada j dosegne $|P|$, pronađeno je pojavljivanje i j se vraća na $pi[j-1]$ da omogući preklapanje.

1. Izračunaj prefiks-funkciju pi za P .
2. $j=0$; prolazi znakove T .
3. Dok konflikt i $j>0$: $j=pi[j-1]$.
4. Ako jednakost: $j++$; ako $j==m$: $ans++$, $j=pi[j-1]$.

Zašto postupak daje tačan rezultat?

Skok na $pi[j-1]$ ne preskače potencijalno podudaranje jer svaki kandidat dužine između nema odgovarajuću prefiks-sufiks strukturu.

Složenost. Vrijeme $O(|T| + |P|)$, memorija $O(|P|)$.

PYTHON 3 • potpuno rješenje

```
t = input().strip()
p = input().strip()
m = len(p)
pi = [0] * m
for i in range(1, m):
    j = pi[i - 1]
    while j and p[i] != p[j]:
        j = pi[j - 1]
```


PYTHON 3 • potpuno rješenje / nastavak

```

    j = pi[j - 1]
    if p[i] == p[j]:
        j += 1
    pi[i] = j
ans = 0
j = 0
for ch in t:
    while j and ch != p[j]:
        j = pi[j - 1]
    if ch == p[j]:
        j += 1
    if j == m:
        ans += 1
        j = pi[j - 1]
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string t, p;
    cin >> t >> p;
    int m = p.size();
    vector<int> pi(m);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j && p[i] != p[j]) j = pi[j - 1];
        if (p[i] == p[j]) j++;
        pi[i] = j;
    }
    long long ans = 0;
    int j = 0;
    for (char c : t) {
        while (j && c != p[j]) j = pi[j - 1];
        if (c == p[j]) j++;
        if (j == m) {
            ans++;
            j = pi[j - 1];
        }
    }
    cout << ans << "\n";
}

```

Važne provjere

Poslije punog poklapanja ne vraćati j na 0 nego na pi[m-1].

Uzorak nije prazan po ograničenju.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$ T \leq 5000$	7
2	Nema preklapanja	5
3	Puna ograničenja	18

ZADATAK 100

Jednaki podstringovi — tačno poređenje

Cilj: Dvostrukim rolling hashom odgovara na veliki broj upita jednakosti podstringova jednake dužine.

Dat je string S i Q upita. Svaki upit daje $l1$ $r1$ $l2$ $r2$, pri čemu su dužine podstringova jednake. Ispiši DA ako su $S[l1..r1]$ i $S[l2..r2]$ jednaki, inače NE. Indeksi su 1-bazni.

Ulaz. S , Q , zatim Q upita.

Izlaz. DA/NE po upitu.

Ograničenja. $1 \leq |S| \leq 300000$; $1 \leq Q \leq 300000$. S sadrži mala slova engleske abecede. Upiti označavaju važeće podstringove jednakih dužina.

Primjer

ULAZ

```
abrakadabra
3
1 3 8 10
1 4 4 7
4 6 6 8
```

IZLAZ

```
DA
NE
NE
```

Razumijevanje i postupak

Umjesto numeričkih hash potpisa s mogućim kolizijama, dodjeljujemo tačne oznake klasama jednakih blokova dužine 1, 2, 4, 8, Blok dužine 2^k određen je parom oznaka svojih polovina. Sortiranjem parova i dodjelom jednake oznake jednakim parovima dobijamo tačnu, a ne vjerovatnosnu jednakost.

Za upit dužine L uzmemo najveći stepen dvojke p koji nije veći od L . Poredimo početne blokove dužine p i završne blokove dužine p . Budući da je 2^p barem L , ta dva bloka zajedno pokrivaju cijeli podstring; njihovo preklapanje nije problem.

1. Prvi red oznaka napravi od znakova stringa.
2. Udvostručuj dužinu bloka; za svaki važeći početak formiraj par oznaka polovina.
3. Sortiraj parove i jednake parove označi istim cijelim brojem.
4. Za svaki upit uporedi oznake početnog i završnog bloka odabrane dužine.

Zašto postupak daje tačan rezultat?

Indukcijom po dužini stepena dvojke oznake su jednake tačno kada su jednaki cijeli blokovi: baza poredi znakove, a korak poredi obje polovine. U upitu su početni i završni blok zajedno pokrili svaki položaj podstringa. Jednakost oba para oznaka zato je ekvivalentna jednakosti svih znakova, bez rizika hash kolizije.

Složenost. Predobrada $O(N \log^2 N)$, upit $O(1)$, memorija $O(N \log N)$. Oznake su u Pythonu smještene u kompaktne nizove array; konkretno vrijeme izvođenja zavisi od ulaza i okruženja.

PYTHON 3 • potpuno rješenje

```
from array import array
s = input().strip()
n = len(s)
q = int(input())
redovi = [array('i', (ord(c) for c in s))]
duzina = 1
while 2 * duzina <= n:
    prethodni = redovi[-1]
    koliko = n - 2 * duzina + 1
    parovi = [(prethodni[i], prethodni[i + duzina], i) for i in range(koliko)]
    parovi.sort()
    novi = array('i', [0]) * koliko
    oznaka = -1
    zadnji = None
    for lijevi, desni, i in parovi:
        kljuc = (lijevi, desni)
        if kljuc != zadnji:
            oznaka += 1
            zadnji = kljuc
        novi[i] = oznaka
    redovi.append(novi)
    duzina *= 2
for _ in range(q):
    l1, r1, l2, r2 = map(int, input().split())
    l1 -= 1
    r1 -= 1
    l2 -= 1
    r2 -= 1
    nivo = (r1 - l1 + 1).bit_length() - 1
    p = 1 << nivo
    a = redovi[nivo]
    jednaki = a[l1] == a[l2] and a[r1 - p + 1] == a[r2 - p + 1]
    print('DA' if jednaki else 'NE')
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
```

C++17 • potpuno rješenje / nastavak

```

int n = s.size(), q;
cin >> q;
vector < vector < int >> redovi(1, vector < int > (n));
for (int i = 0; i < n; i++) redovi [0] [i] = (unsigned char) s [i];
for (int duzina = 1; 2 * duzina <= n; duzina *= 2) {
    const auto & prethodni = redovi.back();
    int koliko = n - 2 * duzina + 1;
    vector < tuple < int, int, int >> parovi;
    for (int i = 0; i < koliko; i++) parovi.push_back({
        prethodni [i], prethodni [i + duzina], i
    });
    sort(parovi.begin(), parovi.end());
    vector < int > novi(koliko);
    int oznaka = - 1;
    pair < int, int > zadnji = {
        - 1, - 1
    };
    ;
    for (auto [lijevi, desni, i] : parovi) {
        pair < int, int > kljuc = {
            lijevi, desni
        };
        ;
        if (kljuc != zadnji) {
            oznaka++;
            zadnji = kljuc;
        }
        novi [i] = oznaka;
    }
    redovi.push_back(move(novi));
}
vector < int > lg(n + 1);
for (int i = 2; i <= n; i++) lg [i] = lg [i / 2] + 1;
while (q--) {
    int l1, r1, l2, r2;
    cin >> l1 >> r1 >> l2 >> r2;
    -- l1;
    -- r1;
    -- l2;
    -- r2;
    int nivo = lg [r1 - l1 + 1], p = 1 << nivo;
    const auto & a = redovi [nivo];
    bool isti = a [l1] == a [l2] && a [r1 - p + 1] == a [r2 - p + 1];
    cout << (isti ? "DA" : "NE") << '\n';
}
}

```

Važne provjere

Oznake se porede samo unutar istog nivoa, odnosno za istu dužinu bloka.

Završni blok uključivog intervala $[l, r]$ počinje na $r - p + 1$.

Nije dovoljno provjeriti samo početni blok kada L nije stepen dvojke.

ZADATAK 101

Sljedeći veći element

Cilj: Monotonim stekom nalazi prvi strogo veći element desno za svaki indeks.

Za svaki element niza a_i odredi vrijednost prvog elementa desno od i koji je strogo veći od a_i . Ako takav ne postoji, ispiši -1.

Ulaz. N , zatim N cijelih brojeva.

Izlaz. N odgovora u jednom redu.

Ograničenja. $N \leq 500000$.

Primjer**ULAZ**

```
6
2 1 2 4 3 5
```

IZLAZ

```
4 2 4 5 5 -1
```

Razumijevanje i postupak

Stek čuva indekse čiji prvi veći element još nije pronađen. Kada dođe $x=a[i]$, dok je x veći od vrijednosti na vrhu, upravo x je prvi veći za taj indeks pa ga skidamo i upisujemo odgovor.

1. $ans=-1$ za sve; st prazan.
2. Za $i=0..N-1$: dok st i $a[i]>a[st[-1]]$, odgovori tom indeksu i pop .
3. Push i .

Zašto postupak daje tačan rezultat?

Element se stavlja na stek jednom i skida najviše jednom. Kada ga novi element skine, taj novi element je prvi desno koji zadovoljava uslov jer su svi između već bili nepovoljni.

Složenost. Vrijeme $O(N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
ans = [-1] * n
st = []
for i, x in enumerate(a):
    while st and x > a[st[-1]]:
        ans[st.pop()] = x
    st.append(i)
print(*ans)
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > a(n), ans(n, - 1);
    for (auto & x : a) cin >> x;
    vector< int > st;
    for (int i = 0; i < n; i++) {
        while (! st.empty() && a [i] > a [st.back()]) {
            ans [st.back()] = a [i];
            st.pop_back();
        }
        st.push_back(i);
    }
    for (int i = 0; i < n; i++) cout << ans [i] << (i + 1 == n ? '\n' : ' ');
}

```

Važne provjere

Uslov je >, ne >=.

Stek čuva indekse, jer nam treba pristup vrijednosti i poziciji odgovora.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 2000	7
2	Svi elementi različiti	5
3	Puna ograničenja	13

ZADATAK 102

Maksimum kliznog prozora

Cilj: Monotonim deque-om računa maksimum svakog prozora dužine K u linearnom vremenu.

Dat je niz od N brojeva i K . Za svaki uzastopni segment dužine K ispiši njegov maksimum.

Ulaz. N K , zatim N cijelih brojeva.

Izlaz. $N-K+1$ maksimuma.

Ograničenja. $1 \leq K \leq N \leq 500000$.

Primjer

ULAZ

```
8 3
1 3 -1 -3 5 3 6 7
```

IZLAZ

```
3 3 5 5 6 7
```

Razumijevanje i postupak

Deque čuva indekse kandidata u opadajućem redoslijedu vrijednosti. Prvo izbacimo indekse koji su izašli iz prozora; zatim sa kraja izbacimo vrijednosti $\leq$ novoj jer novi indeks traje duže i nije slabiji. Prednji je maksimum.

1. Za svaki i : izbaci $\text{front} < i-K+1$.
2. Dok deque i $a[\text{back}] \leq a[i]$, pop back.
3. Push i .
4. Kada $i \geq K-1$, odgovor je $a[\text{front}]$.

Zašto postupak daje tačan rezultat?

Prednji indeks je uvijek najveća vrijednost u trenutnom prozoru, a uklanjanje slabijeg kandidata je sigurno jer novi kandidat traje barem jednako dugo i nije manji.

Složenost. Vrijeme $O(N)$, memorija $O(K)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
n, k = map(int, input().split())
a = list(map(int, input().split()))
d = deque()
ans = []
for i, x in enumerate(a):
    while d and d[0] < i - k + 1:
        d.popleft()
```

PYTHON 3 • potpuno rješenje / nastavak

```

while d and a[d[-1]] <= x:
    d.pop()
d.append(i)
if i >= k - 1:
    ans.append(a[d[0]])
print(*ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, k;
    cin >> n >> k;
    vector < long long > a(n);
    for (auto & x : a) cin >> x;
    deque < int > d;
    for (int i = 0; i < n; i++) {
        while (! d.empty() && d.front() < i - k + 1) d.pop_front();
        while (! d.empty() && a [d.back()] <= a [i]) d.pop_back();
        d.push_back(i);
        if (i >= k - 1) cout << a [d.front()] << (i == n - 1 ? '\n' : ' ');
    }
}

```

Važne provjere

Indekse van prozora izbacivati prije čitanja maksimuma.

Deque mora čuvati indekse, ne samo vrijednosti.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 2000	7
2	K ≤ 2	5
3	Puna ograničenja	18

21. Grafovi, povezivanje i najkraći putevi

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
103	Povezane škole
104	Najkraći hodnik
105	Najbliži izlaz
106	Dva tima
107	Najjeftinija ruta
108	Mostovi u mreži
109	Redoslijed preduslova
110	Da li zavisnosti sadrže ciklus?
111	Komponente nakon spajanja
112	Najjeftinija mreža kablova
113	Putevi cijene 0 ili 1
114	Najbliža stanica svakoj tački
115	Vrati najkraći put
116	Može li graf u dvije ekipe?
117	Najduži lanac u DAG-u
118	Svi parovi najkraćih puteva
119	Kritične raskrsnice
120	Kablovi sa obaveznim vezama
121	Broj najkraćih puteva
122	Koliko parova razdvaja most?
123	Najjeftiniji put sa jednim popustom

Zadatak	Naziv
124	Put sa besplatnim portalima
125	Najjeftiniji put između gradova
126	Redoslijed predmeta
127	Podjela u dvije ekipe
128	Koliko zatvorenih regija?
129	Jedna besplatna karta
130	Online prijateljstva
131	Minimalna mreža kablova
132	Broj komponenti poslije svake veze
133	Tačka sastanka
134	Prag povezivanja
135	Najduži put kroz plan

ZADATAK 103

Povezane škole

Cilj: Prebrojati povezane komponente u neusmjerenom grafu pomoću DFS/BFS.

U mreži je N škola. Između nekih parova postoji direktna saradnja, ukupno M veza. Saradnja se može prenositi preko više škola. Koliko odvojenih grupa škola postoji?

Ulaz. U prvom redu N M . U narednih M redova dva broja u i v koji označavaju neusmjerenu vezu.

Izlaz. Ispisati broj povezanih komponenti.

Ograničenja. 1 $N, M \leq 200000$; nema višestrukih veza ni petlji.

Primjer

ULAZ

```
6 3
1 2
2 3
5 6
```

IZLAZ

```
3
```

Razumijevanje i postupak

Svaki put kada pronađemo neposjećen čvor, on započinje novu povezanu komponentu. Jednim BFS/DFS obilaskom označimo sve čvorove koji su s njim povezani.

1. Napraviti listu susjedstva.
2. `visited=false` za sve.
3. Za svaki čvor: ako nije posjećen, povećati broj komponenti i pokrenuti BFS/DFS.

Složenost. $O(N+M)$ vrijeme i $O(N+M)$ memorija.

PYTHON 3 • potpuno rješenje

```
from collections import deque
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
vis = [False] * n
comp = 0
```

PYTHON 3 • potpuno rješenje / nastavak

```

for s in range(n):
    if vis[s]:
        continue
    comp += 1
    vis[s] = True
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if not vis[v]:
                vis[v] = True
                q.append(v)
print(comp)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < int >> g(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back(v);
        g [v].push_back(u);
    }
    vector < char > vis(n, 0);
    int comp = 0;
    for (int s = 0; s < n; s++) if (! vis [s]) {
        comp++;
        queue < int > q;
        q.push(s);
        vis [s] = 1;
        while (! q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g [u]) if (! vis [v]) vis [v] = 1, q.push(v);
        }
    }
    cout << comp << "\n";
}

```

ZADATAK 104

Najkraći hodnik

Cilj: Naći najkraći broj ivica između dva čvora u neponderisanom grafu.

Školski kompleks ima N prostorija i M dvosmjernih hodnika. Svaki prolazak kroz jedan hodnik računa se kao jedan korak. Odredi najmanji broj hodnika od prostorije S do T ili -1 ako put ne postoji.

Ulaz. U prvom redu N M S T , zatim M veza u v .

Izlaz. Ispisati najkraći broj hodnika ili -1.

Ograničenja. 1 N, M 200000.

Primjer

ULAZ

```
5 5 1 5
1 2
2 3
3 5
1 4
4 5
```

IZLAZ

```
2
```

Razumijevanje i postupak

BFS je tačan jer sve ivice imaju jednaku težinu 1. Kada prvi put posjetimo čvor, njegova distanca je minimalna.

1. Izgraditi listu susjedstva.
2. $\text{dist}[S]=0$, S staviti u red.
3. Za svaki neobiđeni susjed v postaviti $\text{dist}[v]=\text{dist}[u]+1$.
4. Ispisati $\text{dist}[T]$.

Složenost. $O(N+M)$ vrijeme i memorija.

PYTHON 3 • potpuno rješenje

```
from collections import deque
n, m, s, t = map(int, input().split())
s -= 1
t -= 1
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
```

PYTHON 3 • potpuno rješenje / nastavak

```

    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
d = [-1] * n
d[s] = 0
q = deque([s])
while q:
    u = q.popleft()
    for v in g[u]:
        if d[v] == -1:
            d[v] = d[u] + 1
            q.append(v)
print(d[t])

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;
int main() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    -- s;
    -- t;
    vector < vector < int >> g(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back(v);
        g [v].push_back(u);
    }
    vector < int > d(n, - 1);
    queue < int > q;
    d [s] = 0;
    q.push(s);
    while (! q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g [u]) if (d [v] == - 1) {
            d [v] = d [u] + 1;
            q.push(v);
        }
    }
    cout << d [t] << "\n";
}

```


ZADATAK 105

Najbliži izlaz

Cilj: Pokrenuti BFS istovremeno iz više izvora.

U mreži $R \times C$ postoje slobodna polja ., zidovi # i više izlaza E. Za svako slobodno polje treba odrediti udaljenost do najbližeg izlaza. Za potrebe takmičenja traži se samo najveća od tih minimalnih udaljenosti; ako neko slobodno polje ne može doći ni do jednog izlaza, ispisati -1.

Ulaz. U prvom redu R C, zatim R redova mreže.

Izlaz. Ispisati najveću udaljenost do najbližeg izlaza ili -1 ako postoji nedostupno slobodno polje.

Ograničenja. 1 $R, C \leq 1000$; postoji barem jedan E.

Primjer

ULAZ

```
3 5
E...
.###.
....E
```

IZLAZ

```
3
```

Razumijevanje i postupak

Umjesto BFS-a iz svakog polja, sve izlaze stavimo u red na početku sa distancom 0. BFS zatim širi „talas“ i svakom polju dodjeljuje udaljenost do najbližeg izlaza. Ovo je ekvivalentno pokretanju svih BFS-ova paralelno, ali linearno.

1. Sve E staviti u red i $dist=0$.
2. Pokrenuti BFS kroz sva nezidna polja.
3. Ako neko . ostane na -1, odgovor je -1.
4. Inače uzeti maksimum svih distanci.

Složenost. $O(R \cdot C)$ vrijeme i memorija.

PYTHON 3 • potpuno rješenje

```

from collections import deque
r, c = map(int, input().split())
g = [input().strip() for _ in range(r)]
d = [[-1] * c for _ in range(r)]
q = deque()
for i in range(r):
    for j in range(c):
        if g[i][j] == 'E':
            d[i][j] = 0
            q.append((i, j))
for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
    pass
while q:
    x, y = q.popleft()
    for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        nx, ny = (x + dx, y + dy)
        if 0 <= nx < r and 0 <= ny < c and (g[nx][ny] != '#') and \
            (d[nx][ny] == -1):
            d[nx][ny] = d[x][y] + 1
            q.append((nx, ny))
ans = 0
for i in range(r):
    for j in range(c):
        if g[i][j] != '#':
            if d[i][j] == -1:
                print(-1)
                raise SystemExit
            ans = max(ans, d[i][j])
print(ans)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <algorithm>
using namespace std;
int main() {
    int R, C;
    cin >> R >> C;
    vector < string > g(R);
    for (auto & s : g) cin >> s;
    vector < vector < int >> d(R, vector < int > (C, - 1));
    queue < pair < int, int >> q;
    for (int i = 0; i < R; i++) for (int j = 0; j < C; j++) if (g [i] [j]
        == 'E') {
        d [i] [j] = 0;
        q.push({
            i, j
        });
    }
}

```

C++17 • potpuno rješenje / nastavak

```

int dx [4] = {
    1, - 1, 0, 0
}
, dy [4] = {
    0, 0, 1, - 1
}
;
while (! q.empty()) {
    auto [x, y] = q.front();
    q.pop();
    for (int k = 0; k < 4; k++) {
        int nx = x + dx [k], ny = y + dy [k];
        if (nx >= 0 && nx < R && ny >= 0 && ny < C && g [nx] [ny] !=
            '#' && d
                [nx] [ny] == - 1) {
            d [nx] [ny] = d [x] [y] + 1;
            q.push({
                nx, ny
            });
        }
    }
}
int ans = 0;
for (int i = 0; i < R; i++) for (int j = 0; j < C; j++) if (g [i] [j]
    != '#') {
    if (d [i] [j] == - 1) {
        cout << - 1 << "\n";
        return 0;
    }
    ans = max(ans, d [i] [j]);
}
cout << ans << "\n";
}

```

ZADATAK 106

Dva tima

Cilj: Provjeriti može li se graf obojiti sa dvije boje.

N učenika imaju M parova koji ne smiju biti u istoj ekipi. Treba ih podijeliti u dvije ekipe tako da svaki takav par bude u različitim ekipama. Odredi je li podjela moguća.

Ulaz. U prvom redu N M, zatim M parova u v.

Izlaz. Ispisati DA ako je moguća, inače NE.

Ograničenja. 1 N,M 200000.

Primjer

ULAZ

```
5 4
1 2
2 3
3 4
4 5
```

IZLAZ

```
DA
```

Razumijevanje i postupak

Graf je bipartitan ako se njegovi čvorovi mogu obojiti sa dvije boje tako da svaka ivica spaja različite boje. BFS/DFS dodjeljuje susjedu suprotnu boju; sukob nastaje kada dva susjedna čvora dobiju istu boju.

1. color=-1 za sve.
2. Za svaku komponentu pokrenuti BFS i obojiti početak sa 0.
3. Susjed dobija 1-color[u].
4. Ako susjed već ima istu boju kao u, odgovor NE.

Složenost. $O(N+M)$ vrijeme i memorija.

PYTHON 3 • potpuno rješenje

```
from collections import deque
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
```

PYTHON 3 • potpuno rješenje / nastavak

```

    g[v].append(u)
color = [-1] * n
for s in range(n):
    if color[s] != -1:
        continue
    color[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if color[v] == -1:
                color[v] = 1 - color[u]
                q.append(v)
            elif color[v] == color[u]:
                print('NE')
                raise SystemExit
print('DA')

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < int >> g(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back(v);
        g [v].push_back(u);
    }
    vector < int > c(n, - 1);
    for (int s = 0; s < n; s++) if (c [s] == - 1) {
        queue < int > q;
        q.push(s);
        c [s] = 0;
        while (! q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g [u]) {
                if (c [v] == - 1) c [v] = 1 - c [u], q.push(v);
                else if (c [v] == c [u]) {
                    cout << "NE\n";
                    return 0;
                }
            }
        }
    }
}

```

C++17 • potpuno rješenje / nastavak

```
        }  
    }  
}  
cout << "DA\n";  
}
```

ZADATAK 107

Najjeftinija ruta

Cilj: Naći najkraći put u grafu sa nenegativnim težinama.

Između N mjesta postoji M dvosmjernih puteva. Svaki put ima trošak w . Odredi najmanji trošak od mjesta S do T ili -1 ako nije povezano.

Ulaz. U prvom redu N M S T . U narednih M redova u v w .

Izlaz. Ispisati minimalan trošak ili -1 .

Ograničenja. $1 \leq N, M \leq 200000$, $0 \leq w \leq 10^9$.

Primjer

ULAZ

```
4 5 1 4
1 2 5
2 4 4
1 3 2
3 4 20
2 3 1
```

IZLAZ

```
7
```

Razumijevanje i postupak

BFS više nije dovoljan jer putevi nemaju jednaku cijenu. Dijkstrin algoritam uvijek obrađuje trenutno najbliži nefinalizirani čvor. Prioritetni red omogućava $O((N+M) \log N)$. Kada iz reda izvučemo zastarjelu veću distancu, preskačemo je.

1. $\text{dist}[S]=0$, ostalo beskonačno.
2. U min-priority queue ubaciti $(0, S)$.
3. Za svaku ivicu pokušati relaksaciju $\text{dist}[v] > \text{dist}[u] + w$.
4. Ispisati $\text{dist}[T]$ ili -1 .

Složenost. $O((N+M) \log N)$ vrijeme i $O(N+M)$ memorija.

PYTHON 3 • potpuno rješenje

```
import heapq
n, m, s, t = map(int, input().split())
s -= 1
t -= 1
g = [[] for _ in range(n)]
for _ in range(m):
    u, v, w = map(int, input().split())
```

PYTHON 3 • potpuno rješenje / nastavak

```

    u -= 1
    v -= 1
    g[u].append((v, w))
    g[v].append((u, w))
INF = 10 ** 30
d = [INF] * n
d[s] = 0
pq = [(0, s)]
while pq:
    du, u = heapq.heappop(pq)
    if du != d[u]:
        continue
    for v, w in g[u]:
        nd = du + w
        if nd < d[v]:
            d[v] = nd
            heapq.heappush(pq, (nd, v))
print(-1 if d[t] == INF else d[t])

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <queue>
#include <tuple>
#include <limits>
using namespace std;
int main() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    -- s;
    -- t;
    vector < vector < pair < int, long long >> > g(n);
    while (m--) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        -- u;
        -- v;
        g[u].push_back({
            v, w
        });
        g[v].push_back({
            u, w
        });
    }
    const long long INF = numeric_limits < long long >::max() / 4;
    vector < long long > d(n, INF);
    d[s] = 0;
    priority_queue < pair < long long, int >, vector < pair < long long,
        int >>,

```


C++17 • potpuno rješenje / nastavak

```
        greater < pair < long long, int >> > pq;
    pq.push({
        0, s
    });
    while (! pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != d [u]) continue;
        for (auto [v, w] : g [u]) if (d [v] > du + w) {
            d [v] = du + w;
            pq.push({
                d [v], v
            });
        }
    }
    cout << (d [t] == INF ? - 1 : d [t]) << "\n";
}
```

ZADATAK 108

Mostovi u mreži

Cilj: Prepoznati mostove u neusmjerenom grafu pomoću vremena ulaska i low vrijednosti.

U mreži N računara postoji M dvosmjernih veza. Veza je kritična ako njenim uklanjanjem broj povezanih komponenti mreže poraste. Odredi broj kritičnih veza.

Ulaz. U prvom redu N M , zatim M parova u v .

Izlaz. Ispisati broj mostova.

Ograničenja. 1 $N, M \leq 200000$; nema petlji ni višestrukih ivica.

Primjer**ULAZ**

```
5 5
1 2
2 3
3 1
3 4
4 5
```

IZLAZ

```
2
```

Razumijevanje i postupak

Tokom DFS-a $tin[u]$ je vrijeme ulaska u u , a $low[u]$ najmanji tin dostupan iz podstabla u koristeći nula ili više stabloskih i najviše jednu povratnu ivicu. Za DFS ivicu $u-v$, ako $low[v] > tin[u]$, iz podstabla v ne postoji povratak do u ili nekog njegovog pretka bez te ivice, pa je $u-v$ most.

1. Pokrenuti DFS iz svake komponente.
2. Postaviti $tin[u]=low[u]=timer++$.
3. Za povratnu ivicu ažurirati $low[u]$ sa $tin[v]$.
4. Poslije DFS djeteta v : $low[u]=\min(low[u], low[v])$; ako $low[v] > tin[u]$, povećati odgovor.

Složenost. $O(N+M)$ vrijeme i $O(N+M)$ memorija.

PYTHON 3 • potpuno rješenje

```
import sys
sys.setrecursionlimit(1000000)
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
```

PYTHON 3 • potpuno rješenje / nastavak

```

    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
tin = [-1] * n
low = [0] * n
timer = 0
bridges = 0

def dfs(u, p):
    global timer, bridges
    tin[u] = low[u] = timer
    timer += 1
    for v in g[u]:
        if v == p:
            continue
        if tin[v] != -1:
            low[u] = min(low[u], tin[v])
        else:
            dfs(v, u)
            low[u] = min(low[u], low[v])
            if low[v] > tin[u]:
                bridges += 1
for i in range(n):
    if tin[i] == -1:
        dfs(i, -1)
print(bridges)

```

C++17 • potpuno rješenje

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
vector < vector < int >> g;
vector < int > tin, low;
int timer_ = 0;
long long bridges = 0;
void dfs(int u, int p) {
    tin [u] = low [u] = timer_++;
    for (int v : g [u]) {
        if (v == p) continue;
        if (tin [v] != - 1) low [u] = min(low [u], tin [v]);
        else {
            dfs(v, u);
            low [u] = min(low [u], low [v]);
            if (low [v] > tin [u]) bridges++;
        }
    }
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

```

C++17 • potpuno rješenje / nastavak

```
int n, m;
cin >> n >> m;
g.assign(n, {
})
);
tin.assign(n, - 1);
low.resize(n);
while (m--) {
    int u, v;
    cin >> u >> v;
    -- u;
    -- v;
    g [u].push_back(v);
    g [v].push_back(u);
}
for (int i = 0; i < n; i++) if (tin [i] == - 1) dfs(i, - 1);
cout << bridges << "\n";
}
```

ZADATAK 109

Redoslijed preduslova

Cilj: Učenik primjenjuje topološko sortiranje i prioritetni red za leksikografski najmanji valjan poredak.

Postoji N tema i M usmjerenih zavisnosti $a \rightarrow b$, što znači da se tema a mora obraditi prije teme b . Pretpostavlja se da ciklusa nema. Ispiši leksikografski najmanji mogući redoslijed.

Ulaz. Prvi red N M , zatim M redova a b .

Izlaz. Ispisati N brojeva - redoslijed tema.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
4 4
1 2
1 3
2 4
3 4
```

IZLAZ

```
1 2 3 4
```

Razumijevanje i postupak

Kahnov algoritam drži sve čvorove ulaznog stepena 0. Da bi redoslijed bio leksikografski najmanji, među trenutno dostupnim čvorovima uvijek uzimamo najmanji.

1. Izračunaj indegree svakog čvora.
2. Sve čvorove sa indegree 0 stavi u min-heap.
3. Vadi najmanji u , dodaj ga rezultatu i smanji indegree susjeda.
4. Susjed koji padne na 0 ulazi u heap.

Složenost. Vrijeme $O((N+M) \log N)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
import heapq
N, M = map(int, input().split())
g = [[] for _ in range(N)]
ind = [0] * N
for _ in range(M):
    a, b = map(int, input().split())
    a -= 1
    b -= 1
    g[a].append(b)
    ind[b] += 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

h = [i for i in range(N) if ind[i] == 0]
heapq.heapify(h)
order = []
while h:
    u = heapq.heappop(h)
    order.append(u + 1)
    for v in g[u]:
        ind[v] -= 1
        if ind[v] == 0:
            heapq.heappush(h, v)
print(*order)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M;
    cin >> N >> M;
    vector < vector < int >> g(N);
    vector < int > ind(N);
    while (M--) {
        int a, b;
        cin >> a >> b;
        -- a;
        -- b;
        g[a].push_back(b);
        ++ ind[b];
    }
    priority_queue < int, vector < int >, greater < int >> pq;
    for (int i = 0; i < N; ++ i) if (! ind[i]) pq.push(i);
    vector < int > ord;
    while (! pq.empty()) {
        int u = pq.top();
        pq.pop();
        ord.push_back(u + 1);
        for (int v : g[u]) if (-- ind[v] == 0) pq.push(v);
    }
    for (int i = 0; i < N; ++ i) {
        if (i) cout << ' ';
        cout << ord[i];
    }
    cout << "\n";
}

```

Važne provjere

Obični red daje neki topološki poredak, ali ne nužno leksikografski najmanji.

Smjer ivice $a \rightarrow b$ mora odgovarati “a prije b”.

ZADATAK 110

Da li zavisnosti sadrže ciklus?

Cilj: Učenik koristi DFS bojenje 0/1/2 za detekciju ciklusa u usmjerenom grafu.

Za usmjereni graf odredi postoji li usmjereni ciklus.

Ulaz. Prvi red N M, zatim M usmjerenih ivica a b.

Izlaz. Ispisati DA ako ciklus postoji, inače NE.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
4 4
1 2
2 3
3 1
3 4
```

IZLAZ

```
DA
```

Razumijevanje i postupak

Boja 0 znači neobrađen, 1 znači čvor je trenutno na rekurzivnom putu, a 2 znači potpuno završen. Ivica prema čvoru boje 1 zatvara ciklus.

1. Pokreni DFS iz svakog neobrađenog čvora.
2. Na ulazu postavi boju 1.
3. Ako vidiš susjeda boje 1, ciklus postoji.
4. Po završetku postavi boju 2.

Složenost. Vrijeme $O(N+M)$; memorija $O(N+M)$, uz rekurzivni stek $O(N)$.

PYTHON 3 • potpuno rješenje

```
import sys
sys.setrecursionlimit(1000000)
N, M = map(int, input().split())
g = [[] for _ in range(N)]
for _ in range(M):
    a, b = map(int, input().split())
    g[a - 1].append(b - 1)
color = [0] * N

def dfs(u):
    color[u] = 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

    for v in g[u]:
        if color[v] == 1:
            return True
        if color[v] == 0 and dfs(v):
            return True
    color[u] = 2
    return False
print('DA' if any((color[i] == 0 and dfs(i) for i in range(N))) else 'NE')

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
vector < vector < int >> g;
vector < int > color;
bool dfs(int u) {
    color [u] = 1;
    for (int v : g [u]) {
        if (color [v] == 1) return true;
        if (color [v] == 0 && dfs(v)) return true;
    }
    color [u] = 2;
    return false;
}
int main() {
    int N, M;
    cin >> N >> M;
    g.resize(N);
    color.assign(N, 0);
    while (M--) {
        int a, b;
        cin >> a >> b;
        g [a - 1].push_back(b - 1);
    }
    for (int i = 0; i < N; ++ i) if (! color [i] && dfs(i)) {
        cout << "DA\n";
        return 0;
    }
    cout << "NE\n";
}

```

Važne provjere

U neusmjerenom grafu pravilo je drugačije zbog roditeljske ivice.

Boju 2 postaviti tek nakon obrade svih potomaka.

ZADATAK 111

Komponente nakon spajanja

Cilj: Učenik koristi DSU (union-find) sa kompresijom putanje i spajanjem po veličini.

N računara je na početku nepovezano. Nakon M dvosmjernih veza odredi broj povezanih komponenti i veličinu najveće komponente.

Ulaz. Prvi red N M, zatim M parova a b.

Izlaz. Ispisati: broj_komponenti najveća_veličina.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
6 3
1 2
2 3
4 5
```

IZLAZ

```
3 3
```

Razumijevanje i postupak

DSU ne čuva cijeli graf. Svaki skup ima predstavnika. find pronalazi predstavnika, a union spaja dva različita skupa. Kompresija putanje i spajanje manjeg pod veći daju gotovo konstantno vrijeme.

1. $\text{parent}[i]=i$, $\text{size}[i]=1$.
2. Za svaku vezu nađi korijene.
3. Ako su različiti, spoji manji skup pod veći, smanji broj komponenti i ažuriraj najveću veličinu.

Složenost. Amortizirano $O((N+M) \alpha(N))$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
N, M = map(int, input().split())
parent = list(range(N))
size = [1] * N

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x
comp, best = (N, 1)
for _ in range(M):
```

PYTHON 3 • potpuno rješenje / nastavak

```

a, b = map(int, input().split())
a -= 1
b -= 1
a, b = (find(a), find(b))
if a != b:
    if size[a] < size[b]:
        a, b = (b, a)
    parent[b] = a
    size[a] += size[b]
    comp -= 1
    best = max(best, size[a])
print(comp, best)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector< int > p, sz;
    DSU(int n) : p(n), sz(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int f(int x) {
        return p[x] == x ? x : p[x] = f(p[x]);
    }
    int unite(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b) return sz[a];
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return sz[a];
    }
};
int main() {
    int N, M;
    cin >> N >> M;
    DSU d(N);
    int comp = N, best = 1;
    while (M--) {
        int a, b;
        cin >> a >> b;
        -- a;
        -- b;
        int ra = d.f(a), rb = d.f(b);
        if (ra != rb) {
            best = max(best, d.unite(ra, rb));
        }
    }
}

```

C++17 • potpuno rješenje / nastavak

```
        -- comp;
    }
}
cout << comp << " " << best << "\n";
}
```

Važne provjere

Ako su čvorovi već u istoj komponenti, broj komponenti se ne smanjuje.

Veličina se ažurira samo pri stvarnom spajanju.

ZADATAK 112

Najjeftinija mreža kablova

Cilj: Učenik primjenjuje Kruskalov algoritam i DSU za minimalno razapinjuće stablo.

N škola treba povezati kablovima. Postoji M mogućih veza sa cijenama. Pretpostavi da je graf povezan. Odredi minimalnu ukupnu cijenu da sve škole budu međusobno povezane.

Ulaz. Prvi red N M. Zatim M redova a b w.

Izlaz. Ispisati minimalnu ukupnu cijenu.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$, $0 \leq w \leq 10^9$.

Primjer

ULAZ

```
4 5
1 2 3
1 3 5
2 3 1
2 4 6
3 4 2
```

IZLAZ

```
6
```

Razumijevanje i postupak

Kruskal sortira ivice po težini i uzima najjeftiniju ivicu koja ne pravi ciklus. DSU brzo provjerava da li su krajevi već povezani.

1. Sortiraj ivice po w.
2. Za svaku ivicu, ako su krajevi u različitim DSU skupovima, uzmi je.
3. Dodaj cijenu i spoji skupove.
4. Zaustavi se nakon N-1 uzetih ivica.

Složenost. Vrijeme $O(M \log M)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
N, M = map(int, input().split())
edges = []
for _ in range(M):
    a, b, w = map(int, input().split())
    edges.append((w, a - 1, b - 1))
edges.sort()
p = list(range(N))
sz = [1] * N
```

PYTHON 3 • potpuno rješenje / nastavak

```

def find(x):
    if p[x] != x:
        p[x] = find(p[x])
    return p[x]
ans = used = 0
for w, a, b in edges:
    a, b = (find(a), find(b))
    if a == b:
        continue
    if sz[a] < sz[b]:
        a, b = (b, a)
    p[b] = a
    sz[a] += sz[b]
    ans += w
    used += 1
    if used == N - 1:
        break
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct D {
    vector < int > p, s;
    D(int n) : p(n), s(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int f(int x) {
        return p [x] == x ? x : p [x] = f(p [x]);
    }
    bool u(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b) return false;
        if (s [a] < s [b]) swap(a, b);
        p [b] = a;
        s [a] += s [b];
        return true;
    }
};
;
int main() {
    int N, M;
    cin >> N >> M;
    vector < tuple < long long, int, int >> e;
    while (M--) {
        int a, b;
        long long w;
        cin >> a >> b >> w;
        e.push_back({
            w, a - 1, b - 1
        })
    }
}

```

C++17 • potpuno rješenje / nastavak

```
    );  
    }  
    sort(e.begin(), e.end());  
    D d(N);  
    long long ans = 0;  
    int used = 0;  
    for (auto [w, a, b] : e) if (d.u(a, b)) {  
        ans += w;  
        if (++used == N - 1) break;  
    }  
    cout << ans << "\n";  
}
```

Važne provjere

MST minimizira zbir izabranih ivica, ne pojedinačne najkraće puteve od jednog čvora.

Ne uzimati ivicu koja spaja već povezane komponente.

ZADATAK 113

Putevi cijene 0 ili 1

Cilj: Učenik primjenjuje 0-1 BFS sa deque strukturom.

U neusmjerenom grafu svaka ivica ima cijenu 0 ili 1. Odredi minimalnu cijenu puta od S do T.

Ulaz. Prvi red N M S T. Zatim M redova a b w, gdje je w 0 ili 1.

Izlaz. Ispisati najkraću cijenu ili -1 ako T nije dostupan.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
5 6 1 5
1 2 0
2 3 1
1 4 1
4 5 0
3 5 0
2 5 1
```

IZLAZ

```
1
```

Razumijevanje i postupak

Kod težina samo 0 i 1 nije potreban heap. Ako relaksiramo ivicu težine 0, novi čvor ide na početak deque-a; za težinu 1 ide na kraj. Time čuvamo red približno po rastućoj udaljenosti.

1. $\text{dist}[S]=0$, ostalo INF; S u deque.
2. Vadi s lijeve strane.
3. Ako $\text{dist}[u]+w$ poboljšava $\text{dist}[v]$, ažuriraj.
4. Za $w=0$ dodaj v lijevo, za $w=1$ desno.

Složenost. Vrijeme $O(N+M)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
N, M, S, T = map(int, input().split())
S -= 1
T -= 1
g = [[] for _ in range(N)]
for _ in range(M):
    a, b, w = map(int, input().split())
    a -= 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

    b -= 1
    g[a].append((b, w))
    g[b].append((a, w))
INF = 10 ** 30
d = [INF] * N
d[S] = 0
q = deque([S])
while q:
    u = q.popleft()
    for v, w in g[u]:
        if d[u] + w < d[v]:
            d[v] = d[u] + w
            (q.appendleft if w == 0 else q.append)(v)
print(-1 if d[T] == INF else d[T])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M, S, T;
    cin >> N >> M >> S >> T;
    -- S;
    -- T;
    vector < vector < pair < int, int >> > g(N);
    while (M--) {
        int a, b, w;
        cin >> a >> b >> w;
        -- a;
        -- b;
        g[a].push_back({
            b, w
        });
        g[b].push_back({
            a, w
        });
    }
    const long long INF = 4e18;
    vector < long long > d(N, INF);
    deque < int > q;
    d[S] = 0;
    q.push_back(S);
    while (! q.empty()) {
        int u = q.front();
        q.pop_front();
        for (auto [v, w] : g[u]) if (d[u] + w < d[v]) {
            d[v] = d[u] + w;
            if (w == 0) q.push_front(v);
        }
    }
}

```


C++17 • potpuno rješenje / nastavak

```
        else q.push_back(v);
    }
}
cout << (d [T] == INF ? - 1 : d [T]) << "\n";
}
```

Važne provjere

0-1 BFS važi samo kada su težine tačno 0 ili 1.

Deque nije obični queue: koristi se i push_front.

ZADATAK 114

Najbliža stanica svakoj tački

Cilj: Učenik pokreće multi-source BFS iz više izvora istovremeno.

U neusmjerenom neponderisanom grafu dato je K stanica. Za svaki čvor posmatra se udaljenost do najbliže stanice. Odredi najveću od tih udaljenosti. Pretpostavi da je svaki čvor dostižan iz barem jedne stanice.

Ulaz. Prvi red N M K . Drugi red K stanica. Zatim M ivica.

Izlaz. Ispisati maksimalnu udaljenost do najbliže stanice.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
6 5 2
1 6
1 2
2 3
3 4
4 5
5 6
```

IZLAZ

```
2
```

Razumijevanje i postupak

Ako sve izvore stavimo u BFS red sa udaljenošću 0, talasi se šire istovremeno. Prvi talas koji stigne do čvora nužno dolazi od najbližeg izvora.

1. $\text{dist} = -1$ za sve.
2. Sve stanice postavi na 0 i stavi u isti red.
3. Pokreni obični BFS.
4. Ispiši $\max(\text{dist})$.

Složenost. Vrijeme $O(N+M)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
N, M, K = map(int, input().split())
sources = [x - 1 for x in map(int, input().split())]
g = [[] for _ in range(N)]
for _ in range(M):
    a, b = map(int, input().split())
    a -= 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

    b -= 1
    g[a].append(b)
    g[b].append(a)
d = [-1] * N
q = deque()
for s in sources:
    if d[s] == -1:
        d[s] = 0
        q.append(s)
while q:
    u = q.popleft()
    for v in g[u]:
        if d[v] == -1:
            d[v] = d[u] + 1
            q.append(v)
print(max(d))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M, K;
    cin >> N >> M >> K;
    vector < vector < int >> g(N);
    vector < int > d(N, - 1);
    queue < int > q;
    for (int i = 0; i < K; ++ i) {
        int s;
        cin >> s;
        -- s;
        if (d [s] == - 1) {
            d [s] = 0;
            q.push(s);
        }
    }
    while (M--) {
        int a, b;
        cin >> a >> b;
        -- a;
        -- b;
        g [a].push_back(b);
        g [b].push_back(a);
    }
    while (! q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g [u]) if (d [v] == - 1) {
            d [v] = d [u] + 1;

```

C++17 • potpuno rješenje / nastavak

```
        q.push(v);
    }
}
cout << * max_element(d.begin(), d.end()) << "\n";
}
```

Važne provjere

Ne pokretati K odvojenih BFS-ova - to može biti presporo.

Svi izvori moraju u red prije širenja.

ZADATAK 115

Vrati najkraći put

Cilj: Učenik proširuje BFS tako da osim udaljenosti rekonstruiše i stvarnu putanju.

U neusmjerenom neponderisanom grafu pronađi najkraći put od S do T. Ako put postoji, ispiši broj ivica i zatim čvorove puta. Ako ne postoji, ispiši -1.

Ulaz. Prvi red N M S T. Zatim M ivica.

Izlaz. Ako postoji: prvi red udaljenost, drugi red put. Inače -1.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
5 5 1 5
1 2
2 5
1 3
3 4
4 5
```

IZLAZ

```
2
1 2 5
```

Razumijevanje i postupak

Kada BFS prvi put otkrije v iz u, postavimo $\text{parent}[v]=u$. Nakon što stignemo do T, pratimo roditelje unazad do S i okrenemo dobijenu listu.

1. $\text{dist}[S]=0$, $\text{parent}=-1$.
2. Pri prvom otkrivanju v postavi dist i parent.
3. Ako T ostane neotkriven, -1.
4. Inače prati parent od T do -1 i obrni put.

Složenost. Vrijeme $O(N+M)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
N, M, S, T = map(int, input().split())
S -= 1
T -= 1
g = [[] for _ in range(N)]
for _ in range(M):
    a, b = map(int, input().split())
    a -= 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

    b -= 1
    g[a].append(b)
    g[b].append(a)
d = [-1] * N
par = [-1] * N
q = deque([S])
d[S] = 0
while q:
    u = q.popleft()
    for v in g[u]:
        if d[v] == -1:
            d[v] = d[u] + 1
            par[v] = u
            q.append(v)
if d[T] == -1:
    print(-1)
else:
    path = []
    v = T
    while v != -1:
        path.append(v + 1)
        v = par[v]
    path.reverse()
    print(d[T])
    print(*path)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M, S, T;
    cin >> N >> M >> S >> T;
    -- S;
    -- T;
    vector < vector < int >> g(N);
    while (M--) {
        int a, b;
        cin >> a >> b;
        -- a;
        -- b;
        g[a].push_back(b);
        g[b].push_back(a);
    }
    vector < int > d(N, - 1), par(N, - 1);
    queue < int > q;
    d[S] = 0;
    q.push(S);
    while (! q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u]) if (d[v] == - 1) {
            d[v] = d[u] + 1;

```

C++17 • potpuno rješenje / nastavak

```

        par [v] = u;
        q.push(v);
    }
}
if (d [T] == - 1) {
    cout << - 1 << "\n";
    return 0;
}
vector < int > p;
for (int v = T; v != - 1; v = par [v]) p.push_back(v + 1);
reverse(p.begin(), p.end());
cout << d [T] << "\n";
for (int i = 0; i < (int) p.size(); ++ i) {
    if (i) cout << ' ';
    cout << p [i];
}
cout << "\n";
}

```

Važne provjere

Roditelj se postavlja samo pri prvom otkrivanju čvora.

Put se dobije unazad, pa ga treba obrnuti.

ZADATAK 116

Može li graf u dvije ekipe?

Cilj: Učenik testira bipartitnost BFS/DFS bojenjem.

N učenika povezano je M parova sukoba. Treba ih podijeliti u dvije ekipe tako da nijedan sukobljeni par ne bude u istoj ekipi. Odredi da li je to moguće.

Ulaz. Prvi red N M. Zatim M neusmjerenih ivica.

Izlaz. Ispisati DA ili NE.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
5 4
1 2
2 3
3 4
4 5
```

IZLAZ

```
DA
```

Razumijevanje i postupak

Graf je bipartitan ako se može obojiti sa dvije boje tako da svaka ivica spaja različite boje. Svaka nova komponenta počinje proizvoljnom bojom.

1. color=-1.
2. Za svaki neobojeni čvor pokreni BFS i dodijeli boju 0.
3. Susjed dobija 1-color[u].
4. Ako susjed već ima istu boju kao u, odgovor je NE.

Složenost. Vrijeme $O(N+M)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
N, M = map(int, input().split())
g = [[] for _ in range(N)]
for _ in range(M):
    a, b = map(int, input().split())
    a -= 1
    b -= 1
    g[a].append(b)
    g[b].append(a)
color = [-1] * N
```


PYTHON 3 • potpuno rješenje / nastavak

```

for s in range(N):
    if color[s] != -1:
        continue
    color[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if color[v] == -1:
                color[v] = color[u] ^ 1
                q.append(v)
            elif color[v] == color[u]:
                print('NE')
                raise SystemExit
print('DA')

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M;
    cin >> N >> M;
    vector < vector < int >> g(N);
    while (M--) {
        int a, b;
        cin >> a >> b;
        -- a;
        -- b;
        g [a].push_back(b);
        g [b].push_back(a);
    }
    vector < int > c(N, - 1);
    for (int s = 0; s < N; ++ s) if (c [s] == - 1) {
        queue < int > q;
        c [s] = 0;
        q.push(s);
        while (! q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g [u]) if (c [v] == - 1) {
                c [v] = c [u] ^ 1;
                q.push(v);
            }
            else if (c [v] == c [u]) {
                cout << "NE\n";
                return 0;
            }
        }
    }
    cout << "DA\n";
}

```

Važne provjere

Graf može biti nepovezan, zato treba pokrenuti BFS iz svake komponente.

Neparan ciklus onemogućava bipartitnost.

ZADATAK 117

Najduži lanac u DAG-u

Cilj: Učenik računa najduži put u usmjerenom acikličkom grafu dinamičkim programiranjem po topološkom poretku.

Za DAG odredi maksimalan broj ivica na usmjerenom putu.

Ulaz. Prvi red N M , zatim M ivica a b .

Izlaz. Ispisati dužinu najdužeg puta u broju ivica.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
5 5
1 2
1 3
2 4
3 4
4 5
```

IZLAZ

```
3
```

Razumijevanje i postupak

U topološkom poretku svi prethodnici čvora dolaze ranije. Zato kada obrađujemo u , njegova najbolja dužina je konačna i možemo relaksirati $dp[v] = \max(dp[v], dp[u] + 1)$.

1. Kahnovim algoritmom dobij topološki poredak.
2. dp inicijalno 0.
3. Za svaki u u poretku, za svaku ivicu $u \rightarrow v$ ažuriraj $dp[v]$.
4. Odgovor je $\max(dp)$.

Složenost. Vrijeme $O(N+M)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
N, M = map(int, input().split())
g = [[] for _ in range(N)]
ind = [0] * N
for _ in range(M):
    a, b = map(int, input().split())
    a -= 1
    b -= 1
    g[a].append(b)
    ind[b] += 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

q = deque((i for i in range(N) if ind[i] == 0))
dp = [0] * N
while q:
    u = q.popleft()
    for v in g[u]:
        dp[v] = max(dp[v], dp[u] + 1)
        ind[v] -= 1
        if ind[v] == 0:
            q.append(v)
print(max(dp))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M;
    cin >> N >> M;
    vector < vector < int >> g(N);
    vector < int > ind(N), dp(N);
    while (M--) {
        int a, b;
        cin >> a >> b;
        -- a;
        -- b;
        g [a].push_back(b);
        ++ ind [b];
    }
    queue < int > q;
    for (int i = 0; i < N; ++ i) if (! ind [i]) q.push(i);
    while (! q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g [u]) {
            dp [v] = max(dp [v], dp [u] + 1);
            if (-- ind [v] == 0) q.push(v);
        }
    }
    cout << * max_element(dp.begin(), dp.end()) << "\n";
}

```

Važne provjere

Ovaj jednostavni DP vrijedi zato što nema ciklusa.

Dužina se ovdje mjeri brojem ivica, ne brojem čvorova.

ZADATAK 118

Svi parovi najkraćih puteva

Cilj: Učenik primjenjuje Floyd-Warshall kada je N umjeren, a broj upita velik.

U ponderisanom neusmjerenom grafu treba odgovoriti na Q upita o najkraćoj udaljenosti između parova čvorova.

Ulaz. Prvi red N M Q . Zatim M ivica a b w . Zatim Q parova s t .

Izlaz. Za svaki upit ispisati udaljenost ili -1 ako put ne postoji.

Ograničenja. $1 \leq N \leq 400$, $0 \leq M \leq N(N-1)/2$, $Q \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
4 4 2
1 2 5
2 3 2
1 4 10
3 4 1
1 4
2 4
```

IZLAZ

```
8
3
```

Razumijevanje i postupak

Floyd-Warshall postepeno dozvoljava čvorove $1..k$ kao unutrašnje čvorove puta. Prijelaz je $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$. Nakon $O(N^3)$ predobrade svaki upit je $O(1)$.

1. $d[i][i] = 0$, ostalo INF ; ivice postavi na minimalnu težinu.
2. Za k , pa i , pa j pokušaj put preko k .
3. Odgovori direktno iz $d[s][t]$.

Složenost. Vrijeme $O(N^3 + Q)$; memorija $O(N^2)$.

PYTHON 3 • potpuno rješenje

```
N, M, Q = map(int, input().split())
INF = 10 ** 30
d = [[INF] * N for _ in range(N)]
for i in range(N):
    d[i][i] = 0
for _ in range(M):
    a, b, w = map(int, input().split())
    a -= 1
    b -= 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

    if w < d[a][b]:
        d[a][b] = d[b][a] = w
for k in range(N):
    for i in range(N):
        dik = d[i][k]
        if dik == INF:
            continue
        for j in range(N):
            if dik + d[k][j] < d[i][j]:
                d[i][j] = dik + d[k][j]
for _ in range(Q):
    s, t = map(int, input().split())
    x = d[s - 1][t - 1]
    print(-1 if x == INF else x)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M, Q;
    cin >> N >> M >> Q;
    const long long INF = 4e18;
    vector < vector < long long >> d(N, vector < long long > (N, INF));
    for (int i = 0; i < N; ++ i) d [i] [i] = 0;
    while (M--) {
        int a, b;
        long long w;
        cin >> a >> b >> w;
        -- a;
        -- b;
        d [a] [b] = d [b] [a] = min(d [a] [b], w);
    }
    for (int k = 0; k < N; ++ k) for (int i = 0; i < N; ++ i) if (d [i] [k] <
        INF) for
        (int j = 0; j < N; ++ j) if (d [k] [j] < INF) d [i] [j] = min(d [i]
            [j], d [i]
            [k] + d [k] [j]);
    while (Q--) {
        int s, t;
        cin >> s >> t;
        long long x = d [s - 1] [t - 1];
        cout << (x == INF ? - 1 : x) << "\n";
    }
}

```

Važne provjere

$O(N^3)$ je prihvatljivo samo za relativno mali N .

Kod više ivica između istih čvorova čuva se najmanja težina.

ZADATAK 119

Kritične raskrsnice

Cilj: Učenik proširuje low-link ideju sa mostova na artikulacijske tačke.

U neusmjerenom grafu čvor je kritičan ako njegovo uklanjanje (zajedno sa incidentnim ivicama) povećava broj povezanih komponenti. Odredi broj kritičnih čvorova.

Ulaz. Prvi red N M , zatim M ivica.

Izlaz. Ispisati broj artikulacijskih tačaka.

Ograničenja. $1 \leq N, M \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
5 4
1 2
2 3
2 4
4 5
```

IZLAZ

```
2
```

Razumijevanje i postupak

Za neroot čvor u , ako postoji DFS djetete v sa $low[v] \geq tin[u]$, podstablo v nema povratni put iznad u , pa uklanjanje u odvaja taj dio. Korijen DFS stabla je artikulacija samo ako ima najmanje dva DFS djeteta.

1. Pokreni DFS iz svake komponente.
2. Računaj tin i low kao kod mostova.
3. Za neroot u označi kritičnim ako $low[v] \geq tin[u]$ za neko djetete v .
4. Root je kritičan ako ima više od jednog DFS djeteta.

Složenost. Vrijeme $O(N+M)$; memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
import sys
sys.setrecursionlimit(1000000)
N, M = map(int, input().split())
g = [[] for _ in range(N)]
for _ in range(M):
    a, b = map(int, input().split())
    a -= 1
    b -= 1
    g[a].append(b)
```

PYTHON 3 • potpuno rješenje / nastavak

```

    g[b].append(a)
tin = [-1] * N
low = [0] * N
cut = [False] * N
timer = 0

def dfs(u, p=-1):
    global timer
    tin[u] = low[u] = timer
    timer += 1
    children = 0
    for v in g[u]:
        if v == p:
            continue
        if tin[v] != -1:
            low[u] = min(low[u], tin[v])
        else:
            dfs(v, u)
            low[u] = min(low[u], low[v])
            if p != -1 and low[v] >= tin[u]:
                cut[u] = True
            children += 1
    if p == -1 and children > 1:
        cut[u] = True
for i in range(N):
    if tin[i] == -1:
        dfs(i)
print(sum(cut))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
vector < vector < int >> g;
vector < int > tin, low;
vector < char > cut;
int timer = 0;
void dfs(int u, int p = - 1) {
    tin [u] = low [u] = timer++;
    int ch = 0;
    for (int v : g [u]) {
        if (v == p) continue;
        if (tin [v] != - 1) low [u] = min(low [u], tin [v]);
        else {
            dfs(v, u);
            low [u] = min(low [u], low [v]);
            if (p != - 1 && low [v] >= tin [u]) cut [u] = 1;
            ++ ch;
        }
    }
    if (p == - 1 && ch > 1) cut [u] = 1;
}
int main() {

```


C++17 • potpuno rješenje / nastavak

```
int N, M;
cin >> N >> M;
g.resize(N);
while (M--) {
    int a, b;
    cin >> a >> b;
    -- a;
    -- b;
    g[a].push_back(b);
    g[b].push_back(a);
}
tin.assign(N, - 1);
low.resize(N);
cut.assign(N, 0);
for (int i = 0; i < N; ++ i) if (tin[i] == - 1) dfs(i);
cout << count(cut.begin(), cut.end(), 1) << "\n";
}
```

Važne provjere

Root DFS stabla ima posebno pravilo.

Uslov za artikulaciju je $\geq$, dok je za most $\text{low}[v] > \text{tin}[u]$.

ZADATAK 120

Kablovi sa obaveznim vezama

Cilj: Kombinuje DSU i Kruskal uz unaprijed spojene komponente.

N računara treba povezati uz minimalan dodatni trošak. Dato je M mogućih kablova sa cijenama i K postojećih besplatnih veza koje su već izgrađene. Odredi minimalan dodatni trošak da cijela mreža postane povezana, ili -1 ako nije moguće.

Ulaz. N M K; zatim M redova u v w; zatim K redova u v.

Izlaz. Minimalan dodatni trošak ili -1.

Ograničenja. $N \leq 200000$, $M \leq 300000$.

Primjer

ULAZ

```
4 4 1
1 2 5
2 3 2
3 4 4
1 4 10
1 2
```

IZLAZ

```
6
```

Razumijevanje i postupak

Postojeće veze prvo besplatno spajamo u DSU-u. Zatim standardni Kruskal sortira samo kablove koji se plaćaju i uzima one koji spajaju različite komponente.

1. DSU inicijalizacija.
2. Union za K postojećih veza.
3. Sortiraj M kablova po cijeni.
4. Za svaku ivicu: ako union uspije, dodaj cijenu.
5. Na kraju provjeri da je ostala jedna komponenta.

Zašto postupak daje tačan rezultat?

Kruskalov exchange argument ostaje važeći nakon kontrakcije već povezanih komponenti. Besplatne postojeće veze možemo smatrati ivicama težine 0 koje su unaprijed odabrane.

Složenost. Vrijeme $O(M \log M + (N+M) \alpha(N))$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

class DSU:

    def __init__(self, n):
        self.p = list(range(n))
        self.sz = [1] * n

    def f(self, x):
        while x != self.p[x]:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x

    def u(self, a, b):
        a, b = (self.f(a), self.f(b))
        if a == b:
            return False
        if self.sz[a] < self.sz[b]:
            a, b = (b, a)
        self.p[b] = a
        self.sz[a] += self.sz[b]
        return True

N, M, K = map(int, input().split())
E = []
for _ in range(M):
    u, v, w = map(int, input().split())
    E.append((w, u - 1, v - 1))
d = DSU(N)
comps = N
for _ in range(K):
    u, v = map(int, input().split())
    comps -= d.u(u - 1, v - 1)
ans = 0
for w, u, v in sorted(E):
    if d.u(u, v):
        ans += w
        comps -= 1
print(ans if comps == 1 else -1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct D {
    vector<int> p, s;
    D(int n) : p(n), s(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int f(int x) {
        return p[x] == x ? x : p[x] = f(p[x]);
    }
    bool u(int a, int b) {
        a = f(a);
        b = f(b);

```

C++17 • potpuno rješenje / nastavak

```

        if (a == b) return false;
        if (s[a] < s[b]) swap(a, b);
        p[b] = a;
        s[a] += s[b];
        return true;
    }
}
;
int main() {
    int N, M, K;
    cin >> N >> M >> K;
    vector < tuple < long long, int, int >> e;
    while (M--) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        e.push_back({
            w, --u, --v
        });
    }
    D d(N);
    int comps = N;
    while (K--) {
        int u, v;
        cin >> u >> v;
        if (d.u(--u, --v))-- comps;
    }
    sort(e.begin(), e.end());
    long long ans = 0;
    for (auto [w, u, v] : e) if (d.u(u, v)) {
        ans += w;
        -- comps;
    }
    cout << (comps == 1 ? ans : - 1) << "\n";
}

```

Važne provjere

Postojeće veze ne dodaju cijenu.

Ne pretpostaviti da M kablova garantuje povezivost.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	K=0	8
2	N≤2000,M≤5000	7
3	Puna ograničenja	15

ZADATAK 121

Broj najkraćih puteva

Cilj: Uz BFS računa i broj najkraćih puteva u neusmjerenom neponderisanom grafu.

U neusmjerenom grafu od 1 do N odredi udaljenost od 1 do N i broj različitih najkraćih puteva modulo 1 000 000 007. Ako put ne postoji, ispiši -1 0.

Ulaz. N M, zatim M ivica.

Izlaz. dist broj_puteva.

Ograničenja. N,M≤300000.

Primjer

ULAZ

```
5 6
1 2
1 3
2 4
3 4
4 5
2 5
```

IZLAZ

```
2 1
```

Razumijevanje i postupak

BFS daje slojeve udaljenosti. $ways[v]$ je broj najkraćih puteva do v . Kada v vidimo prvi put, nasljeđuje $ways[u]$. Ako kasnije do v dolazimo sa iste prethodne udaljenosti $dist[u]+1=dist[v]$, dodajemo $ways[u]$.

1. $dist[1]=0, ways[1]=1$.
2. BFS.
3. Prvi dolazak: postavi $dist$ i $ways$.
4. Jednako kratak dodatni dolazak: saberi $ways$ modulo MOD.

Zašto postupak daje tačan rezultat?

Svaki najkraći put do v završava ivicom iz nekog u sa $dist[u]=dist[v]-1$. BFS obrađuje upravo te prethodnike, a $ways[v]$ sabira disjunktne izbore posljednje ivice.

Složenost. Vrijeme $O(N+M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
MOD = 1000000007
N, M = map(int, input().split())
g = [[] for _ in range(N)]
for _ in range(M):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
d = [-1] * N
w = [0] * N
d[0] = 0
w[0] = 1
q = deque([0])
while q:
    u = q.popleft()
    for v in g[u]:
        if d[v] == -1:
            d[v] = d[u] + 1
            w[v] = w[u]
            q.append(v)
        elif d[v] == d[u] + 1:
            w[v] = (w[v] + w[u]) % MOD
print(-1 if d[-1] < 0 else d[-1], w[-1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
int main() {
    int N, M;
    cin >> N >> M;
    vector < vector < int >> g(N);
    while (M--) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector < int > d(N, -1);
    vector < long long > w(N);
    queue < int > q;
    d[0] = 0;
    w[0] = 1;
    q.push(0);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u]) {

```

C++17 • potpuno rješenje / nastavak

```

        if (d [v] == - 1) {
            d [v] = d [u] + 1;
            w [v] = w [u];
            q.push(v);
        }
        else if (d [v] == d [u] + 1) w [v] = (w [v] + w [u]) % MOD;
    }
}
cout << (d [N - 1] < 0 ? - 1 : d [N - 1]) << ' ' << w [N - 1] << "\n";
}

```

Važne provjere

Ne sabirati puteve koji daju veću udaljenost.

ways se inicijalizuje samo na izvoru.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Stablo	5
2	N,M<=5000	5
3	Puna ograničenja	10

ZADATAK 122

Koliko parova razdvaja most?

Cilj: Kombinuje DFS low-link sa veličinama podstabala za doprinos svakog mosta.

Dat je povezan neusmjeren graf sa N čvorova. Za svaku ivicu koja je most odredi koliko neuređenih parova čvorova postaje nepovezano njenim uklanjanjem. Ispiši zbir tih vrijednosti preko svih mostova.

Ulaz. N M , zatim M ivica.

Izlaz. Zbir doprinosa svih mostova.

Ograničenja. $N \leq 200000$, $M \leq 300000$.

Primjer

ULAZ

```
5 5
1 2
2 3
3 4
4 5
3 5
```

IZLAZ

```
10
```

Razumijevanje i postupak

U DFS stablu, ako je ivica parent-v most, uklanjanje odvaja podstablo v veličine $sz[v]$ od ostatka $N-sz[v]$. Broj razdvojenih neuređenih parova je njihov proizvod.

1. DFS čuva tin , low i sz .
2. Za tree-edge $u-v$ nakon povratka ažuriraj $low[u]$ i $sz[u]$.
3. Ako $low[v] > tin[u]$, dodaj $sz[v] * (N - sz[v])$.
4. Za paralelne ivice identifikovati ivicu ID-em.

Zašto postupak daje tačan rezultat?

Low-link uslov tačno karakteriše most. Kada je ivica most, jedina veza između podstabla v i ostatka prolazi kroz nju, pa svaki izbor jednog čvora sa svake strane daje razdvojeni par, ukupno $sz * (N - sz)$.

Složenost. Vrijeme $O(N+M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

import sys
sys.setrecursionlimit(1000000)
N, M = map(int, input().split())
g = [[] for _ in range(N)]
for eid in range(M):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append((v, eid))
    g[v].append((u, eid))
tin = [-1] * N
low = [0] * N
sz = [0] * N
timer = 0
ans = 0

def dfs(u, pe=-1):
    global timer, ans
    tin[u] = low[u] = timer
    timer += 1
    sz[u] = 1
    for v, eid in g[u]:
        if eid == pe:
            continue
        if tin[v] != -1:
            low[u] = min(low[u], tin[v])
        else:
            dfs(v, eid)
            sz[u] += sz[v]
            low[u] = min(low[u], low[v])
            if low[v] > tin[u]:
                ans += sz[v] * (N - sz[v])
dfs(0)
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int N;
vector < vector < pair < int, int >> > g;
vector < int > tin, low, sz;
int tim = 0;
long long ans = 0;
void dfs(int u, int pe = - 1) {
    tin [u] = low [u] = tim++;
    sz [u] = 1;
    for (auto [v, id] : g [u]) {
        if (id == pe) continue;
        if (tin [v] != - 1) low [u] = min(low [u], tin [v]);
        else {
            dfs(v, id);
            sz [u] += sz [v];
        }
    }
}

```

C++17 • potpuno rješenje / nastavak

```

        low [u] = min(low [u], low [v]);
        if (low [v] > tin [u]) ans += 1LL * sz [v] * (N - sz [v]);
    }
}
int main() {
    int M;
    cin >> N >> M;
    g.assign(N, {
    });
    for (int id = 0; id < M; id++) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back({
            v, id
        });
        g [v].push_back({
            u, id
        });
    }
    tin.assign(N, - 1);
    low.resize(N);
    sz.resize(N);
    dfs(0);
    cout << ans << "\n";
}

```

Važne provjere

Kod neusmjerenog grafa preskače se samo parent edge ID, ne svaki čvor roditelj.

Proizvod traži 64-bitni tip.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Graf je stablo	7
2	$N \leq 3000$	6
3	Puna ograničenja	12

ZADATAK 123

Najjeftiniji put sa jednim popustom

Cilj: Proširuje Dijkstru stanjem iskorištenog popusta.

U usmjerenom grafu putujemo od 1 do N. Jednom tokom puta smijemo izabrati jednu ivicu cijene w i platiti $\text{floor}(w/2)$. Odredi minimalan ukupan trošak. Popust se može i ne mora iskoristiti.

Ulaz. N M, zatim M usmjerenih ivica u v w.

Izlaz. Minimalni trošak ili -1 ako N nije dostižan.

Ograničenja. $N \leq 200000, M \leq 300000, w \leq 1e9$.

Primjer

ULAZ

```
4 5
1 2 8
2 4 8
1 3 5
3 4 20
2 3 1
```

IZLAZ

```
12
```

Razumijevanje i postupak

Stanje je (čvor, used). Iz stanja $\text{used}=0$ preko svake ivice imamo dva prelaza: puna cijena i sniženje uz prelazak u $\text{used}=1$. Iz $\text{used}=1$ imamo samo punu cijenu.

1. $\text{dist}[N][2], \text{dist}[1][0]=0$.
2. Priority queue Dijkstra.
3. Relaksiraj normalni prelaz.
4. Ako $\text{used}=0$, relaksiraj i $w//2$ u drugi sloj.
5. Odgovor $\min(\text{dist}[N][0], \text{dist}[N][1])$.

Zašto postupak daje tačan rezultat?

Prošireni graf ima $2N$ stanja i nenegativne težine, pa Dijkstra daje optimalnu udaljenost. Svaki stvarni put sa izborom najviše jedne snižene ivice odgovara tačno jednom putu u proširenom grafu.

Složenost. Vrijeme $O((N+M) \log N)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

import heapq
INF = 10 ** 30
N, M = map(int, input().split())
g = [[] for _ in range(N)]
for _ in range(M):
    u, v, w = map(int, input().split())
    g[u - 1].append((v - 1, w))
d = [[INF, INF] for _ in range(N)]
d[0][0] = 0
pq = [(0, 0, 0)]
while pq:
    du, u, z = heapq.heappop(pq)
    if du != d[u][z]:
        continue
    for v, w in g[u]:
        if du + w < d[v][z]:
            d[v][z] = du + w
            heapq.heappush(pq, (d[v][z], v, z))
        if z == 0 and du + w // 2 < d[v][1]:
            d[v][1] = du + w // 2
            heapq.heappush(pq, (d[v][1], v, 1))
ans = min(d[-1])
print(-1 if ans == INF else ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 4e18;
int main() {
    int N, M;
    cin >> N >> M;
    vector < vector < pair < int, ll >> > g(N);
    while (M--) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        g[--u].push_back({
            --v, w
        });
    }
    vector < array < ll, 2 >> d(N, array < ll, 2 > {
        INF, INF
    });
    using S = tuple < ll, int, int >;
    priority_queue < S, vector < S >, greater < S >> pq;
    d[0][0] = 0;
    pq.push({
        0, 0, 0
    });
}

```

C++17 • potpuno rješenje / nastavak

```

);
while (! pq.empty()) {
    auto [du, u, z] = pq.top();
    pq.pop();
    if (du != d[u][z]) continue;
    for (auto [v, w] : g[u]) {
        if (du + w < d[v][z]) {
            d[v][z] = du + w;
            pq.push({
                d[v][z], v, z
            });
        }
        if (! z && du + w / 2 < d[v][1]) {
            d[v][1] = du + w / 2;
            pq.push({
                d[v][1], v, 1
            });
        }
    }
}
ll ans = min(d[N - 1][0], d[N - 1][1]);
cout << (ans >= INF / 2 ? - 1 : ans) << "\n";
}

```

Važne provjere

Ne označavati čvor visited bez razlikovanja used.

Dijeljenje je cjelobrojno floor.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Graf je DAG	6
2	N,M<=5000	7
3	Puna ograničenja	12

ZADATAK 124

Put sa besplatnim portalima

Cilj: Prepoznaje graf sa težinama 0/1 i koristi 0-1 BFS.

Dat je usmjeren graf sa N čvorova i M ivica. Svaka ivica ima cijenu 0 ili 1. Odredi minimalnu cijenu puta od 1 do N .

Ulaz. N M , zatim M redova u v w gdje je w 0 ili 1.

Izlaz. Minimalna cijena ili -1.

Ograničenja. $N \leq 300000$, $M \leq 500000$.

Primjer

ULAZ

```
5 6
1 2 1
1 3 0
3 4 0
4 2 0
2 5 1
4 5 1
```

IZLAZ

```
1
```

Razumijevanje i postupak

Obični BFS minimizira broj ivica, ne zbir težina. Dijkstra bi radila, ali zbog težina samo 0 i 1 možemo koristiti deque: poboljšanje težine 0 ide naprijed, a težine 1 nazad.

1. $\text{dist}[1]=0$, ostalo INF.
2. Izvadi u sa početka deque-a.
3. Za ivicu $u \rightarrow v$ težine w relaksiraj $\text{dist}[v]$.
4. Ako $w=0$ stavi v na početak, inače na kraj.

Zašto postupak daje tačan rezultat?

Deque održava stanja u neopadajućoj privremenoj udaljenosti, analogno Dijkstri, ali bez heap-a jer postoje samo dvije moguće težine.

Složenost. Vrijeme $O(N+M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v, w = map(int, input().split())
    g[u - 1].append((v - 1, w))
INF = 10 ** 18
d = [INF] * n
d[0] = 0
q = deque([0])
while q:
    u = q.popleft()
    for v, w in g[u]:
        nd = d[u] + w
        if nd < d[v]:
            d[v] = nd
            if w == 0:
                q.appendleft(v)
            else:
                q.append(v)
print(-1 if d[-1] == INF else d[-1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < pair < int, int >> > g(n);
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        g [-- u].push_back({
            -- v, w
        });
    }
    const int INF = 1e9;
    vector < int > d(n, INF);
    deque < int > q;
    d [0] = 0;
    q.push_back(0);
    while (! q.empty()) {
        int u = q.front();
        q.pop_front();
        for (auto [v, w] : g [u]) if (d [u] + w < d [v]) {
            d [v] = d [u] + w;
            if (w == 0) q.push_front(v);
        }
    }
}

```

C++17 • potpuno rješenje / nastavak

```

        else q.push_back(v);
    }
}
cout << (d [n - 1] == INF ? - 1 : d [n - 1]) << "\n";
}

```

Važne provjere

Ne označavati čvor trajno visited pri prvom viđenju; može se relaksirati boljom 0-ivicom.

0-ivice moraju ići na početak deque-a.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Sve težine 1	5
2	N,M ≤ 5000	8
3	Puna ograničenja	17

ZADATAK 125

Najjeftiniji put između gradova

Cilj: Koristi Dijkstrin algoritam za najkraći put u grafu sa nenegativnim težinama.

Dat je usmjeren graf sa pozitivnim cijenama putovanja. Odredi minimalan trošak od grada 1 do grada N.

Ulaz. N M, zatim M redova u v w.

Izlaz. Minimalni trošak ili -1.

Ograničenja. $N \leq 200000$, $M \leq 400000$, $1 \leq w \leq 10^9$.

Primjer

ULAZ

```
5 6
1 2 4
1 3 2
3 2 1
2 4 5
3 4 8
4 5 3
```

IZLAZ

```
11
```

Razumijevanje i postupak

Težine više nisu jednake pa BFS nije dovoljan. Priority queue uvijek daje trenutno najmanju udaljenost; iz nje relaksiramo susjede. Stare zapise u heap-u samo preskočimo.

1. $\text{dist}[1]=0$ i $\text{push}(0,1)$.
2. Pop najmanji (du,u) ; ako $du \neq \text{dist}[u]$, preskoči.
3. Za svaku ivicu izračunaj $nd=du+w$ i relaksiraj.
4. Odgovor je $\text{dist}[N]$.

Zašto postupak daje tačan rezultat?

Kada je stanje izvađeno sa najmanjom udaljenošću, nijedan put kroz neobrađene čvorove ne može ga poboljšati jer su sve dodatne težine nenegativne.

Složenost. Vrijeme $O((N+M) \log N)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

import heapq
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v, w = map(int, input().split())
    g[u - 1].append((v - 1, w))
INF = 10 ** 30
d = [INF] * n
d[0] = 0
pq = [(0, 0)]
while pq:
    du, u = heapq.heappop(pq)
    if du != d[u]:
        continue
    for v, w in g[u]:
        nd = du + w
        if nd < d[v]:
            d[v] = nd
            heapq.heappush(pq, (nd, v))
print(-1 if d[-1] == INF else d[-1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < pair < int, ll >> > g(n);
    while (m--) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        g [-- u].push_back({
            -- v, w
        });
    }
    const ll INF = 4e18;
    vector < ll > d(n, INF);
    priority_queue < pair < ll, int >, vector < pair < ll, int >>, greater
        < pair <
            ll, int >> > pq;
    d [0] = 0;
    pq.push({
        0, 0
    });
    while (! pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != d [u]) continue;
    }
}

```

C++17 • potpuno rješenje / nastavak

```

    for (auto [v, w] : g [u]) if (du + w < d [v]) {
        d [v] = du + w;
        pq.push({
            d [v], v
        });
    }
}
cout << (d [n - 1] >= INF / 2 ? - 1 : d [n - 1]) << "\n";
}

```

Važne provjere

Udaljenosti moraju biti 64-bitne.

Dijkstra zahtijeva nenegativne težine.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Sve w=1	5
2	N,M <= 5000	8
3	Puna ograničenja	17

ZADATAK 126

Redoslijed predmeta

Cilj: Nalazi leksikografski najmanji topološki redoslijed pomoću indegree-a i min-heap-a.

N školskih modula imaju M preduvjeta $u \rightarrow v$: modul u mora biti završen prije v. Ispiši leksikografski najmanji valjan redoslijed 1..N. Ako postoji ciklus, ispiši -1.

Ulaz. N M, zatim M usmjerenih ivica u v.

Izlaz. Redoslijed ili -1.

Ograničenja. $N, M \leq 300000$.

Primjer

ULAZ

```
5 4
1 3
2 3
3 4
2 5
```

IZLAZ

```
1 2 3 4 5
```

Razumijevanje i postupak

Kahn algoritam koristi čvorove indegree 0. Da bismo dobili najmanji mogući sljedeći broj, držimo sve trenutno dostupne čvorove u min-heap-u.

1. Izračunaj indegree svih čvorova.
2. Sve indegree=0 stavi u min-heap.
3. Uzimaj najmanji u, dodaj ga u odgovor i smanji indegree susjeda.
4. Ako odgovor nema N čvorova, postoji ciklus.

Zašto postupak daje tačan rezultat?

Svaka emitovana tačka u tom trenutku nema neobrađenog prethodnika. Ako se obrade svi čvorovi, redoslijed poštuje svaku usmjerenu ivicu; inače postoji ciklus.

Složenost. Vrijeme $O((N+M) \log N)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

import heapq
n, m = map(int, input().split())
g = [[] for _ in range(n)]
deg = [0] * n
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    deg[v] += 1
pq = [i for i in range(n) if deg[i] == 0]
heapq.heapify(pq)
ans = []
while pq:
    u = heapq.heappop(pq)
    ans.append(u + 1)
    for v in g[u]:
        deg[v] -= 1
        if deg[v] == 0:
            heapq.heappush(pq, v)
print(*ans) if len(ans) == n else print(-1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < int >> g(n);
    vector < int > d(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g [-- u].push_back(-- v);
        d [v]++;
    }
    priority_queue < int, vector < int >, greater < int >> q;
    for (int i = 0; i < n; i++) if (! d [i]) q.push(i);
    vector < int > a;
    while (! q.empty()) {
        int u = q.top();
        q.pop();
        a.push_back(u + 1);
        for (int v : g [u]) if (-- d [v] == 0) q.push(v);
    }
    if ((int) a.size() != n) cout << - 1 << "\n";
    else {
        for (int i = 0; i < n; i++) cout << a [i] << (i + 1 == n ? '\n' : ' ');
    }
}

```

Važne provjere

Obični queue daje neki topološki red, ne nužno najmanji.

Ciklus se prepoznaje po tome što nije obrađeno svih N čvorova.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Graf je lanac	5
2	$N, M \leq 5000$	8
3	Puna ograničenja	17

ZADATAK 127

Podjela u dvije ekipe

Cilj: BFS/DFS bojenjem provjerava može li se graf podijeliti u dvije grupe bez unutrašnjih konflikata.

N učenika i M parova koji ne smiju biti u istoj ekipi. Odredi može li se svih N učenika podijeliti u dvije ekipe tako da svaki konfliktni par bude u različitim ekipama.

Ulaz. N M, zatim M neusmjerenih ivica.

Izlaz. DA ili NE.

Ograničenja. N,M ≤ 300000.

Primjer

ULAZ

```
5 4
1 2
2 3
3 4
4 5
```

IZLAZ

```
DA
```

Razumijevanje i postupak

Problem je tačno provjera bipartitnosti. Svaku još neoboјenu komponentu započinjemo boјom 0 i susjedima dajemo 1-color. Konflikt istih boја znači nemogućnost.

1. color=-1 za sve.
2. Za svaki neoboјeni start pokreni BFS i postavi boју 0.
3. Susjed dobija suprotnu boју; ako već ima istu boју, ispiši NE.

Zašto postupak daje tačan rezultat?

Ako ikad ivica spaja iste boје, postoji neparni ciklus i biparticipija nije moguća. Ako konflikta nema, svaka ivica ide između dvije klase.

Složenost. Vrijeme $O(N+M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

    g[u].append(v)
    g[v].append(u)
c = [-1] * n
for s in range(n):
    if c[s] != -1:
        continue
    c[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if c[v] == -1:
                c[v] = c[u] ^ 1
                q.append(v)
            elif c[v] == c[u]:
                print('NE')
                raise SystemExit
print('DA')
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < int >> g(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back(v);
        g [v].push_back(u);
    }
    vector < int > c(n, - 1);
    queue < int > q;
    for (int s = 0; s < n; s++) if (c [s] < 0) {
        c [s] = 0;
        q.push(s);
        while (! q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g [u]) {
                if (c [v] < 0) {
                    c [v] = c [u] ^ 1;
                    q.push(v);
                }
                else if (c [v] == c [u]) {
                    cout << "NE\n";
                    return 0;
                }
            }
        }
    }
}
```


C++17 • potpuno rješenje / nastavak

```
        }  
    }  
    }  
    cout << "DA\n";  
}
```

Važne provjere

Pokrenuti BFS iz svake komponente, ne samo iz čvora 1.

Samopetlja odmah znači NE.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Graf je povezan	5
2	$N, M \leq 5000$	7
3	Puna ograničenja	13

ZADATAK 128

Koliko zatvorenih regija?

Cilj: Koristi Kosaraju algoritam za broj jako povezanih komponenti u usmjerenom grafu.

Dat je usmjeren graf. Dva čvora pripadaju istoj zatvorenoj regiji ako se iz svakog može stići do drugog. Odredi broj takvih maksimalnih regija (jako povezanih komponenti).

Ulaz. N M, zatim M usmjerenih ivica.

Izlaz. Broj SCC komponenti.

Ograničenja. N,M ≤ 300000.

Primjer

ULAZ

```
6 7
1 2
2 1
2 3
3 4
4 3
4 5
5 6
```

IZLAZ

```
4
```

Razumijevanje i postupak

Jedan DFS nije dovoljan jer dostižnost nije simetrična. Kosaraju radi dva prolaza: prvo zapisuje čvorove po vremenu završetka, zatim na obrnutom grafu pokreće DFS tim redom unazad; svaki novi DFS daje jednu SCC.

1. Izgradi g i obrnuti rg.
2. Prvi DFS na g: nakon djece dodaj u u order.
3. Reset visited.
4. Prolazi reversed(order); svaki neposjećen u pokreće DFS na rg i povećava odgovor.

Zašto postupak daje tačan rezultat?

Red završetka kondenzacionog DAG-a osigurava da u drugom prolazu ne “pobjegnemo” iz trenutne SCC u još neobrađenu SCC.

Složenost. Vrijeme $O(N+M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

import sys
sys.setrecursionlimit(1000000)
n, m = map(int, input().split())
g = [[] for _ in range(n)]
rg = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    rg[v].append(u)
vis = [0] * n
order = []

def dfs(s, gr, out=False):
    st = [(s, 0)]
    vis[s] = 1
    while st:
        u, i = st[-1]
        if i < len(gr[u]):
            v = gr[u][i]
            st[-1] = (u, i + 1)
            if not vis[v]:
                vis[v] = 1
                st.append((v, 0))
        else:
            st.pop()
            if out:
                order.append(u)
    for i in range(n):
        if not vis[i]:
            dfs(i, g, True)
vis = [0] * n
ans = 0
for s in reversed(order):
    if not vis[s]:
        dfs(s, rg, False)
        ans += 1
print(ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < int >> g(n), rg(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
    }
}

```

C++17 • potpuno rješenje / nastavak

```

    g[u].push_back(v);
    rg[v].push_back(u);
}
vector< char > vis(n);
vector< int > ord;
for (int s = 0; s < n; s++) if (! vis[s]) {
    vector< pair< int, int >> st {
        {
            s, 0
        }
    }
    ;
    vis[s] = 1;
    while (! st.empty()) {
        auto & [u, i] = st.back();
        if (i < (int) g[u].size()) {
            int v = g[u][i++];
            if (! vis[v]) {
                vis[v] = 1;
                st.push_back({
                    v, 0
                });
            }
        }
        else {
            ord.push_back(u);
            st.pop_back();
        }
    }
}
fill(vis.begin(), vis.end(), 0);
int ans = 0;
for (int z = n - 1; z >= 0; z--) {
    int s = ord[z];
    if (vis[s]) continue;
    ans++;
    stack< int > st;
    st.push(s);
    vis[s] = 1;
    while (! st.empty()) {
        int u = st.top();
        st.pop();
        for (int v : rg[u]) if (! vis[v]) {
            vis[v] = 1;
            st.push(v);
        }
    }
}
cout << ans << "\n";
}

```

Važne provjere

U drugom prolazu koristi se obrnuti graf.

Red je obrnuti red završetka, ne red ulaska.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 2000$	8
2	Graf je DAG	7
3	Puna ograničenja	20

ZADATAK 129

Jedna besplatna karta

Cilj: Proširuje Dijkstrino stanje informacijom da li je jednokratna pogodnost iskorištena.

U usmjerenom grafu od 1 do N svaka ivica ima cijenu w . Tokom puta smiješ najviše jednu ivicu proći besplatno. Odredi minimalan trošak.

Ulaz. N M, zatim M redova u v w.

Izlaz. Minimalni trošak ili -1.

Ograničenja. $N \leq 200000$, $M \leq 400000$, $0 \leq w \leq 10^9$.

Primjer

ULAZ

```
4 5
1 2 8
2 4 8
1 3 5
3 4 20
2 3 1
```

IZLAZ

```
5
```

Razumijevanje i postupak

Isti čvor sa neiskorištenom i iskorištenom kartom nije isto stanje. Imamo $\text{dist}[v][0/1]$. Iz sloja 0 preko ivice možemo platiti w i ostati 0 ili platiti 0 i preći u 1; iz sloja 1 plaćamo normalno.

1. $\text{dist}[1][0]=0$.
2. Dijkstra nad parom (čvor,used).
3. Uvijek relaksiraj normalnu cijenu.
4. Ako $\text{used}=0$, relaksiraj i besplatni prelaz u $\text{used}=1$.
5. Odgovor $\min(\text{dist}[N][0], \text{dist}[N][1])$.

Zašto postupak daje tačan rezultat?

Kada je stanje izvađeno sa najmanjom udaljenošću, nijedan put kroz neobrađene čvorove ne može ga poboljšati jer su sve dodatne težine nenegativne.

Složenost. Vrijeme $O((N+M) \log N)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

import heapq
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v, w = map(int, input().split())
    g[u - 1].append((v - 1, w))
INF = 10 ** 30
d = [[INF, INF] for _ in range(n)]
d[0][0] = 0
pq = [(0, 0, 0)]
while pq:
    du, u, z = heapq.heappop(pq)
    if du != d[u][z]:
        continue
    for v, w in g[u]:
        if du + w < d[v][z]:
            d[v][z] = du + w
            heapq.heappush(pq, (du + w, v, z))
        if z == 0 and du < d[v][1]:
            d[v][1] = du
            heapq.heappush(pq, (du, v, 1))
ans = min(d[-1])
print(-1 if ans == INF else ans)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < pair < int, ll >> > g(n);
    while (m--) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        g [-- u].push_back({
            -- v, w
        });
    }
    const ll INF = 4e18;
    vector < array < ll, 2 >> d(n, array < ll, 2 > {
        INF, INF
    });
    using S = tuple < ll, int, int >;
    priority_queue < S, vector < S >, greater < S >> pq;
    d [0] [0] = 0;
    pq.push({
        0, 0, 0
    });
}

```

C++17 • potpuno rješenje / nastavak

```

    );
    while (! pq.empty()) {
        auto [du, u, z] = pq.top();
        pq.pop();
        if (du != d [u] [z]) continue;
        for (auto [v, w] : g [u]) {
            if (du + w < d [v] [z]) {
                d [v] [z] = du + w;
                pq.push({
                    d [v] [z], v, z
                });
            }
        }
        if (! z && du < d [v] [1]) {
            d [v] [1] = du;
            pq.push({
                du, v, 1
            });
        }
    }
}
ll ans = min(d [n - 1] [0], d [n - 1] [1]);
cout << (ans >= INF / 2 ? - 1 : ans) << "\n";
}

```

Važne provjere

Ne birati unaprijed “najskuplju” ivicu bez znanja kojeg puta.

visited mora razlikovati sloj used=0 i used=1.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N,M ≤ 3000	8
2	Graf je DAG	7
3	Puna ograničenja	20

ZADATAK 130

Online prijateljstva

Cilj: Koristi DSU za dinamičko spajanje komponenti i upite povezanosti.

N učenika u početku nisu povezani. Slijedi Q operacija: 1 a b znači dodaj prijateljsku vezu između a i b; 2 a b pita jesu li a i b u istoj povezanoj grupi. Za svaki tip 2 ispiši DA ili NE.

Ulaz. N Q, zatim Q operacija.

Izlaz. Odgovori na upite tipa 2.

Ograničenja. N,Q ≤ 300000.

Primjer

ULAZ

```
5 5
1 1 2
1 2 3
2 1 3
2 1 5
1 4 5
```

IZLAZ

```
DA
NE
```

Razumijevanje i postupak

Ne trebamo čuvati cijeli graf niti raditi BFS nakon svake veze. DSU održava samo komponente: union spaja predstavnike, a find poredi pripadnost.

1. parent[i]=i, size[i]=1.
2. find sa path compression.
3. union po veličini.
4. Za tip 2 poredi find(a)==find(b).

Zašto postupak daje tačan rezultat?

Invariant: dva čvora imaju isti predstavnik ako i samo ako su spojena dosadašnjim union operacijama.

Složenost. Amortizovano $O((N+Q) \alpha(N))$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n, q = map(int, input().split())
p = list(range(n))
sz = [1] * n

def f(x):
    while x != p[x]:
        p[x] = p[p[x]]
        x = p[x]
    return x

def un(a, b):
    a, b = (f(a), f(b))
    if a == b:
        return
    if sz[a] < sz[b]:
        a, b = (b, a)
    p[b] = a
    sz[a] += sz[b]
for _ in range(q):
    t, a, b = map(int, input().split())
    a -= 1
    b -= 1
    if t == 1:
        un(a, b)
    else:
        print('DA' if f(a) == f(b) else 'NE')

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct D {
    vector<int> p, s;
    D(int n) : p(n), s(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int f(int x) {
        return p[x] == x ? x : p[x] = f(p[x]);
    }
    void u(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b) return;
        if (s[a] < s[b]) swap(a, b);
        p[b] = a;
        s[a] += s[b];
    }
};
;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;

```

C++17 • potpuno rješenje / nastavak

```

cin >> n >> q;
D d(n);
while (q--) {
    int t, a, b;
    cin >> t >> a >> b;
    -- a;
    -- b;
    if (t == 1) d.u(a, b);
    else cout << (d.f(a) == d.f(b) ? "DA" : "NE") << "\n";
}
}

```

Važne provjere

DSU odgovara na povezanost, ali ne čuva najkraći put.

Spajati korijene, ne proizvoljne čvorove.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N,Q ≤ 2000	7
2	Samo union operacije pa upiti	5
3	Puna ograničenja	13

ZADATAK 131

Minimalna mreža kablova

Cilj: Kruskalom gradi minimalno razapinjuće stablo i provjerava povezanost.

N računara treba povezati tako da postoji put između svakog para. Dato je M mogućih kablova sa cijenama. Odredi minimalan ukupan trošak ili -1 ako povezivanje nije moguće.

Ulaz. N M, zatim M redova u v w.

Izlaz. Minimalni ukupni trošak ili -1.

Ograničenja. $N \leq 200000$, $M \leq 400000$, $w \leq 10^9$.

Primjer

ULAZ

```
4 5
1 2 4
2 3 2
3 4 3
1 4 10
1 3 5
```

IZLAZ

```
9
```

Razumijevanje i postupak

Najjeftinije ivice prvo daju najbolju šansu, ali ne smijemo napraviti ciklus. Kruskal sortira ivice po cijeni, a DSU kaže spajaju li dvije različite komponente.

1. Sortiraj ivice po w.
2. DSU sa N komponenti.
3. Ako $\text{union}(u,v)$ uspije, dodaj w i smanji broj komponenti.
4. Na kraju treba biti 1 komponenta.

Zašto postupak daje tačan rezultat?

Cut property garantuje da je najlakša ivica koja prelazi odgovarajući rez sigurna za neko minimalno razapinjuće stablo.

Složenost. Vrijeme $O(M \log M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

n, m = map(int, input().split())
e = []
for _ in range(m):
    u, v, w = map(int, input().split())
    e.append((w, u - 1, v - 1))
p = list(range(n))
sz = [1] * n

def f(x):
    while x != p[x]:
        p[x] = p[p[x]]
        x = p[x]
    return x

def un(a, b):
    a, b = (f(a), f(b))
    if a == b:
        return False
    if sz[a] < sz[b]:
        a, b = (b, a)
    p[b] = a
    sz[a] += sz[b]
    return True

ans = 0
used = 0
for w, u, v in sorted(e):
    if un(u, v):
        ans += w
        used += 1
print(ans if used == n - 1 else -1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct D {
    vector<int> p, s;
    D(int n) : p(n), s(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int f(int x) {
        return p[x] == x ? x : p[x] = f(p[x]);
    }
    bool u(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b) return 0;
        if (s[a] < s[b]) swap(a, b);
        p[b] = a;
        s[a] += s[b];
        return 1;
    }
}

```

C++17 • potpuno rješenje / nastavak

```

;
int main() {
    int n, m;
    cin >> n >> m;
    vector < tuple < long long, int, int >> e;
    while (m--) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        e.push_back({
            w, -- u, -- v
        });
    }
    sort(e.begin(), e.end());
    D d(n);
    long long ans = 0;
    int k = 0;
    for (auto [w, u, v] : e) if (d.u(u, v)) {
        ans += w;
        k++;
    }
    cout << (k == n - 1 ? ans : - 1) << "\n";
}

```

Važne provjere

MST ima $N-1$ izabranih ivica samo kada je graf povezan.

Ne sabirati ivicu koja spaja već povezane čvorove.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Graf je već stablo	5
2	$N, M \leq 5000$	8
3	Puna ograničenja	17

ZADATAK 132

Broj komponenti poslije svake veze

Cilj: Prati broj povezanih komponenti tokom online dodavanja ivica.

Imamo N izolovanih čvorova. Dodaje se M neusmjerenih veza jedna po jedna. Nakon svake dodane veze ispiši trenutni broj povezanih komponenti.

Ulaz. N M , zatim M parova u v .

Izlaz. M brojeva, po jedan nakon svake veze.

Ograničenja. $N, M \leq 300000$.

Primjer

ULAZ

```
5 4
1 2
2 3
4 5
1 3
```

IZLAZ

```
4
3
2
2
```

Razumijevanje i postupak

Početno imamo N komponenti. Svaka union operacija koja zaista spaja dva različita DSU korijena smanjuje broj komponenti za jedan; redundantna ivica ga ne mijenja.

1. $components = N$.
2. Za svaku ivicu ako union uspije: $components--$.
3. Ispiši $components$ nakon svake ivice.

Zašto postupak daje tačan rezultat?

Invariant: dva čvora imaju isti predstavnik ako i samo ako su spojena dosadašnjim union operacijama.

Složenost. Amortizovano $O((N+M) \alpha(N))$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n, m = map(int, input().split())
p = list(range(n))
sz = [1] * n
c = n

def f(x):
    while x != p[x]:
        p[x] = p[p[x]]
        x = p[x]
    return x

for _ in range(m):
    a, b = map(int, input().split())
    a = f(a - 1)
    b = f(b - 1)
    if a != b:
        if sz[a] < sz[b]:
            a, b = (b, a)
        p[b] = a
        sz[a] += sz[b]
        c -= 1
print(c)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector<int> p(n), s(n, 1);
    iota(p.begin(), p.end(), 0);
    function<int(int)> f = [&](int x) {
        return p[x] == x ? x : p[x] = f(p[x]);
    };
    ;
    int c = n;
    while (m--) {
        int a, b;
        cin >> a >> b;
        a = f(--a);
        b = f(--b);
        if (a != b) {
            if (s[a] < s[b]) swap(a, b);
            p[b] = a;
            s[a] += s[b];
            c--;
        }
        cout << c << "\n";
    }
}

```


Važne provjere

Samo uspješan union smanjuje broj komponenti.

Na početku je svaka tačka posebna komponenta.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N, M \leq 2000$	5
2	Nema redundantnih veza	5
3	Puna ograničenja	10

ZADATAK 133

Tačka sastanka

Cilj: Koristi svojstvo medijane za minimizaciju zbirne Manhattan udaljenosti.

N učenika nalazi se u tačkama (x_i, y_i) na cjelobrojnoj mreži. Izaberi cjelobrojnu tačku (X, Y) koja minimizira zbir $|x_i - X| + |y_i - Y|$. Ispiši minimalni zbir udaljenosti.

Ulaz. N, zatim N parova x y.

Izlaz. Minimalni zbir Manhattan udaljenosti.

Ograničenja. $N \leq 300000$, $|koordinate| \leq 10^9$.

Primjer

ULAZ

```
4
0 0
2 0
0 2
2 2
```

IZLAZ

```
8
```

Razumijevanje i postupak

Manhattan suma se razdvaja na x-dio i y-dio. U jednoj dimenziji zbir apsolutnih odstupanja minimizira bilo koja medijana. Zato nezavisno biramo medijanu x koordinata i medijanu y koordinata.

1. Sortiraj sve x i sve y.
2. Uzmi $mx = x_sorted[N//2]$, $my = y_sorted[N//2]$.
3. Saberi $|x - mx| + |y - my|$ za sve tačke.

Zašto postupak daje tačan rezultat?

Ako je stanje ispravno poslije prvih i elemenata, lokalno pravilo ga tačno ažurira nakon elementa $i+1$. Indukcijom stanje je tačno na kraju.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n = int(input())
xs = []
ys = []
for _ in range(n):
    x, y = map(int, input().split())
    xs.append(x)
    ys.append(y)
xs.sort()
ys.sort()
mx = xs[n // 2]
my = ys[n // 2]
print(sum((abs(x - mx) for x in xs)) + sum((abs(y - my) for y in ys)))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];
    sort(x.begin(), x.end());
    sort(y.begin(), y.end());
    long long X = x[n / 2], Y = y[n / 2], ans = 0;
    for (long long v : x) ans += llabs(v - X);
    for (long long v : y) ans += llabs(v - Y);
    cout << ans << "\n";
}

```

Važne provjere

Ne koristiti prosjek; apsolutna odstupanja minimizira medijana.

x i y se mogu optimizovati nezavisno.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 2000	7
2	Svi y _i jednaki	8
3	Puna ograničenja	15

ZADATAK 134

Prag povezivanja

Cilj: Sortiranjem ivica i DSU-om nalazi najmanji prag težine pri kojem su dva zadana čvora povezana.

Dat je neusmjeren graf sa težinama ivica. Smijemo koristiti samo ivice težine $\leq X$. Odredi najmanji X za koji postoji put od čvora 1 do čvora N , ili -1 ako put ne postoji ni sa svim ivicama.

Ulaz. N M , zatim M redova u v w .

Izlaz. Minimalni prag X ili -1 .

Ograničenja. $N \leq 200000$, $M \leq 400000$, $w \leq 10^9$.

Primjer

ULAZ

```
5 6
1 2 8
2 5 9
1 3 4
3 4 6
4 5 7
2 3 5
```

IZLAZ

```
7
```

Razumijevanje i postupak

Ako ivice obrađujemo rastuće po težini, nakon obrade svih ivica $\leq w$ DSU tačno predstavlja povezanost pod pragom w . Prvi trenutak kada 1 i N postanu u istoj komponenti daje najmanji mogući prag.

1. Sortiraj ivice po w .
2. DSU početno izolovan.
3. Dodaj ivice grupu po grupu ili redom.
4. Nakon svake težine provjeri $\text{find}(1) == \text{find}(N)$; prva takva težina je odgovor.

Zašto postupak daje tačan rezultat?

Invariant: dva čvora imaju isti predstavnik ako i samo ako su spojena dosadašnjim union operacijama.

Složenost. Vrijeme $O(M \log M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

n, m = map(int, input().split())
e = []
for _ in range(m):
    u, v, w = map(int, input().split())
    e.append((w, u - 1, v - 1))
p = list(range(n))
sz = [1] * n

def f(x):
    while x != p[x]:
        p[x] = p[p[x]]
        x = p[x]
    return x

def un(a, b):
    a, b = (f(a), f(b))
    if a == b:
        return
    if sz[a] < sz[b]:
        a, b = (b, a)
    p[b] = a
    sz[a] += sz[b]
for w, u, v in sorted(e):
    un(u, v)
    if f(0) == f(n - 1):
        print(w)
        break
else:
    print(-1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct D {
    vector< int > p, s;
    D(int n) : p(n), s(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int f(int x) {
        return p[x] == x ? x : p[x] = f(p[x]);
    }
    void u(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b) return;
        if (s[a] < s[b]) swap(a, b);
        p[b] = a;
        s[a] += s[b];
    }
}
;
int main() {

```

C++17 • potpuno rješenje / nastavak

```

int n, m;
cin >> n >> m;
vector < tuple < long long, int, int >> e;
while (m--) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    e.push_back({
        w, -- u, -- v
    });
}
sort(e.begin(), e.end());
D d(n);
for (auto [w, u, v] : e) {
    d.u(u, v);
    if (d.f(0) == d.f(n - 1)) {
        cout << w << "\n";
        return 0;
    }
}
cout << - 1 << "\n";
}

```

Važne provjere

Ne traži se zbir težina puta nego najveća dozvoljena pojedinačna težina.

Prvi prag pri kojem se komponente spoje je optimalan.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N,M ≤ 3000	8
2	Graf je stablo	10
3	Puna ograničenja	22

ZADATAK 135

Najduži put kroz plan

Cilj: Kombinuje topološko sortiranje, DP i rekonstrukciju jednog najdužeg puta u DAG-u.

Dat je usmjeren acikličan graf. Odredi najveći broj ivica na putu od 1 do N i ispiši jedan takav put. Ako N nije dostižan, ispiši -1.

Ulaz. N M, zatim M usmjerenih ivica.

Izlaz. Prvo dužina, zatim čvorovi jednog najdužeg puta.

Ograničenja. N,M ≤ 300000; garantovano DAG.

Primjer

ULAZ

```
6 7
1 2
1 3
2 4
3 4
4 5
3 5
5 6
```

IZLAZ

```
4
1 2 4 5 6
```

Razumijevanje i postupak

Topološki red omogućava da prije obrade u znamo najbolju dužinu puta do u. Za svaku ivicu $u \rightarrow v$ relaksiramo $\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + 1)$ i pamtimos $\text{parent}[v] = u$ kada poboljšamo. Parent zatim rekonstruira put.

1. Kahn/DFS topološki red.
2. $\text{dist} = -\text{INF}$, $\text{dist}[1] = 0$.
3. Prolazi u topološkom redu; za svako v relaksiraj ako je u dostižan.
4. Ako $\text{dist}[N]$ ne postoji $\rightarrow -1$; inače prati parent unazad.

Zašto postupak daje tačan rezultat?

Svaki put u DAG-u prati topološki red, te kada obrađujemo u već znamo najbolju vrijednost svakog puta koji završava u u; relaksacije pokrivaju sve nastavke.

Složenost. Vrijeme $O(N+M)$, memorija $O(N+M)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
n, m = map(int, input().split())
g = [[] for _ in range(n)]
deg = [0] * n
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    deg[v] += 1
q = deque((i for i in range(n) if deg[i] == 0))
ord = []
while q:
    u = q.popleft()
    ord.append(u)
    for v in g[u]:
        deg[v] -= 1
        q.append(v) if deg[v] == 0 else None
NEG = -10 ** 9
d = [NEG] * n
par = [-1] * n
d[0] = 0
for u in ord:
    if d[u] == NEG:
        continue
    for v in g[u]:
        if d[u] + 1 > d[v]:
            d[v] = d[u] + 1
            par[v] = u
if d[-1] == NEG:
    print(-1)
else:
    path = []
    u = n - 1
    while u != -1:
        path.append(u + 1)
        u = par[u]
    path.reverse()
    print(d[-1])
    print(*path)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < int >> g(n);
    vector < int > deg(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
    }
}

```


C++17 • potpuno rješenje / nastavak

```

    g [-- u].push_back(-- v);
    deg [v]++;
}
queue < int > q;
for (int i = 0; i < n; i++) if (! deg [i]) q.push(i);
vector < int > ord;
while (! q.empty()) {
    int u = q.front();
    q.pop();
    ord.push_back(u);
    for (int v : g [u]) if (-- deg [v] == 0) q.push(v);
}
const int NEG = - 1e9;
vector < int > d(n, NEG), p(n, - 1);
d [0] = 0;
for (int u : ord) if (d [u] != NEG) for (int v : g [u]) if (d [u] + 1 >
    d [v]) {
    d [v] = d [u] + 1;
    p [v] = u;
}
if (d [n - 1] == NEG) {
    cout << - 1 << "\n";
    return 0;
}
vector < int > path;
for (int u = n - 1; u != - 1; u = p [u]) path.push_back(u + 1);
reverse(path.begin(), path.end());
cout << d [n - 1] << "\n";
for (int i = 0; i < (int) path.size(); i++) cout << path [i] << (i + 1
    == (int)
    path.size() ? '\n' : ' ');
}

```

Važne provjere

U opštem grafu najduži put je težak; ovdje DAG omogućava DP.

Parent se mijenja samo kada se dist strogo poboljša.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N,M ≤ 3000	7
2	Jedinstven put 1->N	5
3	Puna ograničenja	13

22. Stabla, dinamičko programiranje i napredni izazovi

Zadaci u ovoj cjelini povezuju teoriju s potpunim programima. Prije čitanja razrade pokušajte oblikovati vlastito rješenje. Srodne vježbe uporedite prema ulazu, traženom izlazu i ograničenjima; ista tehnika ne podrazumijeva iste uslove.

Zadatak	Naziv
136	Odabir projekata
137	Kamenčići sa krajeva
138	Minimalni broj novčića
139	Najduži rastući podniz
140	Koliko izmjena dijeli dvije riječi?
141	Stepenice sa zabranama
142	Najveći zbir bez susjeda
143	Najduži zajednički podniz
144	Podjela ekipe
145	Prečnik stabla
146	Veličine podstabala
147	Najbliži zajednički predak
148	Čuvari koji se ne dodiruju
149	Stepenice 1-2-3
150	Najmanji broj kovanica
151	Ruksak za takmičenje
152	Najduži rastući podniz
153	Najduži zajednički podniz
154	Najbolji put kroz matricu
155	Može li se podijeliti jednako?

Zadatak	Naziv
156	Broj inverzija
157	Labirint sa ključevima
158	Putnik kroz stanice
159	Maksimalni protok kroz mrežu

ZADATAK 136

Odabir projekata

Cilj: Formulirati stanje DP-a za ograničen kapacitet i svaki predmet koristiti najviše jednom.

Na sajmu postoji N projekata. Projekat i zahtijeva t_i minuta prezentacije i donosi v_i bodova. Učenik ima ukupno najviše T minuta i svaki projekat može pogledati najviše jednom. Koliko najviše bodova može prikupiti?

Ulaz. U prvom redu N T . Zatim N redova t_i v_i .

Izlaz. Ispisati maksimalan zbir bodova.

Ograničenja. $1 \leq N \leq 200, 1 \leq T \leq 20000, 1 \leq t_i \leq T, 0 \leq v_i \leq 10^9$.

Primjer

ULAZ

```
4 7
3 10
4 11
2 7
5 14
```

IZLAZ

```
21
```

Razumijevanje i postupak

$DP[x]$ je maksimalan broj bodova uz tačno/at most x minuta kapaciteta nakon obrađenih projekata. Pošto se svaki projekat smije uzeti samo jednom, x prolazimo opadajuće; tako isti projekat ne može biti iskorišten više puta u istoj iteraciji.

1. $dp[0..T]=0$.
2. Za svaki (t,v) , za x od T do t : $dp[x]=\max(dp[x], dp[x-t]+v)$.
3. Odgovor je $\max(dp)$.

Složenost. $O(N \cdot T)$ vrijeme i $O(T)$ memorija.

PYTHON 3 • potpuno rješenje

```
n, T = map(int, input().split())
dp = [0] * (T + 1)
for _ in range(n):
    t, v = map(int, input().split())
    for x in range(T, t - 1, -1):
        dp[x] = max(dp[x], dp[x - t] + v)
print(max(dp))
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n, T;
    cin >> n >> T;
    vector< long long > dp(T + 1, 0);
    for (int i = 0; i < n; i++) {
        int t;
        long long v;
        cin >> t >> v;
        for (int x = T; x >= t; x--) dp [x] = max(dp [x], dp [x - t] + v);
    }
    cout << * max_element(dp.begin(), dp.end()) << "\n";
}
```

ZADATAK 137

Kamenčići sa krajeva

Cilj: Rješavati igru optimalnih poteza pomoću DP-a po intervalima.

U nizu je N hrpica kamenčića, a i -ta vrijedi a_i bodova. Dva igrača naizmjenično uzimaju lijevu ili desnu krajnju hrpicu; prvi igrač počinje i oba igraju optimalno. Odredi najveću moguću razliku (bodovi prvog - bodovi drugog) koju prvi može garantovati.

Ulaz. U prvom redu N , u drugom N vrijednosti.

Izlaz. Ispisati optimalnu razliku.

Ograničenja. $1 \leq N \leq 2000$, $|a_i| \leq 10^9$.

Primjer

ULAZ

4

4 7 2 9

IZLAZ

10

Razumijevanje i postupak

Neka $dp[l][r]$ označava najbolju razliku koju igrač na potezu može ostvariti na segmentu $l..r$. Ako uzme lijevo, dobija $a[l]$, ali protivnik zatim može ostvariti $dp[l+1][r]$, pa je neto $a[l] - dp[l+1][r]$. Slično za desno. Uzimamo maksimum.

1. Baza: $dp[i][i] = a[i]$.

2. Za dužine $2..N$ računati $dp[l][r] = \max(a[l] - dp[l+1][r], a[r] - dp[l][r-1])$.

3. Odgovor $dp[0][N-1]$.

Složenost. $O(N^2)$ vrijeme i $O(N^2)$ memorija u jasnoj verziji.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
dp = [[0] * n for _ in range(n)]
for i in range(n):
    dp[i][i] = a[i]
for length in range(2, n + 1):
    for l in range(n - length + 1):
        r = l + length - 1
        dp[l][r] = max(a[l] - dp[l + 1][r], a[r] - dp[l][r - 1])
print(dp[0][n - 1])
```

C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < long long > a(n);
    for (auto & x : a) cin >> x;
    vector < vector < long long >> dp(n, vector < long long > (n));
    for (int i = 0; i < n; i++) dp [i] [i] = a [i];
    for (int len = 2; len <= n; len++) for (int l = 0; l + len <= n; l++) {
        int r = l + len - 1;
        dp [l] [r] = max(a [l] - dp [l + 1] [r], a [r] - dp [l] [r - 1]);
    }
    cout << dp [0] [n - 1] << "\n";
}
```

ZADATAK 138

Minimalni broj novčića

Cilj: Primijeniti jednodimenzionalni DP za neograničen broj elemenata.

Postoji K vrsta novčića vrijednosti c_i . Svaka vrsta se može koristiti neograničeno mnogo puta. Odredi najmanji broj novčića potreban za tačan iznos S ili -1 ako ga nije moguće sastaviti.

Ulaz. U prvom redu K S , u drugom K vrijednosti c_i .

Izlaz. Ispisati minimalan broj novčića ili -1 .

Ograničenja. $1 \leq K \leq 100, 1 \leq S \leq 100000, 1 \leq c_i \leq S$.

Primjer**ULAZ**

```
3 11
1 5 7
```

IZLAZ

```
3
```

Razumijevanje i postupak

$DP[x]$ je minimalan broj novčića za iznos x . Počinjemo sa $dp[0]=0$, ostalo beskonačno. Za svaki x pokušavamo dodati svaki novčić c : ako $x-c$ i $dp[x-c]$ postoji, kandidat je $dp[x-c]+1$.

1. $dp[0]=0$, ostalo INF .
2. Za $x=1..S$, za svaki c ažurirati $dp[x]$.
3. Ako $dp[S]$ ostane INF , ispisati -1 .

Složenost. $O(K \cdot S)$ vrijeme i $O(S)$ memorija.

PYTHON 3 • potpuno rješenje

```
k, S = map(int, input().split())
c = list(map(int, input().split()))
INF = 10 ** 9
dp = [INF] * (S + 1)
dp[0] = 0
for x in range(1, S + 1):
    for v in c:
        if v <= x and dp[x - v] != INF:
            dp[x] = min(dp[x], dp[x - v] + 1)
print(-1 if dp[S] == INF else dp[S])
```


C++17 • potpuno rješenje

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int k, S;
    cin >> k >> S;
    vector<int> c(k);
    for (int & x : c) cin >> x;
    const int INF = 1e9;
    vector<int> dp(S + 1, INF);
    dp[0] = 0;
    for (int x = 1; x <= S; x++) for (int v : c) if (v <= x && dp[x - v]
        != INF) dp
        [x] = min(dp[x], dp[x - v] + 1);
    cout << (dp[S] == INF ? -1 : dp[S]) << "\n";
}
```

ZADATAK 139

Najduži rastući podniz

Cilj: Učenik razumije $O(N \log N)$ algoritam za dužinu LIS-a pomoću niza tails i binarnog pretraživanja.

Za niz A odredi dužinu najdužeg strogo rastućeg podniza. Elementi podniza ne moraju biti susjedni, ali redoslijed mora ostati isti.

Ulaz. Prvi red N . Drugi red N cijelih brojeva.

Izlaz. Ispisati dužinu LIS-a.

Ograničenja. $1 \leq N \leq 2 \cdot 10^5$.

Primjer

ULAZ

```
8
3 1 5 2 6 4 9 7
```

IZLAZ

```
4
```

Razumijevanje i postupak

$\text{tails}[k]$ čuva najmanju moguću završnu vrijednost nekog rastućeg podniza dužine $k+1$. Za novi x nalazimo prvu poziciju $\text{tails}[\text{pos}] \geq x$ i zamjenjujemo je sa x ; ako takve nema, produžujemo listu.

- Počni sa praznim tails.
- Za svaki x nađi lower_bound u tails.
- Zamijeni pronađenu vrijednost ili dodaj x na kraj.
- Dužina tails je odgovor.

Složenost. Vrijeme $O(N \log N)$; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
from bisect import bisect_left
N = int(input())
A = list(map(int, input().split()))
tails = []
for x in A:
    i = bisect_left(tails, x)
    if i == len(tails):
        tails.append(x)
    else:
        tails[i] = x
print(len(tails))
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N;
    cin >> N;
    vector< long long > t;
    while (N--) {
        long long x;
        cin >> x;
        auto it = lower_bound(t.begin(), t.end(), x);
        if (it == t.end()) t.push_back(x);
        else * it = x;
    }
    cout << t.size() << "\n";
}
```

Važne provjere

tails nije nužno stvarni LIS - njegova dužina jeste ispravna.

Za strogo rastući podniz koristi se `lower_bound`; za neopadajući bi se koristio `upper_bound`.

ZADATAK 140

Koliko izmjena dijeli dvije riječi?

Cilj: Učenik primjenjuje Levenshteinovu udaljenost i razumije DP sa tri moguća posljednja poteza.

Dozvoljene su operacije: umetanje jednog znaka, brisanje jednog znaka i zamjena jednog znaka. Odredi minimalan broj operacija kojim se string A pretvara u string B.

Ulaz. Prvi red A, drugi red B.

Izlaz. Ispisati minimalan broj operacija.

Ograničenja. $0 \leq |A|, |B| \leq 2000$.

Primjer

ULAZ

```
kitten
sitting
```

IZLAZ

```
3
```

Razumijevanje i postupak

$dp[i][j]$ je minimalan broj operacija da prvih i znakova A postanu prvih j znakova B. Posljednja operacija je brisanje, umetanje ili zamjena; ako su zadnji znakovi jednaki, zamjena košta 0.

1. Baza: $dp[0][j]=j$ i $dp[i][0]=i$.
2. Prijelaz: $\min(\text{brisi}, \text{umetni}, \text{zamijeni/ostavi})$.
3. Za memoriju su dovoljne prethodna i trenutna vrsta.

Složenost. Vrijeme $O(|A| * |B|)$; memorija $O(|B|)$.

PYTHON 3 • potpuno rješenje

```
A = input()
B = input()
prev = list(range(len(B) + 1))
for i, ca in enumerate(A, 1):
    cur = [i] + [0] * len(B)
    for j, cb in enumerate(B, 1):
        cur[j] = min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (ca != cb))
    prev = cur
print(prev[-1])
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string A, B;
    getline(cin, A);
    getline(cin, B);
    vector < int > prev(B.size() + 1), cur(B.size() + 1);
    iota(prev.begin(), prev.end(), 0);
    for (int i = 1; i <= (int) A.size(); ++ i) {
        cur [0] = i;
        for (int j = 1; j <= (int) B.size(); ++ j) cur [j] = min({
            prev [j] + 1, cur [j - 1] + 1, prev [j - 1] + (A [i - 1] != B
                [j - 1])
        });
        swap(prev, cur);
    }
    cout << prev [B.size()] << "\n";
}

```

Važne provjere

Paziti da zamjena koristi `dp[i-1][j-1]`.

Prazan string je dozvoljen u ograničenjima, zato baze moraju biti ispravne.

ZADATAK 141

Stepenice sa zabranama

Cilj: Dinamičkim programiranjem broji puteve uz zabranjena stajališta.

Do stepenika N može se skočiti za 1 ili 2 stepenika. Neki stepenici su zabranjeni i na njih se ne smije stati. Koliko načina postoji da se dođe sa 0 na N ? Rezultat modulo $1e9+7$.

Ulaz. N K , zatim K različitih zabranjenih indeksa između 1 i $N-1$.

Izlaz. Broj načina modulo $1e9+7$.

Ograničenja. $N \leq 1000000$.

Primjer

ULAZ

7 2

3 5

IZLAZ

2

Razumijevanje i postupak

$dp[i]$ je broj načina da se stigne tačno na i . Ako je i zabranjen, $dp[i]=0$; inače posljednji skok dolazi sa $i-1$ ili $i-2$.

1. $dp[0]=1$.

2. Za $i=1..N$: ako zabranjen $\rightarrow 0$, inače $dp[i]=dp[i-1]+dp[i-2]$.

Zašto postupak daje tačan rezultat?

Svaki legalan put do i završava tačno jednim od dva moguća posljednja skoka. Te dvije grupe puteva se ne preklapaju, pa se brojevi sabiraju.

Složenost. Vrijeme $O(N+K)$, memorija $O(N)$ ili $O(1)+\text{set zabrana}$.

PYTHON 3 • potpuno rješenje

```
MOD = 1000000007
N, K = map(int, input().split())
bad = set(map(int, input().split())) if K else set()
dp = [0] * (N + 1)
dp[0] = 1
for i in range(1, N + 1):
    if i not in bad:
        dp[i] = dp[i - 1] + (dp[i - 2] if i >= 2 else 0)
        dp[i] %= MOD
print(dp[N])
```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
int main() {
    int N, K;
    cin >> N >> K;
    vector< char > bad(N + 1);
    while (K--) {
        int x;
        cin >> x;
        bad [x] = 1;
    }
    vector< long long > dp(N + 1);
    dp [0] = 1;
    for (int i = 1; i <= N; i++) if (! bad [i]) dp [i] = (dp [i - 1] + (i
        >= 2 ? dp
        [i - 2] : 0)) % MOD;
    cout << dp [N] << "\n";
}

```

Važne provjere

Ne zaboraviti `dp[0]=1`.

Zabranjeni stepenik mora imati nula načina.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	K=0	5
2	N<=10000	5
3	Puna ograničenja	10

ZADATAK 142

Najveći zbir bez susjeda

Cilj: Formulirajte jednodimenzionalni DP sa izborom uzmi/preskoči.

Dat je niz vrijednosti. Izaberi podskup indeksa tako da nijedna dva izabrana indeksa nisu susjedna i da zbir bude maksimalan. Dozvoljeno je ne izabrati ništa.

Ulaz. N, zatim N cijelih vrijednosti.

Izlaz. Maksimalan zbir.

Ograničenja. $N \leq 1000000$, $|a_i| \leq 10^9$.

Primjer

ULAZ

```
6
5 -2 7 10 -3 8
```

IZLAZ

```
23
```

Razumijevanje i postupak

Za element x imamo dvije mogućnosti: preskočiti ga i zadržati najbolji zbir do $i-1$, ili uzeti ga, pa prethodni izabrani može biti najviše $i-2$.

1. $prev2 = dp[i-2]$, $prev1 = dp[i-1]$.
2. $new = \max(prev1, prev2 + x)$.
3. Pomjeri dvije varijable.

Zašto postupak daje tačan rezultat?

Svako optimalno rješenje za prefiks ili ne sadrži posljednji element (prvi slučaj), ili ga sadrži i zato ne sadrži prethodni (drugi slučaj). Maksimum pokriva oba iscrpna slučaja.

Složenost. Vrijeme $O(N)$, memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
prev2 = prev1 = 0
for x in a:
    prev2, prev1 = (prev1, max(prev1, prev2 + x))
print(prev1)
```


C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long p2 = 0, p1 = 0;
    while (n--) {
        long long x;
        cin >> x;
        long long nw = max(p1, p2 + x);
        p2 = p1;
        p1 = nw;
    }
    cout << p1 << "\n";
}
```

Važne provjere

Ako zadatak zahtijeva bar jedan izbor, početne vrijednosti bi bile drugačije; ovdje je prazan skup dozvoljen.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Sve vrijednosti nenegativne	5
2	$N \leq 10000$	5
3	Puna ograničenja	10

ZADATAK 143

Najduži zajednički podniz

Cilj: Formulirajte LCS DP i optimizirajte memoriju na dva reda.

Data su dva stringa A i B. Odredi dužinu njihovog najdužeg zajedničkog podniza (znakovi ne moraju biti uzastopni, ali redoslijed se čuva).

Ulaz. Dva reda: A i B.

Izlaz. Dužina LCS-a.

Ograničenja. $|A|, |B| \leq 4000$.

Primjer

ULAZ

ALGORITAM

LOGARITAM

IZLAZ

7

Razumijevanje i postupak

$dp[i][j]$ za prefikse ima klasičan prijelaz: ako su posljednji znakovi jednaki, možemo ih upariti; inače odbacujemo jedan od posljednja dva znaka. Za dužinu su dovoljna dva reda.

1. Neka B bude kraći string.
2. $prev = nule$.
3. Za svaki znak A izgradi cur : ako isti znak $\rightarrow prev[j-1]+1$; inače $\max(prev[j], cur[j-1])$.

Zašto postupak daje tačan rezultat?

Optimalni zajednički podniz prefiksa ili koristi oba posljednja jednaka znaka, ili kada se razlikuju mora izostaviti barem jedan od njih. Prijelaz razmatra upravo te slučajeve.

Složenost. Vrijeme $O(|A| \cdot |B|)$, memorija $O(\min(|A|, |B|))$.

PYTHON 3 • potpuno rješenje

```

A = input().strip()
B = input().strip()
if len(B) > len(A):
    A, B = (B, A)
prev = [0] * (len(B) + 1)
for x in A:
    cur = [0] * (len(B) + 1)
    for j, y in enumerate(B, 1):
        cur[j] = prev[j - 1] + 1 if x == y else max(prev[j], cur[j - 1])
    prev = cur
print(prev[-1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string A, B;
    cin >> A >> B;
    if (B.size() > A.size()) swap(A, B);
    vector<int> p(B.size() + 1, c(B.size() + 1));
    for (char x : A) {
        fill(c.begin(), c.end(), 0);
        for (int j = 1; j <= (int) B.size(); j++) c[j] = (x == B[j - 1] ?
            p[j - 1]
            + 1 : max(p[j], c[j - 1]));
        swap(p, c);
    }
    cout << p.back() << "\n";
}

```

Važne provjere

Podniz nije isto što i podstring.

Kod dva reda `cur[j-1]` pripada trenutnom redu.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Dužine ≤ 200	5
2	Dužine ≤ 1500	10
3	Puna ograničenja uz $O(\min(n, m))$ memorije	15

ZADATAK 144

Podjela ekipe

Cilj: Koristi subset-sum DP za balansiranje dvije ekipe.

N učenika ima vrijednosti snage a_i . Treba ih podijeliti u dvije ekipe tako da apsolutna razlika zbroja snage bude minimalna. Svaki učenik ide u tačno jednu ekipu.

Ulaz. N, zatim N pozitivnih vrijednosti.

Izlaz. Minimalna moguća razlika.

Ograničenja. $N \leq 200$, suma svih $a_i \leq 200000$.

Primjer

ULAZ

```
6
3 1 4 2 2 7
```

IZLAZ

```
1
```

Razumijevanje i postupak

Ako jedna ekipa ima zbir x , druga ima $total - x$, a razlika je $|total - 2x|$. Treba naći dostizni subset-sum x što bliži $total/2$.

1. `reachable[0]=True`.
2. Za svaku vrijednost x ažuriraj sume opadajućim redom.
3. Nađi najveći dostizni $s \leq total/2$.
4. Ispiši $total - 2*s$.

Zašto postupak daje tačan rezultat?

0/1 subset-sum DP označava tačno sve zbroje koji se mogu dobiti nekim podskupom. Za fiksni total funkcija $total - 2s$ opada dok s raste do $total/2$, pa je najbolji najveći dostizni s u toj polovini.

Složenost. Vrijeme $O(N \cdot SUM)$, memorija $O(SUM)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
tot = sum(a)
r = [False] * (tot + 1)
r[0] = True
for x in a:
    for s in range(tot, x - 1, -1):
```

PYTHON 3 • potpuno rješenje / nastavak

```

        if r[s - x]:
            r[s] = True
for s in range(tot // 2, -1, -1):
    if r[s]:
        print(tot - 2 * s)
        break

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < int > a(n);
    int tot = 0;
    for (int & x : a) {
        cin >> x;
        tot += x;
    }
    vector < char > r(tot + 1);
    r [0] = 1;
    for (int x : a) for (int s = tot; s >= x; s--) if (r [s - x]) r [s] = 1;
    for (int s = tot / 2; s >= 0; s--) if (r [s]) {
        cout << tot - 2 * s << "\n";
        break;
    }
}

```

Važne provjere

Sume se ažuriraju opadajuće da se isti učenik ne koristi više puta.

Ograničenje je po sumi, zato pseudo-polinomijalni DP prolazi.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N<=25	5
2	Suma<=5000	8
3	Puna ograničenja	12

ZADATAK 145

Prečnik stabla

Cilj: Koristi svojstvo stabla i dva obilaska za nalaženje najdužeg jednostavnog puta.

Dato je stablo sa N čvorova. Odredi broj ivica na njegovom najdužem jednostavnom putu (prečnik stabla).

Ulaz. N , zatim $N-1$ neusmjerenih ivica.

Izlaz. Dužina prečnika u broju ivica.

Ograničenja. $1 \leq N \leq 300000$.

Primjer

ULAZ

```
6
1 2
2 3
3 4
2 5
5 6
```

IZLAZ

```
4
```

Razumijevanje i postupak

U stablu, najudaljeniji čvor od proizvoljnog starta je jedan kraj nekog prečnika. Zato prvo BFS/DFS od 1 nađe A , a drugi obilazak od A nađe najudaljeniji B ; $\text{dist}(A,B)$ je prečnik.

1. Pokreni BFS od 1 i nađi čvor A sa najvećom udaljenošću.
2. Pokreni BFS od A .
3. Najveća udaljenost u drugom obilasku je odgovor.

Zašto postupak daje tačan rezultat?

Bez ciklusa, svaki čvor osim korijena prvi put se dostiže preko jedinog roditelja, pa postorder/preorder akumulacije ne dvostruko broje čvorove.

Složenost. Vrijeme $O(N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
from collections import deque
n = int(input())
g = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = map(int, input().split())
    u -= 1
```

PYTHON 3 • potpuno rješenje / nastavak

```

    v -= 1
    g[u].append(v)
    g[v].append(u)

def far(s):
    d = [-1] * n
    d[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if d[v] < 0:
                d[v] = d[u] + 1
                q.append(v)
    a = max(range(n), key=d.__getitem__)
    return (a, d[a])
a, _ = far(0)
b, diam = far(a)
print(diam)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < vector < int >> g(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back(v);
        g [v].push_back(u);
    }
    auto far = [&] (int s) {
        vector < int > d(n, - 1);
        queue < int > q;
        d [s] = 0;
        q.push(s);
        while (! q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g [u]) if (d [v] < 0) {
                d [v] = d [u] + 1;
                q.push(v);
            }
        }
        int a = max_element(d.begin(), d.end()) - d.begin();
        return pair < int, int > {
            a, d [a]
        }
    }
}

```

C++17 • potpuno rješenje / nastavak

```

        ;
    }
    ;
    auto [a, x] = far(0);
    auto [b, ans] = far(a);
    cout << ans << "\n";
}

```

Važne provjere

Prečnik se traži u broju ivica, ne broju čvorova.

Ovo svojstvo važi za stablo; u opštem grafu dva BFS-a nisu opšti algoritam za prečnik.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 2000$	7
2	Stablo je lanac	5
3	Puna ograničenja	13

ZADATAK 146

Veličine podstabala

Cilj: Korijeni stablo i postorder akumulacijom računa veličinu svakog podstabla.

Dato je stablo ukorijenjeno u čvoru 1. Za svaki čvor v odredi broj čvorova u njegovom podstablu, uključujući v .

Ulaz. N , zatim $N-1$ ivica.

Izlaz. N brojeva $sz[1] \dots sz[N]$.

Ograničenja. $1 \leq N \leq 300000$.

Primjer

ULAZ

```
5
1 2
1 3
3 4
3 5
```

IZLAZ

```
5 1 3 1 1
```

Razumijevanje i postupak

Najprije formiramo roditelje i red obilaska od korijena. Zatim u obrnutom redoslijedu svaki čvor dodaje svoju veličinu roditelju. List počinje sa 1, a roditelj dobija zbir $1 +$ veličine djece.

1. DFS/BFS od 1: parent i order.
2. $sz[v]=1$ za sve.
3. Prođi reversed(order) bez korijena i dodaj $sz[v]$ u $sz[parent[v]]$.

Zašto postupak daje tačan rezultat?

Bez ciklusa, svaki čvor osim korijena prvi put se dostiže preko jedinog roditelja, pa postorder/preorder akumulacije ne dvostruko broje čvorove.

Složenost. Vrijeme $O(N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
g = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
```

PYTHON 3 • potpuno rješenje / nastavak

```

    g[v].append(u)
p = [-1] * n
order = [0]
for u in order:
    for v in g[u]:
        if v != p[u] and v != 0:
            p[v] = u
            order.append(v)
sz = [1] * n
for v in reversed(order[1:]):
    sz[p[v]] += sz[v]
print(*sz)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < vector < int >> g(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back(v);
        g [v].push_back(u);
    }
    vector < int > p(n, - 1), ord {
        0
    }
    ;
    for (int k = 0; k < (int) ord.size(); k++) {
        int u = ord [k];
        for (int v : g [u]) if (v != p [u] && v != 0) {
            p [v] = u;
            ord.push_back(v);
        }
    }
    vector < int > sz(n, 1);
    for (int i = n - 1; i > 0; i--) sz [p [ord [i]]] += sz [ord [i]];
    for (int i = 0; i < n; i++) cout << sz [i] << (i + 1 == n ? '\n' : ' ');
}

```

Važne provjere

Roditelj se mora fiksirati da se ne vraćamo nazad.

Podstablo zavisi od izabranog korijena 1.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 2000$	5
2	Stablo je lanac	5
3	Puna ograničenja	10

ZADATAK 147

Najbliži zajednički predak

Cilj: Predobrađuje binarne pretke i odgovara na veliki broj LCA upita u $O(\log N)$.

Dato je stablo ukorijenjeno u 1 i Q upita (u,v) . Za svaki upit ispiši njihov najbliži zajednički predak.

Ulaz. N Q , $N-1$ ivica, zatim Q parova u v .

Izlaz. Po jedan LCA za svaki upit.

Ograničenja. $N, Q \leq 200000$.

Primjer

ULAZ

```
7 3
1 2
1 3
2 4
2 5
3 6
6 7
4 5
4 7
6 7
```

IZLAZ

```
2
1
6
```

Razumijevanje i postupak

Tabela $up[k][v]$ čuva 2^k -tog pretka. Prvo dublji čvor podignemo na dubinu drugog. Ako nisu jednaki, podižemo oba od najvećeg k naniže kada im se 2^k -ti preci razlikuju. Njihov neposredni roditelj tada je LCA.

1. BFS/DFS od 1 izračuna parent i depth.
2. $LOG = \lceil \log_2 N \rceil + 1$; popuni $up[0]$, pa $up[k][v] = up[k-1][up[k-1][v]]$.
3. Za upit izjednači dubine bitovima razlike.
4. Podiži oba naniže po k ; vrati $up[0][u]$.

Zašto postupak daje tačan rezultat?

Nakon izjednačavanja dubina LCA se ne može izgubiti. Pri simultanom podizanju zadržavamo oba čvora strogo ispod LCA dok god je moguće; njihov roditelj je zato LCA.

Složenost. Predobrada $O(N \log N)$, upit $O(\log N)$, memorija $O(N \log N)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
n, q = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
LOG = n.bit_length()
up = [[0] * n for _ in range(LOG)]
dep = [0] * n
par = [-1] * n
par[0] = 0
dq = deque([0])
while dq:
    u = dq.popleft()
    for v in g[u]:
        if par[v] == -1:
            par[v] = u
            dep[v] = dep[u] + 1
            dq.append(v)
up[0] = par[:]
for k in range(1, LOG):
    for v in range(n):
        up[k][v] = up[k - 1][up[k - 1][v]]

def lca(a, b):
    if dep[a] < dep[b]:
        a, b = (b, a)
    diff = dep[a] - dep[b]
    for k in range(LOG):
        if diff >> k & 1:
            a = up[k][a]
    if a == b:
        return a
    for k in range(LOG - 1, -1, -1):
        if up[k][a] != up[k][b]:
            a = up[k][a]
            b = up[k][b]
    return up[0][a]
for _ in range(q):
    a, b = map(int, input().split())
    print(lca(a - 1, b - 1) + 1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;

```

C++17 • potpuno rješenje / nastavak

```

cin >> n >> q;
vector < vector < int >> g(n);
for (int i = 1; i < n; i++) {
    int u, v;
    cin >> u >> v;
    -- u;
    -- v;
    g [u].push_back(v);
    g [v].push_back(u);
}
int L = 1;
while ((1 <= L) <= n) L++;
vector < vector < int >> up(L, vector < int > (n));
vector < int > d(n), p(n, - 1), ord {
    0
}
;
p [0] = 0;
for (int k = 0; k < (int) ord.size(); k++) {
    int u = ord [k];
    for (int v : g [u]) if (p [v] == - 1) {
        p [v] = u;
        d [v] = d [u] + 1;
        ord.push_back(v);
    }
}
up [0] = p;
for (int k = 1; k < L; k++) for (int v = 0; v < n; v++) up [k] [v] = up
    [k - 1]
    [up [k - 1] [v]];
auto lca = [&] (int a, int b) {
    if (d [a] < d [b]) swap(a, b);
    int z = d [a] - d [b];
    for (int k = 0; k < L; k++) if (z >> k & 1) a = up [k] [a];
    if (a == b) return a;
    for (int k = L - 1; k >= 0; k--) if (up [k] [a] != up [k] [b]) {
        a = up [k] [a];
        b = up [k] [b];
    }
    return up [0] [a];
}
;
while (q--) {
    int a, b;
    cin >> a >> b;
    cout << lca(-- a, -- b) + 1 << "\n";
}
}

```

Važne provjere

Tabela parent mora za korijen pokazivati na korijen da ne izlazimo iz opsega.

Najprije izjednačiti dubine.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N,Q ≤ 2000	8
2	Stablo je lanac	7
3	Puna ograničenja	20

ZADATAK 148

Čuvari koji se ne dodiruju

Cilj: Primjenjuje DP na stablu za maksimalni nezavisni skup.

Na čvorove stabla želimo postaviti što više čuvara, ali nijedna dva čvora spojena ivicom ne smiju oba imati čuvara. Odredi maksimalan broj čuvara.

Ulaz. N, zatim N-1 ivica.

Izlaz. Maksimalan broj čuvara.

Ograničenja. $1 \leq N \leq 300000$.

Primjer**ULAZ**

```
6
1 2
1 3
2 4
2 5
3 6
```

IZLAZ

```
4
```

Razumijevanje i postupak

Za svaki čvor u računamo $dp1[u]$ kada u ima čuvara i $dp0[u]$ kada ga nema. Ako u ima čuvara, djeca ne smiju; ako ga nema, svako dijete bira bolju od svoje dvije opcije.

1. Korijeni stablo u 1 i napravi postorder.
2. $dp1[u] = 1 + \text{suma } dp0[v]$ za djecu.
3. $dp0[u] = \text{suma } \max(dp0[v], dp1[v])$.
4. Odgovor $\max(dp0[\text{root}], dp1[\text{root}])$.

Zašto postupak daje tačan rezultat?

Zbog jedinstvenih podstabala odluke djece su nezavisne kada je odluka roditelja fiksirana; zato se optimalne vrijednosti mogu sabirati/kombinovati lokalno.

Složenost. Vrijeme $O(N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

n = int(input())
g = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
p = [-1] * n
ord = [0]
for u in ord:
    for v in g[u]:
        if v != p[u] and v != 0:
            p[v] = u
            ord.append(v)
d0 = [0] * n
d1 = [1] * n
for u in reversed(ord):
    for v in g[u]:
        if p[v] == u:
            d1[u] += d0[v]
            d0[u] += max(d0[v], d1[v])
print(max(d0[0], d1[0]))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector < vector < int >> g(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        -- u;
        -- v;
        g [u].push_back(v);
        g [v].push_back(u);
    }
    vector < int > p(n, - 1), ord {
        0
    }
    ;
    for (int k = 0; k < (int) ord.size(); k++) {
        int u = ord [k];
        for (int v : g [u]) if (v != p [u] && v != 0) {
            p [v] = u;
            ord.push_back(v);
        }
    }
    vector < int > d0(n), d1(n, 1);
    for (int k = n - 1; k >= 0; k--) {

```

C++17 • potpuno rješenje / nastavak

```

    int u = ord [k];
    for (int v : g [u]) if (p [v] == u) {
        d1 [u] += d0 [v];
        d0 [u] += max(d0 [v], d1 [v]);
    }
}
cout << max(d0 [0], d1 [0]) << "\n";
}

```

Važne provjere

Nezavisni skup nije isto što i skup listova.

Stanja se računaju nakon djece - postorder.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 25	7
2	Stablo je lanac	8
3	Puna ograničenja	20

ZADATAK 149

Stepenice 1-2-3

Cilj: Definiše jednodimenzionalno DP stanje za broj načina dolaska do pozicije.

Do vrha sa N stepenika može se u jednom potezu preći 1, 2 ili 3 stepenika. Koliko različitih nizova poteza vodi tačno na vrh? Odgovor dati modulo 1 000 000 007.

Ulaz. N.

Izlaz. Broj načina modulo 1 000 000 007.

Ograničenja. $0 \leq N \leq 1000000$.

Primjer

ULAZ

5

IZLAZ

13

Razumijevanje i postupak

Posljednji potez do n može biti dužine 1,2 ili 3. Zato $dp[n] = dp[n-1] + dp[n-2] + dp[n-3]$. Baza $dp[0]=1$ predstavlja prazan niz poteza.

1. $dp[0]=1$.
2. Za $i=1..N$ saberi dostupna prethodna tri stanja.
3. Svaki zbir uzmi modulo MOD.

Zašto postupak daje tačan rezultat?

Princip optimalnosti: optimalno rješenje se sastoji od optimalnih rješenja relevantnih manjih stanja; prijelaz razmatra sve legalne posljednje odluke.

Složenost. Vrijeme $O(N)$, memorija $O(N)$ ili $O(1)$ uz kružne tri vrijednosti.

PYTHON 3 • potpuno rješenje

```
MOD = 1000000007
n = int(input())
dp = [0] * (n + 1)
dp[0] = 1
for i in range(1, n + 1):
    for k in (1, 2, 3):
        if i >= k:
            dp[i] = (dp[i] + dp[i - k]) % MOD
print(dp[n])
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    const int M = 1000000007;
    int n;
    cin >> n;
    vector<int> d(n + 1);
    d[0] = 1;
    for (int i = 1; i <= n; i++) for (int k = 1; k <= 3; k++) if (i >= k) d[i] = (d[i] + d[i - k]) % M;
    cout << d[n] << "\n";
}
```

Važne provjere

dp[0]=1 je važna kombinatorna baza.

Redoslijed poteza je bitan.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N <= 30	5
2	N <= 10000	5
3	Puna ograničenja	10

ZADATAK 150

Najmanji broj kovanica

Cilj: Rješava unbounded coin change za minimalan broj elemenata.

Dato je N vrijednosti kovanica i cilj S . Svaku vrijednost možeš koristiti neograničeno mnogo puta. Odredi najmanji broj kovanica kojim se dobija tačno S ili -1 .

Ulaz. N S , zatim N pozitivnih vrijednosti.

Izlaz. Minimalan broj ili -1 .

Ograničenja. $N \leq 100$, $S \leq 200000$.

Primjer

ULAZ

```
3 11
1 5 7
```

IZLAZ

```
3
```

Razumijevanje i postupak

$dp[x]$ je najmanji broj kovanica za zbir x . Za svaku vrijednost x razmatramo posljednju kovanicu c : ako je $x-c$ dostižan, kandidat je $dp[x-c]+1$.

1. $dp[0]=0$, ostalo INF.
2. Za x od 1 do S , za svaku $c \leq x$ relaksiraj $dp[x]$.
3. Ako $dp[S]$ ostane INF, odgovor -1 .

Zašto postupak daje tačan rezultat?

Princip optimalnosti: optimalno rješenje se sastoji od optimalnih rješenja relevantnih manjih stanja; prijelaz razmatra sve legalne posljednje odluke.

Složenost. Vrijeme $O(N \cdot S)$, memorija $O(S)$.

PYTHON 3 • potpuno rješenje

```
n, S = map(int, input().split())
c = list(map(int, input().split()))
INF = 10 ** 9
dp = [INF] * (S + 1)
dp[0] = 0
for x in range(1, S + 1):
    for v in c:
        if v <= x:
            dp[x] = min(dp[x], dp[x - v] + 1)
print(-1 if dp[S] >= INF else dp[S])
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, S;
    cin >> n >> S;
    vector < int > c(n);
    for (int & x : c) cin >> x;
    const int INF = 1e9;
    vector < int > d(S + 1, INF);
    d [0] = 0;
    for (int x = 1; x <= S; x++) for (int v : c) if (v <= x && d [x - v] <
        INF) d [x]
        = min(d [x], d [x - v] + 1);
    cout << (d [S] >= INF ? - 1 : d [S]) << "\n";
}
```

Važne provjere

Ovo nije 0/1 ruksak: ista vrijednost se može koristiti više puta.

Greedy uzimanje najveće kovanice nije opšte tačno.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$S \leq 1000$	7
2	Postoji kovanica 1	5
3	Puna ograničenja	13

ZADATAK 151

Ruksak za takmičenje

Cilj: Primjenjuje 0/1 knapsack i razumije zašto se kapacitet obilazi opadajuće.

N predmeta ima težinu w_i i vrijednost v_i . Ruksak nosi najviše W . Svaki predmet se može uzeti najviše jednom. Odredi najveću ukupnu vrijednost.

Ulaz. N W , zatim N redova w v .

Izlaz. Maksimalna vrijednost.

Ograničenja. $N \leq 300$, $W \leq 200000$.

Primjer

ULAZ

```
4 7
1 1
3 4
4 5
5 7
```

IZLAZ

```
9
```

Razumijevanje i postupak

$dp[c]$ je najbolja vrijednost sa kapacitetom c nakon obrađenih predmeta. Za predmet (w,v) prolazimo c opadajuće da se isto ažuriranje ne iskoristi ponovo u istoj iteraciji.

1. $dp[0..W]=0$.
2. Za svaki predmet: for $c=W..w$: $dp[c]=\max(dp[c], dp[c-w]+v)$.
3. Odgovor $dp[W]$.

Zašto postupak daje tačan rezultat?

Princip optimalnosti: optimalno rješenje se sastoji od optimalnih rješenja relevantnih manjih stanja; prijelaz razmatra sve legalne posljednje odluke.

Složenost. Vrijeme $O(NW)$, memorija $O(W)$.

PYTHON 3 • potpuno rješenje

```
n, W = map(int, input().split())
dp = [0] * (W + 1)
for _ in range(n):
    w, v = map(int, input().split())
    for c in range(W, w - 1, -1):
        dp[c] = max(dp[c], dp[c - w] + v)
print(dp[W])
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, W;
    cin >> n >> W;
    vector < long long > d(W + 1);
    while (n--) {
        int w;
        long long v;
        cin >> w >> v;
        for (int c = W; c >= w; c--) d [c] = max(d [c], d [c - w] + v);
    }
    cout << d [W] << "\n";
}
```

Važne provjere

Kapacitet mora ići opadajuće za 0/1 verziju.

Vrijednost može tražiti 64-bitni tip.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	N ≤ 25	5
2	W ≤ 5000	8
3	Puna ograničenja	17

ZADATAK 152

Najduži rastući podniz

Cilj: Računa dužinu LIS-a u $O(N \log N)$ pomoću tails i lower_bound.

Dat je niz. Odredi dužinu najdužeg strogo rastućeg podniza, pri čemu izabrani elementi ne moraju biti uzastopni, ali njihov redoslijed mora ostati isti.

Ulaz. N, zatim N cijelih brojeva.

Izlaz. Dužina LIS-a.

Ograničenja. $1 \leq N \leq 300000$.

Primjer

ULAZ

8

10 9 2 5 3 7 101 18

IZLAZ

4

Razumijevanje i postupak

tails[k] čuva najmanji mogući završetak rastućeg podniza dužine k+1. Za x nalazimo prvi tails[x]; zamjenom taj završetak činimo što manjim, ili x produžava strukturu ako je veći od svih.

- Počni praznim tails.
- Za svaki x nađi $p = \text{lower_bound}(\text{tails}, x)$.
- Ako $p == \text{len}$, append; inače tails[p]=x.
- Dužina tails je odgovor.

Zašto postupak daje tačan rezultat?

Manji završetak za istu dužinu nikada nije lošiji za buduća proširenja. Zato zamjena održava mogućnost svih optimalnih nastavaka, a broj elemenata u tails jednak je LIS dužini.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```

from bisect import bisect_left
n = int(input())
a = list(map(int, input().split()))
t = []
for x in a:
    p = bisect_left(t, x)
    if p == len(t):
        t.append(x)
    else:
        t[p] = x
print(len(t))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector< long long > t;
    while (n--) {
        long long x;
        cin >> x;
        auto it = lower_bound(t.begin(), t.end(), x);
        if (it == t.end()) t.push_back(x);
        else * it = x;
    }
    cout << t.size() << "\n";
}

```

Važne provjere

Traži se podniz, ne segment.

Za strogo rastući koristi `lower_bound`, ne `upper_bound`.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 2000$	8
2	Svi brojevi različiti	7
3	Puna ograničenja	15

ZADATAK 153

Najduži zajednički podniz

Cilj: Definiše dvodimenzionalni DP za LCS i optimizuje memoriju na dva reda.

Data su dva stringa A i B. Odredi dužinu najdužeg stringa koji je podniz i A i B (znakovi ne moraju biti uzastopni).

Ulaz. String A, zatim string B.

Izlaz. Dužina LCS-a.

Ograničenja. $1 \leq |A|, |B| \leq 3000$.

Primjer

ULAZ

ABCBADAB
BDCABA

IZLAZ

4

Razumijevanje i postupak

Za prefikse $A[:i]$ i $B[:j]$, ako su posljednji znakovi jednaki možemo ih upariti i dodati 1 na LCS prethodnih prefiksa. Inače optimalno rješenje izostavlja barem jedan od ta dva znaka.

1. prev je DP red za $i-1$, cur za i .
2. Ako $A[i-1] == B[j-1]$: $cur[j] = prev[j-1] + 1$.
3. Inače $cur[j] = \max(prev[j], cur[j-1])$.
4. Nakon reda postavi $prev = cur$.

Zašto postupak daje tačan rezultat?

Princip optimalnosti: optimalno rješenje se sastoji od optimalnih rješenja relevantnih manjih stanja; prijelaz razmatra sve legalne posljednje odluke.

Složenost. Vrijeme $O(|A| \cdot |B|)$, memorija $O(|B|)$.

PYTHON 3 • potpuno rješenje

```

a = input().strip()
b = input().strip()
prev = [0] * (len(b) + 1)
for x in a:
    cur = [0] * (len(b) + 1)
    for j, y in enumerate(b, 1):
        cur[j] = prev[j - 1] + 1 if x == y else max(prev[j], cur[j - 1])
    prev = cur
print(prev[-1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string a, b;
    cin >> a >> b;
    vector<int> p(b.size() + 1), c(b.size() + 1);
    for (char x : a) {
        fill(c.begin(), c.end(), 0);
        for (int j = 1; j <= (int) b.size(); j++) c[j] = (x == b[j - 1] ?
            p[j - 1] + 1 : max(p[j], c[j - 1]));
        swap(p, c);
    }
    cout << p.back() << "\n";
}

```

Važne provjere

Podniz ne mora biti uzastopan.

Kod memorijske optimizacije `cur[j-1]` je tekući red, `prev[j]` prethodni.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Dužine ≤ 200	7
2	A=B	5
3	Puna ograničenja	18

ZADATAK 154

Najbolji put kroz matricu

Cilj: Koristi grid DP za maksimalni zbir uz poteze desno i dolje.

Data je matrica $R \times C$ cijelih vrijednosti. Kreće se iz (1,1) do (R,C) samo desno ili dolje. Odredi najveći mogući zbir posjećenih ćelija.

Ulaz. R C , zatim R redova po C brojeva.

Izlaz. Maksimalni zbir.

Ograničenja. $R \cdot C \leq 1000000$, $|a_{ij}| \leq 10^9$.

Primjer

ULAZ

```
3 4
5 1 2 3
2 10 -5 1
1 2 20 4
```

IZLAZ

```
43
```

Razumijevanje i postupak

Do ćelije (r,c) možemo doći samo odozgo ili slijeva, pa je najbolji zbir vrijednost ćelije plus maksimum ta dva prethodna stanja. Negativne vrijednosti zahtijevaju pravilnu -INF inicijalizaciju granica.

1. Koristi jedan DP red dužine $C+1$ inicijalizovan na -INF; $dp[1]=0$ prije prve ćelije.
2. Za svaku ćeliju j : $dp[j]=a+\max(dp[j],dp[j-1])$.
3. Odgovor $dp[C]$.

Zašto postupak daje tačan rezultat?

Princip optimalnosti: optimalno rješenje se sastoji od optimalnih rješenja relevantnih manjih stanja; prijelaz razmatra sve legalne posljednje odluke.

Složenost. Vrijeme $O(RC)$, memorija $O(C)$.

PYTHON 3 • potpuno rješenje

```

r, c = map(int, input().split())
NEG = -10 ** 30
dp = [NEG] * (c + 1)
dp[1] = 0
for _ in range(r):
    row = list(map(int, input().split()))
    for j, x in enumerate(row, 1):
        dp[j] = x + max(dp[j], dp[j - 1])
print(dp[c])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int R, C;
    cin >> R >> C;
    const long long NEG = - (1LL << 60);
    vector < long long > d(C + 1, NEG);
    d[1] = 0;
    for (int i = 0; i < R; i++) for (int j = 1; j <= C; j++) {
        long long x;
        cin >> x;
        d[j] = x + max(d[j], d[j - 1]);
    }
    cout << d[C] << "\n";
}

```

Važne provjere

Ne inicijalizovati sve dp vrijednosti nulom kada postoje negativni brojevi.

Paziti na redoslijed: `dp[j]` još predstavlja odozgo, `dp[j-1]` već slijeva.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	R,C ≤ 30	5
2	Sve vrijednosti ≥ 0	5
3	Puna ograničenja	15

ZADATAK 155

Može li se podijeliti jednako?

Cilj: Koristi 0/1 subset-sum za odluku postoji li podskup zbira polovine ukupne sume.

Dato je N pozitivnih brojeva. Možemo li ih podijeliti u dvije grupe tako da zbrovi grupa budu jednaki? Svaki broj mora pripasti tačno jednoj grupi.

Ulaz. N , zatim N pozitivnih brojeva.

Izlaz. DA ili NE.

Ograničenja. $N \leq 200$, zbir svih brojeva ≤ 200000 .

Primjer

ULAZ

```
6
1 5 11 5 2 2
```

IZLAZ

```
DA
```

Razumijevanje i postupak

Ako je total neparan, odgovor je odmah NE. Inače trebamo podskup zbira $\text{total}/2$. Boolean DP označava koje sume možemo postići koristeći svaki element najviše jednom.

1. Ako $\text{total} \% 2 == 1 \rightarrow \text{NE}$.
2. $\text{target} = \text{total} // 2$; $\text{dp}[0] = \text{True}$.
3. Za svaki x prolazi s od target naniže i postavi $\text{dp}[s] |= \text{dp}[s-x]$.
4. Odgovor $\text{dp}[\text{target}]$.

Zašto postupak daje tačan rezultat?

Princip optimalnosti: optimalno rješenje se sastoji od optimalnih rješenja relevantnih manjih stanja; prijelaz razmatra sve legalne posljednje odluke.

Složenost. Vrijeme $O(N \cdot \text{SUM})$, memorija $O(\text{SUM})$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))
tot = sum(a)
if tot % 2:
    print('NE')
else:
    t = tot // 2
    dp = [False] * (t + 1)
```

PYTHON 3 • potpuno rješenje / nastavak

```

dp[0] = True
for x in a:
    for s in range(t, x - 1, -1):
        dp[s] |= dp[s - x]
print('DA' if dp[t] else 'NE')

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    int tot = 0;
    for (int & x : a) {
        cin >> x;
        tot += x;
    }
    if (tot & 1) {
        cout << "NE\n";
        return 0;
    }
    int T = tot / 2;
    vector<char> d(T + 1);
    d[0] = 1;
    for (int x : a) for (int s = T; s >= x; s--) d[s] = d[s] || d[s - x];
    cout << (d[T] ? "DA" : "NE") << "\n";
}

```

Važne provjere

Sume se ažuriraju opadajuće.

Brojevi nisu neograničene kovanice; svaki se koristi jednom.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 25$	5
2	Ukupna suma ≤ 5000	7
3	Puna ograničenja	13

ZADATAK 156

Broj inverzija

Cilj: Broji parove $i < j$ sa $a_i > a_j$ u $O(N \log N)$ pomoću merge sorta.

Dat je niz od N cijelih brojeva. Inverzija je par indeksa $i < j$ za koji je $a_i > a_j$. Odredi ukupan broj inverzija.

Ulaz. N , zatim N cijelih brojeva.

Izlaz. Broj inverzija.

Ograničenja. $N \leq 300000$.

Primjer**ULAZ**

```
5
2 4 1 3 5
```

IZLAZ

```
3
```

Razumijevanje i postupak

Dvostruka petlja je $O(N^2)$. U merge sortu su obje polovine već sortirane. Kada $b[j]$ iz desne polovine ide prije $a[i]$ iz lijeve, on je manji od svih preostalih lijevih elemenata, pa odmah dodajemo njihov broj.

1. Rekurzivno riješi lijevu i desnu polovinu.
2. Merge sa pokazivačima i, j .
3. Ako $left[i] \leq right[j]$, uzmi lijevi. Inače dodaj $len(left) - i$ inverzija i uzmi desni.

Zašto postupak daje tačan rezultat?

Svaka inverzija pripada tačno jednom nivou rekurzije: nivou na kojem su njena dva elementa prvi put u različitim polovinama. Merge je zato broji tačno jednom.

Složenost. Vrijeme $O(N \log N)$, memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
a = list(map(int, input().split()))

def solve(a):
    if len(a) <= 1:
        return (a, 0)
    m = len(a) // 2
    L, x = solve(a[:m])
```

PYTHON 3 • potpuno rješenje / nastavak

```

R, y = solve(a[m:])
i = j = 0
c = x + y
z = []
while i < len(L) and j < len(R):
    if L[i] <= R[j]:
        z.append(L[i])
        i += 1
    else:
        z.append(R[j])
        j += 1
    c += len(L) - i
z += L[i:]
z += R[j:]
return (z, c)
print(solve(a)[1])

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
long long sol(vector < long long > & a, vector < long long > & tmp, int l,
    int r) {
    if (r - l <= 1) return 0;
    int m = (l + r) / 2;
    long long c = sol(a, tmp, l, m) + sol(a, tmp, m, r);
    int i = l, j = m, k = l;
    while (i < m && j < r) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else {
            tmp[k++] = a[j++];
            c += m - i;
        }
    }
    while (i < m) tmp[k++] = a[i++];
    while (j < r) tmp[k++] = a[j++];
    for (i = l; i < r; i++) a[i] = tmp[i];
    return c;
}
int main() {
    int n;
    cin >> n;
    vector < long long > a(n), t(n);
    for (auto & x : a) cin >> x;
    cout << sol(a, t, 0, n) << "\n";
}

```

Važne provjere

Broj inverzija može biti oko $N^2/2$, pa treba 64-bitni tip.

Jednaki elementi nisu inverzija jer uslov traži $>$.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 3000$	8
2	Svi elementi različiti	7
3	Puna ograničenja	15

ZADATAK 157

Labirint sa ključevima

Cilj: BFS proširuje stanjem skupa ključeva i provjerava uslove prolaska kroz vrata.

Data je mreža sa S, T, zidovima #, ključevima a,b,c i vratima A,B,C. Na ključ se može stati i tada ostaje trajno prikupljen; kroz vrata X može se proći samo ako imamo odgovarajući ključ x. Odredi minimalan broj poteza od S do T ili -1.

Ulaz. R C, zatim mapa.

Izlaz. Minimalan broj poteza ili -1.

Ograničenja. $R \cdot C \leq 100000$, najviše 3 vrste ključeva.

Primjer

ULAZ

```
4 7
S.a#..T
.#A#.#.
..bB...
#####
```

IZLAZ

```
10
```

Razumijevanje i postupak

Pozicija sama nije dovoljno stanje: dolazak na isto polje sa različitim skupom ključeva mijenja koje su buduće ivice dozvoljene. Bitmask 0..7 kodira a,b,c. BFS nad (r,c,mask) ima jedinične poteze.

1. Na S počni mask=0.
2. Pri ulasku na malo slovo uključi odgovarajući bit.
3. Na veliko slovo dopusti prelaz samo ako bit postoji.
4. visited[r][c][mask]; prvi T je optimalan.

Zašto postupak daje tačan rezultat?

Prošireni graf ima $R \cdot C \cdot 2^K$ stanja i jedinične prelaze, pa BFS daje najkraći legalni put uz tačno modelovane ključeve/vrata.

Složenost. Vrijeme $O(R \cdot C \cdot 8)$, memorija $O(R \cdot C \cdot 8)$.

PYTHON 3 • potpuno rješenje

```

from collections import deque
R, C = map(int, input().split())
g = [input().strip() for _ in range(R)]
for r in range(R):
    for c in range(C):
        if g[r][c] == 'S':
            sr, sc = (r, c)
D = [[[-1] * 8 for _ in range(C)] for __ in range(R)]
D[sr][sc][0] = 0
q = deque([(sr, sc, 0)])
while q:
    r, c, m = q.popleft()
    if g[r][c] == 'T':
        print(D[r][c][m])
        break
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = (r + dr, c + dc)
        if not (0 <= nr < R and 0 <= nc < C) or g[nr][nc] == '#':
            continue
        ch = g[nr][nc]
        nm = m
        if ch in 'abc':
            nm |= 1 << ord(ch) - 97
        if ch in 'ABC' and (not m >> ord(ch) - 65 & 1):
            continue
        if D[nr][nc][nm] < 0:
            D[nr][nc][nm] = D[r][c][m] + 1
            q.append((nr, nc, nm))
else:
    print(-1)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
struct S {
    int r, c, m;
}
;
int main() {
    int R, C;
    cin >> R >> C;
    vector < string > g(R);
    int sr = 0, sc = 0;
    for (int r = 0; r < R; r++) {
        cin >> g[r];
        for (int c = 0; c < C; c++) if (g[r][c] == 'S') sr = r, sc = c;
    }
    vector < vector < array < int, 8 >> > d(R, vector < array < int, 8 >> (C));
    for (auto & row : d) for (auto & a : row) a.fill(-1);
    queue < S > q;
    d[sr][sc][0] = 0;
    q.push({

```

C++17 • potpuno rješenje / nastavak

```

        sr, sc, 0
    }
    );
    int dr [4] = {
        1, - 1, 0, 0
    }
    , dc [4] = {
        0, 0, 1, - 1
    }
    ;
    while (! q.empty()) {
        auto [r, c, m] = q.front();
        q.pop();
        if (g [r] [c] == 'T') {
            cout << d [r] [c] [m] << "\n";
            return 0;
        }
        for (int k = 0; k < 4; k++) {
            int nr = r + dr [k], nc = c + dc [k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C || g [nr] [nc] ==
                '#') continue;
            char ch = g [nr] [nc];
            int nm = m;
            if (ch >= 'a' && ch <= 'c') nm |= 1 << (ch - 'a');
            if (ch >= 'A' && ch <= 'C' && ! (m >> (ch - 'A') & 1)) continue;
            if (d [nr] [nc] [nm] < 0) {
                d [nr] [nc] [nm] = d [r] [c] [m] + 1;
                q.push({
                    nr, nc, nm
                });
            }
        }
    }
    cout << - 1 << "\n";
}

```

Važne provjere

Kod provjere vrata koristi se stari mask `m` prije ulaska u vrata; vrata sama nisu ključ. `visited` mora imati dimenziju mask.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	Nema vrata	5
2	Samo ključ a/A	5
3	Puna ograničenja	10

ZADATAK 158

Putnik kroz stanice

Cilj: Bitmask DP-om rješava mali TSP: posjetiti sve tačke i vratiti se na početak.

Dato je N tačaka sa cjelobrojnim koordinatama, $2 \leq N \leq 16$. Putnik počinje u tački 1, mora posjetiti svaku tačku tačno jednom i vratiti se u 1. Cijena prelaska je Manhattan udaljenost. Odredi minimalnu ukupnu cijenu.

Ulaz. N , zatim N parova $x y$.

Izlaz. Minimalna cijena ture.

Ograničenja. $2 \leq N \leq 16$, $| \text{koordinate} | \leq 10^6$.

Primjer

ULAZ

```
4
0 0
1 0
1 1
0 1
```

IZLAZ

```
4
```

Razumijevanje i postupak

$dp[mask][i]$ je minimalna cijena puta koji počinje u 0, posjeti tačno mask i završava u i. Iz stanja dodajemo novi j. Na kraju iz punog maska dodajemo povratak iz i u 0.

1. INF tabela $2^N \times N$; $dp[1][0]=0$.
2. Za svaki mask koji sadrži 0 i svaki završetak i, pokušaj svaki j koji nije u maski.
3. Relaksiraj $dp[mask | 1 \ll j][j]$.
4. Odgovor $\min_i dp[full][i] + dist(i,0)$.

Zašto postupak daje tačan rezultat?

Svaka ruta odgovara jedinstvenom redoslijedu dodavanja elemenata u masku. Razmatranjem svih prethodnih stanja uz minimum dobijamo optimalnu rutu za svaku masku i kraj.

Složenost. Vrijeme $O(N^2 2^N)$, memorija $O(N 2^N)$.

PYTHON 3 • potpuno rješenje

```

n = int(input())
p = [tuple(map(int, input().split())) for _ in range(n)]

def d(i, j):
    return abs(p[i][0] - p[j][0]) + abs(p[i][1] - p[j][1])
INF = 10 ** 30
S = 1 << n
dp = [[INF] * n for _ in range(S)]
dp[1][0] = 0
for mask in range(S):
    if not mask & 1:
        continue
    for i in range(n):
        cur = dp[mask][i]
        if cur == INF:
            continue
        for j in range(n):
            if not mask >> j & 1:
                nm = mask | 1 << j
                v = cur + d(i, j)
                if v < dp[nm][j]:
                    dp[nm][j] = v
full = S - 1
print(min((dp[full][i] + d(i, 0) for i in range(n))))

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n;
    cin >> n;
    vector < pair < ll, ll >> p(n);
    for (auto & z : p) cin >> z.first >> z.second;
    auto dist = [&] (int i, int j) {
        return llabs(p[i].first - p[j].first) + llabs(p[i].second - p[j].second);
    };
    ;
    int S = 1 << n;
    const ll INF = 4e18;
    vector < vector < ll >> d(S, vector < ll > (n, INF));
    d[1][0] = 0;
    for (int m = 0; m < S; m++) if (m & 1) for (int i = 0; i < n; i++) if
        (d[m][i] < INF) for (int j = 0; j < n; j++) if (! (m >> j & 1)) {
        int nm = m | 1 << j;
        d[nm][j] = min(d[nm][j], d[m][i] + dist(i, j));
    }
}

```


C++17 • potpuno rješenje / nastavak

```

    }
    ll ans = INF;
    for (int i = 0; i < n; i++) ans = min(ans, d [S - 1] [i] + dist(i, 0));
    cout << ans << "\n";
}

```

Važne provjere

$N=16$ je malo upravo zbog faktora 2^N .

Maska mora sadržati početnu tačku 0 u svim relevantnim stanjima.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 10$	7
2	Sve tačke na pravoj	5
3	Puna ograničenja	13

ZADATAK 159

Maksimalni protok kroz mrežu

Cilj: Modelira kapacitete i implementira Edmonds-Karp na rezidualnom grafu.

Dat je usmjeren graf sa izvorom 1 i ponorom N. Svaka ivica ima nenegativan cjelobrojni kapacitet c. Odredi maksimalnu količinu toka koju je moguće poslati od 1 do N.

Ulaz. N M, zatim M redova u v c.

Izlaz. Vrijednost maksimalnog toka.

Ograničenja. $2 \leq N \leq 200$, $M \leq 2000$, $0 \leq c \leq 10^9$.

Primjer

ULAZ

```
4 5
1 2 3
1 3 2
2 3 1
2 4 2
3 4 4
```

IZLAZ

```
5
```

Razumijevanje i postupak

Rezidualni graf čuva koliko još možemo poslati svakom ivicom i koliko možemo “vratiti” prethodno poslani tok. BFS nalazi augmentirajući put sa pozitivnim rezidualnim kapacitetom. Najmanji kapacitet na putu je dodatni tok koji možemo poslati.

1. Izgradi $cap[u][v]$ sabiranjem paralelnih kapaciteta i listu susjeda u oba smjera.
2. BFS od s do t čuva parent samo po ivicama sa $cap > 0$.
3. Ako nema puta - završili smo.
4. Nađi bottleneck, smanji naprijed cap i povećaj nazad; dodaj u flow.

Zašto postupak daje tačan rezultat?

Teorem max-flow/min-cut garantuje optimalnost kada više nema augmentirajućeg puta. Svaka augmentacija čuva ograničenja kapaciteta i konzervaciju toka.

Složenost. Edmonds-Karp: $O(V \cdot E^2)$, memorija $O(V^2 + E)$ u ovoj matričnoj implementaciji.

PYTHON 3 • potpuno rješenje

```

from collections import deque
n, m = map(int, input().split())
cap = [[0] * n for _ in range(n)]
g = [[] for _ in range(n)]
for _ in range(m):
    u, v, c = map(int, input().split())
    u -= 1
    v -= 1
    if cap[u][v] == 0 and cap[v][u] == 0:
        g[u].append(v)
        g[v].append(u)
    cap[u][v] += c
s = 0
t = n - 1
flow = 0
while True:
    par = [-1] * n
    par[s] = s
    q = deque([s])
    while q and par[t] == -1:
        u = q.popleft()
        for v in g[u]:
            if par[v] == -1 and cap[u][v] > 0:
                par[v] = u
                q.append(v)
    if par[t] == -1:
        break
    add = 10 ** 30
    v = t
    while v != s:
        add = min(add, cap[par[v]][v])
        v = par[v]
    v = t
    while v != s:
        u = par[v]
        cap[u][v] -= add
        cap[v][u] += add
        v = u
    flow += add
print(flow)

```

C++17 • potpuno rješenje

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n, m;
    cin >> n >> m;
    vector < vector < ll >> cap(n, vector < ll > (n));
    vector < vector < int >> g(n);
    while (m--) {
        int u, v;

```

C++17 • potpuno rješenje / nastavak

```

    ll c;
    cin >> u >> v >> c;
    -- u;
    -- v;
    if (cap [u] [v] == 0 && cap [v] [u] == 0) {
        g [u].push_back(v);
        g [v].push_back(u);
    }
    cap [u] [v] += c;
}
int s = 0, t = n - 1;
ll flow = 0;
while (true) {
    vector < int > p(n, - 1);
    queue < int > q;
    p [s] = s;
    q.push(s);
    while (! q.empty() && p [t] < 0) {
        int u = q.front();
        q.pop();
        for (int v : g [u]) if (p [v] < 0 && cap [u] [v] > 0) {
            p [v] = u;
            q.push(v);
        }
    }
    if (p [t] < 0) break;
    ll add = LLONG_MAX;
    for (int v = t; v != s; v = p [v]) add = min(add, cap [p [v]] [v]);
    for (int v = t; v != s; v = p [v]) {
        int u = p [v];
        cap [u] [v] -= add;
        cap [v] [u] += add;
    }
    flow += add;
}
cout << flow << "\n";
}

```

Važne provjere

Obrnuta rezidualna ivica je ključna čak i ako nije postojala u originalnom grafu.

Paralelne ivice treba sabrati u matrici kapaciteta.

Podzadaci za trening

Testna grupa	Ograničenje	Bodovi
1	$N \leq 30$, $M \leq 100$	8
2	Graf je slojevit bez povratnih ivica u ulazu	7
3	Puna ograničenja	15

23. UPINITK: dodatni srednjoškolski izazovi 2024

Ova tri zadatka potiču s kantonalnog takmičenja za srednje škole iz 2024. godine. U osnovnoškolskoj zbirci služe isključivo kao dodatni izazovi za učenike koji već sigurno vladaju potrebnim pojmovima. Nisu označeni kao općinski osnovnoškolski zadaci. [A4]

Zadatak	Naziv
160	KLIKERI
161	PJESME
162	LAGNO

ZADATAK 160 / UPINITK ARHIVA

KLIKERI

Kantonalno takmičenje za srednje škole, 2024. • [A4], PDF str. 13.

Premještaj klikere između dvije osobe dok prvi broj ne postane djeljiv drugim. Oboma mora ostati barem jedan.

Ulaz. A i B.

Izlaz. Najmanje premještenih klikera.

Ograničenja. 1 A,B 1000.

Primjer

ULAZ

8 7

IZLAZ

2

Dva klikera prelaze prvoj osobi. Nova raspodjela 10 i 5 zadovoljava djeljivost; nijedno pojedinačno premještanje nije dovoljno.

Razumijevanje i postupak

Ukupan broj $T = A + B$ ne mijenja se. Ako druga osoba na kraju ima b klikera, prva ima $T - b$. Uslov djeljivosti je $(T - b) \% b == 0$.

Za izabrano b potrebno je premjestiti tačno $|b - B|$ klikera: ako b raste, klikeri prelaze drugoj osobi; ako pada, prelaze prvoj. Isprobamo sve b od 1 do $T-1$. Tako oba konačna broja ostaju pozitivna.

1. Sačuvaj ukupan broj T .
2. Za svako b od 1 do $T-1$ provjeri djeljivost $T - b$ sa b .
3. Među ispravnim kandidatima minimiziraj $|b - B|$.

Zašto postupak daje tačan rezultat?

Svaka konačna raspodjela određena je brojem b i pojavljuje se u pretrazi. Za nju apsolutna razlika daje nužan i dovoljan broj premještanja. Najmanja vrijednost među svim dopuštenim raspodjelama zato je optimalna.

Složenost. Vrijeme $O(A+B)$; memorija $O(1)$.

PYTHON 3 • potpuno rješenje

```
import sys
a, b = map(int, sys.stdin.read().split())
ukupno = a + b
odgovor = ukupno
for novo_b in range(1, ukupno):
    if (ukupno - novo_b) % novo_b == 0:
        odgovor = min(odgovor, abs(novo_b - b))
print(odgovor)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    int t = a + b, ans = t;
    for (int nb = 1; nb < t; nb++) if ((t - nb) % nb == 0) ans = min(ans,
        abs(nb - b));
    cout << ans << '\n';
}
```

Važne provjere

Ne dopustiti da druga osoba dobije nulu.

Ne tražiti samo jednake brojeve: djeljivost je širi uslov.

ZADATAK 161 / UPINITK ARHIVA

PJESME

Kantonalno takmičenje za srednje škole, 2024. • [A4], PDF str. 14.

Nađi najraniju riječ teksta do koje se pojavila barem polovina različitih riječi naslova.

Ulaz. N i N riječi naslova; M i M riječi teksta, riječi u zasebnim redovima.

Izlaz. Pozicija prve dovoljne riječi, brojeći od 1.

Ograničenja. N 50; M 10 000; naslovne riječi su različite; rješenje postoji.

Primjer

ULAZ

```
3
more
sunce
put
6
put
i
put
more
je
tu
```

IZLAZ

```
4
```

Riječ put nalazi se i na poziciji 1 i 3, ali vrijedi samo jednom. Druga različita naslovna riječ more dolazi na poziciji 4.

Razumijevanje i postupak

Ponavljanje iste riječi ne povećava broj prepoznatih naslovnih riječi. Zato u skupu čuvamo samo riječi koje još nisu pronađene. Kad pročitamo takvu riječ, uklonimo je i povećamo brojač.

Ako je N neparan, barem polovina znači zaokruživanje naviše. Umjesto realnog dijeljenja provjeravamo $2 \cdot \text{broj } N$. Čim uslov prvi put postane tačan, trenutna pozicija je odgovor. Ne smijemo je kasnije prepisati.

1. U skup smjesti riječi naslova.
2. Redom čitaj tekst i uklanjaj novopronađene naslovne riječi.
3. Sačuvaj prvu poziciju za koju vrijedi $2 \cdot \text{broj } N$.

Zašto postupak daje tačan rezultat?

Nakon i riječi brojač je broj različitih naslovnih riječi u tom prefiksu, jer svaku uklanjamo najviše jednom. Redosljed obrade osigurava da je prvi dovoljan prefiks ujedno najkraći.

Složenost. Očekivano vrijeme $O(N+M)$ za riječi ograničene dužine; memorija $O(N)$.

PYTHON 3 • potpuno rješenje

```
n = int(input())
trazene = {input().strip() for _ in range(n)}
m = int(input())
broj = 0
odgovor = -1
for pozicija in range(1, m + 1):
    rijec = input().strip()
    if rijec in trazene:
        trazene.remove(rijec)
        broj += 1
    if odgovor == -1 and 2 * broj >= n:
        odgovor = pozicija
print(odgovor)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n;
    unordered_set < string > s;
    for (int i = 0; i < n; i++) {
        string w;
        cin >> w;
        s.insert(w);
    }
    cin >> m;
    int broj = 0, ans = - 1;
    for (int i = 1; i <= m; i++) {
        string w;
        cin >> w;
        if (s.erase(w)) broj++;
        if (ans == - 1 && 2 * broj >= n) ans = i;
    }
    cout << ans << '\n';
}
```

Važne provjere

Ponavljanje ne povećava broj različitih prepoznatih riječi.

Za $N=3$ treba pronaći 2 riječi, ne jednu.

ZADATAK 162 / UPINITK ARHIVA

LAGNO

Kantonalno takmičenje za srednje škole, 2024. • [A4], PDF str. 15.

Na prazno polje table stavi crni žeton. Prebroj bijele nizove omeđene novim i postojećim crnim žetonom, u osam smjerova.

Ulaz. Osam redova po osam znakova: . prazno, C crno, B bijelo.

Izlaz. Najviše preokrenutih bijelih žetona; 0 ako ih nema.

Primjer

ULAZ

```
.....
.....
.....
.CBB....
.....
.....
.....
.....
```

IZLAZ

```
2
```

Crni žeton na petom polju četvrtog reda omeđuje dva bijela s postojećim crnim žetonom.

Razumijevanje i postupak

Svako prazno polje isprobavamo kao kandidat. Iz njega gledamo horizontalno, vertikalno i dijagonalno. U jednom smjeru prvo mora doći barem jedan bijeli žeton. Niz tih bijelih vrijedi samo ako se završava crnim žetonom prije izlaska iz table.

Prazno polje ili rub prekidaju mogućnost zarobljavanja. Za jedan kandidat sabiramo rezultate svih smjerova, ali tablu ne mijenjamo: svi smjerovi procjenjuju se prema istom početnom rasporedu. Tek rezultat cijelog kandidata poredi se s dotadašnjim maksimumom.

1. Obiđi sva prazna polja.
2. U svakom od osam smjerova prebroj neposredni niz B.
3. Dodaj taj broj samo ako sljedeće polje postoji i sadrži C.
4. Sačuvaj najveći zbir za jedan potez.

Zašto postupak daje tačan rezultat?

U jednom smjeru pravilo može zahvatiti samo početni neprekinuti niz bijelih žetona. Provjera završnog crnog žetona tačno odlučuje da li je taj niz omeđen. Osam smjerova pokriva sve dopuštene pravce, a obilazak svih praznih polja sve poteze.

Složenost. Za tablu 8×8 vrijeme i memorija su $O(1)$. Za tablu $K \times K$ direktni postupak je $O(K^3)$, uz $O(K^2)$ memorije.

PYTHON 3 • potpuno rješenje

```
a = [input().strip() for _ in range(8)]
smjerovi = [(dr, dk) for dr in (-1, 0, 1)
             for dk in (-1, 0, 1) if (dr, dk) != (0, 0)]
najbolje = 0
for r in range(8):
    for k in range(8):
        if a[r][k] != ".":
            continue
        ukupno = 0
        for dr, dk in smjerovi:
            x, y = r + dr, k + dk
            broj = 0
            while 0 <= x < 8 and 0 <= y < 8 and a[x][y] == "B":
                broj += 1
                x += dr
                y += dk
            if 0 <= x < 8 and 0 <= y < 8 and a[x][y] == "C":
                ukupno += broj
        najbolje = max(najbolje, ukupno)
print(najbolje)
```

C++17 • potpuno rješenje

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector < string > a(8);
    for (auto & s : a) cin >> s;
    int ans = 0;
    for (int r = 0; r < 8; r++) for (int k = 0; k < 8; k++) {
        if (a[r][k] != '.') continue;
        int suma = 0;
        for (int dr = -1; dr <= 1; dr++) for (int dk = -1; dk <= 1; dk++) {
            if (dr == 0 && dk == 0) continue;
            int x = r + dr, y = k + dk, broj = 0;
            while (x >= 0 && x < 8 && y >= 0 && y < 8 && a[x][y] == 'B') {
                broj++;
                x += dr;
                y += dk;
            }
            if (x >= 0 && x < 8 && y >= 0 && y < 8 && a[x][y] == 'C')
                suma += broj;
        }
        ans = max(ans, suma);
    }
    cout << ans << '\n';
}
```

Važne provjere

Bijeli niz bez završnog C ne donosi rezultat.

Ne mijenjati tablu tokom procjene smjerova jednog kandidata.

24. Trening, provjera i pojmovnik

Plan jednog rada na zadatku

Najprije pročitajte tekst bez gledanja programa. Zapišite šta ulazi i šta izlazi. Napravite najmanji smislen primjer, predložite jednostavan algoritam i procijenite njegovu cijenu. Tek tada birajte optimizaciju. Nakon implementacije ponovo pročitajte specifikaciju: mnoge greške su posljedica pogrešnog izlaza, jednakosti na granici ili zanemarenog slučaja.

Tri seta za vježbu — urednički prijedlog

Početni set • 90 minuta

Zadatak	Naziv	Bodovi
17	TRKA	20
21	VISINE	20
22	PLASTENIK	25
23	SEKCIJA	35

Općinski problemski set • 120 minuta

Zadatak	Naziv	Bodovi
25	DOMINE	20
26	SILOSI	20
19	PIZZA	25
24	BAZEN	35

Dodatni izazov • 150 minuta

Zadatak	Naziv	Bodovi
20	BROJKE I SLOVA	20
160	KLIKERI	25
161	PJESME	25
162	LAGNO	30

Ovo su prijedlozi treninga, ne službene propozicije. Brojevi bodova ovdje su urednički raspored po zadacima; ne predstavljaju automatsko bodovanje skrivenih testnih grupa.

Šta je provjereno u programskom paketu

Svih 162 para rješenja provjereno je na prikazanim primjerima. Postojeće vježbe provjerene su na svojim primjerima, a 15 dodatih arhivskih zadataka i na 854 dodatna mala, granična i generisana testna slučaja. Python i C++ rezultati upoređeni su s očekivanim izlazima. Za neke probleme očekivanje je računato nezavisnim sporijim postupkom, npr. sparivanjem polja za domine i neposrednim prebrojavanjem vremena za bazen.

Testiranje ne dokazuje ispravnost na svim dopuštenim ulazima i ne zamjenjuje stručnu recenziju. U pratećem paketu nalaze se izvorni kodovi, ulazi, očekivani izlazi i izvještaji o stvarno provedenim provjerama.

Mali pojmovnik

Pojam	Značenje
Algoritam	Precizan postupak koji za dopušten ulaz daje traženi izlaz.
Invarijanta	Tvrdnja koja ostaje tačna nakon svakog koraka petlje ili postupka.
Prefiks	Početni dio niza ili stringa.
Podniz	Niz dobijen izostavljanjem elemenata uz očuvanje redoslijeda; ne mora biti neprekinut.
Podstring	Neprekinuti dio stringa.
Relaksacija	Pokušaj poboljšanja trenutno poznate udaljenosti preko jedne ivice.
Stanje DP-a	Precizno definisan podproblem čiji odgovor čuvamo.
Kolizija sažetka	Različiti podaci imaju isti hash; jednak potpis zato nije opći dokaz jednakosti.
WA / TLE / RE	Pogrešan odgovor / prekoračeno vrijeme / greška pri izvršavanju.

25. Izvori i porijeklo zadataka

Arhiva takmičenja

[A1] UPINITK. Gradska/općinska takmičenja za osnovne škole, 2021.

<https://upinitk.ba/wp-content/uploads/2021/02/Bilten2.pdf>

Obrađeni zadaci: TRKA, KOVANICE, PIZZA, BROJKE I SLOVA.

[A2] UPINITK. III gradsko/općinsko takmičenje za osnovne škole, 2022.

<https://upinitk.ba/wp-content/uploads/2022/04/Bilten3JTKGradsko.pdf>

Obrađeni zadaci: VISINE, PLASTENIK, SEKCIJA, BAZEN.

[A3] UPINITK. IV gradsko/općinsko takmičenje za osnovne škole, 2023.

<https://upinitk.ba/wp-content/uploads/2022/05/Bilten4T4.pdf>

Obrađeni zadaci: DOMINE, SILOSI, DAN JEDNAKOSTI, BLAGO.

[A4] UPINITK. Kantonalno takmičenje za srednje škole, 2024.

<https://upinitk.ba/wp-content/uploads/2022/05/Bilten-Informatika-2024.pdf>

Obrađeni zadaci: KLIKERI, PJESME, LAGNO.

Pristupne stranice arhive

<https://upinitk.ba/takmicenja/takmicenja-za-osnovne-skole/>

<https://upinitk.ba/takmicenja/takmicenja-za-srednje-skole/>

<https://upinitk.ba/bilteni/>

Jezici i programsko okruženje

[5] Python Software Foundation. The Python Tutorial.

<https://docs.python.org/3/tutorial/>

[6] GNU Project. GCC — Invoking G++.

https://gcc.gnu.org/onlinedocs/gcc/Invoking-G_002b_002b.html

Pripremni rukopis

Preostale vježbe i osnovne teorijske bilješke potiču iz prethodno sastavljenog rukopisa zbirke, koji je za ovo izdanje tematski reorganizovan i tipografski prerađen. Ranije opće napomene o inspiraciji HONI arhivom nisu pretvorene u tvrdnju da je riječ o doslovno preuzetim ili službeno objavljenim HONI zadacima. Za provjereno arhivsko porijeklo u ovom izdanju služe isključivo oznake [A1]–[A4].